Learned Sparse and Hybrid Sparse–Linear Attention for Long-Context LLM Inference

Trainable sparse attention, shared-routing and indexer designs, and KV-cache eviction and compression

A technical review with derivations, numerical experiments and an evidence ledger (covering developments from October 2025 to October 2026)

Research compilation prepared with Claude (Anthropic)

27 October 2026

Abstract

Long-context inference is limited less by arithmetic than by memory: each decoded token must read the key–value (KV) cache of the whole context, and the cache must be stored. Between October 2025 and October 2026 the main response was *learned* sparse attention (DeepSeek Sparse Attention, MiniMax Sparse Attention, Native Sparse Attention and related designs), together with shared-routing schemes that amortise the selector's own cost, hybrids that replace most attention layers by linear-state layers, and KV-cache eviction and quantisation. This report surveys these developments with a unified cost model, derivations, numerical experiments and a critical review of the retrieved sources.

The mathematical core covers a roofline and byte-accounting model of decode, an exact identity for the error of renormalised top-k attention with consequences for budgets, block versus token selection, needle retrieval through a noisy indexer and KL-trained indexers, the amortisation and placement of shared indexers, the recall capacity of linear-state layers, and the effect of rotation and scalar quantisation on attention. Simulations check each formula; a small trained model (Qwen3-0.6B) provides measurements of top-k coverage, cross-layer overlap, key quantisation and eviction.

The main findings are: the index scan, not the sparse attention itself, dominates bytes read at long context in a V3.2-style layer (88% at 128K in the model); sharing the scan across four layers raises the byte reduction from 7.7× to 22.6×; the output error of top-k is proportional to discarded mass with a constant set by value coherence; block selection cannot capture more mass than token selection at equal budget; and author-reported parity with dense baselines is established for average benchmarks at the scales tested, but remains thinly evidenced for aggregation, non-literal retrieval and contexts of 10^6 tokens or more. Vendor claims (notably SubQ) and secondary reports are flagged and not treated as established. Open problems are listed with decisive experiments.

Contents

1 Introduction and Scope **1**

1.1 The problem: attention is the long-context bill . . . 1
1.2 Three families of answers . . . 1
1.3 Thesis and organising questions . . . 2
1.4 Conventions on evidence and provenance . . . 3
1.5 Scope and time frame . . . 4
1.6 Reading guide . . . 4

2 Background and History **6**

2.1 Softmax attention and the decoder-only Transformer . . . 6
2.2 Strand 1: fixed sparse attention patterns (2019–2021) . . . 6
2.3 Strand 2: shrinking the cache by sharing and compressing heads (2019–2024) 7
2.4 Strand 3: exact attention, made fast (2022–2023) . . . 7
2.5 Strand 4: linear attention and state-space models (2020–2025) . . . 8
2.6 Strand 5: training-free sparsity and cache compression (2023–2025) . . . 8
2.7 2025: the shift to trainable sparsity . . . 9
2.8 Timeline . . . 10

3 Cost Models: FLOPs, Bytes and the Roofline **12**

3.1 Dense attention: arithmetic and memory . . . 12
3.1.1 Prefill . . . 12
3.1.2 Decode . . . 13
3.1.3 The roofline . . . 13
3.2 KV-cache memory formulas . . . 13
3.2.1 GQA, MQA, MHA . . . 13
3.2.2 MLA . . . 14
3.2.3 Additional caches introduced by sparse attention . . . 14
3.2.4 Quantized caches . . . 14
3.3 Throughput as a function of batch size . . . 15
3.4 Cost of a sparse decode step with an indexer . . . 15
3.5 Prefill with a token-level indexer . . . 17
3.6 Putting numbers on shared routing . . . 17
3.7 A cautionary comparison with published speedups . . . 18
3.8 Summary of the cost chapter . . . 19

4 Approximation Theory of Sparse Attention **20**
4.1 Definitions . . . 20
4.2 The basic error identity . . . 20
4.3 How much mass does top-k capture? . . . 21
4.3.1 Power-law (Zipf) attention . . . 21
4.3.2 Gaussian-logit attention . . . 22
4.3.3 Output error versus tail mass . . . 23
4.4 Block-sparse selection . . . 24
4.5 Noisy indexers . . . 25
4.6 Indexer training objectives and what they guarantee . . . 26
4.7 Error propagation through depth . . . 27
4.8 What a fixed budget cannot do: aggregation tasks . . . 27
4.9 Measured on a real model . . . 28

5 Linear Attention, State Capacity and Hybrid Cost Models **31**
5.1 From softmax to linear attention . . . 31
5.2 Forgetting, gating and the delta rule . . . 32
5.3 Capacity of a fixed-size state . . . 32
5.3.1 Interference in additive memory . . . 32
5.3.2 Recency bias under decay . . . 33
5.3.3 An information-theoretic bound . . . 34
5.4 Memory and cost of a recurrent layer . . . 34
5.5 Hybrid layer cost models . . . 35
5.6 Choosing the layer ratio . . . 36
5.7 Why hybrids are cheaper to *train into* than to train . . . 36

6 Shared Routing: Amortisation Analysis **38**
6.1 Where can a scan be shared? . . . 38
6.2 Quality of a shared selection . . . 38
6.3 Choosing which layers keep an indexer . . . 39
6.4 Cross-decoder sharing: all layers use one index . . . 40
6.5 Amortisation in the training of the indexer . . . 41
6.6 Real-model test of the sharing premise . . . 41

7 KV-Cache Quantization and Eviction: Error Analysis **44**
7.1 How cache perturbations reach the output . . . 44
7.2 Uniform quantization and the outlier problem . . . 45
7.3 Random rotation and Lloyd–Max: the TurboQuant construction . . . 45
7.4 What TurboQuant reports, and what is contested . . . 47
7.5 Eviction: what a fixed cache cannot do . . . 48

7.5.1 Accumulated-score bias . . . 48
7.5.2 Retention under redundancy: why random eviction can work . . . 48
7.5.3 Pre-RoPE concentration and the TriAttention score . . . 49
7.6 Real-tensor experiments on quantization and eviction . . . 49
7.7 How the savings compose . . . 51

8 Trainable Sparse Attention: Method-by-Method 52
8.1 Native Sparse Attention (NSA) . . . 52
8.2 MoBA: Mixture of Block Attention . . . 53
8.3 InfLLM-V2: dense–sparse switchable attention . . . 54
8.4 DeepSeek Sparse Attention (DSA) . . . 54
8.5 MiniMax Sparse Attention (MSA) . . . 56
8.6 Gated Sparse Attention (GSA) . . . 57
8.7 DeepSeek-V4: compressed sparse attention and heavily compressed attention 58
8.8 SubQ: vendor claims about sparse attention at 12M tokens . . . 59
8.9 Comparison of designs . . . 60

9 Shared Routing and Indexer Designs 61
9.1 Cross-layer sparse attention (CLSA) on YOCO . . . 61
9.2 IndexCache . . . 63
9.3 HySparse and HySparse2: oracle selection and KV sharing . . . 64
9.4 SparDA: Forecast projection and offload-friendly decoupling . . . 65
9.5 HISA and MISA: shrinking the token-level scan without retraining . . . 66
9.6 AsyncTLS: two-level sparsity with asynchronous offload . . . 67
9.7 Token Sparse Attention (TSA) . . . 68
9.8 Synthesis: what is shared, and what is lost . . . 68

10 Hybrid Linear/Sparse Models 70
10.1 Lightning Attention and MiniMax-01 . . . 70
10.2 The MiniMax M2 decision: why full attention . . . 70
10.3 Kimi Linear . . . 71
10.4 Qwen3-Next . . . 71
10.5 Jamba and earlier hybrids . . . 71
10.6 What the systematic study says about ratios . . . 72
10.7 MiniCPM-SALA in detail . . . 72
10.8 Naive-N0.5-Flash and other pure-sparse hybrids . . . 74
10.9 Comparison of hybrid designs . . . 75

11 KV-Cache Eviction and Compression Methods **76**
11.1 A taxonomy . . . 76
11.2 The classical lineage . . . 76
11.3 TriAttention . . . 78
11.4 Random Attention . . . 79
11.5 The critical evaluation: pitfalls of compression . . . 80
11.6 Evaluation hazards specific to eviction . . . 80

12 Systems and Kernels **82**
12.1 Why sparse attention is hard to make fast . . . 82
12.2 Kernel designs reported . . . 83
12.3 The offload regime . . . 84
12.4 Quantisation kernels and the cost of rotation . . . 84
12.5 Linear-attention kernels and serving . . . 85
12.6 Training-side systems . . . 85
12.7 Summary of implementation patterns . . . 86

13 Empirical Review **87**
13.1 Quality: do trained sparse models match dense ones? . . . 87
13.2 Long-context retrieval: what the benchmarks cover . . . 87
13.3 Efficiency claims, by level . . . 88
13.4 Efficiency per training dollar: what the conversions cost . . . 89
13.5 Statistical quality of the evidence . . . 89
13.6 Summary of what the evidence establishes . . . 91

14 Comparing the Approaches **92**
14.1 Organising by the factor attacked . . . 92
14.2 Requirements imposed on training . . . 92
14.3 Failure modes by family . . . 93
14.4 Composition: how the savings stack, and where they do not . . . 93
14.4.1 Multiplicative and non-multiplicative pairs . . . 93
14.4.2 Byte accounting for a reference configuration . . . 94
14.4.3 Interaction effects that arithmetic misses . . . 95
14.5 Decision procedure . . . 96
14.6 Head-to-head: three representative stacks . . . 96

15 Advantages, Disadvantages and Failure Modes **98**
15.1 Learned token- and block-level sparse attention . . . 98
15.1.1 Advantages . . . 98
15.1.2 Disadvantages . . . 98

15.2 Shared routing and cheaper selectors . . . 99
15.3 Hybrid sparse/linear models . . . 100
15.4 Eviction and compression of the cache . . . 100
15.5 Cross-cutting failure modes . . . 100
15.6 Summary scorecard . . . 100

16 Community Perception and Reception **102**
16.1 Episode 1: DSA and the price cut . . . 102
16.2 Episode 2: MiniMax, from lightning attention to full attention to sparse attention . . . 102
16.3 Episode 3: TurboQuant, publicity and dispute . . . 103
16.4 Episode 4: SubQ and the 12-million-token claim . . . 103
16.5 Episode 5: the critical literature on evaluation . . . 104
16.6 Themes . . . 104
16.7 Perception of the technical bets . . . 105

17 Case Studies **106**
17.1 Case 1: DeepSeek V3.2-Exp to V3.2 to V4 . . . 106
17.2 Case 2: MiniMax, M2 to M3 . . . 107
17.3 Case 3: MiniCPM-SALA, a sparse–linear hybrid at 9B . . . 107
17.4 Case 4: SubQ . . . 108
17.5 Case 5: TurboQuant . . . 109
17.6 Case 6: GLM-5.2 and IndexShare . . . 109
17.7 Cross-case lessons . . . 110

18 Open Problems **111**
18.1 Evaluation . . . 111
18.2 Mathematics and theory . . . 111
18.3 Training . . . 112
18.4 Systems . . . 113
18.5 Scientific questions about the models . . . 113
18.6 Claims that would change this report's conclusions . . . 114

19 Conclusion **115**
19.1 What can be said with confidence . . . 115
19.2 What the evidence does not settle . . . 115
19.3 Practical guidance . . . 116
19.4 Limitations of this report . . . 116

A Extended Derivations **117**
A.1 Arithmetic intensity of decode attention . . . 117
A.2 Amdahl bound for end-to-end speedup . . . 117
A.3 Throughput under a capacity limit . . . 118
A.4 Euler–Maclaurin for Zipf-distributed attention . . . 118
A.5 Mass of the top fraction under Gaussian logits . . . 119
A.6 Error of random-value truncation . . . 119
A.7 KL divergence from logit noise . . . 119
A.8 Lloyd–Max quantisation of a Gaussian source . . . 120
A.9 Chunkwise form of linear attention and the gated delta rule . . . 120
A.10 Interference in an additive associative memory . . . 121
A.11 Hierarchical selection: optimal block size . . . 121
A.12 From KL to captured mass: Pinsker and its limits . . . 121

B Experiments: Setup, Results and Code **122**
B.1 Environment . . . 122
B.2 Summary of experiments and results . . . 122
B.3 Limitations specific to the experiments . . . 122
B.4 Script listings . . . 123
B.4.1 exp_a_synthetic_topk.py . . . 123
B.4.2 exp_b_block_needle.py . . . 124
B.4.3 exp_c_quant.py . . . 126
B.4.4 exp_d_linear_capacity.py . . . 127
B.4.5 exp_e_costmodel.py . . . 128
B.4.6 exp_real_qwen.py . . . 130
B.4.7 exp_real_cross_ctrl.py . . . 132
B.4.8 exp_f_real_quant_evict.py . . . 133

C Evidence Ledger **136**
C.1 Known inconsistencies in retrieved sources . . . 138

D Glossary **139**

List of Figures

3.1 Bytes read per layer per decoded token versus context length (log–log). Left: DSA-like MLA layer. Right: GQA layer with Qwen3-8B dimensions and an MSA-like index branch (one index head per KV group, BF16, $d_I = 128$). In both cases the index scan grows linearly while the sparse attention is flat at $k = 2048$ tokens (derived). 16

3.2 Byte reduction versus dense MLA as the indexer pass is shared across more layers (derived from Table 3.3). Diminishing returns set in when the sparse-attention read (2.36 MB) dominates. 18

4.1 Captured attention mass m_k versus budget k for synthetic Zipf (left) and Gaussian-logit (right) attention with $N = 32{,}768$ ([SIM]). The spread across the curves at $k = 2048$ (0.32 to 0.99 on the left) illustrates why a single fixed budget is a poor description of "how sparse attention is". 23

4.2 Left: output error of renormalised top-k attention versus k for Zipf attention ([SIM]). Right: measured error against the incoherent-value prediction (4.6) (circles, on the $y = x$ line) and against the tail mass τ_k (crosses, far below the worst-case scale). 24

4.3 Left: with Gaussian logits at $\sigma = 2, 3$, a noisy indexer with logit-noise τ_n degrades both captured mass (solid) and recall of the oracle set (dashed). Right: probability that a planted needle is among the top 2048 of 32,768 versus its logit boost Δ ([SIM]). 27

4.4 Qwen3-0.6B, 2048-token windows ([SIM]). Left: captured mass versus budget; the band is the 10th–90th percentile over (layer, head) pairs. Right: mean relative output error of renormalised oracle top-k and mean tail mass; the two curves are parallel on the log scale (ratio ≈ 1.8). 29

4.5 Left: m_{128} for every (layer, head). Right: mean over heads of m_{128}, of the mass on the first token (sink) and of the mass on the most recent 64 tokens, by layer ([SIM]). Early layers are local; later layers are sink-dominated. 30

5.1 Associative recall quality versus number of stored pairs for head dimensions $32, 64, 128$ ([SIM]). Dotted vertical lines mark $n = d$. The delta rule is best below $n \approx d$ and loses older items beyond; additive superposition degrades smoothly; softmax lookup is exact because it keeps all pairs. 34

6.1 Overlap (left) and mass ratio (right) versus layer gap, raw and controlled ([SIM], Qwen3-0.6B). 42

7.1 Left: simulated MSE of rotation plus Lloyd–Max quantization versus the information-theoretic lower bound and the upper-bound constant $\sqrt{3}\pi/2$ (the plotted constant $\sqrt{3}\pi/2 \approx 2.72$ is the one implied by the paper's quoted gap of about 2.7 to the lower bound). Right: effect of outlier channels on min–max quantization with and without rotation ([SIM]). 47

List of Tables

1.1 Notation. 5

2.1 Timeline of principal developments referenced in this report (dates from retrieved pages). 11

3.1 KV-cache bytes per token and total size in GiB at several context lengths for a single sequence (derived; configuration files for Qwen3-8B and DeepSeek-V3.2-Exp; 4B transformer and CLSA dimensions from [1]). The indexer-key layout is an assumption. The CLSA row counts only the single global KV cache shared by cross-decoder layers; the self-decoder's 512-token windows and the indexer keys are not included. 14

3.2 Per-layer, per-decoded-token bytes and FLOPs for one DeepSeek-V3.2-style MLA layer ($H = 128$, absorbed MQA form, $c_{KV} = 512$, $c_R = 64$, BF16 latent) under dense attention versus DSA with $k = 2048$, $H_I = 64$, $d_I = 128$ and a 132-byte FP8 index key (assumed). Derived by `exp_e_costmodel.py`. 16

3.3 Effect of sharing one indexer pass across m layers on a DSA-like layer at 128K tokens (derived; $k = 2048$, 132-byte FP8 index keys, 576-wide BF16 latent). "Indexer share" is the fraction of the layer's bytes spent on index keys. 18

4.1 Simulated top-k attention mass m_k for Zipf weights with exponent α and $N = 32{,}768$ ([SIM]; `exp_a_synthetic_topk.py`), and the value of (4.4) at $k = 2048$ (last column). 22

4.2 Top-2048 attention mass for i.i.d. Gaussian logits with scale σ: simulation versus the closed form (4.5) ([SIM], [DER]; `exp_b_block_needle.py`, mean of 8 draws). 23

4.3 Oracle captured mass at a 2048-token budget for Zipf attention ($N = 32{,}768$) under token-granular versus block-granular oracle selection, with heavy tokens placed i.i.d. ($c = 0$) or in contiguous runs of length c aligned to multiples of c ([SIM]; runs of 128 are aligned with blocks of 64 and 128 and therefore lose nothing, an artefact of the toy layout). 25

4.4 Needle selection probability through a noisy indexer: simulated (3000 trials) versus the closed form (4.7); $N = 32{,}768$, $k = 2048$. The last column is the needle's mean dense-softmax attention mass (from the 400-trial run of `exp_a`; it does not depend on τ_n). [SIM], [DER]. 26

4.5 Oracle top-k attention on Qwen3-0.6B ([SIM]; 28 layers × 16 heads, 3 windows, queries with 1025–2048 visible keys). "p10/p90" are percentiles across the 448 (layer, head) pairs of the mean mass; "min" is the worst head; the last two columns give the mean relative output error and its ratio to the tail mass $1 - m_k$. 28

5.1 Mean cosine similarity between recalled and true values for n stored random key–value pairs ([SIM]). "add" is the un-gated additive state, "Δ" is the delta rule (5.2) with $\beta = 1$, "formula" is (5.4), and "softmax" is lookup in an explicit cache. 33

5.2 Decode bytes per token and stored size for a 32-layer hybrid with Qwen3-8B-like attention dimensions ($G = 8$, $d = 128$), sparse budget $k = 2048$, 32 linear heads of 128×128 BF16 state per linear layer, and an index key of 132 bytes per token per sparse layer ("token-idx") or 132/64 bytes ("block-idx", pooled over 64-token blocks) ([DER]; `exp_e_costmodel.py`). The reference row is dense GQA with the same dimensions. 36

6.1 Levers for reducing the selector cost, by factor of (6.1). 39

6.2 Cross-layer sharing of the head-averaged top-k set on Qwen3-0.6B ([SIM]; mean over layers, 1024 queries and three windows). "Raw" includes all positions; "controlled" removes the sink and the last 64 tokens. "ovl." is the set overlap and "ratio" the mass ratio ρ_k. 42

7.1 MSE of rotation plus Lloyd–Max scalar quantization of unit vectors ($d = 128$) from the simulation `exp_c_quant.py` ([SIM]), compared with the values retrieved from the TurboQuant paper and its lower bound. 45

7.2 Inner-product estimation for random unit vector pairs ($d = 128$, 4000 pairs; [SIM]). "Slope" is the regression coefficient of the estimate on the true inner product; $1 - D_b$ predicts $0.884, 0.966, 0.991$ for the MSE-only column. Both settings use the same number of bits per coordinate. 46

7.3 Key quantisation on real Qwen3-0.6B tensors ([SIM]; mean over layers 2, 9, 16, 23; values unquantised). Output errors above 1 mean the attention output is destroyed. Per-layer numbers are in `results/real_quant_evict.json`. 50

7.4 Retained dense attention mass under eviction policies ([SIM]; Qwen3-0.6B, 2048 tokens, four heads per layer). 51

8.1 DeepSeek-V3.1-Terminus versus DeepSeek-V3.2-Exp (the first model with DSA), as listed on the model card (retrieved 27 Oct. 2026; author-reported, single-run). 55

8.2 Design axes of the trainable sparse and compressed attention methods (n/r = not retrieved). Scan terms are those derived or stated in this chapter. 60

9.1 CLSA results at 4B parameters (author-reported). At 32K the CLSA gain over dense YOCO is concentrated in multi-needle tasks (MK1 58.4 vs 53.2; MK2 88.8 vs 84.0). 62

9.2 SparDA accuracy averages from Table 1 of the paper (author-reported). "Sparse" is the original sparse model with its own same-layer selector; InfiniGen is a prior prefetching baseline. 65

9.3 AsyncTLS accuracy (author-reported). Each query retrieves 128 blocks (8192 tokens); token budgets 512 (RULER, 32K), 1024 (LongBench). Four RULER tasks were excluded for GLM-4.7-Flash because of abnormal metrics under the default harness. . . . 67
9.4 Sharing and shrinking strategies for the selector, with idealised factor and the quality cost each introduces. . . . 69

10.1 MiniCPM-SALA training pipeline (author-reported; learning rates and batch sizes omitted). . . . 73
10.2 MiniCPM-SALA standard evaluation (author-reported; n/r = value unreadable in the retrieved text). . . . 73
10.3 MiniCPM-SALA long-context evaluation (author-reported). . . . 74
10.4 Ultra-long RULER as reported in the MiniCPM-SALA paper (Qwen3 values cited by the authors from the Qwen3-Next documentation). . . . 74
10.5 Hybrid and compressed-attention models in the retrieved literature, with the headline claim of each source. . . . 75

11.1 Taxonomy of eviction and selection methods. "Reversible" means the decision can be revised at a later step because the cache is retained (possibly offloaded). 77
11.2 TriAttention versus baselines on AIME (pass rate %, budget 2048 or 512; Table 1 of the paper, author-reported). . . . 78
11.3 Selected rows of Table 1 of the Random Attention paper (author-reported accuracies). Random Attention is significantly ahead of baselines in 31 of 60 comparisons and significantly behind in one. . . . 79

13.1 Quality comparisons of sparse/hybrid models against dense baselines (author-reported). . . . 88
13.2 Long-context benchmarks that appear in the retrieved sparse/hybrid literature and the failure mode each is positioned to detect. . . . 89
13.3 Efficiency claims in the retrieved literature. "n/r" = not retrieved. The "level" column determines what the number means; see text. . . . 90

14.1 Methods by the factor of (14.1) that they change. The last column matters for capacity: methods that do not change what is stored do not reduce the number of sequences that fit on a GPU. . . . 93
14.2 Training requirements of the method families (author-reported budgets, from Chapter 13). . . . 94
14.3 Characteristic failure modes by family. . . . 95
14.4 Stacking idealised changes on a V3.2-like layer at 128K tokens ([DER]). The first three rows come from the generated tables of Chapter 3; rows 4–5 are my arithmetic on top (index keys 132 B/token → halved; main cache ÷4; scale and metadata overheads ignored), not reported by any source. The stored column lists main cache plus index keys. . . . 95
14.5 Qualitative comparison of three architectural directions. . . . 97

15.1 Cross-cutting failure modes of learned sparse and hybrid attention. 101
15.2 Qualitative scorecard. ↓↓ means a large reported or derived reduction, ↓ moderate, ≈ none, ↑ an increase. The last column is a judgement based on Chapters 13 and 16. 101

16.1 Judged perception of technical bets, with the basis for each judgement. 105

17.1 SubQ against the evidence standards applied to the other methods in this report. 108

B.1 Summary of experiments (details in the listed chapters). 123

Chapter 1

Introduction and Scope

1.1 The problem: attention is the long-context bill

A decoder-only Transformer language model processes a sequence of T tokens by letting every position read from the key–value (KV) representations of every earlier position. That operation, softmax attention, is the mechanism that gives the architecture its extraordinary ability to retrieve, copy and compose information from its own context. It is also the mechanism whose cost grows with the length of that context. Two distinct costs matter and they bite in different phases of inference.

During *prefill*, when a prompt of length T is ingested in parallel, the attention score matrix has $\Theta(T^2)$ entries per head and per layer. The arithmetic cost is $\Theta(T^2 d)$ and, for contexts beyond a few tens of thousands of tokens, it overtakes the roughly linear cost of the feed-forward layers and projections. MInference's authors, for instance, report that prefilling a one-million-token prompt on a single A100 for an 8B model takes about thirty minutes [2].

During *decode*, when tokens are produced one at a time, each new token must read the entire KV cache once per layer. The arithmetic per step is only $\Theta(Td)$, but the data volume is also $\Theta(Td)$ bytes per layer, and the arithmetic intensity (FLOPs per byte moved) is a small constant that does not grow with context. On current accelerators this places decode attention firmly on the memory-bandwidth roof: its speed is set by how many bytes of KV cache must stream from high-bandwidth memory (HBM) per token, not by how many multiply-accumulates are performed. The KV cache itself also occupies HBM proportional to T times the number of concurrent sequences, which limits batch size and therefore throughput.

The long-context era has made both costs central. Reasoning models emit tens of thousands of tokens of chain-of-thought; agentic systems accumulate tool outputs and file contents over hundreds of turns; vendors advertise one-million-token windows as a product feature. The economic and engineering question is therefore not whether attention can be made cheaper but *which* cheaper attention preserves what long-context models are used for.

1.2 Three families of answers

The literature this report covers can be organised around three families of answers, which in 2025–2026 began to converge into hybrid designs.

Learned sparse attention. Keep exact softmax attention, but let each query attend to a small, input-dependent subset of keys. If the subset is chosen well, the output is close to the dense output (Chapter 4 quantifies this). The key question is how the subset is chosen. Early training-free methods used heuristics (heavy hitters, query-aware page summaries). The 2025 generation made the selection *trainable*: Native Sparse Attention (NSA) and MoBA

trained the model with sparse attention in the loop; DeepSeek Sparse Attention (DSA) trained a small "lightning indexer" to predict which keys the dense attention would have preferred; MiniMax Sparse Attention (MSA) did the same at block granularity for grouped-query attention. The selection step then becomes a learned retrieval module embedded in each layer.

Shared routing and indexer designs. A learned indexer is not free. For token-level selection it must score every earlier position for every query, which is $\Theta(T)$ per decoded token per layer and $\Theta(T^2)$ over a prefill, so at long context the *selector* becomes the bottleneck of a method whose point was to avoid quadratic work. The October 2025–October 2026 literature responds with ways to amortise or shrink the selector: reuse the same selection across several layers (IndexCache, HySparse, the cross-layer sparse attention CLSA built on YOCO), hierarchically pre-filter blocks (HISA, AsyncTLS), activate only a few indexer heads (MISA), or predict the next layer's selection one layer early so that offloaded KV can be prefetched (SparDA).

Hybrid sparse–linear and KV-compression designs. A different attack is to shrink what must be read. Linear-attention and state-space layers replace the growing cache by a fixed-size recurrent state; hybrids interleave a minority of exact-attention layers (full or sparse) with a majority of linear layers (Kimi Linear, Qwen3-Next-style models, MiniCPM-SALA). Orthogonally, the KV cache can be compressed in the feature dimension (quantization, as in KIVI and TurboQuant), in the token dimension (eviction, as in H2O, SnapKV and TriAttention), or in the head/latent dimension (GQA, MLA, and the compressed attention of DeepSeek-V4).

These families are not rivals so much as multipliers. A model's decode-time byte count is, to first order, a product of the number of layers that keep a growing cache, the fraction of that cache read per step, the bytes per cached element, and the number of cached elements per token. Different papers attack different factors, and Chapter 14 organises the field by the factor attacked.

1.3 Thesis and organising questions

The central claims that this report develops, each of which is argued with derivations, simulation, or cited measurements, are as follows.

1. **Decode attention is a bytes problem.** The speedup available from any attention variant in the bandwidth-bound regime is bounded by the ratio of bytes read, not of FLOPs (Chapter 3). A method with a large FLOP reduction but an unchanged cache read pattern achieves nothing at decode.
2. **Top-k selection error is controlled by tail mass, and tail mass depends on the distribution.** Dense softmax attention in trained models is heavy-tailed enough that fixed budgets of a few thousand tokens capture most, but not nearly all, of the attention mass (Chapter 4). The residual is exactly what matters for exact recall and, more subtly, for tasks where the answer is spread over many weak attenders.

3. **The indexer is the new bottleneck, and sharing it is the principal lever.** Once the main attention is $O(k)$ per token, the selector's $O(T)$ scan dominates; cross-layer reuse reduces this by the sharing factor (Chapters 6 and 9).
4. **Hybrids trade a recall guarantee for a state-size guarantee.** A fixed-size state can store at most a bounded amount of information; hybrid models need enough exact-attention layers for retrieval-type tasks (Chapters 5 and 10).
5. **KV compression is composable with sparsity but not free from distribution shift.** Quantization has well-understood worst-case distortion; eviction has a decision-time vs. use-time mismatch that creates failure modes (Chapters 7 and 11).
6. **Evidence quality is uneven.** Many headline numbers come from the authors of the method on their own benchmarks and kernels; a few come from vendors with limited independent verification (Chapters 13 and 16).

1.4 Conventions on evidence and provenance

A central discipline of this document is to separate what was *read from a source*, what was *derived* here, what was *simulated* here, and what is a *vendor or press claim* not independently verified. Four superscript tags are used throughout.

[DER] A derivation or analytical cost model produced for this report. These are mathematics, not measurements; their inputs (model dimensions) are cited, but the conclusions are not found in the cited papers unless stated.

[SIM] A numerical experiment run for this report (code in Appendix B and in the accompanying directory). Where the experiment uses a real model (Qwen3-0.6B on CPU) this is stated explicitly; where it uses synthetic distributions that is also stated. Simulations illustrate mechanisms; they are not benchmarks of the cited systems.

[VENDOR] A claim made by a company or its press materials and not independently verified, or an author-reported number that the retrieved page itself flagged as inconsistent.

[2ND] A claim retrieved only from a secondary source (blog, news article, aggregator) rather than the primary paper or model card.

Unmarked numbers attributed to a paper are the authors' own reported results as retrieved from the arXiv abstract or the full-text (HTML/PDF) at the time of writing (27 October 2026). Where the retrieved text of a paper was truncated or the abstract and body disagreed, that is noted. All text is paraphrased; no passage is reproduced at length. A consolidated ledger of every empirical claim and its status is given in Appendix C.

What this report does not do. It does not re-run any of the cited systems; no GPU was available for this work. The only model-in-the-loop experiments are on a 0.6B-parameter model on a CPU, using up to 2048-token sequences, and are reported in Chapters 4, 6 and 7 as mechanism checks. Claims about kernel speedups at 128K–1M tokens are therefore always the cited authors' claims.

1.5 Scope and time frame

The brief for this report is the period October 2025 to October 2026, but the methods of that period make sense only against the 2019–2025 lineage, which Chapter 2 summarises. Several of the most-discussed events in the period are:

- the release of DeepSeek-V3.2-Exp (end of September 2025) with DSA and the associated API price reduction, followed by the DeepSeek-V3.2 technical report (December 2025) [3];
- the Kimi Linear paper (30 October 2025) [4] and MiniMax's public explanation of why its M2 model uses full attention (autumn 2025) [5];
- the cluster of shared-index and cross-layer papers of early 2026: HySparse (3 February), Token Sparse Attention (3 February), MiniCPM-SALA (12 February), IndexCache (12 March), HISA (30 March);
- KV compression work: TurboQuant (first posted April 2025, widely publicised in March 2026 with an ICLR 2026 venue), TriAttention (April 2026), AsyncTLS (April 2026) and the critical follow-ups, including Random Attention (September 2026);
- the mid-2026 wave: SparDA and CLSA (3–4 June), MiniMax Sparse Attention (11 June), DeepSeek-V4 (technical report listed on arXiv in June 2026), the SubQ launch and controversy (May–June 2026), and HySparse2 (September 2026).

1.6 Reading guide

Chapters 3–7 are mathematical and self-contained; a reader mainly interested in the methods can skim the statements of the propositions and the worked examples and proceed to Chapters 8–11. Chapter 12 treats kernels and serving. Chapters 13–15 collect and critique the empirical evidence. Chapters 16–17 cover reception and case studies, and Chapters 18–19 the open problems. Appendices contain extended derivations, experiment details, the evidence ledger and a glossary.

Notation. Table 1.1 fixes the notation used throughout.

Symbol	Meaning
T	context length (number of cached tokens)
d, d_h	head dimension
H, G	number of query heads, number of KV heads (groups) in GQA
L	number of layers
b	bytes per cached element (2 for BF16, 1 for FP8, $\frac{1}{2}$ for 4-bit)
k	token budget for top-k selection
B	block size (tokens per block) in block-sparse methods
p_i	softmax attention weight of a query on key i
m_k	attention mass captured by the k largest weights: $\sum_{r \le k} p_{(r)}$
$\tau_k = 1 - m_k$	tail mass
H_I, d_I	indexer heads and indexer head dimension
c_{KV}, c_R	MLA latent width and decoupled-RoPE width
$S_t \in \mathbb{R}^{d_v \times d_k}$	recurrent state of a linear-attention layer
β_t, α_t	delta-rule write strength, gate (decay)
W	sliding-window length
m	number of layers sharing one indexer pass
BW, P	HBM bandwidth (B/s), peak arithmetic throughput (FLOP/s)

Table 1.1: Notation.

Chapter 2

Background and History

This chapter traces how the current designs arose. It is organised chronologically within five strands: (i) fixed sparse patterns, (ii) cache-size reductions through head sharing and latent compression, (iii) systems work on exact attention, (iv) recurrent and linear-attention alternatives, and (v) inference-time, training-free sparsity and cache compression. The final section describes the shift in 2025 from heuristics to *trainable* sparsity, which is the proper starting point for the rest of the report.

2.1 Softmax attention and the decoder-only Transformer

The Transformer [6] defines, for queries $Q \in \mathbb{R}^{T\times d}$, keys $K \in \mathbb{R}^{T\times d}$ and values $V \in \mathbb{R}^{T\times d_v}$, the attention output

$$O = \mathrm{softmax}\Big(\frac{QK^\top}{\sqrt{d}} + M\Big)V, \tag{2.1}$$

where M is a causal mask. Multi-head attention runs H such maps in parallel with learned projections. Two properties of (2.1) drive everything that follows. First, the weights form a probability simplex per query, so the output is a convex combination of value vectors; any truncation of the support is therefore an approximation whose error can be bounded in terms of the omitted mass (Chapter 4). Second, the keys and values of past tokens are independent of future queries in a causal model, so they can be cached; this converts the per-token cost of generation from recomputing the whole prefix to reading it.

Positional information is injected by rotary embeddings (RoPE) [7] in most open models of the period; Chapter 11 will use the fact that RoPE rotates queries and keys by position-dependent angles, which matters for eviction scoring, and Chapter 10 describes hybrid positional-encoding choices (NoPE in some sparse layers) that interact with length extrapolation.

2.2 Strand 1: fixed sparse attention patterns (2019–2021)

Sparse Transformers [8] factorised the attention matrix into strided and local components so that each position attended to $O(\sqrt{T})$ positions, lowering the cost from quadratic to $T\sqrt{T}$ and showing that models could be trained on sequences of tens of thousands of steps. Longformer [9] combined sliding-window attention with a small number of global tokens that attend everywhere, producing linear scaling for document tasks; this window-plus-global template is the ancestor of "sink plus local window" designs. Reformer [10] replaced exact neighbour search with locality-sensitive hashing so that only keys hashed to the same bucket were compared; it is the first prominent example of *content-dependent* sparse attention, and the idea of using a cheap proxy to find candidate keys survives in the indexers of 2025–2026.

These methods share a feature that later work reacted against: the pattern was either fixed (not learned) or learned by a hashing scheme not tied to the model's own attention. Practitioners who tried to *retrofit* fixed sparse patterns onto pretrained dense models typically found large quality losses, because the dense model's heads had learned to attend wherever the content required. MiniMax's account of an early attempt to convert a model to hybrid sliding-window attention is a recent statement of this experience: quality degraded as the context grew, which the authors regarded as unacceptable for agentic use [5].

2.3 Strand 2: shrinking the cache by sharing and compressing heads (2019–2024)

The KV cache per token in a standard multi-head model is $2LHd_hb$ bytes. Multi-query attention (MQA) [11] reduced this by a factor of H by sharing one key head and one value head across all query heads; grouped-query attention (GQA) [12] interpolates, with G KV heads for H query heads, and provided a recipe for uptraining multi-head checkpoints with about 5% of the original pre-training compute. GQA became the default in open models. DeepSeek-V2 [13] introduced Multi-head Latent Attention (MLA), which caches a low-rank latent vector (plus a small decoupled RoPE key) per token instead of per-head keys and values; the authors report a 93.3% KV-cache reduction relative to their dense 67B model together with 5.76× the maximum generation throughput. MLA and GQA reduce *bytes per token*; they do not reduce the number of tokens read. They are therefore orthogonal to sparsity, a point DeepSeek's sparse attention exploits by being built on MLA in its multi-query form [3].

A second branch reduced the cache by changing the architecture's layer graph. YOCO (You Only Cache Once) [14] splits the decoder into a self-decoder with efficient (sliding-window or recurrent) attention that produces one global KV cache and a cross-decoder whose layers all attend to that single cache. The global cache is thus paid once rather than once per layer; the authors report one-million-token extension with near-perfect needle retrieval. The same layout became the host for cross-layer sparse attention (CLSA) in 2026 (Chapter 9).

2.4 Strand 3: exact attention, made fast (2022–2023)

FlashAttention [15] showed that exact attention is bound by reads and writes between HBM and on-chip SRAM, not by arithmetic, and eliminated the materialisation of the $T \times T$ score matrix by tiling with an online softmax. The IO-aware viewpoint is the mindset behind the later kernels in this report: whether a sparse method speeds up wall-clock depends on whether it reduces HBM traffic *and* maps onto tensor-core tiles. PagedAttention [16] attacked the other side of the memory problem, allocating KV cache in fixed-size pages to avoid fragmentation, and reported two- to four-fold throughput improvements at equal latency. Paging also provides the natural granularity for query-aware page selection (Quest) and block-sparse selection in the 2025–2026 methods: a "block" in NSA/MoBA/MSA is in effect a page whose summary is used for routing.

Hydragen [17] and prefix-caching systems exploit shared prefixes across requests; they reduce redundant reads when many sequences share a long common prompt, a regime that

matters for agentic workloads and which interacts with sparse selection (selected blocks differ per query, which complicates sharing).

2.5 Strand 4: linear attention and state-space models (2020–2025)

Katharopoulos et al. [18] observed that replacing the softmax by a kernel feature map ϕ allows the causal sum to be computed recurrently: with state $S_t = S_{t-1} + v_t\phi(k_t)^\top$, the output is $S_t\phi(q_t)$ and the per-token cost is independent of T. The cost is a state of fixed size; whatever is stored in it is a compressed summary, and early linear Transformers were less accurate than softmax models. Later work introduced data-dependent gating and forgetting (Mamba [19]), a structured-duality view connecting SSMs and attention (Mamba-2 [20]), the delta rule as an error-correcting write (DeltaNet [21]) and its gated variant (Gated DeltaNet [22]). Lightning Attention-2 [23] supplied an IO-aware block algorithm for causal linear attention, and MiniMax-01 [24] scaled it in a 456B-parameter (45.9B active) MoE model with a one-million-token training context and four-million-token inference extrapolation claim. Jamba [25] interleaved Transformer and Mamba layers with MoE and reported strong results at up to 256K context on a single 80GB GPU.

A theoretical counterpoint came from Jelassi et al. [26], who proved that a two-layer Transformer can copy exponentially long strings while any fixed-size-state model is capacity-limited, and demonstrated a gap on copying and retrieval tasks with pretrained models. This result is the formal backbone of the claim, repeated in this report, that recurrent layers cannot by themselves provide exact long-range recall and that a hybrid needs retrieval-capable layers.

Hybridisation became the norm for production linear-attention models. Kimi Linear [4] (October 2025) combines a refined gated delta layer (KDA) with MLA in a layerwise hybrid and reports that under identical training recipes it outperforms full MLA across short-context, long-context and RL settings, with up to 75% KV-cache reduction and up to 6× decoding throughput at 1M tokens (author claims). A systematic study of hybrids [27] trained 72 models and found that recall improves as the share of full-attention layers increases, particularly below a 3:1 linear-to-full ratio, recommending 3:1 to 6:1 with HGRN-2 or Gated DeltaNet to reach Transformer-level recall.

2.6 Strand 5: training-free sparsity and cache compression (2023–2025)

The decode bottleneck motivated a large literature of inference-time methods that require no retraining, which this report treats as the baseline against which trainable methods are judged.

Eviction. StreamingLLM [28] found that the first few tokens attract disproportionate attention ("attention sinks") and that keeping them together with a recent window stabilises generation to millions of tokens, reporting up to 22.2× speedup over sliding-window recomputation. H2O [29] keeps "heavy hitter" tokens by cumulative attention score plus recent

tokens; with 20% heavy hitters the authors report up to 29× throughput over certain offloading baselines. SnapKV [30] selects per-head clusters of important prompt keys using an observation window at the end of the prompt, and reports 3.6× faster generation and 8.2× better memory efficiency at 16K tokens.

Query-aware retrieval of pages. Quest [31] stores per-page min/max key statistics and, at each decode step, upper-bounds the query–page score to choose which pages to load, reporting self-attention speedups up to 2.23× and latency reductions up to 7.03×. DuoAttention [32] classifies heads as "retrieval" heads (needing the full cache) or "streaming" heads (needing only sinks and a window), consistent with the mechanistic finding that a small set of retrieval heads is responsible for copying facts from context [33]. SeerAttention [34] learns a gating network that predicts block-level attention maps from pooled queries and keys, an early *trainable* sparse-attention design that anticipates DSA and MSA. MInference [2] identifies three recurring sparse patterns in long-context attention and builds dynamic sparse prefill kernels, reporting up to 10× prefill latency reduction on an A100.

Quantization. KIVI [35] quantizes keys per channel and values per token to 2 bits without tuning and reports 2.6× peak memory reduction, up to 4× larger batch and 2.35–3.47× throughput. TurboQuant [36], discussed in Chapter 7, is the most prominent successor, with provable distortion guarantees.

Evaluation and the retrieval debate. Alongside these methods came a literature questioning what "long context" models actually use. "Lost in the Middle" [37] showed that model accuracy depends strongly on where in the context the relevant document is placed, falling when it is in the middle. RULER [38] extended the needle-in-a-haystack test to 13 tasks including multi-key and multi-value retrieval, tracing and aggregation, and found that although almost all models achieved near-perfect vanilla needle retrieval, most degraded sharply with length; of the models claiming at least 32K context, only about half maintained satisfactory performance at 32K. NoLiMa [39] removed literal overlap between question and needle and found that at 32K tokens 11 of 13 models fell below 50% of their short-context baseline (GPT-4o dropped from 99.3% to 69.7%). Li et al. [40] compared retrieval-augmented generation (RAG) with long-context models and found that long-context models won on average when resources permitted, with RAG far cheaper, and proposed routing queries between the two. These evaluations matter here because sparse and hybrid architectures are precisely the places where passing a simple needle test can hide failures on RULER-style multi-key tasks or NoLiMa-style non-literal retrieval.

2.7 2025: the shift to trainable sparsity

By early 2025 three observations had accumulated. (a) Training-free sparsity usually loses accuracy at aggressive budgets, and the loss is task dependent: the large study of Nawrot et al. ("The Sparse Frontier") [41] on six training-free methods at up to 128K tokens and sparsity up to 0.95 concluded that larger sparse models beat smaller dense ones at equal cost, that longer sequences tolerate higher sparsity, and that fixed-budget production methods are therefore suboptimal; they also found that fine-grained per-query estimation during

prefill is impractical without suitable kernels, while token-to-page selection during decode generalises better. (b) If a model is trained with sparse attention in the loop it can adapt its representations to the sparsity pattern, and the sparse model can match or exceed the dense one. (c) Hardware-aligned (blockwise, GQA-group-shared) selection is necessary for wall-clock gains.

Native Sparse Attention (NSA) [42] (February 2025) formalised these into a design with three branches (compressed blocks, selected blocks, sliding window) and Triton kernels, reporting speedups up to 9.0× forward, 6.0× backward and 11.6× decoding at 64K tokens versus FlashAttention-2 on A100. MoBA [43] applied a mixture-of-experts view to attention blocks and reported deployment in Kimi's long-context service. InfLLM-V2 [44] (September 2025) argued that NSA's extra parameters slow convergence and proposed a dense–sparse switchable design that reuses the dense parameters, enabling short-to-long adaptation; it underlies MiniCPM4.1 and the 2026 MiniCPM-SALA hybrid. DeepSeek-V3.2-Exp (29 September 2025) put a fine-grained, token-level learned indexer into a production-scale MoE model and cut its API prices by more than half [45, 46]. The period this report covers begins here.

2.8 Timeline

Table 2.1 lists the principal events used in the rest of the report, with dates as given on the retrieved pages. For the arXiv-listed items the date is that of first submission unless noted.

Date	Item	Note
Nov 2019	MQA [11]	single KV head
Apr 2019	Sparse Transformers [8]	strided factorisation
Jun 2020	Linear Transformers [18]	kernel feature map
May 2022	FlashAttention [15]	IO-aware exact attention
May 2023	GQA [12]	grouped KV heads
Jun–Sep 2023	H2O, StreamingLLM [28, 29]	eviction, sinks
Dec 2023	Mamba [19]	selective SSM
May 2024	DeepSeek-V2 (MLA), YOCO [13, 14]	latent cache, shared cache
Jun 2024	Quest [31]	query-aware pages
Apr 2024	RULER [38]	multi-task long-context test
Jan 2025	MiniMax-01 [24]	lightning attention at scale
Feb 2025	NSA, MoBA [42, 43]	trainable block sparsity
Sep 2025	InfLLM-V2; DeepSeek-V3.2-Exp [44, 45]	switchable sparse; DSA in production
Oct 2025	Kimi Linear [4]	KDA + MLA hybrid
Dec 2025	DeepSeek-V3.2 report [3]	DSA documented
Jan–Feb 2026	GSA, HySparse, TSA, MiniCPM-SALA [47–50]	gating, oracle selection, hybrids
Mar 2026	IndexCache, HISA [51, 52]	indexer cost reduction
Apr 2026	TriAttention, AsyncTLS [53, 54]	eviction; async two-level
May–Jun 2026	MISA, SubQ launch [55, 56]	indexer heads; vendor claims
Jun 2026	SparDA, CLSA, MSA, DeepSeek-V4 [1, 57–59]	decoupled/ shared / block / compressed
Sep 2026	Random Attention, HySparse2 [60, 61]	eviction critique; two-level sharing

Table 2.1: Timeline of principal developments referenced in this report (dates from retrieved pages).

Chapter 3

Cost Models: FLOPs, Bytes and the Roofline

This chapter builds the cost models used in the rest of the report. Everything here is derived for this report (tag [DER]) from standard operation counts; the model dimensions are taken from public configuration files (retrieved from the Hugging Face model repositories for Qwen3-8B, DeepSeek-V3 and DeepSeek-V3.2-Exp), and hardware constants are stated as assumptions. The numerical tables are generated by `experiments/exp_e_costmodel.py` and are reproduced verbatim from its output.

3.1 Dense attention: arithmetic and memory

3.1.1 Prefill

Consider one attention layer with H query heads of dimension d and a causal prompt of length T. For one head, forming $QK^\top$ costs $2T^2d$ multiply-accumulate FLOPs if computed densely, the same for PV; causal masking halves both. Hence

$$\mathrm{FLOPs}^{\mathrm{dense}}_{\mathrm{prefill}} = 2\,H\,d\,T^2 \quad \text{(per layer, causal)}, \tag{3.1}$$

which is $\Theta(T^2d)$. FlashAttention [15] leaves (3.1) unchanged but reduces the HBM traffic from $\Theta(T^2)$ (materialising scores) to $\Theta(Td)$ per head.

The remaining layer computation is linear in T: projections and the feed-forward network cost about $2N_{\mathrm{layer}}T$ FLOPs where N_{layer} is the parameter count of those components. The crossover at which attention FLOPs equal the rest of the layer is

$$2HdT^2 = 2N_{\mathrm{layer}}T \implies T^* = N_{\mathrm{layer}}/(Hd). \tag{3.2}$$

Worked example 3.1 (Qwen3-8B prefill crossover)**.** [DER]The Qwen3-8B configuration has 36 layers, $H = 32$ query heads and head dimension $d = 128$ (so $Hd = 4096$), with 8 KV heads. Taking the total parameter count as roughly 8.2×10^9 (an approximation; non-embedding parameters are somewhat fewer) and assuming that all of it participates per token (dense model), $N_{\mathrm{layer}} \approx 8.2 \times 10^9/36 = 2.28 \times 10^8$, and $T^* \approx 2.28 \times 10^8/4096 \approx 5.6 \times 10^4$. So for prompts longer than about 55K tokens the attention score FLOPs exceed the projection/FFN FLOPs; at $T = 131{,}072$, attention is $2 \cdot 4096 \cdot 36 \cdot T^2 = 5.07 \times 10^{15}$ FLOP against $2 \cdot 8.2 \times 10^9 \cdot T = 2.15 \times 10^{15}$ FLOP for the rest, i.e. attention is about 70% of prefill FLOPs. For a mixture-of-experts model with few active parameters the crossover is much earlier, which is why sparse attention is attractive for MoE models: DeepSeek-V4-Flash activates 13B of 284B parameters [59].

3.1.2 Decode

At decode, a layer processes one query token against T cached keys and values. For one KV head shared by H/G query heads (GQA), reading the cache costs $2dTb$ bytes (K and V, d elements each, T tokens, b bytes per element) per KV head and the arithmetic is $4(H/G)dT$ FLOPs for the H/G query heads that share it. Over all G KV heads,

$$\text{Bytes}_{\text{dec}}^{\text{dense}} = 2\,G\,d\,T\,b, \qquad \text{FLOPs}_{\text{dec}}^{\text{dense}} = 4\,H\,d\,T, \qquad I^{\text{dense}} = \frac{\text{FLOPs}}{\text{Bytes}} = \frac{2H}{G\,b}. \tag{3.3}$$

The arithmetic intensity I is independent of T; for BF16 ($b = 2$) it equals the GQA group size H/G (4 for Qwen3-8B). Multi-head attention ($G = H$) has $I = 1$ FLOP per byte.

3.1.3 The roofline

A kernel's attainable time on an accelerator is bounded below by the larger of its arithmetic time and its memory time:

$$t \;\geq\; \max\Big(\frac{\text{FLOPs}}{\text{P}},\; \frac{\text{Bytes}}{\text{BW}}\Big), \tag{3.4}$$

and the kernel is memory-bound when $I < \text{P/BW}$, the *ridge point*. As an illustrative assumption (public vendor figures, not measured here) take an H100 SXM-class device with BW = 3.35 TB/s and P = 989 TFLOP/s dense BF16, giving a ridge point of 295 FLOP/byte.[DER] Dense GQA decode attention at $I = 4$ is therefore about 74× below the ridge, and is limited to roughly 1.4% of peak arithmetic throughput even if the kernel streams perfectly. Reducing the FLOPs of such a kernel without reducing its bytes yields essentially no speedup.

Proposition 3.2 (Bytes bound on decode speedup)**.** *Let a baseline kernel read B_0 bytes and a modified kernel read B_1 bytes, both memory-bound with identical achieved bandwidth. Then the speedup is B_0/B_1, independent of the FLOP counts. If the modified kernel additionally incurs a fixed-time overhead t_o (selection, launch gaps, host synchronisation) the speedup is $B_0/(B_1 + \text{BW}\,t_o)$.*

Proof. Direct from (3.4) with the memory term dominant. □

The second statement explains a pattern visible in the reported results: "operator" speedups quoted in papers are often larger than end-to-end speedups because they exclude the part of the step that sparse attention does not touch (weights, other layers) and the overheads of selection.

3.2 KV-cache memory formulas

3.2.1 GQA, MQA, MHA

The cache per token across all layers for a model with L layers, G KV heads of dimension d and b bytes per element is

$$\text{KV}_{\text{GQA}} = 2\,L\,G\,d\,b \;\; \text{bytes/token}. \tag{3.5}$$

MQA is $G = 1$; MHA is $G = H$.

Configuration	B/token	32K	128K	512K	1M (GiB)
Qwen3-8B GQA, BF16	147,456	4.50	18.00	72.00	144.00
Qwen3-8B GQA, 4-bit (ideal, no scales)	36,864	1.12	4.50	18.00	36.00
DeepSeek-V3.2 MLA latent, BF16	70,272	2.14	8.58	34.31	68.62
DSA indexer keys, FP8 (assumed 132 B/layer)	8,052	0.25	0.98	3.93	7.86
4B transformer (32L, 4 KV heads), BF16	65,536	2.00	8.00	32.00	64.00
4B YOCO/CLSA shared global KV, BF16	2,048	0.06	0.25	1.00	2.00

Table 3.1: KV-cache bytes per token and total size in GiB at several context lengths for a single sequence (derived; configuration files for Qwen3-8B and DeepSeek-V3.2-Exp; 4B transformer and CLSA dimensions from [1]). The indexer-key layout is an assumption. The CLSA row counts only the single global KV cache shared by cross-decoder layers; the self-decoder's 512-token windows and the indexer keys are not included.

3.2.2 MLA

MLA caches, per layer and token, a compressed latent $c_{KV} \in \mathbb{R}^{c_{KV}}$ and a decoupled rotary key of width c_R:

$$\mathrm{KV}_{\mathrm{MLA}} = L\,(c_{KV} + c_R)\,b \quad \text{bytes/token.} \tag{3.6}$$

For DeepSeek-V3 and V3.2-Exp the configuration files give $L = 61$, $c_{KV} = 512$, $c_R = 64$, so (3.6) with BF16 is $61 \cdot 576 \cdot 2 = 70{,}272$ bytes/token, about 68.6 KiB. Qwen3-8B, by (3.5) with $L = 36$, $G = 8$, $d = 128$, $b = 2$, requires 147,456 bytes/token, i.e. 144 KiB (more than twice the MLA figure despite having a far smaller model).[DER]

3.2.3 Additional caches introduced by sparse attention

A token-level indexer needs its own key per token and layer: d_I elements per index key (single head in DSA and CLSA, one shared head in MSA). For DSA, the indexer runs in FP8 and has $d_I = 128$ (config file: `index_head_dim`= 128, `index_n_heads`= 64, `index_topk`= 2048). The exact storage layout of the production implementation was not examined; this report assumes one FP8 key (128 bytes) plus a 4-byte scale per token and layer, 132 bytes. That assumption is flagged in Table 3.1.[DER]

3.2.4 Quantized caches

A b_q-bit cache has $b = b_q/8$ bytes per element plus per-group metadata (scales, zero-points); for group size g and 16-bit scale and zero point the effective bits per element are $b_q + 32/g$. Table 3.1 neglects metadata ("ideal").

Worked example 3.3 (Capacity-limited batch for Qwen3-8B at 128K)**.** [DER]On an 80 GB accelerator, BF16 weights of an 8.2B-parameter model take about 16.4 GB, leaving roughly 63 GB for KV cache after activation workspace is ignored. A single 128K-token sequence occupies $147{,}456 \times 131{,}072 = 19.3$ GB, so at most $\lfloor 63/19.3 \rfloor = 3$ concurrent sequences fit. Per decode step the aggregate bytes moved are weights (16.4 GB, once per step regardless of batch) plus $3 \times 19.3 = 58$ GB of cache, a 75-GB step costing $75/3350 \approx 22$ ms at the stated bandwidth, yielding about $3/0.022 \approx 134$ tokens/s across the batch. Cutting the cache read by 10× (e.g. with a 2048-token budget plus indexer) cuts the step to $\approx 16.4 + 5.8 = 22$ GB, 6.6 ms, and the cache memory by 10× only if the *stored* cache is also reduced (which

requires eviction or compression; sparse selection alone leaves the stored cache unchanged). This distinction between *bytes read* and *bytes stored* recurs: token-level sparse attention, DSA included, reduces reads but not storage, which is why offloading (SparDA, AsyncTLS) and shared-KV (YOCO, HySparse) designs appear alongside it.

3.3 Throughput as a function of batch size

Let W be the weight bytes read per step (assumed independent of batch), B_s the batch size, and $c(T)$ the cache bytes read per sequence per step. Decode step time is $t(B_s) = (W+B_s c(T))/\mathrm{BW}$ in the bandwidth-bound regime and throughput is B_s/t. For fixed memory M available for cache, the maximum feasible batch is $B_s^{\max} = \lfloor M/\mathrm{KV}(T) \rfloor$ where $\mathrm{KV}(T)$ is the stored bytes per sequence. Throughput at the feasible batch is therefore

$$\Theta(T) = \frac{B_s^{\max}\,\mathrm{BW}}{W + B_s^{\max} c(T)}. \tag{3.7}$$

Two regimes appear.[DER] When $B_s^{\max} c(T) \ll W$, increasing batch is nearly free and throughput scales with $B_s^{\max}$ (so reducing *stored* bytes matters, since it raises $B_s^{\max}$). When $B_s^{\max} c(T) \gg W$, throughput saturates at $\mathrm{BW}/c(T)$ (so reducing *read* bytes matters). At long context the second regime applies for dense attention (Example 3.3) and the first for sparse attention with large stored caches.

This model rationalises why SparDA reports a 5.3× decode-throughput gain over a non-offload sparse baseline: its offload design lets the batch grow beyond what stored cache would otherwise permit, while the sparse read volume keeps $c(T)$ small [57]. (The model is qualitative; SparDA's numbers are measured, not predicted by (3.7).)

3.4 Cost of a sparse decode step with an indexer

For a token-level top-k method with an indexer of H_I heads of dimension d_I and index-key storage of b_I bytes per element, one layer reads

$$\underbrace{H_I^{\mathrm{kv}}\, d_I\, b_I\, T}_{\text{index keys}} + \underbrace{c_{\mathrm{tok}}\, \min(k, T)\, b}_{\text{selected KV}} \quad \text{bytes}, \tag{3.8}$$

where H_I^{kv} is the number of distinct index-key heads (1 in DSA, MSA and CLSA) and c_{tok} is the bytes of attention state per token (e.g. $c_{KV} + c_R$ for MLA). The corresponding FLOP count is $2H_I d_I T$ for scoring plus the sparse attention FLOPs. The first term is linear in T with a small constant; the second is constant in T once $T > k$. The crossover at which the indexer reads more bytes than the sparse attention is

$$T_{\mathrm{idx}} = \frac{c_{\mathrm{tok}}\, b\, k}{H_I^{\mathrm{kv}} d_I b_I}. \tag{3.9}$$

For DSA-like MLA ($c_{\mathrm{tok}} = 576$, $b = 2$, $k = 2048$, 132 B index keys): $T_{\mathrm{idx}} = 576{\cdot}2{\cdot}2048/132 = 17{,}873$ tokens, so beyond $\approx$ 18K tokens the indexer *reads more bytes than the attention it feeds*. At 128K the model gives 17.3 MB for the indexer and 2.36 MB for the sparse MLA read per layer (Table 3.2), a ratio of 7.3 : 1; the indexer is 88% of the layer's bytes.[DER] The indexer is cheap in bytes per token and cheap in precision (FP8; DeepSeek-V4 moves it

	Dense MLA		DSA total		of which indexer	
T	MB	GFLOP	MB	GFLOP	MB	GFLOP
8K	9.4	2.28	3.4	0.70	1.1	0.13
32K	37.7	9.13	6.7	1.11	4.3	0.54
128K	151.0	36.51	19.7	2.72	17.3	2.15
512K	604.0	146.03	71.6	9.16	69.2	8.59
1M	1208.0	292.06	140.8	17.75	138.4	17.18

Table 3.2: Per-layer, per-decoded-token bytes and FLOPs for one DeepSeek-V3.2-style MLA layer ($H = 128$, absorbed MQA form, $c_{KV} = 512$, $c_R = 64$, BF16 latent) under dense attention versus DSA with $k = 2048$, $H_I = 64$, $d_I = 128$ and a 132-byte FP8 index key (assumed). Derived by `exp_e_costmodel.py`.

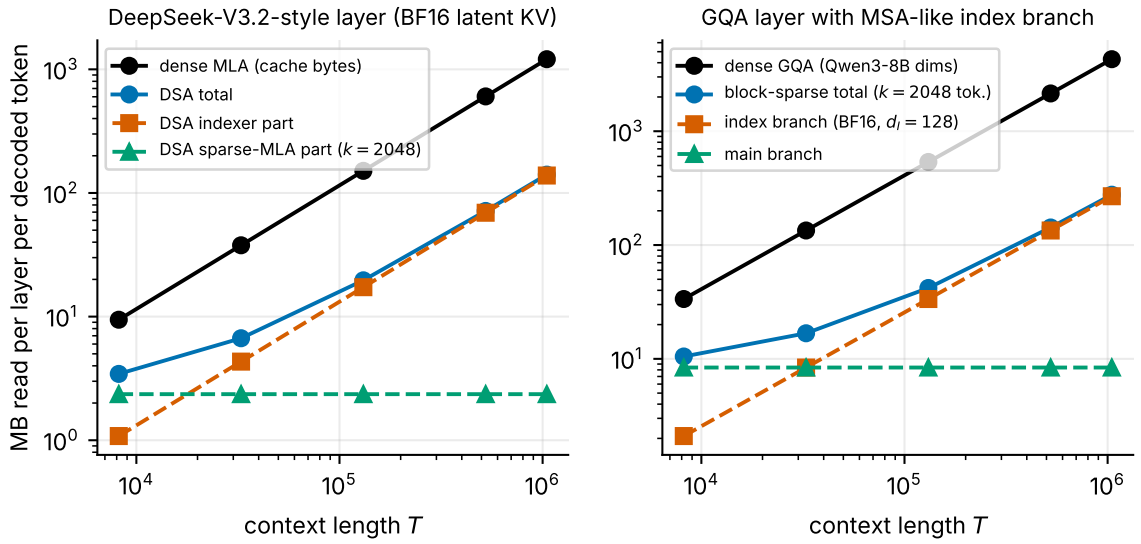


Figure 3.1: Bytes read per layer per decoded token versus context length (log–log). Left: DSA-like MLA layer. Right: GQA layer with Qwen3-8B dimensions and an MSA-like index branch (one index head per KV group, BF16, $d_I = 128$). In both cases the index scan grows linearly while the sparse attention is flat at $k = 2048$ tokens (derived).

to FP4 [59]), but it scans everything. This is the quantitative basis for the claim that the indexer becomes the bottleneck of learned sparse attention, and for the whole shared-indexer literature.

Remark 3.4 (MLA decode is near compute-bound). Equation (3.3) concerns GQA. In the absorbed MLA form, the $H = 128$ query heads attend to one shared 576-dimensional latent key and 512-dimensional latent value per token, so FLOPs are $2H(576 + 512)T$ and bytes are $576 \cdot 2 \cdot T$, an arithmetic intensity of 241.8 FLOP/byte, about 82% of the assumed ridge point.[DER] Dense MLA decode is thus almost balanced between compute and bandwidth, in contrast to GQA's $I = 4$. This is relevant to interpretation: for MLA, the FLOP reduction of top-k attention (from 36.5 GFLOP to 0.57 GFLOP per layer at 128K) is itself valuable, whereas for GQA only the byte reduction matters. DSA's published emphasis on complexity $O(Lk)$ in FLOPs [3] is consistent with that.

3.5 Prefill with a token-level indexer

During prefill a token-level indexer scores all earlier positions for every query: causally, $H_I d_I T^2$ multiply-accumulates $= 2 \cdot \frac{1}{2} H_I d_I T^2$ FLOPs, i.e. $H_I d_I T^2$ FLOPs, compared with $2HdT^2$ for dense attention (3.1). The ratio is

$$\frac{\text{FLOPs}_{\text{indexer}}}{\text{FLOPs}_{\text{dense attn}}} = \frac{H_I d_I}{2Hd}. \tag{3.10}$$

For DSA, $H_I d_I = 64 \cdot 128 = 8192$, and dense absorbed-MLA prefill costs $2H(c_{KV} + c_R + c_{KV})T^2/2 = 139{,}264\,T^2$ FLOPs per layer ($H = 128$, 576-wide scores and 512-wide values), so the ratio is $8192/139{,}264 = 5.9\%$ before accounting for the indexer running in FP8 at higher arithmetic throughput. This matches the qualitative statement in the DeepSeek-V3.2 report that the indexer is $O(L^2)$ but needs far less computation than the main attention [3]. The sparse part costs $2H(1088)Tk = 2.8 \times 10^5\, Tk$ FLOPs, i.e. $5.7 \times 10^8\, T$ at $k = 2048$. The indexer's quadratic term $8192\,T^2$ overtakes it when $T > 5.7 \times 10^8/8192 \approx 7 \times 10^4$: beyond about 70K tokens the indexer's FLOPs exceed those of the sparse attention it controls (in raw FLOP count, ignoring the FP8 advantage, which roughly doubles the crossover in time).[DER] Thus at 128K–1M prefill, *the selector, not the attention, is the dominant attention-related cost*, a fact the follow-up literature (HISA, MISA, IndexCache) takes as its premise.

Block-sparse methods (MSA, NSA, MoBA, InfLLM-V2) reduce the indexer's cost by a factor of the block size B: scores are computed against one pooled key per block, so the indexer scan is T/B per query. For $B = 128$ the scan is 128 times cheaper than token-level scoring at the price of coarser selection (Chapter 4).

Proposition 3.5 (Selection complexity)**.** *Let the selection method score n_s candidates per query and then select the top k_s. Using a heap or partial sort, selection costs $O(n_s + k_s \log k_s)$ per query on a sequential machine and $\Theta(n_s)$ bytes if the scores are streamed. Hence per-layer selection over a prefill of length T costs $\Theta(Tn_s)$; with token-level scoring $n_s = \Theta(T)$ and with block pooling $n_s = \Theta(T/B)$. A two-stage hierarchy that first selects κ blocks and then scores the κB tokens inside them has $n_s = T/B + \kappa B$ per query, minimised at $B = \sqrt{T/\kappa}$ with value $2\sqrt{\kappa T}$.*

Proof. The first two statements are standard. For the hierarchy, minimise $T/B + \kappa B$ over B: the derivative $-T/B^2 + \kappa = 0$ gives $B = \sqrt{T/\kappa}$ and the value $\sqrt{\kappa T} + \sqrt{\kappa T}$. □

For $T = 10^6$ and $\kappa = 16$, the optimal block size is about 250 and the number of scored candidates per query drops from 10^6 to about 8000, a 125× reduction.[DER] This is the arithmetic behind hierarchical indexers such as HISA [52] and AsyncTLS [54], though those papers use fixed block sizes (64 tokens in AsyncTLS) rather than the $\sqrt{T/\kappa}$ optimum, and the real cost model also has memory-traffic terms that the count above ignores.

3.6 Putting numbers on shared routing

Suppose a group of m consecutive layers reuses one indexer pass. The per-layer expected bytes become

$$\bar{B}(m) = \frac{B_{\text{idx}}}{m} + B_{\text{sparse}}, \tag{3.11}$$

Share group m	Bytes/layer (MB)	GFLOP/layer	Indexer share of bytes	Byte reduction vs dense
1	19.66	2.72	88%	7.7×
2	11.01	1.64	79%	13.7×
4	6.68	1.11	65%	22.6×
8	4.52	0.84	48%	33.4×
16	3.44	0.70	31%	43.9×
61	2.64	0.61	11%	57.1×

Table 3.3: Effect of sharing one indexer pass across m layers on a DSA-like layer at 128K tokens (derived; $k = 2048$, 132-byte FP8 index keys, 576-wide BF16 latent). "Indexer share" is the fraction of the layer's bytes spent on index keys.

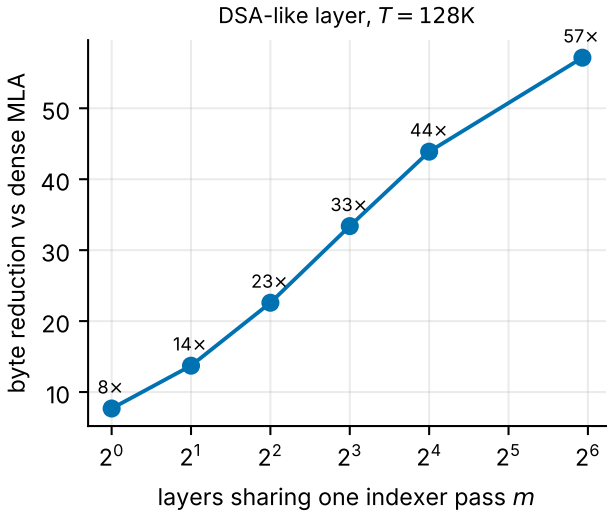


Figure 3.2: Byte reduction versus dense MLA as the indexer pass is shared across more layers (derived from Table 3.3). Diminishing returns set in when the sparse-attention read (2.36 MB) dominates.

and the indexer's share of the per-layer total is $\frac{B_{\text{idx}}/m}{B_{\text{idx}}/m+B_{\text{sparse}}}$. Table 3.3 evaluates (3.11) at 128K tokens for the DSA-like layer: with $m = 4$, the byte reduction over dense MLA rises from 7.7× to 22.6×; with a single indexer for all 61 layers it is 57×. Chapter 6 derives this more carefully and tests the premise (that adjacent layers select overlapping tokens) on a real model.

3.7 A cautionary comparison with published speedups

The cost model predicts, for a 128K context and ideal bandwidth-bound kernels, a 7.7× attention-byte reduction for single-pass DSA and 22.6× with $m = 4$ sharing. Published numbers are smaller, and the gap illustrates Proposition 3.2. IndexCache removes about 75% of indexer computations on a 30B DSA model and reports up to 1.82× prefill and 1.48× decode speedups [51]; those are end-to-end figures including weights, MoE layers and communication. The model's byte reduction applies to the attention component only. If attention were a fraction f_a of the baseline step time, the end-to-end speedup from an attention speedup s is $1/((1 - f_a) + f_a/s)$ (Amdahl's law). Setting this to 1.48 with $s = 22.6/7.7 \approx 2.9$ (the extra gain of sharing at $m = 4$ relative to single-pass DSA) gives $f_a \approx 0.7$,

a plausible figure for long-context decode; but this back-calculation is only a consistency check, not an explanation, since IndexCache's configuration (which layers are shared, MoE shape, batch size) differs from the idealised layer. The same Amdahl argument cautions against comparing, for example, MSA's "28.4× per-token attention compute reduction at 1M" with its measured 7.6× decode speedup [58]: the first counts attention FLOPs, the second is wall-clock for the whole decode step with a bandwidth-bound regime and non-attention work.

3.8 Summary of the cost chapter

- Dense decode attention has arithmetic intensity $2H/(Gb)$ FLOP/byte, far below the accelerator ridge point; only bytes matter.
- Learned sparse attention converts $\Theta(T)$ cache reads into $\Theta(k)$ reads *plus* a $\Theta(T)$ index scan; the scan dominates beyond $T_{\text{idx}} \approx k\, c_{\text{tok}} b/(d_I b_I)$ tokens, about 18K for DSA-like dimensions.
- Stored bytes (capacity) and read bytes (bandwidth) are different resources: sparse attention helps the latter; eviction, quantization, MLA and shared-KV designs help the former.
- Block pooling, hierarchy and cross-layer sharing each reduce the scan, by factors B (block pooling), up to $\frac{1}{2}\sqrt{T/\kappa}$ (two-stage hierarchy, Proposition 3.5) and m (cross-layer sharing) respectively.

Chapter 4

Approximation Theory of Sparse Attention

Sparse attention replaces the softmax over all T cached positions by a softmax over a selected subset S. This chapter makes precise what is lost. Statements marked [DER] are derived here; [SIM] marks results of simulations whose code is in Appendix B. The results are elementary, but they are the vocabulary in which sparse-attention claims are best compared: "captured attention mass", "tail mass" and "selection recall" appear, under various names, in the papers of Chapters 8 and 9.

4.1 Definitions

Fix one query and one head. Let $p \in \Delta^{T-1}$ be the softmax attention weights, $v_i \in \mathbb{R}^{d_v}$ the values, and $o = \sum_i p_i v_i$ the dense output. For a subset $S \subseteq [T]$ define

$$m(S) = \sum_{i \in S} p_i, \qquad \tau(S) = 1 - m(S), \qquad \hat{p}_i^S = \begin{cases} p_i / m(S) & i \in S \\ 0 & i \notin S \end{cases}, \qquad \hat{o}_S = \sum_{i \in S} \hat{p}_i^S v_i. \tag{4.1}$$

$\hat{o}_S$ is the output of sparse attention restricted to S with the softmax renormalised over S (which is what sparse kernels compute, since they take a softmax over the selected scores only). The *oracle top-k set* $S_k^\star$ consists of the k largest weights; write $m_k = m(S_k^\star)$ and $\tau_k = 1 - m_k$. Any real selector (indexer, block score, heuristic) returns some S with $|S| \le k$ and $m(S) \le m_k$. The ratio $m(S)/m_k$ is the *selection efficiency* of the selector; the overlap $|S \cap S_k^\star|/k$ is its *recall*.

4.2 The basic error identity

Proposition 4.1 (Truncation identity)**.** *For any non-empty S with $m = m(S) \in (0, 1]$,*

$$o - \hat{o}_S = \tau \, (\bar{o}_{S^c} - \hat{o}_S), \qquad \bar{o}_{S^c} = \tfrac{1}{\tau} \sum_{i \notin S} p_i v_i, \tag{4.2}$$

and consequently, if $\|v_i\| \le V$ for all i,

$$\|o - \hat{o}_S\| \le 2\tau V, \qquad \|o - \hat{o}_S^{raw}\| \le \tau V \quad \textit{where } \hat{o}_S^{raw} = \sum_{i \in S} p_i v_i. \tag{4.3}$$

Both bounds are tight in the worst case.

Proof. Write $o = m\hat{o}_S + \tau \bar{o}_{S^c}$ (split the sum over S and S^c). Then $o - \hat{o}_S = (m-1)\hat{o}_S + \tau \bar{o}_{S^c} = \tau(\bar{o}_{S^c} - \hat{o}_S)$. Both $\bar{o}_{S^c}$ and $\hat{o}_S$ are convex combinations of vectors of norm at most V, so their difference has norm at most $2V$. For the raw (unnormalised) sum, $o - \hat{o}_S^{\text{raw}} = \tau \bar{o}_{S^c}$ with norm at most τV. Tightness: take $v_i = +Ve$ on S and $v_i = -Ve$ on S^c. □

Corollary 4.2 (Distribution distances)**.** *The total-variation distance between p and $\hat{p}^S$ is $\frac{1}{2}\|p - \hat{p}^S\|_1 = \tau$, and $\mathrm{KL}(\hat{p}^S \,\|\, p) = -\log m(S)$, while $\mathrm{KL}(p \,\|\, \hat{p}^S) = +\infty$ whenever $\tau > 0$.*

Proof. $\sum_{i\in S} |p_i - p_i/m| = \tau$ and $\sum_{i\notin S} p_i = \tau$, so the ℓ_1 distance is 2τ. For the KL, $\sum_{i\in S} \frac{p_i}{m} \log \frac{p_i/m}{p_i} = -\log m$. The reverse direction has mass of p on points where $\hat{p}^S$ is zero. □

The identity has three practical consequences.

(i) Error is linear in tail mass, with a direction-dependent constant. The constant $\|\bar{o}_{S^c} - \hat{o}_S\|$ is the distance between what the discarded keys would have contributed and what the retained ones do. If the discarded values are similar to the retained ones (redundant context), the error is much smaller than $2\tau V$. If they are dissimilar, the bound is approached. This is the reason top-k errors measured as relative output error in real models come out well below the worst-case bound (Section 4.9).

(ii) The training objective of learned indexers is the right one. DSA, MSA, CLSA and SparDA all train the indexer by minimising a KL divergence between the (aggregated) dense attention distribution and the indexer's softmax distribution [1, 3, 57, 58]. By Corollary 4.2, driving the *forward* KL $\mathrm{KL}(p\|q)$ to zero forces the indexer's mass to cover the support of p, which is the mass-covering behaviour wanted from a selector. (The reverse KL, mode seeking, would be satisfied by a selector that finds just the largest weights.)

(iii) The lower bound on any selector is τ_k. No selector with budget k can beat the oracle: $\tau(S) \geq \tau_k$. Hence the oracle tail mass τ_k is a floor on the achievable approximation error (in the worst case over values), and the distribution of p determines whether a budget of a few thousand is adequate.

4.3 How much mass does top-k capture?

4.3.1 Power-law (Zipf) attention

Suppose the sorted weights follow $p_{(r)} = r^{-\alpha}/Z_N$ with $Z_n = \sum_{r\leq n} r^{-\alpha}$. Then $m_k = Z_k/Z_N$. The Euler–Maclaurin expansion gives, for all $\alpha > 0$,

$$Z_n = \begin{cases} \dfrac{n^{1-\alpha}}{1-\alpha} + \zeta(\alpha) + O(n^{-\alpha}), & \alpha \neq 1, \\ \ln n + \gamma + O(1/n), & \alpha = 1, \end{cases} \qquad m_k = \frac{Z_k}{Z_N}, \tag{4.4}$$

[DER] where ζ is the (analytically continued) Riemann zeta function and $\gamma \approx 0.5772$. Table 4.1 shows that (4.4) reproduces the simulated masses to three digits at $k = 2048$ (e.g. $\alpha = 1$: $(7.625 + 0.577)/(10.397 + 0.577) = 0.747$). Three regimes follow. For $\alpha < 1$, $m_k \approx (k/N)^{1-\alpha}$ to leading order and $\tau_k \to 1$ as $N \to \infty$ at fixed k: no fixed budget suffices. For $\alpha = 1$, $m_k \approx \ln k / \ln N$, again slowly degrading. For $\alpha > 1$, $Z_N \to \zeta(\alpha)$ and the tail mass is $\tau_k \approx k^{1-\alpha}/((\alpha-1)\zeta(\alpha))$ independent of N; a target tail ϵ requires $k \approx [(\alpha-1)\zeta(\alpha)\epsilon]^{-1/(\alpha-1)}$, e.g. $\alpha = 1.5$ ($\zeta(1.5) = 2.612$), $\epsilon = 0.05$ gives $k \approx 235$ (simulation: $\tau_{256} = 0.044$).[SIM] Because

α	$k = 64$	256	1024	2048	8192	closed form (k=2048)
0.6	0.071	0.133	0.241	0.322	0.569	0.322
0.8	0.199	0.302	0.438	0.521	0.728	0.521
1.0	0.432	0.558	0.684	0.747	0.874	0.747
1.2	0.688	0.794	0.874	0.907	0.960	0.907
1.5	0.909	0.956	0.980	0.987	0.996	0.987

Table 4.1: Simulated top-k attention mass m_k for Zipf weights with exponent α and $N = 32{,}768$ ([SIM]; `exp_a_synthetic_topk.py`), and the value of (4.4) at $k = 2048$ (last column).

the required k grows as a steep power of $1/\epsilon$ when α is near 1, a head with $\alpha = 1.2$ would need $k \approx 6 \times 10^4$ for a 10% tail in the $N \to \infty$ limit (from the formula), beyond any production budget, while a head with $\alpha = 1.5$ is covered by a few hundred.

4.3.2 Gaussian-logit attention

If the pre-softmax scores are i.i.d. $z_i \sim \mathcal{N}(0, \sigma^2)$ (a crude model of "random" content), the weights are log-normal and the top-k mass has a closed form.

Proposition 4.3 (Mass of the top fraction under Gaussian logits)**.** *Let $q = k/N$ and let $N \to \infty$ with q fixed. Then*

$$m(q) \;\to\; \Phi\big(\sigma - \Phi^{-1}(1-q)\big), \tag{4.5}$$

where Φ is the standard normal CDF.

Proof. The top qN scores are those above the threshold $t = \sigma\Phi^{-1}(1-q)$. By the law of large numbers the captured mass is $\mathbb{E}[e^z \mathbf{1}\{z > t\}]/\mathbb{E}[e^z]$. For $z \sim \mathcal{N}(0, \sigma^2)$ exponential tilting gives $\mathbb{E}[e^z \mathbf{1}\{z > t\}] = e^{\sigma^2/2} \Pr[\mathcal{N}(\sigma^2, \sigma^2) > t] = e^{\sigma^2/2}\Phi(\sigma - t/\sigma)$, and $\mathbb{E}[e^z] = e^{\sigma^2/2}$. □

The striking feature of (4.5) is that it depends only on the *fraction* $q = k/N$ kept, not on N. A fixed budget k therefore captures less and less as N grows: with $\sigma = 3$ the mass of the top 2048 is 0.929 at $N = 32{,}768$ ($q = 1/16$) but 0.72 at $N = 262{,}144$, and 0.38 at $N = 2^{22}$ ($q = 4.9 \times 10^{-4}$, $\Phi^{-1}(1-q) = 3.30$). Table 4.2 confirms the formula by simulation. In asymptotics, $\Phi^{-1}(1-q) \sim \sqrt{2\ln(1/q)}$, so to keep the captured mass constant as N grows the logit spread must grow like $\sqrt{2\ln N}$.[DER]

Real attention is not i.i.d. Gaussian: trained models produce score distributions with a few outliers (sinks, relevant needles, local tokens) on top of a diffuse background. The empirical finding of Nawrot et al. that longer sequences *tolerate higher sparsity* [41] is consistent with this: if the relevant mass is carried by a roughly fixed number of positions regardless of N (the "needle plus local plus sinks" picture), a fixed budget suffices at all lengths; if the mass is spread like a log-normal bulk, it does not. Which picture holds depends on the head and the task, and it is exactly what the coverage measurements in CLSA's Table 4 begin to quantify: at a 2048-token budget, the fraction of dense attention captured was 76–84% on three domains, increasing with budget (62–91% for StarCoder from 512 to 4096) [1].

	$N = 4096$		$N = 32768$		$N = 262144$	
σ	sim	formula	sim	formula	sim	formula
1	0.842	0.841	0.297	0.297	0.079	0.078
2	0.977	0.977	0.682	0.679	0.334	0.338
3	0.999	0.999	0.927	0.929	0.724	0.720
4	1.000	1.000	0.990	0.993	0.926	0.943

Table 4.2: Top-2048 attention mass for i.i.d. Gaussian logits with scale σ: simulation versus the closed form (4.5) ([SIM], [DER]; `exp_b_block_needle.py`, mean of 8 draws).

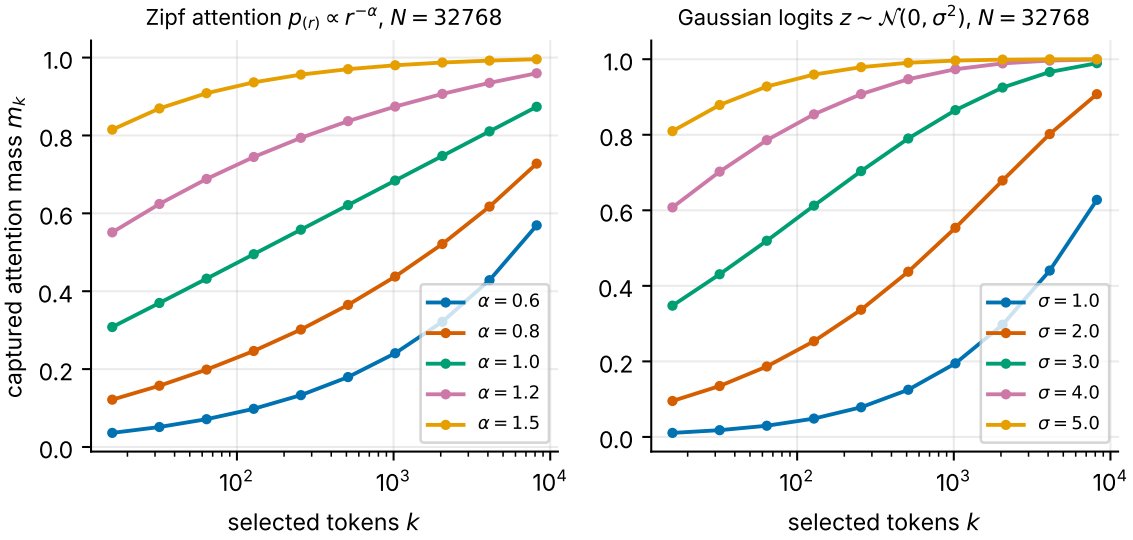


Figure 4.1: Captured attention mass m_k versus budget k for synthetic Zipf (left) and Gaussian-logit (right) attention with $N = 32{,}768$ ([SIM]). The spread across the curves at $k = 2048$ (0.32 to 0.99 on the left) illustrates why a single fixed budget is a poor description of "how sparse attention is".

4.3.3 Output error versus tail mass

Proposition 4.1 gives a *worst-case* bound linear in tail mass. For typical data the error is much smaller, and a short calculation shows why. Suppose the values v_i are independent, zero-mean, with $\mathbb{E}\|v_i\|^2 = 1$ ("incoherent" values). From $o - \hat{o}_S = \sum_{i \notin S} p_i v_i - \frac{\tau}{m} \sum_{i \in S} p_i v_i$ one gets

$$\mathbb{E}\|o - \hat{o}_S\|^2 = \sum_{i \notin S} p_i^2 + \left(\frac{\tau}{m}\right)^2 \sum_{i \in S} p_i^2, \tag{4.6}$$

[DER]so the error is governed by the ℓ_2 norm of the weights (a measure of how *concentrated* the discarded mass is), not by the ℓ_1 tail mass. A tail made of many tiny weights averages out. Figure 4.2 (right) compares (4.6) with the measured errors for 50 combinations of exponent and budget: the ratio measured/predicted lies between 0.91 and 1.19 (mean 1.05) with a single draw of random values per case, while measured error divided by tail mass spans 0.03 to 0.51.[SIM] Real values are not incoherent; they are correlated, particularly across neighbouring tokens, so the real error lies between this optimistic regime and the worst case (4.3). Section 4.9 reports the real-model situation.

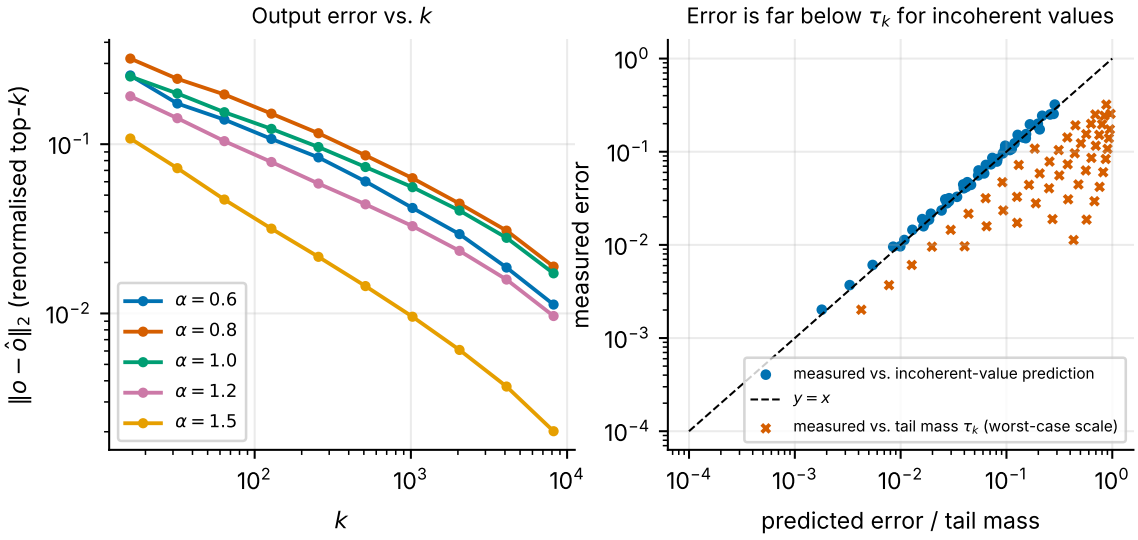


Figure 4.2: Left: output error of renormalised top-k attention versus k for Zipf attention ([SIM]). Right: measured error against the incoherent-value prediction (4.6) (circles, on the $y = x$ line) and against the tail mass τ_k (crosses, far below the worst-case scale).

4.4 Block-sparse selection

Block methods choose $n = k/B$ blocks of B contiguous tokens. Define the block mass $P_j = \sum_{i \in \text{block } j} p_i$. The best a block selector can do is pick the n blocks with the largest P_j, capturing $m_n^{\text{blk}} = \sum_{j \le n} P_{(j)}$.

Proposition 4.4 (Block selection never beats token selection at equal token budget)**.** *For the same total number of selected tokens $k = nB$, $m_n^{blk} \le m_k$, with equality iff the k heaviest tokens occupy exactly n whole blocks.*

Proof. A set of n blocks is a particular set of k tokens, whose mass is at most that of the k heaviest tokens. □

The loss from block granularity depends on how the heavy tokens are laid out. If they are scattered (one heavy token per block), a block selector spends $B - 1$ tokens per heavy token on junk; if heavy tokens are contiguous (a local span, a paragraph that matters) there is no loss. Simulation (Table 4.3) shows the effect for Zipf-1.0 attention at a 2048-token budget: with independently placed heavy tokens, block oracles with $B = 16, 64, 128$ capture $0.53, 0.42, 0.35$ of the mass against 0.747 for the token oracle; with heavy tokens arranged in contiguous runs of 32 the $B = 64$ block oracle captures 0.69; with runs of 128 aligned to blocks it captures the full 0.747.[SIM] The qualitative lesson is that block sparsity is efficient only if relevance is spatially clustered, and that smaller blocks recover more of the token oracle at the cost of more block scores (Proposition 3.5).

This is not merely a simulation artefact. HySparse2 reports that, at matched budget, switching from block-level to token-level selection raised RULER-v2 at 32K from 49.56 to 56.13, two-needle MRCR-v2 from 12.94 to 21.08 and GraphWalks from 29.38 to 34.92, while NoLiMa (38.43 vs 40.27) and BBH (60.70 vs 61.93) got slightly worse [61]; the authors flagged that their comparison is not perfectly isolated (different number of KV heads and

α	run length c	token oracle	block B=16	block B=64	block B=128
0.8	i.i.d.	0.521	0.296	0.219	0.187
0.8	8	0.521	0.458	0.344	0.293
0.8	32	0.521	0.521	0.459	0.396
0.8	128	0.521	0.521	0.521	0.521
1.0	i.i.d.	0.747	0.530	0.419	0.354
1.0	8	0.747	0.699	0.588	0.523
1.0	32	0.747	0.747	0.693	0.636
1.0	128	0.747	0.747	0.747	0.747
1.2	i.i.d.	0.907	0.760	0.648	0.585
1.2	8	0.907	0.879	0.807	0.762
1.2	32	0.907	0.907	0.879	0.847
1.2	128	0.907	0.907	0.907	0.907

Table 4.3: Oracle captured mass at a 2048-token budget for Zipf attention (N = 32,768) under token-granular versus block-granular oracle selection, with heavy tokens placed i.i.d. ($c = 0$) or in contiguous runs of length c aligned to multiples of c ([SIM]; runs of 128 are aligned with blocks of 64 and 128 and therefore lose nothing, an artefact of the toy layout).

full-attention layers between baselines). Retrieval tasks with scattered needles benefit from token granularity; tasks where relevant context is contiguous or semantic benefit from the smoothing of blocks.

4.5 Noisy indexers

A learned indexer produces scores s_i that approximate, but do not equal, the true logits z_i. Model the error as additive Gaussian noise of scale τ_n in logit units: $s_i = z_i + \tau_n \varepsilon_i$.

Proposition 4.5 (Needle retrieval through a noisy indexer)**.** *Let background logits be $z_i \sim \mathcal{N}(0,1)$ for $i \neq i^*$, and let the needle have logit $z_{i^*} = \Delta + \mathcal{N}(0,1)$. The indexer selects the top fraction $q = k/N$ by s. As $N \to \infty$ with q fixed, the probability that the needle is selected converges to*

$$\Pr[i^* \in S] = \Phi\Big(\frac{\Delta}{\sqrt{1+\tau_n^2}} - \Phi^{-1}(1-q)\Big). \tag{4.7}$$

Proof. Background scores are $\mathcal{N}(0, 1+\tau_n^2)$ and the selection threshold converges to its $(1-q)$-quantile $t = \sqrt{1+\tau_n^2}\,\Phi^{-1}(1-q)$. The needle's score is $\mathcal{N}(\Delta, 1+\tau_n^2)$, so it exceeds t with probability $\Phi\big((\Delta - t)/\sqrt{1+\tau_n^2}\big)$. □

Table 4.4 compares (4.7) with 3000-trial simulations at N = 32,768, k = 2048 (q = 1/16, $\Phi^{-1}(15/16) = 1.534$): all twelve cells agree with the closed form to within about three binomial standard errors (the largest deviation, $\Delta = 4, \tau_n = 2$, is 3.2 standard errors: simulated 0.63 versus predicted 0.60; finite-N effects are expected at that level).[SIM]

The formula quantifies a recurring worry about indexers: the *signal-to-selection-noise ratio* $\Delta/\sqrt{1+\tau_n^2}$ must exceed $\Phi^{-1}(1-q)$ for retrieval to be likely. For a budget fraction $q = 1/16$ and an indexer whose noise equals the background spread ($\tau_n = 1$) a needle must stand out by $\Delta > 2.17$ logits just to be a coin flip, and $\Delta = 4$ for 90%. For $q = 2048/10^6$ (a million-token context with the same budget) $\Phi^{-1}(1-q) = 2.87$, and a $\tau_n = 1$ indexer needs

Δ	τ_n	simulated	formula (4.7)	s.e.	dense needle mass
2	1	0.439	0.452	0.009	0.0002
2	2	0.255	0.261	0.008	0.0002
2	3	0.176	0.184	0.007	0.0002
4	1	0.903	0.902	0.005	0.0017
4	2	0.630	0.601	0.009	0.0016
4	3	0.387	0.394	0.009	0.0015
6	1	0.997	0.997	0.001	0.0130
6	2	0.881	0.875	0.006	0.0131
6	3	0.624	0.642	0.009	0.0122
8	1	1.000	1.000	0.000	0.0801
8	2	0.977	0.980	0.003	0.0759
8	3	0.833	0.840	0.007	0.0704

Table 4.4: Needle selection probability through a noisy indexer: simulated (3000 trials) versus the closed form (4.7); $N = 32{,}768$, $k = 2048$. The last column is the needle's mean dense-softmax attention mass (from the 400-trial run of `exp_a`; it does not depend on τ_n). [SIM], [DER].

$\Delta \approx 4.1$ for 50% and $\Delta \approx 5.9$ for 90%. In other words *as the context grows, a fixed-budget sparse attention needs ever stronger needles*, a logarithmic-in-N law whose rate is controlled by indexer noise.[DER] The actual dense-attention weight of the needle also matters (it receives e^{Δ} times the weight of a typical background token): a needle that is selected with probability 0.9 but whose attention mass is 0.0017 (the $\Delta = 4$ row of Table 4.4) contributes little to the output even if selected. Missing it is harmless to the *output*; but if the task reads the needle through a downstream layer that amplifies it, the miss can be catastrophic. This two-sided character of sparse errors (small mass, large downstream effect) is the core of the "lossy exact recall" critique.

4.6 Indexer training objectives and what they guarantee

The learned-indexer recipe trains scores to mimic attention. Let $p_t^{(h)}$ be the dense attention distribution of head h at query t, $\bar{p}_t = \frac{1}{H}\sum_h p_t^{(h)}$ the head average, and $q_t = \mathrm{softmax}(I_{t,\cdot})$ the indexer's distribution. DSA minimises $\sum_t \mathrm{KL}(\bar{p}_t \| q_t)$ over the full prefix during the dense warm-up and over the selected set during sparse training [3]; CLSA averages over all layers and heads [1]; MSA aligns to the group-averaged main-branch distribution over selected tokens [58]; SparDA restricts the KL to the target's top-k set with the remainder lumped into one bucket [57].

Proposition 4.6 (Selection from a KL-optimal indexer)**.** *Suppose* $\mathrm{KL}(\bar{p}\|q) \le \varepsilon$. *Let* S *be the top-*k *set of* q. *Then for any set* A, $\bar{p}(A) \ge q(A) - \sqrt{\varepsilon/2}$ *(Pinsker). In particular, if the top-*k *set of* q *has* $q(S) \ge 1-\delta$ *then* $\bar{p}(S) \ge 1 - \delta - \sqrt{\varepsilon/2}$.

Proof. By Pinsker's inequality, $\mathrm{TV}(\bar{p}, q) \le \sqrt{\varepsilon/2}$, and $|\bar{p}(A) - q(A)| \le \mathrm{TV}$. □

The proposition links what the indexer *believes* it captures to what the dense model attends, but it has two limitations. First, it is about the *head-averaged* distribution; an individual head may have a very different distribution (a retrieval head that attends to a single position the average hides). Second, it controls ℓ_1 error only: a needle with weight

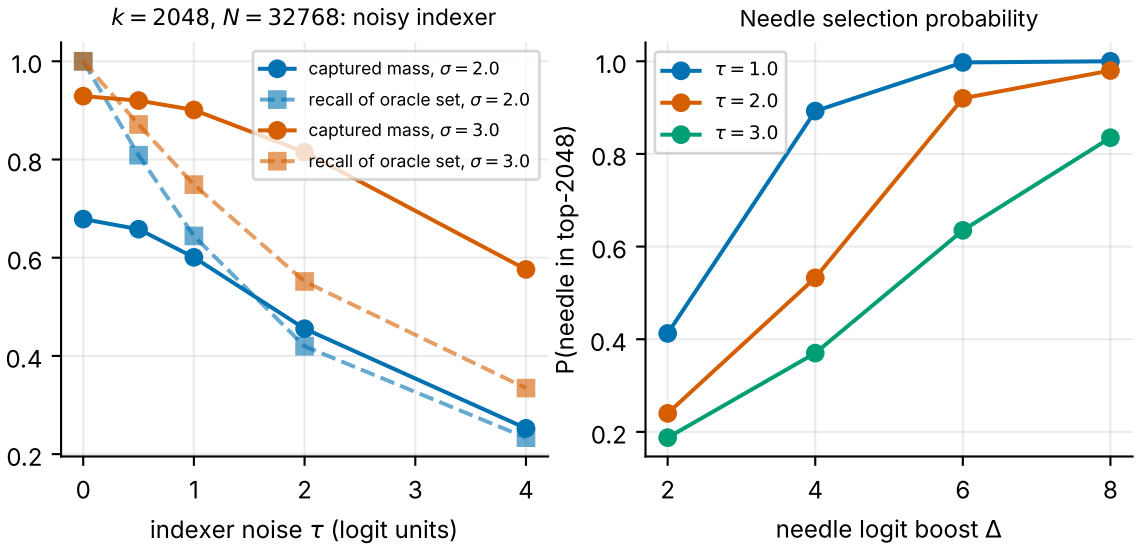


Figure 4.3: Left: with Gaussian logits at $\sigma = 2, 3$, a noisy indexer with logit-noise τ_n degrades both captured mass (solid) and recall of the oracle set (dashed). Right: probability that a planted needle is among the top 2048 of 32,768 versus its logit boost Δ ([SIM]).

10^{-3} in the average is invisible to a small ε yet decisive for the one head that reads it. The mismatch between averaged targets and per-head needs is why GSA, HySparse and HISA discuss head-specific and oracle selection, and why a shared indexer (one target for all heads of all layers) is more of a compromise than it first appears.[DER]

4.7 Error propagation through depth

A small error at one layer perturbs the residual stream, which feeds all later layers. A first-order model: if the layer-ℓ attention output error has norm e_ℓ and later layers are λ_ℓ-Lipschitz maps of the residual stream, the output perturbation is bounded by $\sum_\ell e_\ell \prod_{j>\ell} \lambda_j$. For $\lambda_j \le 1 + \eta$ with small η this is approximately $\sum_\ell e_\ell$ times a growth factor $(1+\eta)^{L-\ell}$. The bound is loose and does not distinguish benign from harmful perturbations, but it conveys that (i) errors accumulate with depth, so sparsifying every layer at 1% error each is not a 1% error overall, and (ii) which layers are sparsified matters: layers with large residual-stream gain are more dangerous. Token Sparse Attention exploits (ii) empirically: it applies sparsification only in layers with small *inter-layer representation drift*, i.e. small relative change between a layer's input and output hidden states, and reports sharp accuracy drops when the drift threshold is raised from 0.5 to 0.9 (RULER average falls to 79.16) [49]. Similarly, MoBA's authors found that keeping the last several layers dense improved SFT loss [43], and MiniCPM-SALA leaves the first and last layers unconverted [50].[DER]

4.8 What a fixed budget cannot do: aggregation tasks

A needle task needs one key. Aggregation tasks (count, summarise, average, "list all") need contributions from many keys, each with small weight. If a query must read r keys with similar weight w each, the useful mass is rw and the budget must cover all r; if $r > k$ or if

k	mean m_k	p10	p90	min (head)	mean rel. output err.	err. / tail
16	0.799	0.554	0.974	0.139	0.281	1.92
32	0.845	0.645	0.984	0.211	0.207	1.88
64	0.887	0.726	0.989	0.311	0.145	1.83
128	0.926	0.812	0.993	0.447	0.091	1.78
256	0.959	0.892	0.997	0.619	0.049	1.76
512	0.984	0.957	0.999	0.806	0.020	1.80
1024	0.997	0.993	1.000	0.956	0.003	1.89

Table 4.5: Oracle top-k attention on Qwen3-0.6B ([SIM]; 28 layers × 16 heads, 3 windows, queries with 1025–2048 visible keys). "p10/p90" are percentiles across the 448 (layer, head) pairs of the mean mass; "min" is the worst head; the last two columns give the mean relative output error and its ratio to the tail mass $1 - m_k$.

the indexer fails to rank them near the top (because individually they are weak), the output mixes in whatever else is selected. RULER's aggregation tasks (common-words extraction, frequent-words extraction) and tracing tasks (variable tracking) are of this kind [38]. Sparse attention has a systematic handicap there that is absent from the needle test. This is a derivation-level argument; empirical evidence is mixed: MiniCPM-SALA reports 92.65/89.37 on RULER at 64K/128K [50], and CLSA reports a RULER 32K average of 53.1 versus 52.3 for dense YOCO and 46.2 for a Transformer, with gains concentrated in multi-needle subtasks [1]. The absence of widely reported degradations on aggregation tasks at the scales tested is notable but does not exclude them at larger contexts or for tasks outside RULER; Chapter 15 returns to this.

4.9 Measured on a real model

The synthetic distributions above isolate mechanisms but say nothing about how concentrated real attention is. I therefore measured the attention of a small trained model. [SIM] The setup (`exp_real_qwen.py`): Qwen3-0.6B (28 layers, 16 query heads, 8 KV heads, head dimension 128; weights and configuration from the Hugging Face hub) is run in FP32 with eager attention on a CPU, on three 2048-token windows of English prose (a public-domain novel from Project Gutenberg, starting at token offsets 0, 6000 and 14000). For every layer, every head and each of the 1024 query positions $t = 1024, \ldots, 2047$ (so that each query sees between 1025 and 2048 keys) the code records the attention row p, the top-k mass m_k for $k \in \{16, \ldots, 1024\}$ and the relative output error $\|o - \hat{o}_{S_k^\star}\|/\|o\|$ of renormalised oracle top-k attention (Proposition 4.1), then averages over queries and the three windows. This is a *very small model on short context*; it demonstrates what is measurable, not what DSA-scale models do. In particular, 2048 tokens is far below the contexts at which sparse attention is used.

Observations. (1) *Concentration is high but heterogeneous.* A budget of 128 keys (roughly 6–12% of those visible) captures on average 92.6% of the mass, 256 keys capture 95.9%; but the worst head at $k = 128$ captures only 44.7%, the 10th percentile is 81.2%, and about 77% of heads exceed 90% at $k = 128$. Only 0.2% of (layer, head) pairs fall below 50% at $k = 128$. The mean across heads therefore overstates the safety of a uniform budget: the distribution

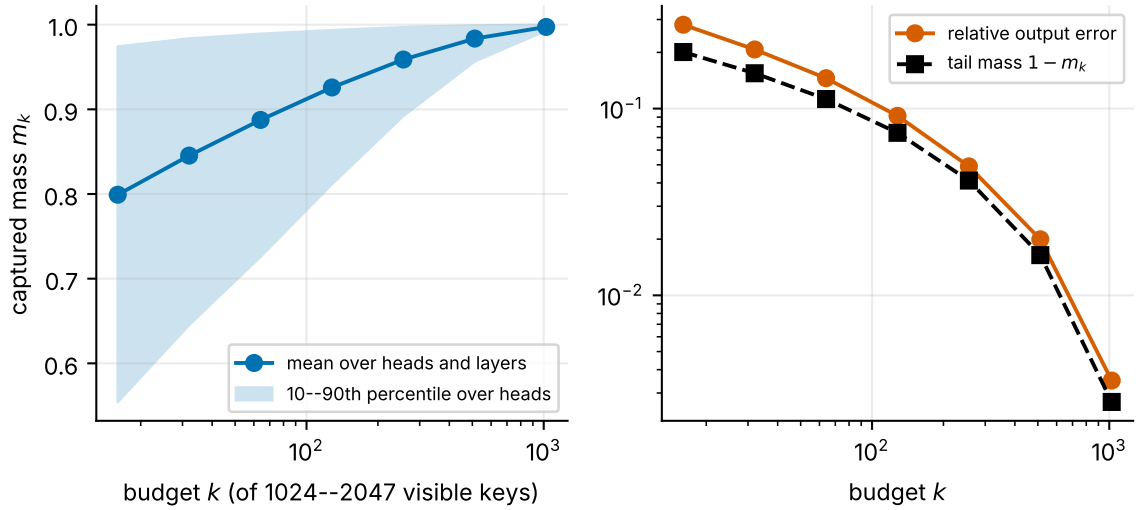


Figure 4.4: Qwen3-0.6B, 2048-token windows ([SIM]). Left: captured mass versus budget; the band is the 10th–90th percentile over (layer, head) pairs. Right: mean relative output error of renormalised oracle top-k and mean tail mass; the two curves are parallel on the log scale (ratio ≈ 1.8).

has a lower tail of diffuse heads (Figure 4.4, shading). (2) *Output error is proportional to the tail mass.* The ratio of mean relative output error to tail mass is nearly constant at 1.76–1.92 over two orders of magnitude of k (Table 4.5), as predicted by the identity (4.2): $\|o - \hat{o}\| = \tau \|\bar{o}_{S^c} - \hat{o}_S\|$. The constant means $\|\bar{o}_{S^c} - \hat{o}_S\| \approx 1.8\|o\|$, i.e. the discarded values' average differs from the retained average by about twice the norm of the output; this is far from the incoherent-values regime of (4.6), where the tail average would nearly vanish. Real values in the tail are coherent (systematically different from retained values), so the error is a substantial fraction of the worst-case scale rather than much below it. (3) *Sinks and locality are visible, and layer dependent* (Figure 4.5). The first token receives 44% of all attention mass on average, and more than 50% in 43.5% of (layer, head) pairs; layers 0–2 have essentially no sink mass and instead put 56–72% of their mass on the last 64 tokens, while layers 20–27 put 61–77% on the sink and only 13–23% on the last 64 tokens. A policy of "sink plus recent window" (StreamingLLM) therefore already retains a large fraction of the mass in this model, and the *budget actually used for retrieval* is what remains. Per-layer m_{128} ranges from 0.83 (layer 1) to 0.98 (layer 27).

Caveats. Short context, a single small model, one text genre, and windows that begin mid-document (so the "first token" is arbitrary text, which still acts as a sink). Heads and layers of a model like DeepSeek-V3.2 will differ; CLSA's measurement of 76–84% coverage at 2048 tokens for 32K contexts in a 4B model [1] is the closest published analogue and shows lower coverage at much longer context, consistent with the Gaussian-logit scaling of Proposition 4.3: at fixed budget, coverage falls as the context grows.

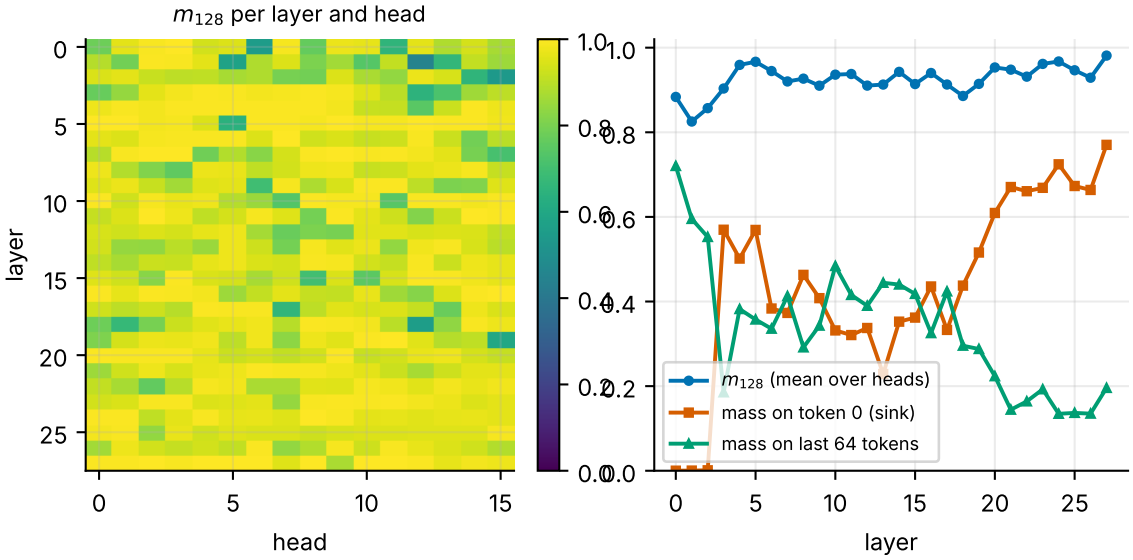


Figure 4.5: Left: m_{128} for every (layer, head). Right: mean over heads of m_{128}, of the mass on the first token (sink) and of the mass on the most recent 64 tokens, by layer ([SIM]). Early layers are local; later layers are sink-dominated.

Chapter 5

Linear Attention, State Capacity and Hybrid Cost Models

The second half of the long-context toolbox replaces some of the attention layers by recurrent layers whose state does not grow. This chapter gives the recurrences, proves the associated cost and capacity statements, tests the capacity analysis by simulation ([SIM]), and builds the hybrid cost model used in the comparisons of later chapters ([DER]).

5.1 From softmax to linear attention

Replace $\exp(q^\top k/\sqrt{d})$ by a kernel $\phi(q)^\top \phi(k)$. The causal output at position t becomes

$$o_t = \frac{\sum_{i\le t} \phi(q_t)^\top \phi(k_i)\, v_i}{\sum_{i\le t} \phi(q_t)^\top \phi(k_i)} = \frac{S_t\, \phi(q_t)}{z_t^\top \phi(q_t)}, \qquad S_t = \sum_{i\le t} v_i \phi(k_i)^\top, \quad z_t = \sum_{i\le t} \phi(k_i), \tag{5.1}$$

which is the construction of Katharopoulos et al. [18]. The state $S_t \in \mathbb{R}^{d_v \times d_k}$ obeys the recurrence $S_t = S_{t-1} + v_t \phi(k_t)^\top$. Modern variants usually drop the denominator and use normalisation layers instead, which is the form analysed here (without loss of generality for the capacity arguments).

Three equivalent computation forms. The same causal outputs can be computed in three ways.

1. *Parallel form*: $O = (QK^\top \odot M)V$ with M the causal mask, $\Theta(T^2 d)$ FLOPs, used for short sequences or as a reference.
2. *Recurrent form*: update S_t and read $o_t = S_t q_t$, $\Theta(d_k d_v)$ FLOPs and bytes per token, independent of T; this is the decode form.
3. *Chunkwise form*: split the sequence into chunks of length C; within a chunk use the parallel form (cost $\Theta(C^2 d)$ per chunk), and carry the state S across chunks, adding the inter-chunk contribution $Q_{[c]} S_{c-1}^\top$ and updating $S_c = S_{c-1} + V_{[c]}^\top K_{[c]}$. Total cost $\Theta(TCd + Td_k d_v)$, linear in T, with matrix-multiply structure suitable for tensor cores. Lightning Attention-2 [23] is a blocked IO-aware instance of this idea.

Proposition 5.1 (Equivalence of the three forms)**.** *For an un-normalised linear attention with identity feature map the three forms compute the same function.*

Proof. Expand $o_t = \sum_{i\le t} (q_t^\top k_i) v_i = \big(\sum_{i\le t} v_i k_i^\top\big) q_t = S_t q_t$ for the recurrent form. For the chunkwise form, split $\sum_{i\le t}$ into i in earlier chunks (giving $S_{c-1} q_t$) and i in the same chunk with $i \le t$ (the intra-chunk causal product). □

[SIM] A float64 implementation with $T = 256$, $d = 32$, chunk $C = 32$ reproduces the parallel result with maximum absolute difference 2.8×10^{-13} (recurrent) and 2.6×10^{-13} (chunkwise), consistent with machine precision (`exp_d_linear_capacity.py`). This is a unit test of the algebra, not a performance result.

5.2 Forgetting, gating and the delta rule

Pure accumulation $S_t = S_{t-1} + v_t k_t^\top$ never forgets, and the state saturates with superposed associations. Two refinements dominate modern designs.

Decay / gating. $S_t = \alpha_t S_{t-1} + v_t k_t^\top$ with $\alpha_t \in (0, 1)$ data-dependent (Mamba's selective SSM [19], gated linear attention, and the scalar-decay form used in Lightning Attention). Decay bounds the influence of old tokens by $\prod \alpha_j$ and makes the state an exponentially weighted memory.

Delta rule. Instead of adding $v_t k_t^\top$ blindly, correct the state's current prediction for key k_t:

$$S_t = S_{t-1} + \beta_t \left(v_t - S_{t-1} k_t\right) k_t^\top = S_{t-1}\left(I - \beta_t k_t k_t^\top\right) + \beta_t v_t k_t^\top, \tag{5.2}$$

which is a step of online gradient descent on $\frac{1}{2}\|S k_t - v_t\|^2$ with learning rate β_t [21]. With $\|k_t\| = 1$ and $\beta_t = 1$, $S_t k_t = v_t$ exactly: the state is overwritten along k_t so that the new association is stored without interference *in that direction*. The gated delta rule [22] (Gated DeltaNet) adds a decay α_t so that memory can be cleared quickly:

$$S_t = \alpha_t \, S_{t-1}\left(I - \beta_t k_t k_t^\top\right) + \beta_t v_t k_t^\top. \tag{5.3}$$

(The retrieved abstract of the Gated Delta Networks paper states that gating enables rapid memory erasure and the delta rule enables targeted updates, but shows no equations; the explicit form (5.3) is written here from the standard presentation of that paper and should be checked against it.) Kimi Delta Attention (KDA), the core of Kimi Linear, refines Gated DeltaNet with finer-grained (per-channel) gating and a specialised diagonal-plus-low-rank chunkwise algorithm [4].

5.3 Capacity of a fixed-size state

5.3.1 Interference in additive memory

Store n pairs (k_i, v_i) with random unit vectors in $\mathbb{R}^d$ in the accumulated state $S = \sum_i v_i k_i^\top$. Reading with key k_j gives $\hat{v}_j = S k_j = v_j + \sum_{i \neq j} (k_i^\top k_j) v_i$.

Proposition 5.2 (Cosine of recall under superposition)**.** *For independent random unit keys and values in $\mathbb{R}^d$, the cosine similarity between $\hat{v}_j$ and v_j concentrates around*

$$\cos(\hat{v}_j, v_j) \approx \frac{1}{\sqrt{1 + (n-1)/d}}. \tag{5.4}$$

d	n	add	Δ	formula	softmax
32	8	0.901	0.926	0.906	1.000
32	32	0.716	0.680	0.713	1.000
32	64	0.584	0.452	0.580	1.000
32	128	0.450	0.248	0.449	1.000
32	256	0.333	0.126	0.334	1.000
64	8	0.953	0.971	0.949	1.000
64	32	0.822	0.839	0.821	1.000
64	64	0.708	0.673	0.710	1.000
64	128	0.585	0.452	0.579	1.000
64	256	0.448	0.247	0.448	1.000
128	8	0.976	0.987	0.974	1.000
128	32	0.899	0.928	0.897	1.000
128	64	0.822	0.844	0.819	1.000
128	128	0.708	0.672	0.708	1.000
128	256	0.578	0.447	0.578	1.000

Table 5.1: Mean cosine similarity between recalled and true values for n stored random key–value pairs ([SIM]). "add" is the un-gated additive state, "Δ" is the delta rule (5.2) with $\beta = 1$, "formula" is (5.4), and "softmax" is lookup in an explicit cache.

Proof. The inner products $c_{ij} = k_i^\top k_j$ for $i \neq j$ are approximately $\mathcal{N}(0, 1/d)$ and independent of the values. The noise term $\sum_{i \neq j} c_{ij} v_i$ is a sum of $n-1$ nearly orthogonal unit-norm vectors with random coefficients, so its squared norm concentrates at $(n-1)/d$. The signal v_j has squared norm 1 and is orthogonal in expectation to the noise. Hence $\cos \approx 1/\sqrt{1 + (n-1)/d}$. □

[SIM] Table 5.1 and Figure 5.1 compare (5.4) with simulation. At $d = 64$ and $n = 64$ stored pairs the prediction is 0.710 and the simulation gives 0.708; at $d = 32$, $n = 64$: predicted 0.581, simulated 0.584; at $d = 128$, $n = 256$: predicted 0.578, simulated 0.578. The delta rule, which is better for $n \lesssim d$ (cosine 0.97 at $d = 64, n = 8$ versus 0.95 for plain accumulation), is *worse* on average once $n > d$ because it overwrites older associations to make room for newer ones: at $d = 64$, $n = 256$ the average recall cosine is 0.25 for the delta rule versus 0.45 for the additive state. A softmax KV lookup over the same n pairs (with a sharp temperature) recalls perfectly (cosine 1.00) at every n, because it stores all n pairs explicitly. These simulations isolate the principle: *a fixed-size associative state degrades gracefully but inevitably as more distinct associations are stored, while an explicit cache does not.*

5.3.2 Recency bias under decay

Gating trades capacity for recency. With the gated delta rule (5.3) at $d = 64$ and $n = 128$ stored pairs, average recall of the *oldest* 16 items versus the *newest* 16 is, for $\alpha = 1.0$: 0.15/0.90; $\alpha = 0.99$: 0.06/0.91; $\alpha = 0.97$: 0.006/0.90; $\alpha = 0.9$: 0.004/0.81.[SIM] Even weak decay (0.99) erases most of what was written 100+ tokens earlier at the level of exact recall; a model must therefore learn to *choose* what to write (via β_t and the key) and to hold important content in directions that are not overwritten. Real models do learn data-dependent gates, and the reported hybrid results show that they reach competitive accuracy; the simulation

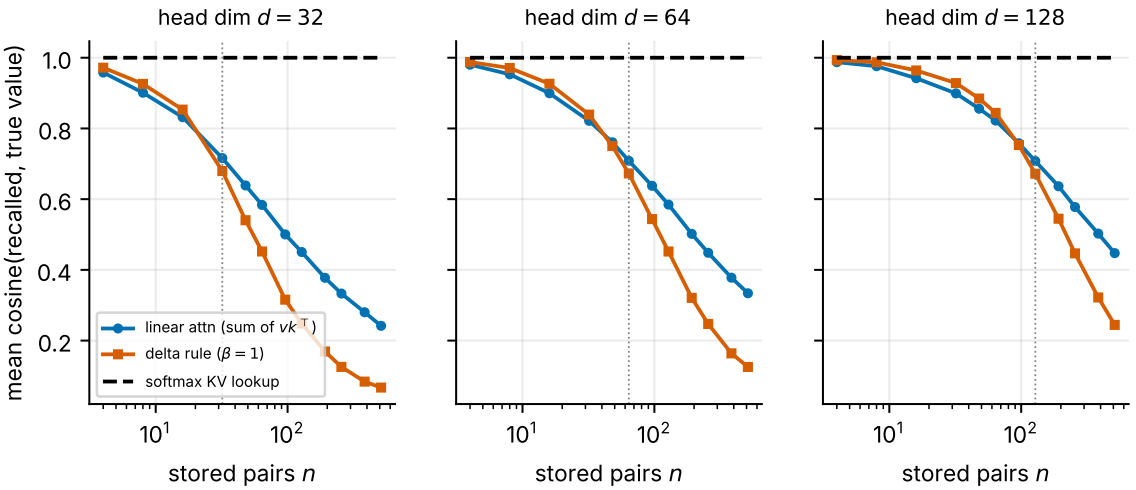


Figure 5.1: Associative recall quality versus number of stored pairs for head dimensions $32, 64, 128$ ([SIM]). Dotted vertical lines mark $n = d$. The delta rule is best below $n \approx d$ and loses older items beyond; additive superposition degrades smoothly; softmax lookup is exact because it keeps all pairs.

is a worst-case (random, incompressible) workload where there is nothing to learn.

5.3.3 An information-theoretic bound

Let a layer's state have $s = H_\ell d_k d_v$ scalars of b_s bits. It cannot store more than sb_s bits. To copy a uniformly random token sequence of length T over a vocabulary of size V requires at least $T \log_2 V$ bits. Hence an exact-copy ability of length T needs $T \leq sb_s / \log_2 V$ for that layer alone (the information could be spread over layers). For a layer with $H_\ell = 32$ heads, $d_k = d_v = 128$, 16-bit state, $V = 131{,}072 = 2^{17}$:

$$T \leq \frac{32 \cdot 128 \cdot 128 \cdot 16}{17} \approx 4.9 \times 10^5 \text{ tokens (information-theoretic ceiling per layer).} \quad (5.5)$$

[DER]This is far above practical capacity (the superposition analysis suggests useful recall degrades once $n \sim d_k = 128$ per head), but it shows the ceiling is a finite number independent of T, whereas the KV cache grows without bound. Jelassi et al. [26] prove the sharper statement that a two-layer Transformer can copy exponentially long strings while fixed-state models are capped by state size, and demonstrate the gap on pretrained models; this is the theoretical reason hybrid architectures retain a fraction of exact-attention layers.

5.4 Memory and cost of a recurrent layer

The state of one layer is $H_\ell d_k d_v b_s$ bytes regardless of T. With 32 heads of 128×128 and BF16 storage that is 1 MiB per layer per sequence; with the FP32 state precision that some kernels require, 2 MiB. MiniMax's account of its decision to keep full attention in M2 lists precision sensitivity of linear attention and difficulty of low-precision state storage among the infrastructure obstacles [5], so the BF16 figure should be regarded as a lower bound. Decode reads and writes the state once per token: $2H_\ell d_k d_v b_s$ bytes (read+write), independent of T, a few MB per layer.

Prefill cost is $\Theta(Td_kd_v)$ FLOPs per head in the recurrent/chunkwise form (plus $\Theta(TCd)$ within chunks), compared with $\Theta(T^2d)$ for softmax. At decode, one token costs about $4dT$ FLOPs per head with softmax and $2d_kd_v$ with a recurrent layer, so for 128-dimensional heads the FLOP break-even is at only $T \approx d_kd_v/(2d) = 64$ tokens. MiniMax's post makes the same point at the level of whole layers, noting that in theory linear attention becomes more efficient than full attention in compute and memory beyond a few thousand tokens, a threshold that is not long by current standards [5]. The practical difficulty is that linear layers are memory-bound in training, and prefix caching and speculative decoding need new solutions for recurrent state [5]; in sparse attention the analogous obstacles are kernel-level (Chapter 12).

5.5 Hybrid layer cost models

Consider an L-layer model with a fraction f of layers that keep a growing cache (sparse or full attention) and a fraction $1-f$ of linear layers. Let a sparse layer read $c_s(T) = 2Gd\,\min(k,T)b + \iota T$ bytes per decoded token, where ι is the index-key bytes per token per layer (zero for full attention with $k = T$, in which case $c_s = 2GdTb$), and a linear layer read/write $c_\ell = 2H_\ell d_k d_v b_s$ bytes. Then

$$c_{\text{hyb}}(T) = Lf\,c_s(T) + L(1-f)\,c_\ell, \qquad \text{Stored}_{\text{hyb}}(T) = Lf\,(2Gdb+\iota)\,T + L(1-f)H_\ell d_k d_v b_s. \tag{5.6}$$

The ratio of the decode bytes of the hybrid to those of a fully dense GQA model with the same dimensions is, for $T \gg k$,

$$\frac{c_{\text{hyb}}}{c_{\text{dense}}} \approx f\Big(\frac{k}{T} + \frac{\iota}{2Gdb}\Big) + (1-f)\frac{c_\ell}{2GdbT}. \tag{5.7}$$

[DER]Three terms are visible: the sparse layers' budget fraction k/T (vanishing at long T), the index-scan fraction $\iota/(2Gdb)$ (a constant that does not vanish with T; for FP8 index keys of 132 bytes and $2Gdb = 4096$ it is 3.2%), and the recurrent layers' share (vanishing with T). At very long context the index scan sets a floor of $f \cdot 3.2\%$ of the dense bytes in this configuration, which is why block-pooled indexing (dividing ι by the block size) matters most for the hybrid with the most sparse layers.

Worked example 5.3 (1M-token decode for a 1:3 hybrid)**.** [DER]From Table 5.2, a 32-layer model with one sparse layer for every three linear layers (8 sparse, 24 linear; $f = 0.25$) reads about 1.2 GB per decoded token at $T = 1$M with token-level FP8 index keys and about 0.11 GB with block-pooled index keys. A dense GQA model with the same dimensions reads 137 GB. The corresponding byte ratios are 115× and 1250× respectively. These are idealised attention-only figures: the weight reads of a 8B model (≈ 16 GB per step at BF16) already exceed the hybrid's attention bytes, so measured end-to-end speed-ups are far smaller. MiniCPM-SALA, which has this 1:3 ratio, reports up to 3.5× inference speed over Qwen3-8B at 256K tokens on one A6000D GPU (time to first token 51.6 s versus 180.8 s, non-quantised), and Kimi Linear reports up to 6× decode throughput at 1M tokens [4, 50]. The gap between 10^2–10^3× attention-byte reductions and 3–6× end-to-end gains is a direct illustration of Proposition 3.2 and Amdahl's law.

Sparse layers	Sparse:linear	Decode MB/token @128K		Decode MB/token @1M		Stored GiB @1M
		token-idx	block-idx	token-idx	block-idx	(token-idx)
1.000	32:0	822	277	4698	338	132.12
0.500	16:16	428	155	2366	186	66.08
0.250	8:24	231	94	1200	110	33.05
0.125	4:28	132	64	617	72	16.54
0.062	2:30	83	49	325	53	8.29
0.000	0:32	34	34	34	34	0.03
dense GQA (ref.)	32 full	17180		137439		128

Table 5.2: Decode bytes per token and stored size for a 32-layer hybrid with Qwen3-8B-like attention dimensions ($G = 8$, $d = 128$), sparse budget $k = 2048$, 32 linear heads of 128×128 BF16 state per linear layer, and an index key of 132 bytes per token per sparse layer ("token-idx") or 132/64 bytes ("block-idx", pooled over 64-token blocks) ([DER]; `exp_e_costmodel.py`). The reference row is dense GQA with the same dimensions.

5.6 Choosing the layer ratio

What fraction of layers should keep exact (sparse or full) attention? Three considerations interact.

Quality: recall. The 72-model study of Wang et al. [27] found language-modelling perplexity roughly flat across linear-to-full ratios while recall improved as full-attention layers were added, especially below 3:1, and recommended 3:1–6:1 with Gated DeltaNet or HGRN-2. At 1:3 sparse-to-linear, MiniCPM-SALA reports RULER 92.65 at 64K and 89.37 at 128K, above Qwen3-8B's 80.53/71.74 (author-reported, measured with the authors' harness) [50]. HySparse uses the opposite extreme: in its 80B MoE model only 5 of 49 layers are full attention, with the other layers sparse and borrowing the full layer's selection and KV, giving a nearly 10× KV-cache reduction [48].

Cost. Equation (5.6) is linear in f, so each additional sparse layer adds a fixed number of bytes (budget plus index scan) at decode, and the stored cache grows linearly with f.

Where to put the exact layers. MiniCPM-SALA keeps the first and last layers unconverted and uses a layer-selection algorithm from the HypeNet work rather than uniform interleaving [50]; the MiniMax post argues that retrieval and induction heads form early in pretraining and are hard to relocate by later conversion, so identifying which layers must stay exact is difficult [5]. This is a hypothesis of the post, not an established result; the MiniCPM-SALA approach of selecting layers algorithmically is one empirical response.

5.7 Why hybrids are cheaper to *train into* than to train

The conversion approach reuses a pretrained Transformer. MiniCPM-SALA reports that its continual training framework reduces cost to about 25% of the 8T tokens used to train MiniCPM-4.0 from scratch (about 2T tokens in total across conversion, stable training, short decay, long decay and SFT stages), i.e. about 75% cheaper (author-reported) [50]. The stages

are instructive: a 1.3B-token architecture-conversion stage trains only the linear layers with sparse attention off at 0.5K context; then 314.6B and 1006.6B tokens of full training at 4K; then 215.7B tokens of long-context decay at 32K/160K/520K with sparse attention on; then SFT at 64K and 140K. About two-thirds of the tokens (1.32T of $\approx$ 1.96T) are spent at 4K context *before* sparse attention is switched on, because InfLLM-V2's dense–sparse switchability lets the sparse layers be trained dense. This is the central reason the MiniCPM family pairs sparse attention with the InfLLM-V2 design: it avoids paying long-context training cost for the bulk of the tokens.

Chapter 6

Shared Routing: Amortisation Analysis

Chapter 3 showed that the index scan becomes the dominant per-layer cost of token-level learned sparse attention at long context. This chapter analyses the family of remedies that share or shrink the scan, derives the cost and quality relations, gives an optimal-placement algorithm for shared indexers, and tests the underlying premise (adjacent layers attend to overlapping tokens) on a real model. Designs are described in Chapter 9; here the focus is on mathematics and measurement.

6.1 Where can a scan be shared?

A selector's work for one decoded token can be written as a product of factors:

$$\underbrace{T}_{\text{candidates}} \times \underbrace{H_I}_{\text{indexer heads}} \times \underbrace{d_I}_{\text{dim.}} \times \underbrace{b_I}_{\text{bytes/elt}} \times \underbrace{L_{\text{idx}}}_{\text{layers that run an indexer}} . \tag{6.1}$$

Table 6.1 associates each published remedy with the factor it reduces and whether it reduces *bytes read*, *FLOPs*, or both. The distinction matters in the bandwidth-bound decode regime.

The levers compose multiplicatively in bytes when they act on different factors: sharing across $m = 4$ layers, FP8-to-FP4 index keys and a $4\times$ shorter candidate list (CSA) would reduce the scan by $4{\cdot}2{\cdot}4 = 32\times$. No paper in the retrieved set combines all of them (DeepSeek-V4 combines compression with a FP4 indexer; GLM-5.2 combines DSA with IndexShare), so the combined effect remains a calculation rather than a measurement.[DER]

6.2 Quality of a shared selection

Let layer l have attention distribution $p^{(l)}$ (for a fixed query token, head-averaged or per head) and let the anchor layer $a < l$ have distribution $p^{(a)}$ and selected set $S_a = \text{top-}k(p^{(a)})$ (or the top-k set of the anchor's indexer, which approximates it). The shared layer attends over S_a.

Proposition 6.1 (Mass captured by a shared selection)**.**

$$p^{(l)}(S_a) \;\geq\; m_k^{(a)} - \mathrm{TV}\big(p^{(l)}, p^{(a)}\big), \qquad m_k^{(l)} - p^{(l)}(S_a) \;\leq\; \big(m_k^{(l)} - m_k^{(a)}\big)^+ + \mathrm{TV}\big(p^{(l)}, p^{(a)}\big), \tag{6.2}$$

where $m_k^{(\cdot)}$ is the oracle top-k mass of the respective layer.

Proof. For any set S, $|p^{(l)}(S) - p^{(a)}(S)| \leq \mathrm{TV}(p^{(l)}, p^{(a)})$, and $p^{(a)}(S_a) = m_k^{(a)}$. The second inequality follows by subtracting the first from $m_k^{(l)}$. □

Lever	Factor reduced in (6.1)	Bytes?	FLOPs?	Examples
Layer sharing	$L_{\text{idx}} \to L_{\text{idx}}/m$	yes ($\div m$)	yes ($\div m$)	IndexCache, CLSA, HySparse
Hierarchical candidates	$T \to T/B + \kappa B$	yes	yes	HISA, AsyncTLS
Compressed candidates	$T \to T/m_c$	yes	yes	NSA, CSA ($m_c = 16, 4$), MoBA (B)
Head activation	$H_I \to r$	no (shared key read once)	yes ($\div H_I/r$)	MISA
Group-level indexer	one head per GQA group	no	yes	MSA, SparDA
Index-key precision	$b_I \downarrow$	yes	no (unless HW)	DSA FP8, V4 FP4
Lookahead	(hides latency)	no	no	SparDA, AsyncTLS

Table 6.1: Levers for reducing the selector cost, by factor of (6.1).

The proposition says that sharing costs at most the total-variation distance between the layers' attention distributions, plus any difference in how concentrated they are. It is an upper bound on the penalty, loose when the layers differ mainly in the tail. A more useful empirical quantity is the *mass ratio*

$$\rho_k(a \to l) = \frac{\mathbb{E}\, p^{(l)}(S_k^\star(p^{(a)}))}{\mathbb{E}\, m_k^{(l)}}, \tag{6.3}$$

the fraction of the layer's own oracle top-k mass that the anchor's set recovers; Section 6.6 measures it. If the shared layers' attention were independent of the anchor's, ρ_k would fall to about k/T divided by the mass concentration (i.e. to random chance). In the opposite limit of identical distributions $\rho_k = 1$.

Sharing is not the same as the layers being similar. A shared layer is penalised only through the tokens it *needs*, weighted by its own attention. If a layer's mass is almost all on the sink and local window, sharing a set that contains the sink and local window costs nothing even if the layers differ elsewhere. This is both a strength (many layers are easy) and a pitfall for evaluation: measures such as overlap of top-k sets or ρ_k must be reported *after* removing the positions that every layer attends to, or the sharing premise will look better than it is. Section 6.6 reports both versions.

6.3 Choosing which layers keep an indexer

The problem solved by IndexCache's greedy search can be posed exactly. Let layers be $0, \ldots, L-1$; choose a set of anchor (Full) layers $\mathcal{A} \ni 0$ with $|\mathcal{A}| = f$; every non-anchor layer b uses the nearest preceding anchor $\phi_{\mathcal{A}}(b)$. Suppose a calibration set gives a penalty

Algorithm 1 Optimal anchor placement for shared indexers (additive penalties)

Input: penalties $c(a, b)$ for $0 \le a \le b < L$; number of anchors f.
Precompute $\mathrm{seg}(a, e) = \sum_{b=a}^{e} c(a, b)$ for all $a \le e$ ▷ $O(L^2)$ via prefix sums
$D[1][e] \leftarrow \mathrm{seg}(0, e)$ for $e = 0, \dots, L-1$
for $s = 2, \dots, f$ **do**
    for $e = s-1, \dots, L-1$ **do**
        $D[s][e] \leftarrow \min_{s-1 \le a \le e}\{D[s-1][a-1] + \mathrm{seg}(a, e)\}$, recording the minimising a
Output: $D[f][L-1]$ and, by backtracking, the anchor set $\mathcal{A}$. ▷ time $O(fL^2)$

$c(a, b) \ge 0$ for layer b using the selection of anchor $a \le b$ (e.g. the increase in language-modelling loss when only layer b uses a's indices, or $1 - \rho_k(a \to b)$), with $c(a, a) = 0$. If penalties are *additive* across layers, the optimal placement minimises $\sum_b c(\phi_{\mathcal{A}}(b), b)$ and is found by dynamic programming.

Proposition 6.2. *Algorithm 1 returns a placement of minimum total penalty under additive penalties, in $O(fL^2)$ time and $O(fL)$ space.*

Proof. Segments $[a_s, a_{s+1}-1]$ between consecutive anchors partition the layers and their costs are independent under additivity; this is the classical optimal segmentation ("k-segment") problem solved by the stated recursion on the last segment. □

For $L = 61$ and $f = 16$ (a 75% reduction, $m_{\mathrm{avg}} \approx 3.8$) the DP evaluates about $16 \cdot 61^2/2 \approx 3 \times 10^4$ cells after $O(L^2)$ penalty evaluations: trivial compared with the model evaluations needed to estimate $c(a, b)$. The practical difficulty is estimating $c(a, b)$ (each requires a forward pass with a modified routing) and the non-additivity of real penalties (two nearby shared layers interact). IndexCache's greedy search and the training-aware distillation loss can be viewed as the heuristic and the repair for these issues; whether the exact DP gives better placements than greedy, once interaction terms are modelled, is not reported in the retrieved material. The fixed periodic pattern used in GLM-5.2 (`full, shared, shared, shared`) corresponds to a uniform segmentation with segment length 4, optimal when $c(a, b)$ depends only on $b - a$ and the layers are exchangeable.[DER]

6.4 Cross-decoder sharing: all layers use one index

CLSA is the limiting case $m = L/2$ of cross-layer sharing: one indexer for all cross-decoder layers. With a single target (the average attention distribution over all layers and heads), the indexer minimises $\mathrm{KL}(\bar{p} \| q)$ where $\bar{p} = \frac{1}{L'H} \sum_{l,h} p^{(l,h)}$. By the convexity of KL in its first argument, this is a mixture target; its optimum $q^\star = \bar{p}$ (Gibbs' inequality) is the *average* attention, and the captured mass for layer l, head h is $p^{(l,h)}(S)$ with $S = \text{top-}k(\bar{p})$. For heads whose distributions are very different from the average (a head that attends to a single position that the average hides), the penalty is large; Proposition 4.6 bounds only the averaged error. Whether the model compensates by learning to place its important information where the average attention would find it is the question that the end-to-end sparse adaptation stage answers empirically; the CLSA paper's evidence (loss difference ≤ 0.0054 at a 2048 budget; multi-needle RULER gains) says that, at 4B scale, it does.[DER]

A useful corollary: if each head (l, h) has captured mass $\geq 1 - \varepsilon_{l,h}$ under its own top-k and the average distribution mixes heads with weights $w_{l,h}$, then $\bar{p}(S) \geq 1 - \sum w_{l,h}\varepsilon_{l,h}$ for S the union of the heads' sets; but $|S|$ can be as large as $\sum_{l,h} k$, so with disjoint needs the single-index budget would have to grow with the number of heads. The shared budget of 2048 is evidently adequate for the evaluated tasks because heads substantially overlap (sink, local, a handful of relevant spans).

6.5 Amortisation in the training of the indexer

An indexer that serves m layers must predict their average attention; the KL target has lower entropy-variance across layers if the layers are similar, and the *irreducible* loss floor is

$$\min_q \ \tfrac{1}{m}\sum_{j=1}^{m} \mathrm{KL}(p^{(j)} \| q) = \tfrac{1}{m}\sum_{j} \mathrm{KL}\left(p^{(j)} \| \bar{p}\right) =: \mathrm{JS}_m, \tag{6.4}$$

the (generalised) Jensen–Shannon divergence of the layers' distributions, attained at $q = \bar{p}$ (a standard identity). Sharing is therefore free exactly when $\mathrm{JS}_m \approx 0$. This gives a principled *diagnostic*: compute the Jensen–Shannon divergence of candidate groups of layers on calibration data; groups with low JS can share an indexer at low cost. I am not aware of a published paper that computes it; IndexCache's calibration search (via LM loss) and overlap statistics (70–100%) are proxies.[DER]

6.6 Real-model test of the sharing premise

To test the premise that adjacent layers attend to overlapping tokens, I used the same Qwen3-0.6B runs (Section 4.9; `exp_real_qwen.py` and the control run `exp_real_cross_ctrl.py`; three 2048-token windows; queries at positions 1024–2047). [SIM] For each layer the attention is averaged over the 16 heads, giving one distribution per query that plays the role of a single shared indexer's target (as in CLSA). For a gap g the set S_l = top-k of layer l is compared with the set of layer $l + g$ by (i) the *overlap* $|S_l \cap S_{l+g}|/k$, and (ii) the *mass ratio* ρ_k of (6.3), the head-averaged attention mass of layer $l + g$ falling on S_l divided by the mass on its own top-k set. Both are averaged over layers, queries and windows.

Control for trivially shared positions. Every layer attends to the first token (sink) and, in many layers, to the most recent tokens; a shared set that contains these has high overlap by construction. The *controlled* variant removes the sink (position 0) and the 64 most recent positions from the candidate pool of both layers, and measures overlap and mass ratio on the remaining positions (mass ratio relative to the layer's own top-k among the remaining positions). These excluded positions hold on average 78% of the head-averaged attention mass (range 56% at layer 2 to 97% at layer 27, over the three windows).

Observations. (1) *Raw numbers look favourable.* At gap 1 a shared top-64 set recovers 94% of the mass of a layer's own top-64 set, and at gap 8 still 87% (Table 6.2), with an overlap of 0.61 and 0.51. Such figures resemble the 70–100% adjacent-layer overlap reported (second-hand) for DSA models [2ND]. (2) *Most of this is the sink and the local window.* After

	top-64				top-256			
	raw		controlled		raw		controlled	
Gap g	ovl.	ratio	ovl.	ratio	ovl.	ratio	ovl.	ratio
1	0.613	0.940	0.444	0.700	0.628	0.961	0.579	0.809
2	0.597	0.915	0.424	0.681	0.614	0.943	0.562	0.793
4	0.570	0.887	0.379	0.624	0.582	0.919	0.525	0.756
8	0.508	0.870	0.296	0.502	0.516	0.902	0.450	0.664

Table 6.2: Cross-layer sharing of the head-averaged top-k set on Qwen3-0.6B ([SIM]; mean over layers, 1024 queries and three windows). "Raw" includes all positions; "controlled" removes the sink and the last 64 tokens. "ovl." is the set overlap and "ratio" the mass ratio ρ_k.

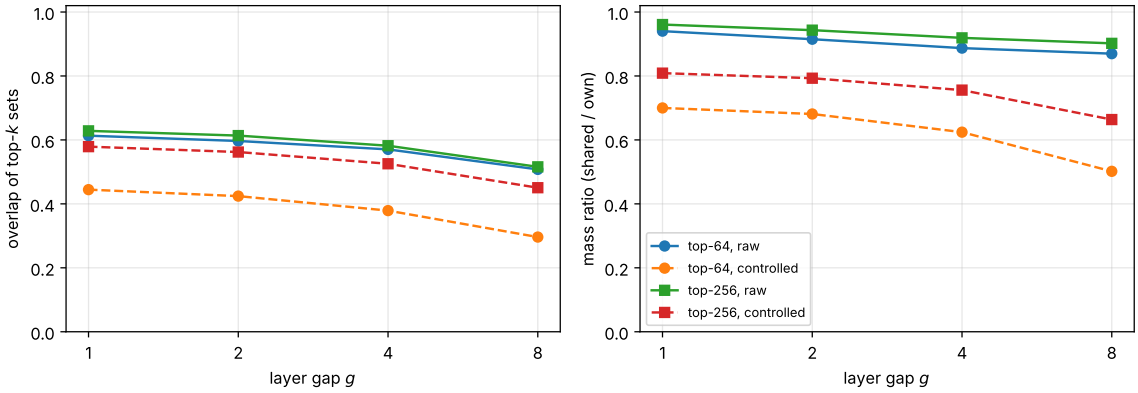


Figure 6.1: Overlap (left) and mass ratio (right) versus layer gap, raw and controlled ([SIM], Qwen3-0.6B).

removing them, the overlap falls to 0.44 (gap 1) and 0.30 (gap 8) for top-64 and the mass ratio to 0.70 and 0.50: a shared set then recovers only about half to 70% of the non-trivial mass. For top-256 the controlled ratio is 0.81 at gap 1 and 0.66 at gap 8. The decay with gap is monotone and gradual rather than abrupt: layers four to eight apart share less than adjacent ones, in line with the practice of sharing within groups of about four. (3) *The non-trivial remainder is small.* The controlled top-64 sets carry about 10% of the mass of the queries on average (top-256: 16%), so even a sizeable relative loss in the controlled ratio corresponds to a modest absolute loss of mass (about 3 points for top-64 at gap 1). The practical consequence, in this model, is that sharing is cheap mostly because most mass is on positions that every layer retains anyway. (4) *Per-head comparison is much worse, but is not the relevant one.* Comparing individual heads of layers 6 → 12 and 12 → 18 (same head index; top-64, controlled) gave a mean overlap of 0.17 and 0.20 and a mean mass ratio of 0.29 and 0.34 (per-head overlaps ranged from 0.05 to 0.37). Head indices do not correspond across layers, so these numbers measure how much a *different head* selects the same tokens, and are best read as showing that a per-head index cannot be shared naively across heads of different layers; a shared head-averaged index, as in CLSA, is the correct analogue.

Reading against the literature. The result is consistent with IndexCache's finding that sharing across a few adjacent layers costs little and that more aggressive sharing requires care (calibration search or training) [51], and with CLSA's choice to train a single indexer on the average attention of all layers and heads [1]: a trained model can organise its attention so

that the shared index is sufficient, which a model trained without sharing, as here, does not do. Because Qwen3-0.6B was not trained with a sharing objective, the numbers in Table 6.2 are a *lower bound on what a model trained for sharing would achieve*, subject to the usual caveats (short context, small model, single genre, no downstream metric). [SIM]

Chapter 7

KV-Cache Quantization and Eviction: Error Analysis

Sparse selection reduces the bytes *read*; compression reduces the bytes *stored* (and, since reads come from storage, also those read). This chapter develops the error analysis for the two compression operations most relevant to the period under review: quantization of keys and values (KIVI, TurboQuant) and eviction of tokens (H2O, SnapKV, TriAttention, Random Attention). Derivations are tagged [DER]; simulations [SIM].

7.1 How cache perturbations reach the output

The cache enters attention through logits $z_i = q^\top k_i/\sqrt{d}$ and values v_i. Quantization perturbs both.

Key errors perturb logits. If key k_i is replaced by $k_i + e_i$ with an error vector of per-coordinate variance σ_e^2 (independent of q), the logit error is $\delta_i = q^\top e_i/\sqrt{d}$ with variance $s^2 = \|q\|^2\sigma_e^2/d$. The attention distribution changes from p to $p'_i \propto p_i e^{\delta_i}$.

Proposition 7.1 (Attention KL from logit noise)**.** *For small logit perturbations δ,*

$$\mathrm{KL}(p \,\|\, p') \;\approx\; \tfrac{1}{2}\,\mathrm{Var}_{i\sim p}(\delta_i). \tag{7.1}$$

If the δ_i are independent with variance s^2, then $\mathrm{Var}_p(\delta) \approx s^2(1 - \sum_i p_i^2) \le s^2$.

Proof. $p'_i = p_i e^{\delta_i}/Z$ with $Z = \mathbb{E}_p[e^\delta]$. Then $\mathrm{KL}(p\|p') = \log Z - \mathbb{E}_p[\delta]$. Expanding $\log \mathbb{E}_p[e^\delta] = \mathbb{E}_p\delta + \frac{1}{2}\mathrm{Var}_p(\delta) + O(\delta^3)$ gives (7.1). For independent noise, $\mathrm{Var}_p(\delta)$ has expectation $s^2(1 - \sum p_i^2)$ (the variance of a weighted average of independent variables, with $\sum p_i^2$ the collision probability). □

Value errors average out. If value v_i is replaced by $v_i + \epsilon_i$ with independent errors of per-coordinate variance σ_v^2, the output error $\sum_i p_i\epsilon_i$ has variance $d_v\sigma_v^2\sum_i p_i^2$, so it *shrinks* when attention is diffuse ($\sum p_i^2$ small) and is largest when attention is concentrated on a single token ($\sum p_i^2 = 1$). Key errors, in contrast, matter most when the logits are large in magnitude ($\|q\|$ large) and attention is spread across many tokens.[DER] These two facts rationalise the asymmetric treatment in KIVI (per-channel quantization for keys, which have a few outlier channels with large magnitude that dominate $\|q\cdot e\|$, and per-token quantization for values) [35].

Eviction perturbs the support. Eviction deletes entries; the effect is exactly the truncation of Chapter 4 with S the retained set, but now the set is fixed *before* the future queries are known.

b	Simulated MSE	Quoted bound (paper)	Lower bound 4^{-b}	Ratio sim./lower
1	0.3606	0.36	0.2500	1.44
2	0.1158	0.117	0.0625	1.85
3	0.0340	0.03	0.0156	2.18
4	0.0094	0.009	0.0039	2.40

Table 7.1: MSE of rotation plus Lloyd–Max scalar quantization of unit vectors ($d = 128$) from the simulation `exp_c_quant.py` ([SIM]), compared with the values retrieved from the TurboQuant paper and its lower bound.

7.2 Uniform quantization and the outlier problem

A b-bit uniform quantizer over a range $[-R, R]$ has step $\Delta = 2R/(2^b - 1)$ and, for inputs uniform within a step, error variance $\Delta^2/12$ per coordinate. For a vector with a few outlier channels of magnitude ρR_0 ($\rho \gg 1$) where the other channels have scale R_0, setting $R = \rho R_0$ (to avoid clipping) inflates the step for all channels by ρ and the MSE by ρ^2. Per-channel scaling avoids this for outlier *channels* (fixed positions); outlier *tokens* are handled by per-token scales; rotations remove the problem by making the coordinates statistically alike.[DER]

7.3 Random rotation and Lloyd–Max: the TurboQuant construction

TurboQuant [36] is built from three ideas, which I restate with the numbers retrieved from the paper.

(1) Random rotation. For a unit vector $x \in \mathbb{R}^d$ and a Haar-random rotation Π, the coordinates of Πx are exchangeable with a Beta-type marginal that is close to $\mathcal{N}(0, 1/d)$ for large d and nearly independent across coordinates. Hence all vectors, whatever their outliers, are transformed to a canonical coordinate distribution.

(2) Optimal scalar quantization of the canonical marginal. Because the marginal is known, a Lloyd–Max quantizer can be designed offline for b bits. For a standard normal marginal the normalised distortion D_b of the optimal b-bit scalar quantizer gives, for a unit vector, vector MSE D_b. [SIM] My re-implementation (random orthogonal matrix from QR of a Gaussian matrix, $d = 128$, 4000 unit vectors, Lloyd–Max fitted by 200 iterations on 2×10^6 Gaussian samples) measures $D_b = 0.3606, 0.1158, 0.0340, 0.00938$ for $b = 1, \ldots, 4$ (Table 7.1). TurboQuant's Theorem 1 gives upper bounds for MSE distortion of order 4^{-b} times a constant, which the retrieved page states as $\sqrt{3\pi}/2$ but which, given that the same page puts the gap to the lower bound at about 2.7, must be $\sqrt{3}\,\pi/2 \approx 2.72$; the tighter small-$b$ values quoted are 0.36, 0.117, 0.03, 0.009 for $b = 1, \ldots, 4$, in agreement with the simulation. The information-theoretic lower bound is 4^{-b} (Theorem 3), so the simulated quantizer is 1.44×, 1.85×, 2.18×, 2.40× above the bound at $b = 1, \ldots, 4$.

Outlier channels. I tested the claim that rotation neutralises outliers by constructing vectors with four channels scaled 25× (then normalised) and comparing (i) per-vector min–max

	MSE-only (b bits)			MSE ($b-1$) + 1-bit QJL residual		
b	slope	RMSE	mean err.	slope	RMSE	mean err.
2	0.885	0.0299	3×10^{-4}	1.005	0.0678	-7×10^{-4}
3	0.965	0.0162	4×10^{-4}	0.999	0.0381	-5×10^{-5}
4	0.991	0.0085	1×10^{-4}	0.999	0.0204	9×10^{-6}

Table 7.2: Inner-product estimation for random unit vector pairs ($d = 128$, 4000 pairs; [SIM]). "Slope" is the regression coefficient of the estimate on the true inner product; $1 - D_b$ predicts $0.884, 0.966, 0.991$ for the MSE-only column. Both settings use the same number of bits per coordinate.

uniform quantization with no rotation against (ii) rotation plus Lloyd–Max. At $2, 3, 4$ bits the MSE is $1.377, 0.264, 0.0558$ for (i) versus $0.123, 0.0382, 0.0111$ for (ii): rotation reduces error by $11\times$ at 2 bits, $6.9\times$ at 3 and $5.0\times$ at 4.[SIM] This is a controlled synthetic test and the min–max baseline is weak; stronger baselines (per-channel scaling, as in KIVI) address the same outliers differently. It nonetheless exhibits the mechanism.

(3) Inner-product correction. MSE-optimal quantizers are biased for inner products: for a centroid (Lloyd–Max) quantizer with reconstruction $\hat{x}$, $\mathbb{E}[\hat{x}^\top x] = \mathbb{E}\|\hat{x}\|^2 = 1 - D_b$ for a unit vector, so inner products with any fixed y are shrunk by the factor $1 - D_b$.[DER] Table 7.2 confirms: the regression slope of estimated against true inner products is $0.885, 0.965, 0.991$ for $b = 2, 3, 4$ against $1 - D_b = 0.884, 0.966, 0.991$.[SIM]

TurboQuant's remedy is to quantize at $b - 1$ bits and add a one-bit Quantized Johnson–Lindenstrauss (QJL) sketch of the residual $r = x - \hat{x}$. With a Gaussian matrix $S \in \mathbb{R}^{d \times d}$ (rows s_j), the estimator

$$\widehat{\langle r, y \rangle} = \frac{\sqrt{\pi/2}}{d} \|r\| \sum_{j=1}^{d} (s_j^\top y) \operatorname{sign}(s_j^\top r) \tag{7.2}$$

is unbiased because $\mathbb{E}[(s^\top y) \operatorname{sign}(s^\top r)] = \sqrt{2/\pi}\, \langle y, r \rangle / \|r\|$ for Gaussian s. Its variance is at most $\frac{\pi}{2}\|r\|^2\|y\|^2/d$ (each summand has second moment $\frac{\pi}{2}\|r\|^2\|y\|^2$ and the d rows are independent).[DER] With $\|r\|^2 \approx D_{b-1}$, the added variance is about $\frac{\pi}{2} D_{b-1}\|y\|^2/d$, larger than the variance of the biased MSE-only estimator, which is the price of unbiasedness. In the simulation (Table 7.2), the unbiased variant has slope ≈ 1.000 but root-mean-square error $0.068, 0.038, 0.020$ versus $0.030, 0.016, 0.0085$ for MSE-only at $b = 2, 3, 4$; i.e. 2.0–2.4× larger RMSE.[SIM]

Remark 7.2 (Bias versus variance in attention). For *softmax* attention what matters is the logit error distribution: a uniform shrink of all logits by $1 - D_b$ is a temperature change, which is mild (at 3 bits, 3.4%), and its effect can be absorbed by a scale. Noise, in contrast, enters Proposition 7.1 directly through s^2. The simulation therefore suggests, as a hypothesis not tested here on a real model, that the QJL stage may hurt rather than help softmax attention at fixed bits: it spends one bit per coordinate to remove a benign bias at the price of 2–2.4× more logit noise. A second-hand report that independent implementers found the QJL stage to hurt token-matching accuracy and that the first stage (PolarQuant) did most of the useful work [2ND] [62] is consistent with this reading, but I did not verify it from the implementers' repositories.

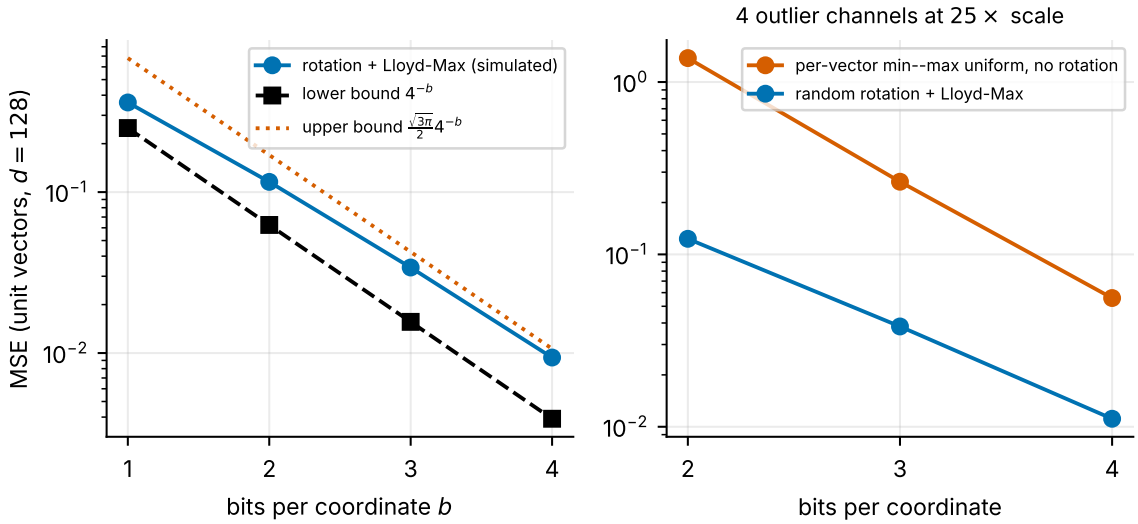


Figure 7.1: Left: simulated MSE of rotation plus Lloyd–Max quantization versus the information-theoretic lower bound and the upper-bound constant $\sqrt{3}\pi/2$ (the plotted constant $\sqrt{3}\pi/2 \approx 2.72$ is the one implied by the paper's quoted gap of about 2.7 to the lower bound). Right: effect of outlier channels on min–max quantization with and without rotation ([SIM]).

7.4 What TurboQuant reports, and what is contested

Paper-reported results [36]. On LongBench with Llama-3.1-8B-Instruct, average scores: full cache (16 bits) 50.06; KIVI at 3 bits 48.50 and at 5 bits 50.16; PolarQuant at 3.9 bits 49.78; TurboQuant at 2.5 bits 49.44 and at 3.5 bits 50.06. For Ministral-7B-Instruct: full 49.89, TurboQuant 2.5 bits 49.62. The 2.5- and 3.5-bit settings use a split into outlier and regular channels: for example 32 outlier channels at 3 bits and 96 regular at 2 bits gives 2.5 bits per channel on average. On needle-in-a-haystack with Llama-3.1-8B-Instruct the recall scores are 0.997 (full), 0.997 (TurboQuant at more than 4× compression), 0.995 (PolarQuant), 0.981 (KIVI), 0.895 (PyramidKV) and 0.858 (SnapKV), with the eviction baselines run at a 0.25 memory ratio. The paper states compression of at least 4.5× for these runs.

Vendor blog framing. Google's blog describes the method as quantizing the KV cache to 3 bits without training or fine-tuning with no drop in accuracy, KV memory reduced by at least 6× on needle tasks, and 4-bit TurboQuant computing attention logits up to 8× faster than 32-bit unquantized keys on H100 GPUs, with the caption indicating the baseline is an optimised JAX implementation [63][VENDOR]. The comparison basis (32-bit keys, JAX) is not the natural baseline of a BF16 production decoder, so the 8× figure should not be read as a speedup over FlashAttention or a fused BF16 kernel.

Independent reports (second-hand). A May 2026 blog survey counts at least ten community implementations within a month, reports roughly 4.7× compression for 3-bit on an Apple M4 Max (short of the 6× headline) and notes the vLLM integration was not merged at time of writing [2ND] [62]. The same post relays an accusation by the RaBitQ authors that TurboQuant mischaracterised RaBitQ (random rotations, optimality of guarantees) and compared against a degraded single-threaded port; it says a complaint was reportedly filed with

ICLR's ethics process and that Google had not publicly responded [2ND]. I have not verified any element of that dispute from primary sources; it is included because it is part of how the work was received (Chapter 16), not as a finding.

7.5 Eviction: what a fixed cache cannot do

Proposition 7.3 (Unavoidable worst case for query-agnostic eviction)**.** *Let an eviction policy retain a set S with $|S| < T$ chosen without knowledge of future queries. For generic keys and any $\epsilon > 0$ there exists a query whose attention places weight at least $1 - \epsilon$ on a single evicted key, so that $\tau(S) \geq 1 - \epsilon$ and, by Proposition 4.1, the output error can be made of the order of $\|v_j - \hat{o}_S\|$.*

Proof. Some key $j \notin S$ exists. For generic keys there is a direction q with $q^\top k_j > q^\top k_i$ for every $i \neq j$ (the key k_j is an extreme point of the key set); scaling q makes $p_j \geq 1 - \epsilon$. □

The proposition is trivial but it clarifies the nature of the evidence: eviction methods cannot be assessed by worst-case guarantees, only by how often real queries are served. This is the content of the "Pitfalls of KV cache compression" paper, which tested five eviction methods (StreamingLLM, SnapKV, TOVA, H2O, K-Norm) on Llama-3.1 8B and Qwen2.5 14B with IFEval and found that some instructions degrade much faster than others, that system-prompt leakage worsens, and that three factors interact: the compression method, instruction order, and eviction bias [64]. (No numbers were available on the retrieved abstract page.)

7.5.1 Accumulated-score bias

H2O ranks tokens by accumulated attention $A_i(t) = \sum_{s \leq t} p_{s,i}$ [29]. Under uniform causal attention ($p_{s,i} = 1/s$ for $s \geq i$) a token of age position i accumulates $A_i(T) = \sum_{s=i}^{T} 1/s \approx \ln(T/i)$, so the ranking is purely by age, favouring the oldest tokens. Real attention is not uniform, but the bias persists: older tokens have been exposed to more queries. This is the "eviction bias" named in [64] and is why H2O-style methods reserve a recent window and why attention sinks (large mass on the first tokens) are retained by construction.[DER]

7.5.2 Retention under redundancy: why random eviction can work

Random Attention [60] evicts uniformly at random within each KV head, protects the whole prompt and a 64-token recent buffer, and matches the best scored baseline in their tests while giving higher throughput. A simple model explains how this can be viable. Let each of G KV heads independently retain a fraction $\rho = B/T$ of the generated entries. A fact stored once in the cache survives in at least one head with probability $1 - (1 - \rho)^G$. At $4\times$ compression ($\rho = 1/4$) with $G = 8$ KV heads (Qwen3-8B-like), that is $1 - 0.75^8 = 0.90$; with $G = 4$, 0.68. If the model restates the fact r times in its reasoning trace, the probability that *no* surviving copy exists in any head falls as $(1 - \rho)^{rG}$, e.g. $(0.75)^{16} = 0.010$ for $r = 2$, $G = 8$.[DER] The paper attributes its result to precisely these two redundancies, restating in text and per-head copies, and measures with a planted-fact probe on Qwen3-4B that reading the value from the single best head succeeds in about 3% of trials, two heads 60%, three 83%

and all eight 99% [60]. Those probe numbers measure *readability by head subsets*, a different quantity from the survival probability above, but both show that redundancy across heads can rescue information that a single head would lose. The paper also reports the limits: facts stated once and never restated (a passcode probe) are not recovered by random eviction at all, while R-KV recovers it 84% of the time, and short generations (1k in, 8k out) do not benefit from compression [60].

The existence result is more important than the mechanism: *a method with no importance signal matches methods with elaborate signals on reasoning benchmarks.* Either the benchmarks are insensitive to the quality of the signal, or the redundancy in chain-of-thought is so large that signals are unnecessary at 4× compression; the paper's own statistical caveat (AIME has 30 problems, run-to-run standard deviation about ±5 points) means many differences are not significant.

7.5.3 Pre-RoPE concentration and the TriAttention score

RoPE rotates the f-th two-dimensional block of q and k by the angle $\theta_f \cdot \text{position}$. For a query at position p_q and key at p_k the logit decomposes as

$$z = \sum_f \|q_f\| \, \|k_f\| \cos\big(\theta_f (p_q - p_k) + \phi_{q,f} - \phi_{k,f}\big), \tag{7.3}$$

[DER] with $\phi_{\cdot,f}$ the pre-rotation phases. TriAttention [53] observes that in the pre-RoPE space the query (and key) vectors of most heads cluster tightly around stable non-zero centers, measured by the mean resultant length $R_f = \|\mathbb{E} q_f\| / \mathbb{E}\|q_f\|$ near 1. If q_f is replaced by its centre $\bar{q}_f$, (7.3) becomes a fixed trigonometric series in the offset $\Delta = p_q - p_k$ with coefficients depending only on the key's per-band norm and phase: each key can be scored by how well it matches the distances at which queries (with that centre) prefer to attend, *without observing recent queries.* This avoids the problem that post-RoPE queries rotate with position, so that only a small window of recent queries is representative. The paper adds a second signal based on norms, weighted by $(1 - R_f)$, to cover heads with weak concentration, averages over future offsets $1, 2, 4, \ldots, 2^{16}$, and prunes to the budget every 128 tokens [53]. The derivation above is a re-derivation of the first step from the RoPE definition; the exact scoring formula is the paper's.

7.6 Real-tensor experiments on quantization and eviction

The synthetic quantisation study (Section 7.3) used i.i.d. Gaussian vectors with injected outliers. To check how the conclusions carry to real tensors I re-used the post-RoPE queries, keys and values saved from the Qwen3-0.6B runs of Section 4.9 ([SIM]; `exp_f_real_quant_evict.py`; layers 2, 9, 16, 23; 16 query heads sharing 8 KV heads; 2048 tokens; queries at positions 1024–2047). Keys were quantised to $b \in \{2, 3, 4\}$ bits by three schemes with values left at full precision: (i) *naive per-token* min–max uniform quantisation of each key vector; (ii) *naive per-channel* min–max over the whole 2048-token axis (the KIVI-style layout for keys, but non-causal and without a full-precision residual window, which flatters it on scale estimation and ignores streaming); (iii) *rotation + Lloyd–Max*: normalise, apply one fixed random orthogonal matrix, quantise each coordinate with the Gaussian Lloyd–Max codebook of Section 7.3, rotate back and rescale (the MSE stage of TurboQuant-style schemes; the QJL

	mean KL (nats)			relative output error		
Scheme	2 bit	3 bit	4 bit	2 bit	3 bit	4 bit
Naive per-token	13.19	6.94	1.90	3.10	1.60	0.76
Naive per-channel	0.81	0.117	0.024	1.12	0.325	0.134
Rotation + Lloyd–Max	1.60	0.585	0.175	1.11	0.538	0.277

Table 7.3: Key quantisation on real Qwen3-0.6B tensors ([SIM]; mean over layers 2, 9, 16, 23; values unquantised). Output errors above 1 mean the attention output is destroyed. Per-layer numbers are in `results/real_quant_evict.json`.

residual stage and any bit-packing are not included, and the norm is kept in full precision). The quantities reported are the KL divergence from the full-precision attention row to the quantised one and the relative error of the attention output, averaged over heads, queries and the four layers.

Observations. (1) At 2 bits every scheme fails on this model: the relative output error is above 1. At 4 bits, per-token quantisation still has a mean KL of 1.9 nats, dominated by layers 2 and 9, whereas per-channel quantisation gives 0.024 nats and 13% output error. (2) The reason is outlier channels: the ratio of the largest to the median per-channel mean $|k|$ is 94 (layer 2), 22 (layer 9), 8 (layer 16) and 12 (layer 23). A min–max scale per token is set by the outlier channel and wastes the bit budget on the remainder. (3) *Rotation helped relative to per-token quantisation (e.g. 4 bits: KL 0.175 versus 1.90) but was worse than per-channel min–max in this test (KL 0.175 versus 0.024).* This is not what the synthetic experiment alone would suggest and I do not claim it generalises: the per-channel baseline here uses scales estimated from the whole sequence (including later tokens) and the keys of this model are strongly channel-structured, so a per-channel scale captures structure that a data-oblivious rotation spreads out and then quantises with a fixed Gaussian codebook that does not match the post-rotation distribution of a few strong channels. A fair comparison with the published TurboQuant would also need its full pipeline (QJL stage, its codebook choice, and the pre-RoPE or post-RoPE key placement they use) and the end-task metrics they report; the result here only says that for post-RoPE keys of a small model, the MSE-only rotation scheme is not automatically superior to a simple outlier-aware scale. [SIM]

Eviction policies on the same tensors. Using the dense attention matrices of layers 9 and 16 (heads 0, 5, 10, 15; 2048 tokens), I simulated four cache policies under a budget of B tokens and measured the mean fraction of the *dense* attention mass that falls on the retained set for query positions 1024–2047. The policies are StreamingLLM (4 sinks plus the most recent $B - 4$), an H2O-style accumulated-attention score (protecting 4 sinks and 32 recent tokens, accumulating renormalised attention over the cached set), random eviction (with the same protected tokens) and a per-query oracle top-B (not an eviction policy: it is the upper bound if the cache could be re-selected for every query).

In this short-context test the simple sink-plus-window policy retains *more* mass than the H2O-style accumulated score and than random eviction, and all irreversible policies sit 5–15 points below the per-query oracle at the same budget. The H2O-style policy losing to random is consistent with the accumulated-score bias analysed in Section 7.5.1 (older tokens accumulate more mass, recent tokens that matter later are evicted early) and with

Policy (layer)	$B = 128$ L9	$B = 128$ L16	$B = 256$ L9	$B = 256$ L16	$B = 512$ L9	$B = 512$ L16
Sink + recent window	0.891	0.850	0.920	0.901	0.950	0.948
H2O-style accumulated score	0.873	0.798	0.898	0.831	0.931	0.878
Random (protected sink/recent)	0.885	0.831	0.908	0.875	0.936	0.913
Oracle top-B per query	0.966	0.955	0.983	0.980	0.994	0.994

Table 7.4: Retained dense attention mass under eviction policies ([SIM]; Qwen3-0.6B, 2048 tokens, four heads per layer).

the findings of Chen et al. [64] and Wang et al. [60] that eviction heuristics are fragile; but the test here is tiny (two layers, four heads, one text), retained mass is only a proxy for task quality, and no multi-turn or retrieval workload is evaluated. The gap between the oracle and the best policy quantifies what query-dependent *selection* (retaining the full cache and choosing per query) can recover that *eviction* cannot, at the price of storing the whole cache.

7.7 How the savings compose

A model's cache bytes per token factorise as $\mathrm{KV} = L \cdot G \cdot 2d \cdot b \cdot \rho$, where ρ is the retained token fraction. Each of the layer count (hybrids), head count (GQA/MLA), bits (quantization) and token fraction (eviction, with sparse selection acting on reads) can be reduced. Published reductions are: GQA vs MHA H/G (e.g. 4× for 32/8); MLA 93.3% in DeepSeek-V2 [13]; hybrid linear 75% in Kimi Linear [4]; KIVI-style 2-bit 2.6× peak memory [35]; TurboQuant $\geq$ 4.5× (paper) [36]; HySparse nearly 10× at its 80B scale [48]; DeepSeek-V4-Pro about 10% (Flash 7%) of V3.2's KV cache at 1M tokens [59]; TriAttention 10.7× at matched accuracy on AIME25 [53]. Naively multiplying these overstates what can be achieved, for three reasons: (i) errors compound (Chapter 4, error propagation), (ii) the compressions act on correlated degrees of freedom (a quantized cache evicted by a score computed on dequantized values), and (iii) the reductions were measured in different regimes (short vs. long context, different benchmarks, different baselines). Whether quantization (TurboQuant-like) *composes* with learned sparse selection (the indexer sees quantized or unquantized keys?) is, to my knowledge, not systematically studied in the papers retrieved for this report, and is listed as an open problem in Chapter 18.

Chapter 8

Trainable Sparse Attention: Method-by-Method

This chapter describes the principal trainable sparse-attention designs: NSA, MoBA, InfLLM-V2, DeepSeek Sparse Attention (DSA), MiniMax Sparse Attention (MSA), Gated Sparse Attention (GSA), and the compressed-sparse attention of DeepSeek-V4. A short treatment of the vendor-claimed SubQ closes the chapter. For each method the section gives the mechanism and cost structure (with derivations tagged [DER] where I have added analysis), the training procedure, the results reported by the authors, and the limitations stated or evident. Section 8.9 tabulates the designs.

8.1 Native Sparse Attention (NSA)

Mechanism. NSA [42] computes, for each query, three attention outputs in parallel and combines them with a learned gate: (i) attention over *compressed* blocks, where each block of $l = 32$ tokens (stride $d = 16$) is compressed by a learnable map into one key–value pair; (ii) attention over *selected* blocks of size $l' = 64$, where $n = 16$ blocks are chosen per query, including one fixed initial block and two local blocks; and (iii) a sliding window of $w = 512$ tokens. The block-selection scores are derived from the attention over the compressed representation and, under GQA, shared across the query heads of a group so that all heads in a group load the same blocks (hardware alignment). The authors implement the kernels in Triton.

Cost structure. The decode step must read the compressed keys and values (one per stride $d = 16$ tokens), the selected blocks, and the window. For cache length s, the paper's Table 4 reports tokens loaded per step of $2048, 2560, 3584, 5632$ at $s = 8192, 16384, 32768, 65536$, i.e. expected speedups of $4\times, 6.4\times, 9.1\times, 11.6\times$ over full attention. These numbers match

$$\text{tokens read}(s) = \frac{s-l}{d} + n\,l' + w \;\approx\; \frac{s}{16} + 1024 + 512, \tag{8.1}$$

[DER](for $s = 65{,}536$: $4094 + 1024 + 512 = 5630 \approx 5632$). The expression exposes NSA's asymptotics: the compressed branch scales as s/d, so NSA's decode cost is still *linear* in context with slope $1/16$ and the speedup saturates at $d = 16\times$ as $s \to \infty$ at fixed n, l', w. At $s = 2^{20}$ the formula gives ≈ 67K tokens read versus 10^6 dense, a speedup of $\approx 15.6\times$.[DER] A consequence is that NSA's scan term is milder than a token-level indexer's (ιT with ι the index bytes) but is not constant.

Training and results. The reported model is a 27B-parameter (3B active) MoE with 30 layers, GQA with 4 groups and 64 heads, pretrained at 8K context on 260–270B tokens

(the paper states both figures), then continued at 32K with YaRN [42]. On nine general benchmarks the average is 0.456 for NSA versus 0.443 for full attention, with NSA higher on 7 of 9; on LongBench the average is 0.469 for NSA versus 0.437 for full attention, 0.423 for exact top-k, 0.392 for Quest, 0.383 for InfLLM and 0.303 for H2O (baselines at a 2560-token budget); needle-in-a-haystack is perfect at 64K; and after SFT on 10B tokens of 32K reasoning traces, AIME accuracy at 8K/16K generation limits is 0.121/0.146 for NSA-R versus 0.046/0.092 for full attention. Speedups on A100 with Triton kernels at 64K are up to 9.0× forward, 6.0× backward and 11.6× decode versus FlashAttention-2.

Interpretation and caveats. The result that NSA *exceeds* full attention on several benchmarks has been read as evidence that sparsity acts as a useful inductive bias; it may also reflect noise in 27B-scale single-run comparisons, small differences on weak benchmarks (the averages differ by 0.013), or the particular training recipe. The 64K evaluation context is short by the standards of later work. Of the designs in this chapter, NSA has the largest number of extra components (compression networks, three branches, gating), and its authors' successor InfLLM-V2 argues this slows convergence [44].

8.2 MoBA: Mixture of Block Attention

Mechanism. MoBA [43] splits the context into n blocks of size $B = N/n$. Each query computes an affinity with each block as the inner product of the query with the mean-pooled keys of the block, keeps the top-k blocks, and attends to all keys in those blocks (the current block is always included, subject to the causal mask). It has the same parameters as full attention, so layers can switch between full and MoBA attention in training or fine-tuning.

Cost structure. The scan reads T/B pooled keys per query (a factor B below token-level scanning; B ranges from 512 to 4096 in the paper's experiments); the attention reads kB tokens. With $B = 4096$, top-12 (their Llama-8B-1M conversion) the budget is 49,152 tokens, i.e. 62.5% sparsity at 128K but 95% at 1M. Block pooling by the mean is a coarse relevance signal: by Proposition 4.4 a block selector can at best match the token oracle, and the mean-key score is not even the optimal block score (Quest uses an upper bound from min/max key statistics).

Reported results. Scaling-law fits at 8K context with block size 512 and top-3 (up to 81.25% sparsity) show validation loss within about 10^{-3} of full attention ($2.625\,C^{-0.063}$ versus $2.622\,C^{-0.063}$). At 32K and up to 95.31% sparsity the trailing-token loss is slightly higher at every scale, with a gap that narrows with compute (fits $1.546\,C^{-0.108}$ versus $1.464\,C^{-0.097}$). Coarser blocks (2 of 8) trailed finer ones by about 10^{-2} loss. A 1.5B model trained on 30B tokens with MoBA for the first 90% and full attention for the last 10% nearly matched full attention in position-wise loss, with no spike at the switch. Llama-8B-1M-MoBA (block 4096, top-12, last 3 of 32 layers full) matches full attention: RULER at 128K 0.7818 vs 0.7849, LongBench at 32K 0.4828 vs 0.4821, with needle-in-a-haystack satisfactory up to 1M, attention-layer prefill up to 6.5× faster at 1M and ≈ 16× less attention compute at 10M tokens at 95.31% sparsity. The authors state that MoBA is deployed for Kimi's long-context requests [43].

Remarks. MoBA's practical significance is the *conversion* result (an existing 8B model converted with limited extra training stays close to full attention) and the layerwise hybrid recommendation (keep the last layers dense). The scaling-law data hint at a real cost: at long context and extreme sparsity the loss is consistently a little higher, a result that fits the tail-mass picture of Chapter 4.

8.3 InfLLM-V2: dense–sparse switchable attention

InfLLM-V2 [44] makes the sparse pattern a *mode* of the same attention parameters: short sequences use dense attention; long sequences switch to block-sparse. It adds no parameters, which keeps the standard recipe of pre-training short and fine-tuning long, and the authors position it against NSA, which they argue adds parameters that slow convergence. They report about 4× speedup over dense attention while retaining 98.1% of long-context-understanding performance and 99.7% of chain-of-thought-reasoning performance, and used it to train and open-source MiniCPM4.1 [44]. (Only the abstract was retrieved; the fine details of the block selection and kernels are not reported here.) Its significance for the period is as the sparse half of MiniCPM-SALA (Chapter 10), where the dense/sparse switchability allows the bulk of the training tokens to be spent at 4K context with sparse attention off [50]. SparDA's evaluation uses MiniCPM4.1-8B and NOSA-8B as its two "sparse-pretrained 8B models" [57]; MiniCPM4.1 is the InfLLM-V2 model.

8.4 DeepSeek Sparse Attention (DSA)

The lightning indexer. DSA [3] adds to each layer a small indexer that scores each earlier token s for query token t:

$$I_{t,s} = \sum_{j=1}^{H_I} w_{t,j}^{I} \ \mathrm{ReLU}\big(q_{t,j}^{I} \cdot k_{s}^{I}\big), \tag{8.2}$$

where the per-head weights $w_{t,j}^{I}$ and the index queries $q_{t,j}^{I}$ are projections of the query token's hidden state and k_s^I is an index key from the key token's hidden state. The ReLU (instead of softmax) is chosen for throughput, and the indexer has few heads and can run in FP8. Attention then runs over the top-k key–value entries by $I_{t,\cdot}$. In the released model $k = 2048$; the configuration file of DeepSeek-V3.2-Exp lists $H_I = 64$ indexer heads of dimension 128. DSA is built on the multi-query mode of MLA, so each latent key–value vector is shared by all 128 query heads (decoding uses this mode; the V3.1-Terminus checkpoint used the multi-head mode for training and prefill).

Why ReLU? A softmax over index heads would require a normalising pass over T scores per head, serialising the computation; ReLU turns the score into a sum of non-negative terms computed independently per key. The learned weights $w_{t,j}^{I}$ restore the ability to emphasise different indexer heads depending on the query token.[DER]

Continued training. DSA was added to an already long-context-extended model (V3.1-Terminus, 128K). Stage 1 (dense warm-up) trains only the indexer with all other parameters frozen, minimising KL divergence to the main attention distribution, for 1000 steps of 16

Benchmark (model-card values)	V3.1-Terminus	V3.2-Exp
MMLU-Pro	85.0	85.0
GPQA-Diamond	80.7	79.9
Humanity's Last Exam	21.7	19.8
LiveCodeBench	74.9	74.1
AIME 2025	88.4	89.3
HMMT 2025	86.1	83.6
Codeforces (rating)	2046	2121
Aider-Polyglot	76.1	74.5
BrowseComp	38.5	40.1
BrowseComp-zh	45.0	47.9
SimpleQA	96.8	97.1
SWE Verified	68.4	67.8
SWE-bench Multilingual	57.8	57.9
Terminal-bench	36.7	37.7

Table 8.1: DeepSeek-V3.1-Terminus versus DeepSeek-V3.2-Exp (the first model with DSA), as listed on the model card (retrieved 27 Oct. 2026; author-reported, single-run).

sequences of 128K tokens (2.1B tokens) at learning rate 10^{-3}. Stage 2 (sparse training) optimises all parameters, selecting 2048 KV tokens per query, for 15,000 steps of 480 sequences of 128K tokens (943.7B tokens) at 7.3×10^{-6}, while the indexer continues to be trained on the selected tokens [3]. Roughly 0.95T tokens are thus spent adapting a model that had been pre-trained on trillions of tokens, a small fraction of pre-training but a substantial compute cost.

Reported quality. DeepSeek reports that V3.2-Exp matches V3.1-Terminus on its benchmarks [45]. The model card table (Table 8.1) shows a mixed picture: increases on AIME 2025 (88.4 → 89.3), Codeforces (2046 → 2121), BrowseComp (38.5 → 40.1), BrowseComp-zh (45.0 → 47.9), SimpleQA, SWE-bench Multilingual and Terminal-bench, and decreases on GPQA-Diamond (80.7 → 79.9), Humanity's Last Exam (21.7 → 19.8), LiveCodeBench (74.9 → 74.1), HMMT 2025 (86.1 → 83.6), Aider-Polyglot (76.1 → 74.5) and SWE Verified (68.4 → 67.8); MMLU-Pro is unchanged at 85.0. The technical report adds that AA-LCR (long-context reasoning) is four points higher in reasoning mode and that Fiction.liveBench is consistently higher, without a numeric table on the retrieved page [3]. Taken together, these are consistent with "no substantial degradation" but not with "no change". Single-run differences of one or two points on most benchmarks are within typical noise for these evaluations, and the pattern of small losses on knowledge-heavy benchmarks (GPQA, HLE) and gains on search/agentic ones is not obviously attributable to attention sparsity.

Reported efficiency and prices. The technical report plots cost per million tokens against position (0–128K) for prefilling and decoding on H800 GPUs at $2 per GPU-hour; the retrieved text gives the axes but no exact values [3]. On the commercial side, DeepSeek announced that API prices fell by more than half at the release of V3.2-Exp [45]; a news report lists the new rates per million tokens as $0.28 for uncached input (from $0.56), $0.028 for cached input (from $0.07) and $0.42 for output (from $1.68) [2ND] [46]. Price is not cost, and the announcement bundles a release strategy with the architecture; nonetheless the price

cut was the first widely visible evidence that a learned sparse attention could be deployed at scale with the economics improving, and is cited as such in the community (Chapter 16).

Subsequent adoption. DSA subsequently appeared in GLM-5, GLM-5.2 (with IndexShare), Tencent Hy4-preview (770B total, 49B active) and a NaiveAI model (Naive-N0.5-Flash, 309B) that pairs DSA with GQA; the last is described as having 48 layers, of which 9 use DSA (4 KV heads, a 16-head indexer, up to 2048 selected tokens) and 39 use 128-token sliding-window attention, with no full-attention layers [2ND] [65].

Cost structure and limitations. By the analysis of Chapter 3, DSA's decode cost per layer is dominated by the index scan beyond $\approx$ 18K tokens (88% of the layer's bytes at 128K under my assumptions), and its prefill indexer FLOPs exceed the sparse-attention FLOPs beyond $\approx$ 70K tokens. The DeepSeek-V3.2 report itself acknowledges that the indexer remains $O(L^2)$ and is merely much cheaper than the main attention [3]. The stored cache is not reduced (MLA aside); all the compression comes from selective reads. These facts motivate the entire next chapter.

8.5 MiniMax Sparse Attention (MSA)

Mechanism. MSA [58] is a block-sparse design for GQA. Each layer has an *Index Branch* with one index query head per GQA group and a single index key head shared across groups; it scores all visible key tokens, *max-pools* the scores within each block, and selects the top-k blocks per group, always including the local block. A *Main Branch* then runs ordinary softmax attention over the tokens of the selected blocks, with each query head keeping its own query projection. Per-query attention cost becomes $O(kB_k)$ with B_k the block size. The index branch is trained by a KL loss to the group-averaged main-branch attention distribution over the selected tokens (with stop-gradient on the target and on the inputs, so the loss only updates the index projections); training begins with an indexer-warm-up stage under full attention. MSA can be trained from scratch or converted "near-losslessly" from a pretrained GQA checkpoint.

Kernels. The authors describe an exp-free top-k selection kernel for the small-k regime, and a "KV-outer" sparse-attention ordering in which each selected KV block gathers its queries to fill tensor-core tiles, with a pre-scheduled chunking and a two-phase combine that avoids atomic updates; in training, the log-sum-exp for the sparse KL loss is computed in the forward pass and persistent load balancing is used in the backward pass.

Reported results. The principal experimental model is a 109B-parameter MoE trained natively multimodal from scratch on a 3T-token budget, with performance on par with GQA on downstream benchmarks. At 1M context the per-token attention compute is reduced 28.4×, and the co-designed kernel gives 14.2× prefill and 7.6× decode wall-clock speedups on H800 GPUs [58]. The released production model MiniMax-M3 is reported by a secondary source to be a 428B-parameter MoE with about 23B active parameters, 1M context, blocks of 128 tokens with 16 selected (a 2048-token budget), pre-trained on more than 100T tokens, and to use uncompressed KV (in contrast to MLA) [2ND] [66]. The same source notes that

the MSA paper validates the attention design on a different 109B model rather than on M3 itself, and lists vendor-reported benchmark results (e.g. SWE-bench Pro 59.0%) next to independent ones (an Artificial Analysis index of 55 for the reasoning variant, tied with GPT-5.5) [2ND] [VENDOR].

The 28.4× figure and the index-scan ceiling. A compute ratio of the form $T/(k+\eta T)$, where k is the sparse-attention budget in tokens and η the index-scan cost per cached token relative to attention cost per token, tends to $1/\eta$ as $T \to \infty$. If $k = 2048$ and the ratio is 28.4 at $T = 10^6$, then $\eta \approx (10^6/28.4 - 2048)/10^6 \approx 0.033$, i.e. the claim would be close to the ceiling $1/\eta \approx 30$.[DER] This is an inference (I did not retrieve the paper's cost model and assume that the figure counts FLOPs in this form), but it illustrates the general point: reported "$N\times$ at 1M" numbers for token-scanning methods are often close to the scan-limited ceiling, which is what shared and hierarchical indexing then try to raise.

Notable reversal. MiniMax's own blog explained in late 2025 why M2 was a full-attention model, citing evaluation gaps and infrastructure immaturity for efficient attention [5]; M3 with MSA reverses that decision [2ND] [66]. It is, in my reading, the clearest example in the period of a lab that tried both hybrid linear attention (MiniMax-01 [24]), then full attention (M2), then sparse attention (M3).

8.6 Gated Sparse Attention (GSA)

GSA [47] (two authors, Jan. 2026) combines a DSA-like indexer with the gating of "gated attention". The indexer replaces the ReLU with a sigmoid factor, $I_{t,s} = \sum_{j=1}^{H_I} \sigma(h_t W_j^{Iw})\, \sigma(q_{t,j}^I \cdot k_s^I + b_j^I)$ with $H_I = 4$, so scores lie in $(0, 4)$ and can be read as a soft count of heads that consider the position relevant. A per-query adaptive budget scales with indexer score variance relative to a moving average, clamped to $[256, 4096]$ around a base of 2048. A value gate and an output gate (per head and query) are added, at about 4.4% extra parameters per layer.

Reported results (1.7B parameters, 24 layers, 400B SlimPajama tokens, 4K training context, YaRN extension to 128K): WikiText-103 perplexity 6.03 (standard), 6.02 (sparse only), 5.76 (gated only), 5.70 (GSA); average downstream accuracy 54.7/55.0/56.4/57.1; RULER at 128K 31.7/36.8/58.8/62.2; attention to the first token 46.7% (standard) versus 3.9% (GSA); loss spikes per 100K steps 12.3 versus 0.3; efficiency at 128K relative to standard: prefill 0.09× time, decode 0.13× time, memory 0.97× [47]. The ablation on the token budget: $k = 512, 1024, 2048, 4096$ gives perplexity $5.89, 5.78, 5.70, 5.69$ and speedups $22\times, 16\times, 12\times, 8\times$.

How to read these numbers. Three observations from the paper's own tables. (a) Most of the quality gain is attributable to gating, not sparsity: the sparse-only perplexity (6.02) is almost identical to the standard (6.03), while gated-only gives 5.76. (b) The large RULER-128K differences should be read in the context of a 4K-trained model extended by YaRN; the gated-only model achieves 58.8 against 31.7 for the standard model, so the dominant effect is extrapolation behaviour, consistent with gating mitigating attention-sink artefacts. (c) The theoretical sections contain loosely argued claims (the retrieved summary of the paper

flagged that the 12.8× speedup does not follow cleanly from the stated complexity, and that the proofs use a 7B-class configuration while the experiments use 1.7B); and the paper's abstract speedup of 12–16× versus "sparse-only baselines" is not what its efficiency table (relative to standard attention) shows. The paper is a small-scale, two-author study and should be treated as suggestive, not as validated at frontier scale. It is included because its core observation, that gating and sparsity address *complementary* weaknesses (the former stability and sinks, the latter cost), reappears in frontier models: DeepSeek-V4 adds learnable sink logits to the softmax denominator [59], and MiniCPM-SALA adds output gates after both attention types [50].

8.7 DeepSeek-V4: compressed sparse attention and heavily compressed attention

DeepSeek-V4 [59] (the arXiv listing carries identifier 2606.19348 and a submission date of 26 April 2026, which are inconsistent with each other; I use the report as listed) combines two attention types in an interleaved stack. *Compressed Sparse Attention* (CSA) compresses every $m = 4$ tokens into one KV entry along the sequence and then applies a DSA-style lightning indexer (64 query heads, head dimension 128) to choose the top-k compressed entries ($k = 512$ for V4-Flash, 1024 for V4-Pro). *Heavily Compressed Attention* (HCA) compresses every $m' = 128$ tokens into one entry and keeps dense attention over them. Both attention types include a sliding window of $n_{\text{win}} = 128$ tokens over uncompressed entries and learnable sink logits added to the softmax denominator, so that attention scores need not sum to one. V4-Flash has 43 layers (the first two pure sliding-window, then interleaved CSA/HCA); V4-Pro has 61 layers (the first two HCA, then interleaved). The models are MoE (V4-Pro 1.6T total / 49B active; V4-Flash 284B / 13B) with 1M-token native context, pre-trained on more than 32T tokens, using Muon and manifold-constrained hyper-connections. KV entries are BF16 for the RoPE dimensions and FP8 for the rest; the indexer runs in FP4.

Reported efficiency. At 1M tokens V4-Pro needs about 27% of the single-token inference FLOPs and 10% of the KV cache of DeepSeek-V3.2 (V4-Flash: about 10% of FLOPs and 7% of KV cache), and the KV cache is about 2% of that of a BF16 GQA-8 baseline with head dimension 128 [59].

Structure of the savings. [DER] Relative to DSA, CSA does two things. It shortens the *indexer's scan* from T to T/m entries (4× fewer at $m = 4$), and it reduces stored cache by the same factor because only compressed entries are stored. The cache per token per layer is then c/m for CSA and c/m' for HCA, where c is the bytes per compressed entry. If CSA and HCA layers are present in roughly equal numbers, the average cache per token per layer is $\frac{1}{2}(c/4 + c/128) = 0.129c$. The reported figure of 2% of a BF16 GQA-8 baseline (4096 bytes per token per layer, i.e. about 82 bytes) would then correspond to $c \approx 635$ bytes, which is compatible with an MLA-like entry of 512 FP8-ish plus 64 RoPE BF16 elements plus overhead. This is a consistency check built on assumptions about the entry width and layer mix that I could not verify from the retrieved text; it is not a replication of the paper's calculation. What it shows is that no exotic mechanism is needed to explain a ∼50×

cache reduction relative to GQA-8: compression by 4–128 combined with FP8 storage and MLA-style width suffices.

Remarks. DeepSeek-V4's design is the clearest signal in the period that the frontier lab that introduced learned token-level sparsity chose to *add compression* in front of it rather than only share the indexer. CSA's top-k of 512–1024 compressed entries covers 2048–4096 tokens, a similar reach to V3.2's 2048, but selects among compressed summaries rather than raw tokens, a form of the hierarchy of Proposition 3.5 applied architecturally. The retrieved text contains no long-context benchmark numbers beyond the statement that V4-Pro-Max performs strongly on synthetic and real 1M-token tasks and surpasses Gemini-3.1-Pro on academic benchmarks (a vendor claim [VENDOR]).

8.8 SubQ: vendor claims about sparse attention at 12M tokens

Subquadratic, a Miami-based startup, emerged from stealth in May 2026 with SubQ, described as a model using dynamic, input-dependent sparse attention to avoid dense attention, with a context of up to 12M tokens [56][VENDOR]. Claims reported by MIT Technology Review include: 56× faster than FlashAttention-based models in a test by Appen that the article describes as a theoretical-speed baseline, not task quality; 89.7% on LiveCodeBench; 98% on needle-in-a-haystack at 6M and 12M tokens (Appen); and a cost comparison in which running the RULER-128 test on Anthropic's Opus 4.6 cost $2,600 and on SubQ $8, a figure from the company. The company has not explained its selection method in detail, access was limited to a waitlist, and critics noted that the model reused weights of an Alibaba Qwen model to bootstrap training, which sits uneasily with a claim of a fully reinvented architecture. A former OpenAI researcher, Will Depue, was quoted as saying sparse attention has been widely tried and is very hard to make work and that the public evidence does not yet support the claim that the quadratic bottleneck is solved; Appen's research director welcomed the results but noted that company-provided claims carry less weight than independent ones. The article describes only the Appen evaluation as independent verification, and the company's CTO acknowledged that releasing third-party benchmarks at launch would have reduced scepticism [56]. A secondary aggregator page headlined a "roughly 1,000×" efficiency claim whose basis I could not trace [2ND] [67].

Assessment. None of the SubQ numbers can be treated as established. Even taken at face value, they are on tasks (needle retrieval at 98%) that do not distinguish sparse methods from each other or from dense models; Chapter 4 shows that a perfect needle result is compatible with large mass loss. The relevant standard is that of the other methods in this chapter: published training recipe, open weights or kernels, and third-party evaluation on multi-key and non-literal retrieval tasks (RULER, NoLiMa-style). The episode is treated in Chapter 16 as a case of the community's reaction to vendor claims.

Method	Granularity	Selection signal	Budget	Scan per decoded token	Training
NSA	compressed blocks + selected 64-token blocks + window 512	attention over compressed blocks, shared across GQA group	16 blocks (1024 tok.) + window	$\approx T/16$ compressed keys	from scratch; extra parameters
MoBA	blocks 512–4096	query · mean-pooled block key	top-3 to top-12 blocks	T/B pooled keys	from scratch or convert; no extra parameters
InfLLM-V2	blocks	(abstract only)	(n/r)	(n/r)	switchable dense/sparse; no extra parameters
DSA	token	lightning indexer (64 heads, ReLU, FP8)	top-2048 tokens	T index keys	indexer warm-up + sparse continued training
MSA	128-token blocks (M3)	index branch: 1 head/GQA group, max-pooled	16 blocks (M3)	T index keys (1 head)	indexer warm-up; from scratch or convert
GSA	token	gated sigmoid indexer (4 heads), adaptive k	256–4096 around 2048	T index keys	two-phase; +4.4% params
CSA (V4)	compressed entries (m=4)	lightning indexer on compressed keys	top-512/1024 entries + window 128	$T/4$ compressed index keys	full pre-training with it
HCA (V4)	compressed entries (m'=128)	none (dense over compressed)	all $T/128$	$T/128$ entries	full pre-training

Table 8.2: Design axes of the trainable sparse and compressed attention methods (n/r = not retrieved). Scan terms are those derived or stated in this chapter.

8.9 Comparison of designs

Three recurring design choices. First, *where the scan cost lives*: NSA and CSA scan compressed summaries ($T/16$, $T/4$); MoBA scans pooled keys (T/B); DSA, MSA and GSA scan every token's index key (T) with different head counts (64, 1 per group, 4). Second, *how the selector is trained*: NSA end-to-end through gating of three branches; DSA, MSA, GSA, SparDA and CLSA by KL distillation from the model's own attention; MoBA by the language-modelling loss alone with a non-learned (mean-key) gate. Third, *whether to start dense*: InfLLM-V2, MSA and DSA are designed so that a dense model can be converted; NSA and MoBA were reported mainly from scratch (with a conversion result for MoBA).

Chapter 9

Shared Routing and Indexer Designs

Chapter 8 ended with the observation that token-level learned selection leaves a linear scan in every layer. This chapter reviews the designs from late 2025 to late 2026 that reduce or hide that scan, organised by what is shared or shrunk: across layers (IndexCache, HySparse, CLSA), across heads (GQA-group sharing, MISA), across scan candidates (HISA, AsyncTLS), and across time (SparDA's forecast, AsyncTLS's pipeline). Chapter 6 supplies the amortisation mathematics and the real-model test of the underlying premise.

9.1 Cross-layer sparse attention (CLSA) on YOCO

Architecture. CLSA [1] (Sun, Zhang, Dong, Wang, Wei; arXiv 4 June 2026) is built on YOCO [14]. The model is split into a self-decoder (16 layers, sliding-window attention with window 512) that produces one shared KV cache, and a cross-decoder (16 layers) whose layers all read that cache. A *single-head* indexer projects the shared hidden states into index queries and keys of dimension d_{idx} (not reported in the retrieved text), computes token-level scores, and selects the top-k tokens *once*; all cross-decoder layers reuse this selection. Each layer retains its own multi-head queries and feed-forward network and attends only to the selected positions. The activated-token budget is at most 2048.

Training. The indexer is trained to match the attention distribution averaged over all layers and heads (a single target, stop-gradient), by KL divergence. Stage 1 trains the indexer alone with the backbone frozen; stage 2 jointly optimises the language-modelling loss plus $\lambda = 0.1$ times the distillation loss. The 4B-parameter model (hidden size 2560, 32 layers, 20 heads, 4 KV heads) is pre-trained dense with 8M tokens per step at 8192 context for 125,000 updates, extended to 32,768 context for 10,000 steps, then adapted to sparse attention for $2500 + 2500$ steps at 32K context (peak LR 3×10^{-4} then 3×10^{-5}).[1] The Transformer baseline uses RoPE with base 500,000; the YOCO variants use RNoPE (RoPE with base 10,000 in sliding-window attention, NoPE in global attention).

Complexities (Table 1 of the paper). With N the sequence length, L layers, D hidden size, W_1 the local window, W_2 the number of selected tokens and η the indexer cost per query–key pair: KV memory $O((N + W_1 L)D)$ for CLSA, the same as dense YOCO; prefill $O(\frac{L}{2} W_1 N D)$; decode $O(\frac{L}{2}(W_1 + W_2)D + \eta N)$, against $O(\frac{L}{2}(N + W_1)D)$ for dense YOCO [1]. The decode expression makes the amortisation explicit: the scan ηN is paid once per token, not once per layer, and the cross-decoder's attention cost $\frac{L}{2} W_2 D$ no longer depends on N.

[1] The pre-training data mixture is not stated in the retrieved text.

Benchmark	Transformer	YOCO dense	YOCO CLSA
ARC-C	0.453	0.461	0.465
BBH	0.420	0.411	0.418
GSM8K	0.434	0.430	0.470
HellaSwag	0.667	0.676	0.674
HumanEval	0.384	0.396	0.396
MMLU	0.527	0.519	0.513
DROP	0.366	0.387	0.391
WinoGrande	0.638	0.630	0.616
RULER avg. 16K	64.4	62.7	62.9
RULER avg. 32K	46.2	52.3	53.1

Table 9.1: CLSA results at 4B parameters (author-reported). At 32K the CLSA gain over dense YOCO is concentrated in multi-needle tasks (MK1 58.4 vs 53.2; MK2 88.8 vs 84.0).

Results. (Author-reported; Tables 2–4 of the paper.)

The attention-coverage study (Table 4) measures the fraction of dense attention mass covered by the selected tokens: at budgets of 512/1024/2048/4096, StarCoder 62.3/76.3/84.1/90.7%, Books 51.4/65.1/76.3/84.1% and ArXiv 55.8/67.7/80.7/89.6%, with loss differences at 2048 tokens relative to dense of -0.0004 (StarCoder), $+0.0054$ (Books) and $+0.0026$ (ArXiv). Validation loss from 8K to 32K on Books, ArXiv and StarCoder nearly overlaps between dense and CLSA models. Efficiency claims: at 128K context, about $7.6\times$ Transformer decode throughput and $17.1\times$ overall throughput; prefill for both YOCO variants is much faster than the Transformer; standalone top-k routing at 128K is expensive, but with amortisation the top-k stage costs about 0.08 ms per layer; per-layer latency at 128K is lowest for CLSA compared with DSA, IndexCache and HySparse. Exact values were in appendices not included in the retrieved text [1].

Reading the results. Several points deserve emphasis.

- The coverage figures (76–84% at 2048) are *lower* than one might expect for a method described as nearly lossless. This is consistent with the tail-mass picture: roughly a fifth of the dense attention mass lies outside the selected set, yet the loss difference is within 0.006 nats. A plausible reading is that the discarded mass is made of near-redundant weights (Section 4.9, (4.6)); the paper does not offer a stronger explanation in the retrieved text.
- The quality comparisons are against a 4B dense YOCO and Transformer trained by the same authors on a common recipe, a good design for internal validity but a single scale and a single seed. Differences of 0.01–0.02 on benchmarks like MMLU or WinoGrande are within typical run-to-run variation at this scale. The GSM8K gain (0.430 to 0.470) is the largest and is not explained.
- The RULER comparison is at 16K and 32K only; the efficiency claims are at 128K. The paper's long-context *quality* evidence beyond the training length (32K) is therefore thin; claims at 128K rest on throughput, not accuracy.
- YOCO changes the memory profile as well as the compute: with a single global cache, stored KV is $L/2$ times smaller than for a Transformer with the same G and d (by

the architecture) before any sparsity. The headline 17.1× overall throughput therefore combines the architecture (YOCO), the sparse attention, and larger feasible batches, and cannot be attributed to the routing alone; the paper compares against dense YOCO in places, which isolates the sparse contribution.

A cost-model check. [DER] Using the dimensions above (32 layers, 4 KV heads, $d = 128$, BF16), a Transformer stores 65,536 B/token (8.0 GiB at 128K) while the YOCO global cache is 2048 B/token (0.25 GiB at 128K); the attention bytes read per decoded token at 128K are 8.59 GB for the Transformer and, for CLSA with 512-token windows in the self-decoder, 2048 tokens read by 16 cross layers, and a 128-d BF16 single-head index key per token, 0.117 GB, a 73× reduction in attention bytes (Table of Chapter 3, `clsa_decode_bytes`). Applying the capacity-limited throughput model (3.7) with an assumed 80 GB / 3.35 TB/s device and 8 GB of BF16 weights gives a feasible batch of 8 (Transformer) versus $\sim$ 226 (CLSA; 0.32 GB per sequence including windows and index keys) and a throughput ratio of $\approx 63\times$ at 128K. This is far above the reported 17.1×, as it should be: the model ignores prefill, the finite batch sizes used in benchmarks, kernel efficiency, attention FLOPs and the top-k kernel. The reported figures are consistent with the model only in the sense that they are below its ceiling; they cannot be used to validate it. The model is an upper bound on what the architecture could deliver, and the relevant question is which losses dominate.

9.2 IndexCache

IndexCache [51] (Bai et al.; arXiv 12 March 2026) starts from the observation that DSA's indexer runs in every layer even though the selected indices of adjacent layers are highly similar. It designates a few *Full* layers that compute their own top-k indices and has the remaining *Shared* layers reuse the indices of the nearest preceding Full layer. Two ways of choosing the configuration are offered. *Training-free*: a greedy search chooses which layers keep their indexers by minimising language-modelling loss on a calibration set, with no weight updates. *Training-aware*: a multi-layer distillation loss trains each retained indexer to match the averaged attention distributions of the layers it serves, so that even simple interleaved patterns approach full-indexer accuracy.

Results. On a 30B DSA model, removing about 75% of indexer computations has negligible quality loss and gives up to 1.82× prefill speedup and 1.48× decode speedup over standard DSA (the latter figure quoted at 200K context in a secondary description) [2ND] [68]; preliminary results on the production-scale GLM-5 are described as consistent [51]. A secondary summary reports adjacent layers' top-k sets overlapping by 70–100% [2ND] and that GLM-5.2 deploys the pattern `full, shared, shared, shared` across a 78-layer stack with a claimed 2.9× reduction in per-token FLOPs at 1M tokens [2ND][VENDOR], introduced during continued mid-training on 128K-token sequences [68]. A vLLM documentation page for "index cache" indicates serving-framework support, though the retrieved listing showed only titles.

Trade-off. A shared layer cannot select positions based on its own current hidden state; the cost is selection mismatch, quantified by the sharing penalty of Chapter 6. IndexCache's

training-aware variant addresses the mismatch by making the retained indexer's target the *average* over its group.

9.3 HySparse and HySparse2: oracle selection and KV sharing

HySparse [48] (Xiaomi MiMo team, arXiv 3 February 2026) interleaves each full-attention layer with several sparse-attention layers, with the sparse layers taking both their token selection *and their KV cache* from the preceding full-attention layer. This addresses two limitations at once: the full layer acts as an *oracle* of token importance (its actual attention scores, rather than an extra proxy indexer), and sharing its KV cache means the sparse layers store nothing. In an 80B MoE with 49 layers of which only 5 are full attention, KV-cache storage falls by nearly 10× [48]; this is the arithmetic consequence of $49/5 = 9.8$ (only layers that own a cache store one).[DER] The authors report consistent outperformance over both full attention and hybrid sliding-window (Hybrid SWA) baselines in all settings (7B dense and 80B MoE); the retrieved abstract gives no benchmark values.

HySparse2. The September 2026 follow-up [61] (arXiv 22 September 2026) has two levels of sharing. At the outer level ("KV Bridging"), a YOCO-style self-decoder with sliding-window attention and a cross-decoder with sparse attention share KV caches, bridging only full-attention layers. At the inner level ("KV Reuse"), selection is *token-level* instead of block-level with a recent-token window always included (budget: the most recent 128 tokens plus the 1024 top-scoring tokens outside the window). Prefill can exit after the self-decoder, skipping the cross-decoder layers for the prompt. On an 80B-A3B MoE with about 100B additional post-training tokens and context extended to 256K, the authors report averaged gains over HySparse of +11.30 points on MRCR-v2 and +19.81 on RULER-v2; at 256K, RULER-v2 is 58.45 versus 32.61 for HySparse. In the 32K ablation at matched budget, token- versus block-level selection raised RULER-v2 from 49.56 to 56.13, two-needle MRCR-v2 from 12.94 to 21.08 and GraphWalks from 29.38 to 34.92, but lowered NoLiMa (38.43 vs 40.27) and BBH (60.70 vs 61.93). The comparison is not a controlled isolation: HySparse2 uses one KV head versus four for the baselines, and Hybrid SWA has nine full-attention layers versus five for each HySparse model.

Significance. The HySparse line is notable for what it removes: it needs no learned indexer at all in the sparse layers (the oracle is the preceding full layer's attention), a different way of avoiding the scan. The cost is that the full layers, which see the whole context densely, remain $O(T)$ per decoded token and become the dominant term as the fraction of full layers f grows; with $f = 5/49 \approx 10\%$ the byte cost per token is roughly 0.1 of dense (before the sparse layers' budget reads). HySparse2's move to token-level selection and the YOCO-style bridge shows that *the HySparse and CLSA ideas converge*: shared KV, shared selection, cross-decoder sparse layers.

Model	Dense	Sparse	InfiniGen	SparDA
MiniCPM4.1-8B (avg.)	63.5	61.4	57.7	61.7
NOSA-8B (avg.)	52.4	49.4	45.6	51.7

Table 9.2: SparDA accuracy averages from Table 1 of the paper (author-reported). "Sparse" is the original sparse model with its own same-layer selector; InfiniGen is a prior prefetching baseline.

9.4 SparDA: Forecast projection and offload-friendly decoupling

SparDA [57] (Fu, Xiao, Dong, Han, Villa; arXiv 3 June 2026; code released as NVlabs/SparDA) addresses two problems: the stored KV cache still grows with T (and offloading to CPU memory creates a PCIe bottleneck), and the selection step is still $O(T^2)$. It adds a fourth per-layer projection, the *Forecast* F_l, alongside Q, K, V. F_l scores the compressed keys of layer $l+1$ and selects that layer's top-k blocks; attention itself still uses the regular query Q_{l+1}. Because the forecast is decoupled from the attention query, it can be computed one layer ahead, allowing the CPU-to-GPU transfer of layer $l+1$'s selected blocks to overlap with layer l's computation, and it can use one head per GQA group (a "compact indexer") instead of one per query head. Layer 0 uses a separate same-layer Forecast. The attended set is the union of initial blocks, local blocks and top-k blocks, with block size 64, 1 initial block and a total top-k budget of 96 blocks; compression windows are kernel 32/stride 16 at inference (training targets use kernel 2/stride 1, max-pooled to the inference grid). The selection cost still scales as $O(T^2)$ over a prefill but is reduced by the group-level indexer.

Training. Only the Forecast projections are trained (33.5M parameters, 0.41% of 8B), on ProLong-64K data at 65,536 tokens for 2000 steps with effective batch 32 on 32 H100s (about 48 hours for MiniCPM, 24 for NOSA), using a KL divergence between target and predicted block distributions restricted to the target's top-k set, with the remaining mass lumped into one bucket.

Results (author-reported). On RULER at 32K/64K/96K/128K SparDA beats the original selector by $+1.5, +0.5, +2.1, +1.1$ (MiniCPM) and $+1.7, +3.9, +4.1, +4.3$ (NOSA). Efficiency on H100: prefill at 128K, batch 4, 17,087.6 tokens/s versus 13,661.8 for sparse with offload (1.25×) and 8085.8 for dense (MiniCPM); decode at 128K in the offload regime for MiniCPM: 240.2 vs 167.8 tokens/s (batch 4), 471.2 vs 279.5 (batch 8, 1.69×), 705.3 vs 447.9 (batch 16); NOSA's peak decode speedup is 1.40×. Relative to non-offloaded baselines it reports up to 5.28× over sparse and 9.21× over dense on MiniCPM, which come from larger feasible batches since the non-offloaded baselines run out of memory beyond batch 16 at long context. Block-selection time falls up to 2.50× in prefill at 128K and by more than 2× in decode. In the prefetch ablation (MiniCPM, 128K, batch 64), plain sparse reaches 788.9 tok/s, SparDA without prefetch 696.9, and full SparDA 1000.1: the decoupled forecast alone does not help (it is slightly slower), and the benefit appears only when prefetch overlap is turned on [57].

Limitations stated by the authors. SparDA inherits the accuracy limits of its sparse backbone, does not change the sparse pattern, and was tested only on 8B block-sparse models; extension to token-level DSA-style models is future work [57]. A subtle point is that the forecast uses layer l's hidden state to predict layer $l+1$'s selection; when the selection of $l+1$ depends strongly on information added in layer l's output, forecast accuracy may drop, which is the same penalty as sharing (Chapter 6) in a different guise.

Pipeline condition. [DER] Let t_c be the per-layer compute time for a decode step, S the bytes that must be transferred for the layer's newly needed blocks, and $\mathrm{BW}_{\mathrm{pcie}}$ the host-to-device bandwidth. Without lookahead the layer time is $t_c + S/\mathrm{BW}_{\mathrm{pcie}}$ (selection must finish before transfer starts, and transfer before compute); with a one-layer lookahead the transfer overlaps the previous layer's compute and the layer time becomes $\max(t_c, S/\mathrm{BW}_{\mathrm{pcie}})$. The overlap is complete when $S \le \mathrm{BW}_{\mathrm{pcie}} t_c$. If a fraction ρ of the selected blocks is already resident on the GPU (temporal locality), $S = (1-\rho)S_{\mathrm{sel}}$ and the condition becomes $\rho \ge 1 - \mathrm{BW}_{\mathrm{pcie}} t_c / S_{\mathrm{sel}}$. SparDA's prefetch ablation (a 1.44× gain from 696.9 to 1000.1 at batch 64) is a measurement of how far the real system is from the ideal max versus sum.

9.5 HISA and MISA: shrinking the token-level scan without retraining

HISA [52] (Xu et al., arXiv 30 March 2026, v4 September 2026) replaces the flat scan of DSA's indexer by a two-stage hierarchy. It first filters out irrelevant blocks using pooled block representations, then applies the *original* token-level indexer only within the surviving candidate blocks. It produces the same token-level sparse pattern for the downstream operator and needs no additional training. Applied directly to DeepSeek-V3.2 and GLM-5 without fine-tuning, it stays close to DSA's quality on needle-in-a-haystack and LongBench and clearly outperforms block-sparse baselines; the abstract reports a kernel-level speedup "up to" a figure at 64K context, whose value was missing from the retrieved text.

MISA [55] (Zhou et al., arXiv 8 May 2026) treats the indexer's query heads as a pool of mixture-of-experts. A lightweight router uses block-level statistics to activate only a few heads per query, so only those heads perform the costly token scoring; a hierarchical variant uses this routed pass to build a larger candidate set and re-ranks it with the original DSA indexer. On DeepSeek-V3.2 (64 indexer query heads) MISA activates 8; the paper reports matching the dense DSA indexer on LongBench for DeepSeek-V3.2 and GLM-5 using 8× and 4× fewer indexer heads respectively, recovering more than 92% of the tokens the DSA indexer selects per layer, fully green needle-in-a-haystack heatmaps up to 128K, and about 3.82× faster than DSA's indexer kernel on a single H200 [55].

Cost analysis. [DER] HISA's scan is $T/B + \kappa B$ candidate evaluations (Proposition 3.5); for $T = 128\mathrm{K}$, $B = 64$ and $\kappa = 64$ this is $2048 + 4096 = 6144$, a 21× reduction relative to $T = 131{,}072$ if the evaluation cost per candidate were equal (it is not: block summaries are cheaper to compare than full index keys, and the final stage still needs the index keys of the candidate tokens, accessed non-contiguously). MISA's reduction is at most $H_I/r = 8$ in the scoring FLOPs plus a router cost; the measured 3.82× kernel speedup is below the ideal

Benchmark	Model (context)	Full	Quest	Double Sparsity	AsyncTLS
LongBench avg.	Qwen3-8B (32K)	50.14	47.49	50.10	49.90
	Qwen3-14B (32K)	52.16	49.33	51.10	51.80
	GLM-4.7-Flash (128K)	50.83	46.75	50.73	50.58
RULER avg.	Qwen3-8B	90.45	38.92	90.44	90.54
	Qwen3-14B	86.24	37.89	77.20	87.21
	GLM-4.7-Flash (6 tasks)	85.73	53.32	82.38	81.50

Table 9.3: AsyncTLS accuracy (author-reported). Each query retrieves 128 blocks (8192 tokens); token budgets 512 (RULER, 32K), 1024 (LongBench). Four RULER tasks were excluded for GLM-4.7-Flash because of abnormal metrics under the default harness.

8×, consistent with router and top-k overheads and the fact that index-key reads (one head, shared) are not reduced by head activation. Because MISA reduces the *compute* of scoring (heads) but not the *bytes* of index-key reads (the shared key is read once regardless of the number of active heads), its effect is limited in the bandwidth-bound decode regime, where HISA's candidate reduction (which does cut index-key reads) matters more.

9.6 AsyncTLS: two-level sparsity with asynchronous offload

AsyncTLS [54] (Hu et al.; arXiv 9 April 2026) combines coarse block filtering with fine token selection and an asynchronous offloading engine. The KV cache is split into 64-token blocks; each block is scored with Quest-style min/max key summaries, reformulated as a matrix multiplication so that it runs on Tensor Cores for shared-KV architectures such as MLA, and the top-k_b blocks are kept. Within kept blocks a Double Sparsity-style step selects informative channels (via calibration), projects the keys onto these channels in INT4, approximates and head-averages the attention scores, and picks the top-k_t tokens, which are then attended with full-dimension KV. In the pipeline, token selection at step t uses the block set from step $t-1$; block selection for step $t+1$ runs in parallel with attention and the FFN, and missing blocks are prefetched from host memory with incremental transfer relative to a GPU-resident cache, exploiting temporal locality (selection changes slowly between steps).

Kernel latency is 1.7–6.2× faster than full attention for GQA and 3.3–10.0× for MLA; versus Double Sparsity 1.2–4.0× (GQA) and 1.9–4.0× (MLA); AsyncTLS reaches about 54% (GQA) and 68% (MLA) of Quest's speed. End-to-end latency without offloading at 32K–96K is about 2.3× (Qwen3-8B) and 2.7× (GLM-4.7-Flash) faster than full attention; with offloading at 96K the throughput is 1.84× and 4.70× higher, because AsyncTLS runs batch size 6 while full attention is limited to batch size 1 by memory [54].

Caveats. There is no dedicated ablation of the two levels or of the offloading pipeline; no limitations section; the headline speedup ranges are inconsistent between abstract (1.2–10.0× operator, 1.3–4.7× throughput), introduction (1.8–5.0× and 1.3–1.8×) and experiments; the offload results cover two models at one sequence length; and the incremental-transfer savings rest on an assumed low rate of block changes that is not directly measured. Contexts only reach 96K (128K for GLM-4.7-Flash accuracy), short of the 1M regime motivating most of the field. The most robust finding is the accuracy table: the two-level method stays within

about 1 point of full attention on LongBench and recovers RULER where Quest fails (38.9 vs 90.5 for Qwen3-8B), showing that block summaries alone are insufficient at a 512-token budget while token-level refinement fixes it.

9.7 Token Sparse Attention (TSA)

TSA [49] (arXiv 3 February 2026, v3 29 May 2026) is a training-free prefill method that operates per head: it selects a token subset, gathers the corresponding rows of Q, K, V into smaller dense tensors, runs standard attention (e.g. FlashAttention) on them, then scatters the output back to the full sequence length with zeros for unselected tokens; the residual connection carries unselected tokens forward so they can be re-selected in later layers and heads. Per-layer budgets come from a *dynamic token coverage* threshold τ on the layer-level importance distribution built from the attention of a small set of recent queries, and layers eligible for sparsification are chosen by the inter-layer representation drift $\delta \leq 0.5$, computed once per model. On LLaMA-3.1-8B-Instruct and Mistral-Nemo-12B-Instruct on one A100, RULER averages are essentially unchanged (87.01 $\rightarrow$ 87.02 for LLaMA with FlashAttention; 67.60 $\rightarrow$ 67.37 for Mistral) while 128K attention speed rises $\times$1.36 and $\times$1.22 respectively; combined with other sparse kernels the speedup reaches $\times$2.76 (FlexPrefill) and, in the appendix, $\times$3.49 (X-Attention). The abstract's $\times$3.23 figure is above the main-table maximum, and some individual 128K scores drop by more than 1 point (MInference on LLaMA: 73.63 $\rightarrow$ 72.18); decoding remains dense, and evaluation covers text-only prefill on two models. The method is notable for what it composes with: it is orthogonal to the pattern-based prefill sparsifiers and, being training-free, offers a baseline against which trainable designs should be judged. [49]

9.8 Synthesis: what is shared, and what is lost

The designs fall along a spectrum of how aggressively they pre-commit selection: from the maximal sharing of HySparse and CLSA (a layer's selection is decided elsewhere) through IndexCache (shared with a small group of neighbours) to HISA and MISA (the per-layer selection is preserved but made cheaper). The accompanying quality evidence mirrors this: methods that commit most to sharing (HySparse, CLSA) require training from the start or substantial adaptation, while those that preserve the selection (HISA, MISA, TSA) are training-free. Whether the pre-commitment hurts is an empirical question that depends on the cross-layer similarity of the needed information, which Chapter 6 measures.

Design	What is shared / shrunk	Factor (idealised)	Cost to quality
CLSA (YOCO)	KV cache and indexer across the cross-decoder	$L/2$ on scan; $L/2$ on stored KV	single shared target for all layers/heads
IndexCache	indices from Full to Shared layers	m (e.g. $4 \Rightarrow 75\%$ removed)	shared layers cannot adapt selection
HySparse(2)	selection and KV from last full layer	KV L/L_{full}; no indexer	few dense layers remain $O(T)$
SparDA	selection forecast one layer ahead; one forecast head per GQA group	overlap of transfer; H/G on heads	forecast error; needs backbone training of 0.4% params
HISA	candidates via block pooling	$\approx T/(T/B + \kappa B)$	candidate recall; training-free
MISA	active indexer heads r of H_I	$\leq H_I/r$ in FLOPs	router error; no byte reduction on shared key
AsyncTLS	block filter, then token top-k; temporal locality of selection	operator 1.2–10× (reported)	pipeline relies on slow selection drift
TSA	per-layer coverage threshold; dense per-head compaction	$\leq$ 3.5× prefill (reported)	prefill only; recency of scoring queries

Table 9.4: Sharing and shrinking strategies for the selector, with idealised factor and the quality cost each introduces.

Chapter 10

Hybrid Linear/Sparse Models

Hybrid models interleave exact attention layers (full or sparse) with recurrent or linear-attention layers. This chapter reviews the models and studies of the 2024–2026 period most relevant to sparse–linear hybrids: MiniMax-01 and the MiniMax M2 post-mortem, Kimi Linear, Qwen3-Next, Jamba, the systematic hybrid study, and in detail MiniCPM-SALA, the one model in the retrieved literature that hybridises *sparse* (InfLLM-V2) and linear (Lightning) attention. Mathematical background is in Chapter 5.

10.1 Lightning Attention and MiniMax-01

Lightning Attention-2 [23] is an IO-aware block algorithm for causal linear attention that claims consistent training and inference speed regardless of sequence length. MiniMax-01 [24] scaled it: a 456B-parameter MoE with 45.9B active parameters, 32 experts, lightning attention combined with softmax-attention layers (the ratio was not in the retrieved abstract), a context of up to 1M tokens in training and extrapolation to 4M at inference, and a claimed 20–32× longer context than GPT-4o and Claude-3.5-Sonnet at comparable performance [VENDOR]. The retrieved abstract does not give the softmax-to-linear ratio.

10.2 The MiniMax M2 decision: why full attention

In late 2025 MiniMax published a post explaining why its M2 model, released after the Lightning Attention MiniMax-01, used full attention [5]. The argument is not that efficient attention is wrong, but that in production it was *not yet reliable enough*, for evaluation and infrastructure reasons.

Evaluation gaps. Hybrid lightning-attention models looked as good as full attention on standard benchmarks (MMLU, BBH, MATH, LongBench), all now saturated. At larger scale, weaknesses appeared on complex multi-hop reasoning; the team built proxy metrics and tuned until the hybrid matched full attention on them, but did not confirm that the proxies predict downstream performance at larger scale. Reliable early signals are scarce for harder tasks, the compute for statistically meaningful results grows quickly, and issues such as RL-training precision problems appear only at scale; changing data mixes make experimental conclusions unstable.

Infrastructure gaps. Linear-attention architectures are often memory-bound even in training, wasting GPU compute without heavy IO optimisation; they are more sensitive to numerical precision than full attention (making low-precision state storage a problem); and prefix caching and speculative decoding need new solutions for recurrent backbones.

A negative result on sliding-window hybrids. An early attempt to convert a model to a hybrid with sliding-window attention (mixing layers both between and within layers) degraded as context length grew, which was unacceptable for agentic use. The post's analysis suggests that global attention patterns such as retrieval and induction heads form early in pretraining and are hard to change afterward, and that identifying and preserving them as full attention is difficult.

Outlook in the post. Efficient attention should become more advantageous as context grows and GPU compute growth slows; the team points to better long-context data, evaluation and infrastructure as preparation. The post does not provide benchmark numbers for its own models. Within months MiniMax published MSA and released M3 with sparse attention (Chapter 8), and independent commentary summarised the M2 post as a reason that "full attention still wins" in practice [69] (title only retrieved [2ND]).

10.3 Kimi Linear

Kimi Linear [4] (Kimi Team, arXiv 30 October 2025) introduces Kimi Delta Attention (KDA), which extends Gated DeltaNet with finer-grained gating to make better use of limited recurrent memory, and a chunkwise algorithm using a specialised diagonal-plus-low-rank transition structure for hardware efficiency. The model is a layerwise hybrid of KDA and Multi-Head Latent Attention (MLA), intended as a drop-in replacement for full attention, at 3B activated and 48B total parameters. Reported claims: under an identical training recipe, it outperforms full MLA "with a sizeable margin across all evaluated tasks" in short-context, long-context and RL settings (and is described as the first hybrid to do so under fair comparison); KV-cache usage is reduced by up to 75%; decoding throughput is up to 6× at 1M tokens. The KDA kernel, vLLM implementations and checkpoints were released. (Only the abstract was retrieved, so the KDA:MLA layer ratio is not confirmed here.) A 75% KV reduction is what a 3:1 ratio of KDA to MLA layers would give, since only the MLA layers (1 in 4) store a growing cache; this is an arithmetic consistency check on the claim, not a statement of the actual ratio.[DER]

10.4 Qwen3-Next

Alibaba's Qwen3-Next-80B-A3B combines Gated DeltaNet layers with Gated Attention layers in a very sparse MoE (512 experts, 10 routed per token plus one shared; 80B total, about 3B active), with multi-token prediction for speculative decoding. The vendor claims about 10× cheaper training and about 10× faster inference than Qwen3-32B, with the largest gains at 32K tokens and beyond [VENDOR][2ND] [70]. The hybrid layer ratio was not available on the retrieved page. Qwen3-Next serves in the retrieved literature as a reference: MiniCPM-SALA compares to Qwen3-Next's published RULER numbers at ultra-long lengths (Table 10.4).

10.5 Jamba and earlier hybrids

Jamba [25] interleaves Transformer and Mamba layers (plus MoE) and reports strong results up to 256K tokens with the configuration fitting on a single 80 GB GPU. It is notable

as an early production-scale hybrid that retained some full-attention layers because pure SSMs underperform on retrieval and in-context learning tasks, consistent with the theoretical results of [26]. Hybrids were thus already in use before the period under review; what changed in 2025–2026 was (a) gated delta rules replacing Mamba-style SSMs as the dominant linear component, and (b) the full-attention component itself becoming sparse.

10.6 What the systematic study says about ratios

The 72-model study of Wang et al. [27] (36 models at 340M parameters/20B tokens and 36 at 1.3B/100B tokens, six linear-attention variants, five ratios) is the most controlled evidence on hybrid design in the retrieved material. Its findings: the best standalone linear models do not necessarily make the best hybrids; language modelling is stable across ratios; recall improves with more full-attention layers, especially below 3:1 linear-to-full; selective gating, hierarchical recurrence and controlled forgetting matter; and HGRN-2 or Gated DeltaNet at a ratio between 3:1 and 6:1 attains Transformer-level recall. These are small-scale results (at most 1.3B parameters); the extent to which they transfer to 9B–80B+ models with sparse rather than full attention is open, though MiniCPM-SALA's 1:3 sparse-to-linear choice and the 3:1 ratio that Kimi Linear's reported KV saving would imply fall within or near the recommended range.

10.7 MiniCPM-SALA in detail

MiniCPM-SALA [50] (MiniCPM Team, arXiv 12 February 2026; v2 28 February) is a 9B-parameter hybrid in which InfLLM-V2 sparse-attention layers and Lightning-Attention linear layers are mixed 1:3 (25% sparse, 75% linear).

Design. Layer positions are chosen by the layer-selection algorithm of HypeNet rather than uniform interleaving; the first and last layers remain unconverted. The FFN is retained after every attention block. InfLLM-V2 adds no parameters and switches between dense and sparse modes, easing conversion; Lightning Attention is chosen because its similarity to softmax attention helps preserve pre-trained knowledge. QK-normalisation is applied in both layer types, and output gates are added after both. The *hybrid positional encoding* (HyPE) applies RoPE in the linear layers and removes it (NoPE) from the sparse layers, which the authors say reduces long-distance decay and, they argue, helps length extrapolation.

Training pipeline (Table 10.1). Starting from an intermediate MiniCPM-4.0 checkpoint trained on 7T tokens, the pipeline converts the architecture with the HALO conversion (training only the linear layers, sparse off, 1.3B tokens at 0.5K), then continual stable training (314.6B tokens, 4K), short decay (1006.6B tokens, 4K), long decay with sparse attention on (32K, 160K, 520K contexts; 102.2B, 62.9B, 50.6B tokens), and SFT at 64K (204.5B tokens) and 140K (213.3B tokens). The total is about 2T tokens (the stage totals sum to about 1.96T), roughly 25% of the 8T used to train MiniCPM-4.0 from scratch, which the authors present as a roughly 75% training-cost saving.

Stage	Trainable	Sparse attn	Seq. length	Tokens
Architecture conversion (HALO)	linear attention only	off	0.5K	1.3B
Continual stable training	all	off	4K	314.6B
Short-decay training	all	off	4K	1006.6B
Long-decay training	all	on	32K / 160K / 520K	102.2B / 62.9B / 50.6B
SFT	all	on	64K / 140K	204.5B / 213.3B

Table 10.1: MiniCPM-SALA training pipeline (author-reported; learning rates and batch sizes omitted).

Benchmark	Qwen3-8B	Nemotron-Nano-v2-9B	MiniCPM-4.1-8B	Ministral-3-R-8B	Falcon-H1R-7B	SALA-9B
CMMLU	81.68	61.59	84.72	71.74	63.55	81.55
MMLU-Pro	73.26	71.79	72.70	68.75	70.98	67.04
HumanEval	93.90	93.90	91.46	96.95	96.34	95.12
LCB-v5	56.89	68.26	56.89	65.87	67.66	60.48
LCB-v6	48.57	60.00	51.43	53.71	57.71	52.00
MBPP	81.32	93.39	91.05	n/r	91.05	89.11
AIME24	73.33	71.67	80.83	81.46	86.67	83.75
AIME25	66.67	56.67	72.08	75.00	81.04	78.33
BBH	74.17	74.28	82.68	64.39	63.17	81.55
IFEval	84.66	86.69	77.45	70.06	86.32	76.34
Average	73.45	73.82	76.13	74.21	76.45	76.53

Table 10.2: MiniCPM-SALA standard evaluation (author-reported; n/r = value unreadable in the retrieved text).

Standard benchmarks (Table 10.2; author-reported). The 9B hybrid's average (76.53) is slightly above Qwen3-8B's (73.45), MiniCPM-4.1-8B (76.13) and Falcon-H1R-7B (76.45), with notable gains over Qwen3-8B on AIME24/25 and BBH, and deficits on MMLU-Pro (67.04 vs 73.26) and IFEval (76.34 vs 84.66). The averages are within a few points of each other, and the models differ in training data and recipe; the table shows no systematic loss from the hybrid architecture, but also no clean attribution of differences to it.

Long-context benchmarks (Table 10.3). SALA leads strongly on RULER (92.65 at 64K, 89.37 at 128K versus 80.53/71.74 for Qwen3-8B and 88.77/68.01 for Nemotron-Nano-v2-9B) and NoLiMa (54.54/42.95/23.86 at 32K/64K/128K versus Qwen3-8B 43.40/23.35/11.25), but *not* on MRCR (multi-round coreference), where Ministral-3-R-8B is clearly ahead (for example 64K-2N: 44.02 vs SALA 29.77; 128K-2N: 50.30 vs 28.62) and Qwen3-8B is comparable or better in several cells (128K-8N: 12.15 vs 10.12). The average over all eleven long-context columns is 38.97 for SALA versus 32.02 for Qwen3-8B. Two caveats the reader should hold: (a) Qwen3-8B's configuration file lists a native maximum position of 40,960 tokens, so its 64K/128K numbers rely on context extension, whereas SALA was trained to 520K; (b) the baselines' numbers are measured by the authors, with their harness. The NoLiMa result is of particular interest because it is a non-literal retrieval test on which sparse-plus-linear attention could plausibly fail, and does not.

Ultra-long RULER (Table 10.4). SALA, trained to 520K, scores 89.4/87.1/86.3/81.6 at 128K/512K/1000K/2048K without extrapolation techniques such as YaRN, attributed partly to the NoPE setup of the sparse layers. The Qwen3 figures in the table are cited from the Qwen3-Next documentation, i.e. from a different harness and not a controlled comparison.

Benchmark	Qwen3-8B	Nemotron-Nano-v2-9B	Ministral-3-R-8B	Falcon-H1R-7B	SALA-9B
RULER 64K	80.53	88.77	70.66	56.50	92.65
RULER 128K	71.74	68.01	45.09	36.33	89.37
MRCR 64K-2N	29.20	20.91	44.02	13.18	29.77
MRCR 64K-4N	21.56	13.69	35.80	9.06	20.57
MRCR 64K-8N	17.82	13.24	17.23	6.93	16.56
MRCR 128K-2N	26.50	14.61	50.30	9.17	28.62
MRCR 128K-4N	14.75	12.20	22.66	8.22	19.62
MRCR 128K-8N	12.15	7.55	14.47	7.54	10.12
NoLiMa 32K	43.40	19.69	3.78	14.89	54.54
NoLiMa 64K	23.35	11.82	2.48	9.87	42.95
NoLiMa 128K	11.25	5.80	3.48	4.73	23.86
Average	32.02	25.12	28.18	16.04	38.97

Table 10.3: MiniCPM-SALA long-context evaluation (author-reported).

Model	128K	512K	1000K	2048K
Qwen3-30B-A3B-Instruct-2507	89.1	78.4	72.8	–
Qwen3-235B-A22B-Instruct-2507	93.9	90.9	84.5	–
Qwen3-Next-80B-A3B-Instruct	96.0	86.9	80.3	–
MiniCPM-SALA (9B)	89.4	87.1	86.3	81.6

Table 10.4: Ultra-long RULER as reported in the MiniCPM-SALA paper (Qwen3 values cited by the authors from the Qwen3-Next documentation).

Inference speed and memory. Compared with Qwen3-8B on one A6000D (96 GB) with 1K tokens generated after prefill: at 256K, TTFT falls from 180.8 s to 51.6 s (non-quantised), which the abstract presents as up to 3.5× inference speed; at 1024K SALA's TTFT is 250.3 s non-quantised and 256.9 s with GPTQ INT4, while Qwen3-8B runs out of memory at 512K and 1024K. On an RTX 5090 (32 GB), Qwen3-8B runs out of memory at 128K (non-quantised) and 256K (quantised), while SALA runs to 1024K. From these numbers I derive prefill rates of 262,144/51.6 ≈ 5.1K tokens/s at 256K (versus 1.45K for Qwen3-8B) and 1,048,576/250.3 ≈ 4.2K tokens/s at 1M for SALA.[DER] The 4× increase in length raises TTFT by 4.85×, only mildly super-linear, whereas a quadratic prefill would raise it by up to 16×; the mild super-linearity is consistent with the sparse layers' scan growing with T.

Assessment. MiniCPM-SALA is the strongest retrieved evidence that a hybrid with only 25% exact-attention layers (themselves sparse) retains long-context retrieval ability to over 1M tokens at 9B scale; its limits are that all comparisons are against baselines the authors ran, that MRCR shows non-uniform behaviour, that 9B is below frontier size, and that the training investment (2T tokens) is not small. The ratio of benefit to training cost depends on the comparison: the 75% saving is relative to pre-training from scratch, not to the (much smaller) cost of context extension of a dense model.

10.8 Naive-N0.5-Flash and other pure-sparse hybrids

A NaiveAI model described in a secondary source uses a 9:39 mix of DSA layers (4 KV heads, 16-head indexer) and 128-token sliding-window layers with no full-attention layers at 309B total parameters [2ND] [65]. HySparse (5 full of 49 layers) is the other extreme of

Model	Cheap component	Exact component	Reported headline (author/vendor)
MiniMax-01	Lightning attention	softmax (ratio n/r)	456B/45.9B active; 1M train, 4M inference ctx
Kimi Linear	KDA (gated delta)	MLA (full)	48B/3B active; −75% KV; 6× decode at 1M
Qwen3-Next-80B-A3B	Gated DeltaNet	Gated attention (full)	80B/3B active; ∼10× inference vs Qwen3-32B at 32K+
MiniCPM-SALA	Lightning attention (75%)	InfLLM-V2 sparse (25%)	9B; RULER 89.4 @128K, 81.6 @2048K; 3.5× @256K
HySparse	sparse layers sharing KV	5 full layers of 49	80B MoE; ∼ 10× KV reduction
Jamba	Mamba	softmax	single 80GB GPU, 256K
DeepSeek-V4	compressed (CSA/HCA) + window	none (all compressed)	KV ∼ 10% of V3.2 @1M (Pro)

Table 10.5: Hybrid and compressed-attention models in the retrieved literature, with the headline claim of each source.

sparse hybrids with a handful of dense layers. These show that the "hybrid" space includes sparse–local combinations without any linear layers; the choice between linear and sliding-window layers as the cheap component is a design variable, one whose merits are not resolved in the retrieved evidence (MiniMax's sliding-window experience was negative [5]; Naive's architecture is a counter-example in the sense that it exists, but quality evidence was not retrieved).

10.9 Comparison of hybrid designs

Chapter 11

KV-Cache Eviction and Compression Methods

Chapter 7 developed the error analysis of quantization and eviction. This chapter reviews the methods themselves, from the classical eviction lineage (StreamingLLM, H2O, SnapKV, Quest, DuoAttention) to the 2026 reasoning-era methods (TriAttention, Random Attention) and the critical evaluations. The emphasis is on what each reports, on what workloads, and how comparable the evidence is.

11.1 A taxonomy

Table 11.1 classifies the methods by when the decision is made (prefill or decode), what is decided (token, block, head) and whether the decision is reversible.

The reversibility distinction aligns with Chapter 3: eviction reduces *stored* bytes (and hence read bytes) and is irreversible; selection reduces *read* bytes and is reversible but leaves storage unchanged unless combined with offloading. Proposition 7.3 says that irreversibility carries unavoidable worst-case risk; the empirical question is how often real workloads hit it.

11.2 The classical lineage

StreamingLLM. The key empirical observation is the *attention sink*: the first few tokens receive a disproportionate share of attention regardless of their content. Keeping the first few tokens plus a recent window stabilises perplexity for arbitrarily long streams; the paper reports operation on contexts up to 4M tokens with Llama-2, MPT, Falcon and Pythia and up to 22.2× speedup over sliding-window recomputation [28]. It does not extend the model's *memory*: tokens outside sinks and window are forgotten. The real-tensor measurements of Section 7.6 (and Chapter 6) quantify how much attention mass the first token and the recent window carry in a real model.

H2O. Keeps a budget of recent tokens plus "heavy hitters" with the largest accumulated attention. With 20% heavy hitters the authors report up to 29× throughput improvement over DeepSpeed Zero-Inference and Hugging Face Accelerate and up to 3× over FlexGen on OPT-6.7B and OPT-30B, and up to 1.9× lower latency at equal batch size [29]. The accumulated-score bias discussed in Section 7.5.1 applies.

SnapKV. At the end of the prompt, an observation window of the last few tokens votes (through its attention) for which earlier keys are important; clustered selections per head are kept. It reports a 3.6× generation speedup and 8.2× memory-efficiency improvement

Method	Phase	Unit	Signal	Reversible?
StreamingLLM [28]	decode	token	position (sinks + window)	no
H2O [29]	decode	token	accumulated attention	no
SnapKV [30]	prefill	token (per head, clustered)	observation-window attention	no
Quest [31]	decode	page	query · min/-max key bound	yes (cache kept)
DuoAttention [32]	both	head type	learned gate per head	partly
TriAttention [53]	decode (reasoning)	token	pre-RoPE trigonometric + norm score	no
Random Attention [60]	decode (reasoning)	token	none (random; prompt protected)	no
DSA/MSA/NSA	decode	token/block	learned indexer	yes (cache kept)

Table 11.1: Taxonomy of eviction and selection methods. "Reversible" means the decision can be revised at a later step because the cache is retained (possibly offloaded).

at 16K input tokens, handling up to 380K tokens on one A100-80GB, with performance matching baselines on 16 long-sequence datasets [30]. Its weakness is exposed by later work: its selection is made from the end of the prompt and therefore depends on the last question; when the instructions come first it can lose them (the "prompt fragility" reported by [60] and the instruction-position dependence reported by [64]).

Quest. Stores per-page min/max of keys so that an upper bound of the query–page logit can be computed from the query alone; loads only the top pages. It reports up to 2.23× attention speedup and 7.03× latency reduction at 128K–1M contexts [31]. The upper bound is loose: min/max statistics over 16 or 64 tokens are dominated by outliers, so at small budgets the selected pages are poorly ranked, as AsyncTLS's RULER result for Quest (38.9 for Qwen3-8B at a 512-token budget, against 90.5 for full attention) shows [54]. NSA's LongBench table likewise places Quest (0.392) below full attention (0.437) and below NSA (0.469) at the same 2560-token budget [42].

DuoAttention. Distinguishes "retrieval heads" from "streaming heads" with a learned per-head gate; only retrieval heads keep the full cache, streaming heads keep sinks and a window. It reports memory reductions up to 2.55× (MHA) and 1.67× (GQA), decoding speedup up to 2.18× and 1.50×, prefilling speedups up to 1.73× and 1.63×, and with quantization decodes Llama-3-8B at 3.3M context on one A100 [32]. It is the head-level counterpart of layer-level hybrids and rests on the finding that a small set of retrieval heads performs the copy-from-context function [33]. Because the fraction of retrieval heads is roughly the fraction of dense attention a model needs, DuoAttention provides an empirical bound on how much hybridisation a pretrained model tolerates.

	AIME24				AIME25			
Method	Qwen3	DS-Llama	DS-Qwen	GPT-OSS	Qwen3	DS-Llama	DS-Qwen	GPT-OSS
Full Attention	57.1	50.4	43.8	69.2	40.8	31.4	34.2	60.0
SnapKV	34.6	5.0	34.6	48.3	20.0	6.7	25.0	36.7
R-KV	25.4	25.8	34.6	49.6	17.5	11.2	23.3	39.2
TriAttention	42.1	33.8	42.5	59.2	32.9	19.6	30.0	49.2

Table 11.2: TriAttention versus baselines on AIME (pass rate %, budget 2048 or 512; Table 1 of the paper, author-reported).

11.3 TriAttention

TriAttention [53] targets long chain-of-thought reasoning, where the cache grows during generation. The method (Chapter 7) scores keys using pre-RoPE query centers and a norm term and prunes every 128 tokens to a budget B.

Setup. Models: Qwen3-8B, DeepSeek-R1-Distill-Llama-8B, DeepSeek-R1-Distill-Qwen-7B and GPT-OSS-20B; AIME24, AIME25 (30 problems each, 8 samples per problem) and MATH500; generation up to 32K tokens, temperature 0.6, top-p 0.95; baselines Full Attention, SnapKV and R-KV; default budget 2048 tokens (512 for DS-Llama and MATH500).

On MATH500 (budget 512) TriAttention scores 56.0, 80.6, 79.6, 81.2 across the four models against Full Attention 69.6, 82.4, 87.0, 91.4 and R-KV 46.4, 76.9, 71.6, 77.4. Efficiency (Qwen3-8B, A100, 16K decode, max batch): on MATH500 at budget 1024, 68.4% accuracy (Full 69.6%) with 1405 versus 223 tokens/s, a 6.3× speedup; on AIME25 at budget 3072 it matches Full Attention's 40.8% at 563.5 versus 222.8 tokens/s (2.5×), which the authors express as a 10.7× KV-memory reduction at matched accuracy. Against R-KV at matched accuracy it needs half the budget (1024 vs 2048) with about 85% higher throughput. On a recursive depth-first-search memory task it stays near Full Attention to depth 16 while R-KV falls from about 61% at depth 14 to about 31% at depth 16. Ablations: removing the trigonometric score causes a large drop (AIME24 18.8, AIME25 21.2); calibrating on coding rather than reasoning data gives comparable accuracy. Appendix results: LongBench average 48.1 versus 45.6 for Ada-KV+SnapKV; RULER 66.1; GLM-4.7-Flash (MLA) results. A deployment claim, Qwen3-32B (INT4) on one RTX 4090 for an agentic report-writing task where full attention ran out of memory, is anecdotal.

Reading the evidence. (i) Most experiments are on math reasoning; the authors list coding and agentic evaluation as future work. (ii) The retrieved text reports internal inconsistencies (for example the AIME24 full-method figure is 42.1 in Table 1 but 45.8 in parts of the text), so individual numbers should be checked against the paper. (iii) At the default budget TriAttention is still well below Full Attention on several models (Qwen3 AIME24 42.1 vs 57.1; DS-Llama 33.8 vs 50.4), so the claim of matching Full Attention applies at the larger budgets (e.g. 3072) and for specific models. (iv) No dedicated kernel was used, so the throughput figures include the scoring overhead of a reference implementation; the authors expect improvement with a hardware-aware kernel. (v) The independent follow-up, Random Attention [60], found TriAttention significantly ahead of random on only one of its tests (Qwen3-32B code, about 3 points) and reports 32–43% higher vLLM throughput for random

Model	Method	MATH500	GPQA-D	AIME	HMMT	LCB
Qwen3-4B	Full	0.939	0.562	0.642	0.462	0.807
	Random Attention	0.874	0.530	0.610	0.438	0.744
	TriAttention	0.864	0.533	0.592	0.437	0.755
Phi-4-reasoning	Full	0.922	0.707	0.677	0.444	0.697
	Random Attention	0.910	0.678	0.662	0.430	0.667
	TriAttention	0.891	0.684	0.633	0.431	0.652
Qwen3-32B	Full	0.950	0.703	0.715	0.559	0.886
	Random Attention	0.891	0.683	0.664	0.509	0.806
	TriAttention	0.887	0.683	0.677	0.508	0.834

Table 11.3: Selected rows of Table 1 of the Random Attention paper (author-reported accuracies). Random Attention is significantly ahead of baselines in 31 of 60 comparisons and significantly behind in one.

eviction at matched budget, which questions how much of TriAttention's advantage is its scoring rather than the protection of the prompt (see below). A NVIDIA TensorRT-LLM documentation page for a "TriAttention KV-cache compression config" indicates inclusion in a major serving stack; and a separate arXiv paper on "Precomputing the Future-Offset Average in TriAttention" (July 2026) suggests follow-up engineering interest; I did not read either beyond their titles [2ND].

11.4 Random Attention

Random Attention [60] (Wang et al., 3 September 2026; revised 28 September 2026) removes the importance score. The full prefill (system prompt, chat template, question) is protected from eviction; every other cached position gets an independent uniform random score; each KV head keeps its own top-K, with independent draws per head at every eviction; the $r = 64$ most recent tokens are never scored; eviction runs every r decode steps. Setup: Qwen3-4B, Qwen3-14B (appendix), Qwen3-32B and Phi-4-reasoning (14B); MATH500, GPQA-Diamond, AIME 2025+2026 pooled, HMMT, LiveCodeBench-v6 medium; per-head budgets 1024, 2048, 4096, 4096 and 3072 respectively (compression about 4×, 3× for code), generation limit 32K; baselines SnapKV, R-KV, VaSE and TriAttention at matched budgets.

In vLLM with PagedAttention (K=2048, 1k prompts, 32k generations, one H200) throughput in tokens/s for Qwen3-4B / Phi-4-reasoning / Qwen3-14B / Qwen3-32B is: Full 1296/780/925/346; TriAttention 1494/1212/1303/700; Random Attention 2046/1737/1819/923, i.e. Random Attention is 37–43% faster than TriAttention in these runs, and 1.58×–2.67× faster than full. The authors attribute the advantage to skipping the scoring pass and to serving-time costs (compaction stalls, extra reads or attention materialisation under paged attention). For short generations (1k in, 8k out) compression does not pay off (0.52× full on Qwen3-4B, 0.96× on Qwen3-32B).

The two findings. (1) The prompt is the fragile part: when every method is given the same prompt-protection rule, most gaps shrink (SnapKV, which retains the least prompt,

gains up to 35.2 points on code and 22.5 on Phi-4 GPQA-D; R-KV, which already kept most of the prompt, gains at most about 1.9). Without protection a recency window scores 0.09 on Qwen3-4B GPQA-D and Random Attention 0.23. (2) The reasoning trace protects itself, by text redundancy (the model restates needed values) and head redundancy (Section 7.5.2). The exception is a fact stated once and never restated, which Random Attention never recovers (passcode probe) while R-KV recovers 84% of the time, VaSE about a third, SnapKV and TriAttention almost never.

Limitations (stated or implied). Protecting the whole prompt may consume up to half of the $K = 3072$ budget on code tasks; rare single-mention facts need a signal; AIME/HMMT have 30/60 problems with run-to-run standard deviation $\approx \pm 5$ points on AIME so many differences are not significant; the vLLM comparison covers only TriAttention since other methods lack vLLM ports; compressed vLLM runs were capped at 224 or 96 concurrent requests to avoid preemption and most compressed cells are single runs; the equal-memory comparison reports roughly 10× full-attention throughput for Random Attention on Qwen3-4B at $K = 3072$, but the TriAttention row there uses an unfused reimplementation, so it is not a fair comparison [60].

Interpretation. The result does not show that scoring is useless in general. It shows that, on *reasoning-trace workloads at 3–4× compression with prompt protection*, the evaluated scoring methods add little over randomness; this is both a statement about the methods and about the benchmarks, whose problems are short (the prompt is a small fraction of the budget) and whose traces are redundant. For retrieval from long documents the situation is different, as the other chapters document: the facts are stated once in the prompt, and prompt-resident content is exactly what the paper finds fragile.

11.5 The critical evaluation: pitfalls of compression

"The Pitfalls of KV Cache Compression" [64] (Chen, Geh, Grover, Van den Broeck, Israel; arXiv 30 September 2025; v2 13 May 2026) evaluates StreamingLLM, SnapKV, TOVA, H2O and K-Norm on Llama-3.1 8B and Qwen2.5 14B with IFEval and multi-instruction prompts. It reports that some instructions degrade rapidly under compression until the model effectively ignores them; that system-prompt leakage is a concrete harm (compression affects both leakage and instruction-following); that three factors contribute (compression method, instruction order, and eviction bias); and that simple changes to the eviction policy reduce these effects. This is the direct counterpart to the benchmark-centred papers above: it says that average accuracy conceals qualitatively different failures of the cache, an observation consistent with the derivation of Proposition 7.3. It is also consistent with Random Attention's finding that prompt protection explains most gaps.

11.6 Evaluation hazards specific to eviction

- **Prompt/generation asymmetry.** Methods designed for prefill compression (SnapKV) and for decode compression (R-KV, TriAttention, Random Attention) are

evaluated on different tasks. A method that wins on long-generation math may lose on long-prompt retrieval.

- **Throughput measurement.** Throughput depends heavily on the serving implementation (compaction frequency, paged attention, preemption caps); Random Attention's authors argue that these costs, not the arithmetic of scoring, drive the throughput ordering.
- **Small benchmark sets.** AIME-style sets of 30 problems with 8 samples cannot resolve differences of a few points.
- **Needle tests.** As Chapter 4 shows, a needle test measures the probability that a single high-logit key survives; it is insensitive to tail-mass loss and to aggregation. TurboQuant and others report needle results; RULER and NoLiMa are more informative [38, 39].
- **Multi-turn state.** Evictions made while serving turn t affect turn $t + 1$; most evaluations are single-turn.

Chapter 12

Systems and Kernels

Whether a sparse or hybrid attention design delivers wall-clock speedups depends on kernels, memory layout and serving integration at least as much as on algorithms. This chapter collects what the retrieved papers say about implementation, adds derived arithmetic-intensity and memory analyses ([DER]), and identifies the recurring sources of kernel complexity. It has little to say about vendor-specific microarchitecture beyond what the papers state.

12.1 Why sparse attention is hard to make fast

Dense attention has regular structure: contiguous reads of keys and values, tiles that map onto tensor cores, and a known loop nest (FlashAttention [15]). Sparse attention breaks each of these in a different way.

Irregular reads. Selected tokens are scattered. Reading one token's key in a GQA model with $d = 128$ in BF16 is 256 bytes, below the granularity at which HBM achieves peak bandwidth (typically a 32–128-byte sector within a longer burst). Block-level selection (64–128 tokens) restores contiguity: a 128-token block of keys for one KV head is 32 KiB contiguous. This is the hardware reason why almost every kernel-efficient design (NSA, MoBA, MSA, SparDA, AsyncTLS) selects blocks or pages, and why token-level methods such as DSA must gather rows from a latent cache with many fewer heads (MLA's 576-element rows are 1152 bytes, contiguous enough). It also explains the empirical headline of Nawrot et al. that fine-grained per-query selection during prefill is impractical without suitable kernels, whereas token-to-page selection in decoding is feasible [41].

Low arithmetic intensity. The arithmetic intensity of a sparse decode kernel equals the number of query vectors that *share the loaded keys and values* times $2/b$ FLOP per byte, because each loaded element is used once per sharing query. Consider the following cases.[DER]

- Dense GQA with group size $H/G = 4$, BF16: $I = 4$ FLOP/B (Eq. (3.3)).
- Block-sparse GQA with selection shared by the group (NSA, MSA, SparDA): the same, $I = H/G$ per sequence, but the loaded set is only kB tokens, so total bytes fall.
- Block-sparse with *per-head* selection (selection differs per query head): $I = 1$, i.e. intensity drops by a factor H/G. This is the reason these methods share the selection across the heads of a GQA group (NSA loads all query heads of a group together so they share the same sparse KV blocks; MSA selects per GQA group; SparDA's forecast uses one head per group).
- DSA on MLA in multi-query form: all $H = 128$ query heads share one latent key–value, so $I = 2H(1088/2)/(576 \cdot 2) \approx 242$ FLOP/B (Chapter 3), already near the

ridge point: sparse MLA decode is a compute-sensitive kernel, which is why a FP8 or smaller-precision implementation is attractive.

Poor tensor-core tiles. Tensor-core instructions operate on tiles with a minimum of 16 (or more) rows. A decode step for one sequence supplies only H/G query rows per KV head (e.g. 4), so tiles are mostly padding. Dense GQA decode kernels handle this by splitting the KV sequence across thread blocks ("split-K"); sparse kernels must additionally cope with a variable, data-dependent number of keys per sequence.

Load imbalance. Under causal masking the number of eligible blocks varies with query position; different sequences in a batch have different lengths and different numbers of valid selections. Static partitioning leaves thread blocks idle.

12.2 Kernel designs reported

NSA. Triton kernels for the three branches, with group-level loading of query heads so that all heads of a GQA group share the same selected blocks, reaching 9.0× forward, 6.0× backward and 11.6× decode speedups over FlashAttention-2 at 64K on A100 [42]. The kernel design was important enough that the authors' title includes "Hardware-Aligned". A noteworthy feature is that the *backward* pass is also accelerated, which is what enables sparse pre-training and not only sparse inference.

MSA. Three kernel ideas [58]. (1) An *exp-free top-k* selection: because top-k is invariant under monotone transformations, the exponentials and the normalisation of the softmax can be skipped when selecting; with small k a dedicated kernel can keep a running threshold or a short register-resident list rather than sorting. (2) A *KV-outer* loop order: instead of the usual query-outer loop that, for each query, iterates over the selected KV blocks, the kernel iterates over KV blocks and gathers the queries (within the same group) that selected each block, filling tensor-core tiles with queries that share the loaded block; this reverses the roles to restore data reuse and tile occupancy during prefill. A two-phase combine with pre-scheduled chunking merges partial softmax results without atomics. (3) Training-side kernels compute the log-sum-exp needed by the sparse KL loss in the forward pass and use persistent thread blocks with load balancing in the backward pass. With these the authors report 14.2× prefill and 7.6× decode speedups at 1M on H800 and release the inference kernel.

SparDA. Kernel-level choices include the number of cooperative thread arrays (CTAs) allocated to selection: 16 CTAs work best at small batch sizes and 32 or 64 at larger ones; the adaptive rule uses 16 CTAs for batch size below 32 and 32 otherwise [57]. This detail illustrates the sensitivity of sparse kernels to batch size: the best parallelisation depends on how much work each sequence supplies.

DSA and the V4 indexer. The lightning indexer is designed for FP8 execution (ReLU instead of softmax; few heads) [3]; in DeepSeek-V4 the indexer QK path is executed in FP4 and trained with quantisation-aware training, and the KV entries are stored in mixed BF16

(RoPE dimensions) and FP8 (rest) [59]. The report states that on current hardware the peak FP4×FP8 throughput equals FP8×FP8, with up to one third better efficiency expected on future hardware. A consequence for the cost model: lowering indexer-key precision lowers the bytes of the scan (the dominant term) by 2× per halving of bits, even if arithmetic throughput does not improve.

AsyncTLS. The Quest-style min/max block score is reformulated as a matrix multiplication to run on Tensor Cores for shared-KV architectures (MLA) [54], turning an elementwise operation into a GEMM. The token-level stage works on INT4-quantised projected keys on a subset of channels, a low-precision, reduced-dimension proxy.

IndexCache and serving. IndexCache modifies the model's forward pass so that Shared layers skip the indexer; a vLLM documentation page "index cache" indicates that the feature is supported in that serving framework [51]. In GLM-5.2 the pattern `full, shared, shared, shared` is the deployed configuration (secondary source [2ND] [68]).

12.3 The offload regime

Decode throughput is bounded by the capacity of HBM when the cache is stored there. Offloading the cache to host memory raises capacity by an order of magnitude but replaces the HBM roofline by the PCIe (or NVLink-C2C) roofline for the portion that must move. With sparse attention, only the selected blocks move. Let S_{sel} be the bytes of selected KV per layer per sequence per step; the per-step transfer for L layers and batch B_s is $B_s L S_{\text{sel}}(1-\rho)$ with ρ the hit fraction on the GPU-resident cache. For a PCIe 5.0 x16 link with theoretical 64 GB/s per direction (an assumed hardware figure) and effective $\sim$ 50 GB/s, the budget available to hide a transfer in a 20 ms decode step is about 1 GB. The condition for complete overlap derived in Chapter 9 then becomes a constraint on ρ, which is why temporal locality (AsyncTLS) and forecasting (SparDA) are central: without either, a layer cannot start its transfer before the layer's selection is known.

Worked example 12.1 (Transfer volume of a 6144-token selection)**.** [DER]SparDA's configuration selects up to 96 blocks of 64 tokens, i.e. 6144 tokens. For a GQA layer with $G = 8$, $d = 128$, BF16 (4096 B/token) that is 25.2 MB per layer per sequence if nothing is resident; over 32 layers, 805 MB per sequence per step. At 50 GB/s that is 16 ms per sequence per step: at batch 16 the transfer volume would need 258 ms/step, far above the measured $\approx$ 23 ms/step implied by 705 tokens/s at batch 16 for MiniCPM [57]. The measured throughput therefore implies hit rates $\rho \geq 90\%$ on the GPU-resident portion (local windows and recently used blocks), or smaller KV per token than assumed (MiniCPM4.1-8B has its own GQA configuration which I did not retrieve). The example shows how crucial the resident-set design is; it does not reconstruct SparDA's actual hit rates.

12.4 Quantisation kernels and the cost of rotation

TurboQuant-style schemes apply a random rotation Π to each key. Because attention logits are inner products, one can instead rotate the query once per step: $q^\top k = (\Pi q)^\top (\Pi k)$, so

stored keys are in the rotated basis and the cost is one $d \times d$ rotation per query head per step (16K MACs for $d = 128$), negligible next to scanning T keys.[DER] For values, the output is $\sum_i p_i v_i$, and rotating values at write time requires inverse-rotating the output once per query; likewise negligible. A dense rotation at write time costs d^2 MACs per token and head (16K), which is small relative to the per-token projection FLOPs. Structured rotations are typically used in practice to reduce this cost further; the retrieved TurboQuant abstract describes random rotation without specifying an implementation. The remaining overheads are the lookup-based dequantisation of 2–4 bit codes, which is cheap relative to the 4–8× reduction in bytes, and the outlier-channel bookkeeping. Google reports 4-bit TurboQuant attention-logit computation up to 8× faster than 32-bit keys on H100, against an optimised JAX baseline [63][VENDOR]; the best-case speedup of a bandwidth-bound kernel on a b-bit cache is $16/b$ against BF16 and $32/b$ against FP32, i.e. 4× and 8× at 4 bits, so the reported 8× is at the limit of the FP32 comparison, not an additional gain. This is a back-of-envelope consistency check [DER], not evidence about the implementation.

12.5 Linear-attention kernels and serving

Chunkwise linear-attention kernels (Lightning Attention-2, Gated DeltaNet and KDA kernels) are matrix-multiply based but sequentially dependent across chunks through the state. MiniMax reports that linear-attention models are often memory-bound even in training, are more sensitive to numerical precision than full attention, and require new designs for prefix caching and speculative decoding [5]. Kimi Linear's release includes the KDA kernel and vLLM implementations [4].

Worked example 12.2 (Prefix-cache snapshots for a hybrid)**.** [DER]Prefix caching for a dense model stores KV for shared prefixes. For a recurrent layer the state at the end of the prefix is a snapshot of $H_\ell d_k d_v b_s$ bytes, valid only at that exact prefix length. Suppose 24 linear layers with 32 heads of 128×128 state in BF16 (1 MiB per layer, 24 MiB per snapshot). To allow reuse at any 512-token boundary of a 1M-token prefix one needs 2048 snapshots, i.e. 48 GiB (96 GiB in FP32). By comparison the stored KV of the 8 attention layers of a 1:3 hybrid with $G = 8$, $d = 128$ is $8 \times 4096\ \text{B} \times 10^6 \approx 30$ GiB (plus index keys). Coarse snapshot spacing (e.g. every 8K tokens) lowers the figure to 3 GiB; the trade is losing reuse for prefixes that diverge between snapshots. Prefix caching therefore changes character for hybrids from "store all KV once" to "choose a snapshot spacing", which is part of what MiniMax called a need for new solutions.

12.6 Training-side systems

Sparse training needs backward kernels (NSA reports 6.0× at 64K), a differentiable or distillation signal for the selector, and, for KL-trained indexers, access to the dense attention distribution or its sufficient statistics. MSA computes the log-sum-exp in the forward pass for the sparse KL loss; DSA's dense warm-up stage needs the dense attention distribution over 128K tokens for the main attention, which is expensive but short (2.1B tokens) [3]. For hybrid conversion, MiniCPM-SALA's staged recipe confines long-context pre-training (32K–520K) to 215.7B of ∼1.96T tokens (∼11%), with a further 417.8B tokens of SFT at 64K–140K [50]. Gradient flow through selection is discussed in Chapter 18.

12.7 Summary of implementation patterns

1. Select at block or page granularity to retain contiguous reads; share selection across the heads of a GQA group to retain arithmetic intensity.
2. Make the selector cheap in bytes: low precision (FP8/FP4), few heads, shared across layers, or hierarchical.
3. Reorder loops (KV-outer) or gather queries to fill tensor-core tiles in prefill.
4. Overlap selection and data movement with the previous layer's computation (forecast, asynchronous prefetch).
5. For recurrent layers, size and place state snapshots deliberately.

Chapter 13

Empirical Review

This chapter assembles the quantitative evidence reported by the sources in Chapters 8–11, classifies it by what level of the stack it measures and how it was produced, and states what can and cannot be concluded. All numbers are as reported by the authors of the cited sources; I ran none of the cited systems. The evidence ledger in Appendix C lists each claim with a verification status.

13.1 Quality: do trained sparse models match dense ones?

Table 13.1 gives the principal quality comparisons of a sparse or hybrid model against a dense baseline trained or provided by the same authors.

What the table supports. At 1.5B–27B scale, with matched training recipes, trained sparse attention loses little relative to dense, and in several cases records small gains. The comparison scale and the variance should be kept in mind: a single training run per configuration is typical, and benchmark differences below ∼1 point are within what different seeds produce at these scales. The strongest evidence of parity is the combination of (i) a model-matched comparison (NSA, MoBA, CLSA, MSA, GSA, DSA with its predecessor) and (ii) long-context probes beyond needle retrieval (RULER; LongBench).

What the table does not support. First, there is no like-for-like frontier-scale comparison: for DSA, V3.2-Exp versus V3.1-Terminus is a controlled continuation of the same weights, but both are in-house; for MSA, the main quality claim is at 109B while the production model (M3, 428B) has no dense counterpart. Second, tasks that stress the weakest aspect of sparse selection (multi-key aggregation, non-literal retrieval) appear in only some papers (CLSA: RULER multi-needle; SALA: NoLiMa; HySparse2: RULER-v2, MRCR-v2, GraphWalks). Third, most positive results are at or below the training context length. Evidence for *generalisation to 1M* comes mainly from needle tests and a few RULER results (MiniCPM-SALA at 1000K: 86.3; 2048K: 81.6; MoBA-converted Llama at 128K).

13.2 Long-context retrieval: what the benchmarks cover

RULER's original finding was that most 32K-claiming models do not hold up at 32K on harder tasks [38]; NoLiMa that with lexical overlap removed, 11 of 13 models fell below half their short-context baseline at 32K tokens, with GPT-4o dropping from 99.3% to 69.7% [39]. These results frame the sparse-attention literature: a sparse method is credible if it holds up on those, not merely on the vanilla needle. In that light, MiniCPM-SALA's NoLiMa 32K/64K/128K of 54.54/42.95/23.86 (against Qwen3-8B's 43.40/23.35/11.25 and the others

Source	Scale / data	Comparison	Short-context result	Long-context result
NSA [42]	27B/3B-act MoE; 260–270B tokens; 8K→32K	NSA vs full attention, same recipe	avg. 0.456 vs 0.443 (7/9 higher)	LongBench 0.469 vs 0.437; NIAH 64K perfect; AIME 0.121/0.146 vs 0.046/0.092
MoBA [43]	1.5B scaling; Llama-8B-1M conversion	MoBA vs full attention	loss within $\sim 10^{-3}$ at 8K	RULER 128K 0.7818 vs 0.7849; LongBench 32K 0.4828 vs 0.4821
DSA [3, 71]	DeepSeek-V3-architecture MoE (61 layers, hidden 7168); continued 0.95T tokens	V3.2-Exp vs V3.1-Terminus	see Table 8.1: mixed, ±1–3 pts	AA-LCR +4; Fiction.liveBench higher
CLSA [1]	4B; dense 8K→32K then sparse	vs YOCO dense and Transformer	avg. within ±0.02 except GSM8K +0.04	RULER 32K 53.1 vs 52.3 (YOCO), 46.2 (Transf.)
MSA [58]	109B MoE, 3T tokens	MSA vs GQA	"on par"	(tables not retrieved)
GSA [47]	1.7B; 400B tokens; 4K	GSA vs standard	PPL 5.70 vs 6.03 (gating)	RULER 128K 62.2 vs 31.7 (YaRN-extended)
MiniCPM-SALA [50]	9B hybrid; ∼2T tokens	vs Qwen3-8B etc. (different data)	avg. 76.53 vs 73.45	RULER 128K 89.37 vs 71.74; NoLiMa 128K 23.86 vs 11.25; MRCR lower than best
HySparse2 [61]	80B-A3B MoE; +100B tokens to 256K	vs HySparse, Hybrid SWA	(n/r)	RULER-v2 256K 58.45 vs 32.61
AsyncTLS [54]	training-free, Qwen3-8B/14B, GLM-4.7-Flash	vs full	LongBench 49.9 vs 50.1 (8B)	RULER 90.54 vs 90.45 (8B)
TSA [49]	training-free prefill; LLaMA-3.1-8B	vs FlashAttention	RULER avg. 87.02 vs 87.01	128K: ×1.36 speed

Table 13.1: Quality comparisons of sparse/hybrid models against dense baselines (author-reported).

lower) is the most informative result among the retrieved sparse-plus-linear papers, and its MRCR deficits relative to the best baseline are the most informative *negative* result. The hybrid's attention is 75% linear, so its exact-recall mechanism is concentrated in 25% of layers; NoLiMa-style non-literal retrieval nonetheless works, which suggests that semantic retrieval capability resides in few layers (consistent with the retrieval-head literature [33]).

13.3 Efficiency claims, by level

The wide range of reported speedups reflects what is measured. Table 13.3 lists the claims with the level (operator/kernel, model-level, end-to-end serving), the baseline, the context length and hardware, and the source type.

How to read it. Three principles organise the table.

(i) Operator numbers exceed end-to-end numbers. MSA's 28.4× compute versus 7.6× decode; AsyncTLS's 10× kernel versus ∼ 2.3–2.7× end-to-end; TSA's 3.23× attention versus the model-level gain of 1.36× on its own. Proposition 3.2 and Amdahl's law account for this.

(ii) The baseline decides. A method compared with FlashAttention-2 on A100 in Triton

Benchmark	What it tests	Typical weakness of sparse/hybrid
Needle-in-a-haystack [38]	single literal key	near none (top-k includes the needle if its logit is high)
RULER (13 tasks) [38]	multi-key/multi-value needles, tracing, aggregation	aggregation and multi-key tasks stress tail mass and state capacity
NoLiMa [39]	retrieval without literal overlap	indexers trained on dense attention may rank by lexical similarity; state memories blur
MRCR	multi-round co-reference over long contexts	requires distinguishing near-duplicates; SALA is below best here
LongBench	mixed real tasks (QA, summarisation, code)	usually small differences
AA-LCR, Fiction.liveBench	long-context reasoning; story comprehension	used in DeepSeek's V3.2 evaluation
GraphWalks, RULER-v2, MRCR-v2	multi-hop and harder retrieval variants (HySparse2)	block selection is weak, token selection stronger

Table 13.2: Long-context benchmarks that appear in the retrieved sparse/hybrid literature and the failure mode each is positioned to detect.

(NSA), with a Transformer of a different architecture (CLSA), with a sparse model plus offload (SparDA), or with an unquantised 32-bit JAX implementation (TurboQuant logits) is not comparable to the others. The ratio between the largest and smallest speedups in the table (28.4 vs 1.25) says more about the choice of baseline than about the methods.

(iii) Context length is a hidden parameter. Sparse speedups increase with T (Chapter 3); a 1.48× at 200K and a 7.6× at 1M can be the same method. NSA's own table shows speedup rising from 4× at 8K to 11.6× at 64K (tokens read).

13.4 Efficiency per training dollar: what the conversions cost

Reported adaptation budgets are an underappreciated part of the evidence. DSA: 2.1B tokens indexer warm-up plus 943.7B sparse training tokens on top of a pre-trained model. CLSA: 2×2500 steps at 8M tokens/step = 40B tokens of sparse adaptation after 1T (125,000 × 8M) tokens of dense pre-training. MiniCPM-SALA: ∼2T tokens in total (25% of from-scratch). SparDA: 33.5M trainable parameters, 2000 steps at batch 32 of 64K tokens (≈4.2B tokens), 24–48 hours on 32 H100s. IndexCache: training-free option or a distillation stage (cost not retrieved). HISA, MISA, TSA: none. The wide range (0 to $\sim 10^{12}$ tokens) supports a design principle: the more the method alters what each layer computes (SALA, DSA), the more adaptation it needs; selection-side changes (SparDA, HISA, MISA) can be nearly free but are bounded by the backbone's sparse pattern.[DER]

13.5 Statistical quality of the evidence

- **Seeds and variance.** Almost no source reports seeds or confidence intervals, except the Random Attention paper, which reports significance tests and notes run-to-run standard deviations (about ±5 points on AIME) and that many differences are not

Method	Claim	Level	Baseline	Context / hardware	Source
NSA	9.0× fwd, 6.0× bwd, 11.6× decode	kernel	FlashAttention-2 (Triton)	64K, A100	paper
MoBA	6.5× attention prefill; ∼ 16× compute	attention layer	full attention	1M; 10M	paper
InfLLM-V2	∼ 4×	attention/model (n/r)	dense	(n/r)	abstract
IndexCache	75% indexer ops removed; 1.82× prefill, 1.48× decode	model-level	standard DSA	30B DSA; 200K (2nd)	paper+2nd
CLSA	7.6× decode, 17.1× overall throughput	model-level	Transformer, 4B	128K	paper
MSA	28.4× attn compute; 14.2× prefill; 7.6× decode	kernel+compute	GQA	1M, H800	paper
SparDA	1.25× prefill, 1.7× decode vs sparse+offload; 5.3× decode vs non-offload sparse	model-level	sparse w/ offload	128K, H100	paper
AsyncTLS	op. 1.2–10.0×; e2e 1.3–4.7× (abstract; text inconsistent)	kernel/e2e	full attn, Quest, DS	48K–96K	paper
TSA	up to 3.23× attention (abstract); 1.36–2.76× (main tables)	kernel	FlashAttention, others	128K, A100	paper
MISA	3.82× vs DSA indexer kernel	kernel	DSA indexer	H200	paper
GSA	prefill 0.09×, decode 0.13× time (i.e. ∼ 11×, ∼ 7.7×)	model-level	standard attn	128K, 1.7B	paper
MiniCPM-SALA	up to 3.5× (TTFT 180.8 → 51.6 s)	e2e	Qwen3-8B	256K, A6000D	paper
Kimi Linear	up to 6× decode; −75% KV	model-level	full MLA	1M	paper (abstract)
DeepSeek-V4	27% FLOPs, 10% KV (Pro); 10%, 7% (Flash)	model-level	V3.2	1M	paper
GLM-5.2	2.9× fewer per-token FLOPs	model-level	(n/r)	1M	2nd
HySparse	KV ∼ 10× lower	memory	full attn	80B MoE	paper
TriAttention	2.5× throughput / 10.7× KV (AIME25); 6.3× (MATH500)	model-level	full attention	16K–32K gen., A100	paper
Random Attn	+32–43% vs TriAttention; 1.58–2.67× vs full	serving	vLLM full	32K gen., H200	paper
TurboQuant	≥ 4.5× (paper); ≥ 6× KV (blog); logits 8×	memory/kernel	16-bit; 32-bit JAX	H100	paper+blog
KIVI	2.6× peak memory; 2.35–3.47× throughput	serving	16-bit	(n/r)	abstract
SubQ	56× (theoretical); $8 vs $2,600 RULER-128	vendor	FlashAttn (?)	12M	press

Table 13.3: Efficiency claims in the retrieved literature. "n/r" = not retrieved. The "level" column determines what the number means; see text.

significant; no other retrieved summary mentioned seeds or confidence intervals (the retrieved texts were summaries or abstracts, so this is a statement about what I could see, not about the papers).

- **Internal inconsistencies.** Retrieval summaries flagged inconsistencies in four sources: NSA (pretraining tokens 260B vs 270B), TriAttention (AIME24 figure 42.1 vs 45.8 across text and tables), AsyncTLS (speedup ranges differ across abstract, introduction and results), and TSA (abstract 3.23× vs table maxima); GSA's theory section also has internal mismatches. These do not invalidate the results but call for reading the tables rather than abstracts.
- **Author-run baselines.** In nearly all cases the baselines were run by the authors with their harness (MiniCPM-SALA's baselines for RULER; CLSA's Transformer). This is standard but inflates the chance of favourable tuning.
- **Independent uses.** The most independent evidence consists of third-party applications of the open DSA models: HISA and MISA evaluate on DeepSeek-V3.2 and GLM-5 without retraining and find that the DSA indexer's choices can be approximated by cheaper procedures, which at least establishes that the DSA models' sparsity is learnable and reproducible by other groups' methods; community implementations of TurboQuant (second-hand) and the Random Attention paper's reproduction against TriAttention in vLLM are the others.

13.6 Summary of what the evidence establishes

1. **Established (multiple independent signals):** trained token- or block-level sparse attention with budgets of 2K–8K tokens can be added to or trained within LLMs at 1B–100B+ scale with small average quality changes on standard and long-context benchmarks at contexts up to roughly the training length; decode/prefill speedups of the order of several-fold at 64K–1M tokens are real at kernel level.
2. **Supported but single-source:** that sharing the indexer across layers is nearly free (IndexCache, CLSA, HySparse: three papers, different regimes); that a 1:3 sparse:linear hybrid can retain RULER/NoLiMa performance to 1M and beyond (one paper, 9B); that 3–4× KV eviction is adequate for reasoning traces (several papers, but math-heavy).
3. **Weak or unestablished:** quality at 1M tokens for tasks beyond retrieval; behaviour on multi-turn agent traces; interaction of quantisation with learned selection; vendor claims for SubQ and the 8× logit speedup; the independent replication of any claimed end-to-end speedup at $\geq$1M tokens on production workloads.

Chapter 14

Comparing the Approaches

Chapters 8–11 described the methods one at a time. This chapter compares them along the axes that decide whether a method is usable: which resource it attacks, what it demands from training, what happens to the cache that must be stored versus the cache that must be read, and what happens when methods are combined. It also gives a decision procedure that follows from the cost model of Chapter 3.

14.1 Organising by the factor attacked

The decode-time bytes of a model with a growing cache can be written (Chapter 3) as

$$\underbrace{L_{\text{grow}}}_{\text{layers with a cache}} \times \underbrace{\phi}_{\text{fraction read}} \times \underbrace{T}_{\text{tokens}} \times \underbrace{n_{\text{elt}}}_{\text{elements/token/layer}} \times \underbrace{b}_{\text{bytes/element}} + \text{(selector scan)}. \tag{14.1}$$

Table 14.1 assigns each method family to the factor it changes. The point of the table is to show that most pairs of methods act on *different* factors and can in principle be stacked, whereas methods acting on the same factor compete.

Stored versus read. A central distinction, repeated throughout the report, is between the bytes *read* per step (which sets decode latency in the bandwidth-bound regime) and the bytes *stored* (which sets concurrency and, at large T, whether the sequence fits at all). Learned sparse attention such as DSA reduces the former dramatically (Table 3.2 in Chapter 3: 151 MB to 19.7 MB per layer per token at 128K for a V3.2-style layer [DER]) but *adds* index keys to the latter. Eviction, quantisation, linear-state hybrids and compression reduce what is stored. This is why the 2026 frontier designs (DeepSeek-V4's CSA/HCA, HySparse, CLSA) combine a sparse read pattern with a reduction of the stored cache, rather than relying on sparse reads alone.

14.2 Requirements imposed on training

Two design principles emerge from Table 14.2. First, the farther a method departs from what a dense model computes in each layer, the more adaptation is reported to be needed: replacing 75% of the attention layers with linear attention (SALA) or learning a token selector end to end (DSA) both used hundreds of billions of tokens or more, while methods that only reorganise the selector (HISA, MISA) need none. Second, training-free methods inherit the pattern of the frozen backbone: HISA and MISA approximate the DSA indexer's choices but cannot improve on them.

Factor	Lever	Representative methods	Changes what is *stored*?
L_{grow}	fewer layers keep a growing cache	linear/SSM hybrids (Kimi Linear, MiniCPM-SALA, Qwen3-Next, Jamba); YOCO/CLSA; HySparse; sliding-window layers	yes
ϕ	read a fraction of the cache	NSA, MoBA, DSA, MSA, GSA, Quest, HISA, MISA, AsyncTLS	no (full cache kept; offload possible)
T	keep fewer tokens	StreamingLLM, H2O, SnapKV, TriAttention, Random Attention	yes
n_{elt}	fewer or narrower entries per token	GQA/MQA, MLA, CSA/HCA compression of every m tokens	yes
b	fewer bits per entry	KIVI, TurboQuant, FP8 caches, FP4 index keys	yes
selector scan	make the selector cheaper	IndexCache, CLSA, HISA, MISA, SparDA forecast, group-level indexers	only the index keys

Table 14.1: Methods by the factor of (14.1) that they change. The last column matters for capacity: methods that do not change what is stored do not reduce the number of sequences that fit on a GPU.

14.3 Failure modes by family

Table 14.3 lists the characteristic ways each family fails, with the source of the evidence. These are developed in Chapter 15.

14.4 Composition: how the savings stack, and where they do not

14.4.1 Multiplicative and non-multiplicative pairs

If two methods act on different factors of (14.1), their byte reductions multiply to first order. Using the cost model:[DER]

- *Token-level sparse read* × *shared indexer* (DSA with IndexShare, $m = 4$): 7.7× → 22.6× in bytes read per layer at 128K (Table 3.3 [DER]). The product is not 7.7×4 because the indexer is only part of the traffic: Amdahl's law applies to the byte budget itself.
- *Sparse read* × *quantised cache*: reading k entries of b bytes each, then halving b, halves

Requirement	Methods	Reported adaptation cost	Risk
Train from scratch with the mechanism	NSA, MoBA (option), MSA (option), GSA, DeepSeek-V4, Kimi Linear, MiniMax-01	full pre-training budgets	cannot reuse existing checkpoints
Continue pre-training an existing dense model	DSA (2.1B warm-up + 943.7B tokens), CLSA (40B sparse after 1T dense), MiniCPM-SALA (∼2T total), MoBA conversion	10^9–10^{12} tokens	distribution shift, regression on non-long tasks
Light fine-tuning of a small module	SparDA (33.5M parameters, ≈4.2B tokens), IndexCache with distillation	10^9 tokens or fewer	only as good as the backbone's pattern
None (training-free)	HISA, MISA, TSA, AsyncTLS, Quest, SnapKV, H2O, TriAttention, Random Attention, IndexCache (search variant)	calibration only	quality ceiling set by the frozen model
Post-training quantisation	KIVI, TurboQuant	none (data-oblivious for TurboQuant)	accuracy at low bits; outlier-heavy models

Table 14.2: Training requirements of the method families (author-reported budgets, from Chapter 13).

the sparse bytes but not the index-key bytes unless the index keys are also quantised (DeepSeek-V4's FP4 indexer is such a case). Once the indexer dominates (88% of bytes at 128K), quantising the main cache gives little.

- *Compression of every m tokens × top-k* (CSA): reduces the stored cache and the scan by m simultaneously; quality cost is that selection operates on summaries.
- *Hybrid linear layers × sparse layers* (SALA): the linear layers contribute a constant-size state; the sparse layers contribute $\phi T n_{\text{elt}} b$ plus scan; the sum is dominated by the sparse layers' scan at large T (Table 5.2 [DER]).
- *Eviction × selection*: eviction removes tokens that selection would later need; selection works on a cache that eviction has already thinned. No retrieved paper evaluates this composition (Chapter 18).

14.4.2 Byte accounting for a reference configuration

Table 14.4 puts illustrative numbers to the stack using the model dimensions of Chapter 3 (a V3.2-like MLA layer at 128K). The point is arithmetic, not a measurement: each row applies one idealised change to the row above and recomputes read and stored bytes per layer per decoded token. [DER]

Family	Characteristic failure	Evidence
Block-level selection	the needle is diluted inside a block whose pooled key does not stand out; coarse blocks waste budget	Proposition 4.4 [DER]; MoBA's block-size trade-offs (Chapter 8)
Token-level learned selection	indexer cost grows with T and dominates at long context; selection quality limited by an indexer that sees only a low-dimensional projection	Table 3.2 [DER]; IndexCache/HISA/MISA exist to fix it
Shared selection	layers or heads that need different tokens get the same set	Chapter 6; CLSA ablations
Linear-state layers	finite state: recall capacity limited by d, interference grows with the number of stored items	Proposition 5.2 [SIM]; Repeat After Me [26]
Eviction	irreversible loss of tokens a later query needs; accumulated-score bias; multi-turn degradation	Proposition 7.3 [DER]; Chen et al. [64]; Table 7.4 [SIM]
Quantisation	outlier channels destroy low-bit schemes; attention softmax amplifies logit noise	Table 7.3 [SIM]; Chapter 7

Table 14.3: Characteristic failure modes by family.

Configuration (per layer, $T = 128$K)	MB read	Stored MB	vs dense read
Dense MLA (576 elements, 2 bytes)	151.0	151.0	1×
+ DSA top-2048, single-layer indexer	19.7	151.0 + 17.3	7.7×
+ indexer shared over $m = 4$ layers	6.7	151.0 + 4.3	22.6×
+ index keys at 4 bits (idealised ÷2 vs FP8)	≈4.5	151.0 + 2.2	≈ 34×
+ KV cache at 4 bits (idealised, ÷4 vs BF16)	≈2.8	37.7 + 2.2	≈ 55×

Table 14.4: Stacking idealised changes on a V3.2-like layer at 128K tokens ([DER]). The first three rows come from the generated tables of Chapter 3; rows 4–5 are my arithmetic on top (index keys 132 B/token → halved; main cache ÷4; scale and metadata overheads ignored), not reported by any source. The stored column lists main cache plus index keys.

14.4.3 Interaction effects that arithmetic misses

1. **Quantisation noise perturbs selection.** Selection ranks logits; quantised keys shift logits by noise δ whose variance grows with the key's norm and shrinks with bits (Chapter 7). By the needle formula of Proposition 4.5, a noisier ranking requires a larger budget for the same recall: effective noise is $\sqrt{1+\tau^2}$ with τ the extra noise-to-margin ratio. A sparse-and-quantised system is therefore not simply a product of the two methods' quality losses. No retrieved paper tests the combination on a trained learned-sparse model.
2. **Eviction and sharing.** If a token is evicted from the anchor layer's cache but not from the shared layers' caches (or vice versa), indices become invalid; designs that share caches (YOCO, HySparse) cannot evict per layer.
3. **Prefix caching.** Reusing a cached prefix requires, for linear-state layers, storing states at prefix boundaries (Chapter 12); for sparse layers requires the index keys. The cost

of this storage is not in the headline speedups.

4. **Speculative decoding.** Verifying several tokens in one pass changes the effective T per step and the arithmetic intensity (Chapter 3); sparse selection per token is awkward for multi-token verification. This interaction is flagged as open in Chapter 18.

14.5 Decision procedure

From the cost model one can derive a practical order of questions. This is my synthesis, not a procedure endorsed by any source. [DER]

1. **Is the workload prefill-bound or decode-bound?** Prefill is FLOP-bound: reduce T^2 (sparse prefill: TSA, MInference-style patterns, NSA/DSA training; hybrids). Decode is bandwidth-bound: reduce bytes read per step.
2. **Is capacity (stored bytes) or latency (read bytes) the constraint?** If sequences do not fit or concurrency is low, reduce stored bytes (quantisation, MLA/GQA, CSA/HCA, linear-state layers, eviction). If latency dominates, learned selection pays off, provided the indexer scan has been amortised (sharing, hierarchy).
3. **Can the model be retrained?** If not, choose among training-free selectors (HISA, MISA, Quest, TSA, AsyncTLS) and post-training quantisation; if yes and context is the product, choose between a trained sparse design (DSA, MSA, GSA, CSA) and a hybrid (SALA, Kimi Linear), by the next question.
4. **Does the task need exact recall of arbitrary items?** If yes (code, retrieval, tool use), keep at least some layers with exact, query-dependent access to the full context (sparse or dense); linear-state-only layers cannot guarantee it (Proposition 5.2, Chapter 5). If the task is aggregative (summarise, count, classify over the whole context), top-k budgets will lose tail mass (Chapter 4) and the benefit of sparsity should be re-evaluated on that task.
5. **Is the evidence adequate for the deployment context length?** Most evidence is at or below the training length; extrapolation to 10^6 tokens is a hypothesis for all but a handful of models.

14.6 Head-to-head: three representative stacks

None of the three has demonstrated dominance. The evidence supports the following more limited statements: sparse-dominant designs are the most widely replicated outside their originators (DeepSeek, Zhipu, Tencent and NaiveAI models, third-party kernels and papers such as HISA and MISA), compression-dominant designs have the largest reported cache reduction in a frontier model but a single source, and hybrid-dominant designs have the strongest reported long-context scaling at small scale (9B) but also the strongest public caution (MiniMax's reasons for M2).

	Sparse-dominant (DSA family)	Compression-dominant (DeepSeek-V4)	Hybrid-dominant (SALA, Kimi Linear)
Read per step	$O(k)$ + scan	$O(k')$ + scan of T/m	$O(1)$ state + sparse/full layers
Stored cache	full + index keys	T/m entries	state + fraction of full layers
Exact recall of any token	yes within budget	only via compressed summaries + window	only in attention layers
Training	continued pre-training (DSA); scratch (MSA)	scratch	scratch or conversion
Open evidence	V3.2, GLM-5, third-party studies	vendor report only (retrieved)	single-lab reports
Main risk	indexer cost, selection misses	summary loses details	recall limited by state

Table 14.5: Qualitative comparison of three architectural directions.

Chapter 15

Advantages, Disadvantages and Failure Modes

This chapter consolidates the strengths and weaknesses of the approaches in a form usable for a decision. Each claim is tagged with its provenance: derivation ([DER]), my simulation or measurement ([SIM]), author-reported in a retrieved source (cited), or vendor/secondary ([VENDOR], [2ND]).

15.1 Learned token- and block-level sparse attention

15.1.1 Advantages

1. **Exact softmax over a chosen subset.** The retained keys are attended exactly, so the output equals the dense output up to the discarded tail (Proposition 4.1); no recurrent state needs to remember items. For needles that are selected, recall is exact. [DER]

2. **Large reductions in bytes read at long context.** For a V3.2-style layer at 128K the bytes read per decoded token fall 7.7× (and 22.6× with the indexer shared over four layers) in the cost model of Chapter 3. [DER] Authors report kernel speedups of 7.6× decode (MSA at 1M), 11.6× (NSA at 64K) and corresponding price reductions [42, 45, 58].

3. **Small average quality change when trained with the mechanism.** Table 13.1: NSA, MoBA, DSA, CLSA and MSA each report parity or near-parity with a dense baseline on the authors' benchmarks.

4. **Compatibility with existing dense checkpoints.** DSA and CLSA add sparsity by continued training of a dense model; MoBA and InfLLM-V2 can switch between dense and sparse.

5. **Retained interpretability of the mechanism.** The selection indices can be inspected; the attention-mass captured by the selection can be measured (Table 4.5 [SIM]).

6. **Replication.** DSA has been adopted outside its originator (GLM-5/5.2, Hy4-preview, NaiveAI) and extended by third parties (HISA, MISA) [2ND].

15.1.2 Disadvantages

1. **The selector is itself a long-context operation.** A token-level indexer reads one key per token per indexer layer; at 128K it accounts for 88% of bytes read in the V3.2-

style layer (Table 3.3), and for ≈98% of the layer's bytes at 1M (Table 3.2). [DER]That is the problem that IndexCache, CLSA, HISA, MISA and compression address.

2. **Storage is not reduced.** The full cache (plus index keys) must be kept. Concurrency and capacity are unchanged unless combined with offload (SparDA, AsyncTLS), compression (CSA/HCA) or quantisation.
3. **Tail mass and its effect on the output.** The error of renormalised top-k is proportional to the discarded mass (ratio ≈ 1.8 in the small-model measurement of Table 4.5 [SIM]); on a heterogeneous model a uniform budget leaves the worst heads with low coverage (44.7% at $k = 128$ in the same measurement). Real needs that are spread over many keys (aggregation) cannot be served by small budgets. [DER][SIM]
4. **Recall depends on indexer quality.** A needle in a noisy ranking is retrieved only with probability given by Proposition 4.5; the indexer is trained to imitate dense attention (a KL objective), not task success. [DER]
5. **Block-level methods have a structural handicap.** At equal token budget block selection cannot capture more mass than token selection (Proposition 4.4); a needle among distractors in a block may not elevate the block's pooled score. This is consistent with (but does not prove) the move from NSA's blocks to DSA's tokens in the DeepSeek line. [DER]
6. **Kernel complexity.** Gather-based access has low arithmetic intensity and irregular memory patterns; reported speedups depend on bespoke kernels (Chapter 12).
7. **Training complexity.** The gradient does not pass through the discrete top-k choice; the indexer is trained by a separate distillation loss or through a gating path, with a warm-up phase (DSA) and a risk of train–inference mismatch.
8. **Evidence at the longest contexts is thin.** Most quality evidence is at or below the training length; the standard retrieval benchmarks are easy for sparse methods (Chapter 13).

15.2 Shared routing and cheaper selectors

Advantages. They address the dominant cost of token-level selection: byte reductions of m on the scan when an index is shared over m layers (Table 3.3 [DER]); the reported quality loss is negligible at $m \approx 4$ in the IndexCache and GLM-5.2 reports [2ND]; training-free variants exist (HISA, MISA, IndexCache search).

Disadvantages. Sharing assumes adjacent layers need similar tokens. The measurement on Qwen3-0.6B (Section 6.6) shows that overlap is partly produced by positions all layers attend to (sink, recent tokens); after controlling for those, the overlap is lower. Placement of shared layers is a combinatorial choice (Algorithm 1); interaction effects between nearby shared layers are not modelled. In the most aggressive designs (CLSA, HySparse), one selection serves all heads, which constrains the model to place needed information where the average attention reaches. Forecasting designs (SparDA) rely on predictions that can be wrong; AsyncTLS relies on temporal locality of the selected sets, which may not hold at topic shifts.

15.3 Hybrid sparse/linear models

Advantages. A constant-size recurrent state in most layers reduces both stored and read bytes (Table 5.2 [DER]: a 3:1 linear:sparse split stores 33 GiB at 1M tokens versus 128 GiB for the dense GQA reference in the cost model's configuration); linear layers are cheap in prefill ($O(T)$); strong ultra-long-context results from MiniCPM-SALA (RULER 128K 89.4; 1000K 86.3; 2048K 81.6) are reported by its authors; Kimi Linear reports -75% KV and $6\times$ decode at 1M [4].

Disadvantages. A finite state cannot hold arbitrary amounts of content (Proposition 5.2 [SIM]: with $n > d$ stored associations, recall cosine falls to about $1/\sqrt{n/d}$ for the additive state, and the delta rule is worse than additive beyond $n > d$ in my simulation); the systematic study of hybrids finds that retrieval-heavy tasks need more full-attention layers than language modelling [27]; MRCR-type tasks lag in SALA; MiniMax's account lists evaluation, precision and infrastructure obstacles (prefix caching and speculative decoding for recurrent backbones, memory-bound training) and a failed sliding-window conversion [5]. A hybrid is usually trained from scratch or converted with substantial tokens.

15.4 Eviction and compression of the cache

Advantages. They reduce *stored* bytes and so help concurrency; most are training-free; TriAttention and others report KV reductions of $\sim 10\times$ on reasoning traces (author-reported); TurboQuant needs no training and is data-oblivious.

Disadvantages. Eviction is irreversible; a policy that does not know the future query cannot guarantee retention of what it will need (Proposition 7.3 [DER]). In my short-context measurement an H2O-style score retained less attention mass than a plain sink-plus-window policy and than random eviction at the same budget (Table 7.4 [SIM]). Systematic evaluation finds degradation under multi-instruction or multi-turn settings [64], and the Random Attention result shows that on math reasoning a random policy rivals engineered scores [60]. Quantisation is dependent on outliers: per-token 4-bit failed on real post-RoPE keys of the small model while per-channel scales worked (Table 7.3 [SIM]); rotation-based schemes need their full pipeline to be judged.

15.5 Cross-cutting failure modes

15.6 Summary scorecard

Failure mode	Mechanism	Where it shows
Aggregation over many weak attenders	the answer depends on a sum over many tokens with small weights; top-k drops them	counting, summarisation, frequency tasks (RULER aggregation)
Non-literal retrieval	selection by lexical similarity, state blur	NoLiMa-style tasks
Multi-key / multi-hop	several needles must all be retained	multi-needle RULER, GraphWalks; CLSA's multi-needle gains came from the shared index
Distribution shift at adaptation	a model continued-trained for sparse attention regresses elsewhere	mixed short-context results (Table 8.1)
Drift between training and inference selection	the indexer chooses differently than in training	kernel numerics; low precision indexers
Degenerate attention sinks	selection must retain the sink or output changes	sink + window always included in practice
Long-horizon accumulation	small per-step errors accumulate in long generations	reasoning traces under eviction

Table 15.1: Cross-cutting failure modes of learned sparse and hybrid attention.

Approach	Read bytes	Stored bytes	Exact recall	Retrain?	Independent evidence
Learned token sparse (DSA)	$\downarrow\downarrow$	$\uparrow$ (index keys)	high within budget	continued	moderate
Learned block sparse (NSA, MoBA, MSA)	$\downarrow\downarrow$	$\approx$	medium	scratch/convert	moderate
Shared / cheaper selector	$\downarrow$ (on scan)	$\downarrow$ (YOCO-type) or $\approx$	as base	often no	limited
Compression (CSA/HCA)	$\downarrow\downarrow$	$\downarrow\downarrow$	summary-level	scratch	vendor only
Hybrid linear + sparse	$\downarrow\downarrow$	$\downarrow\downarrow$	in attention layers	scratch/convert	limited
Eviction	$\downarrow$	$\downarrow$	lost for evicted	no	mixed
Quantisation	$\downarrow$	$\downarrow$	as base, noisy	no	mixed

Table 15.2: Qualitative scorecard. $\downarrow\downarrow$ means a large reported or derived reduction, $\downarrow$ moderate, $\approx$ none, $\uparrow$ an increase. The last column is a judgement based on Chapters 13 and 16.

Chapter 16

Community Perception and Reception

This chapter describes how the field and the wider community received the developments of Chapters 8–11. **Method and limits.** The evidence is a small set of secondary sources found by web search (press articles, blog posts, a tutorial-style gallery of architectures, an ICLR-related discussion relayed by a blog) plus the critical papers in the literature itself. I did not systematically survey social-media threads, forums or conference reviews, and I do not quote anonymous commenters. Statements about "the community" are therefore statements about these sources, and each is tagged [2ND] where it relies on a non-primary account. The chapter is organised as a sequence of episodes followed by a synthesis of recurring themes.

16.1 Episode 1: DSA and the price cut

DeepSeek introduced DSA with V3.2-Exp at the end of September 2025 and announced that it would cut API prices by more than half, accompanied by the claim of parity on its benchmarks [45]. A news report listed the new per-million-token rates ($0.28 uncached input, $0.028 cached input, $0.42 output, from $0.56, $0.07 and $1.68 respectively) [2ND] [46]. The reception, as represented in these sources, treated the cut as evidence that sparse attention reduced serving cost, with the usual caveat that price is a business decision and not a cost figure (Chapter 8). The more durable signal was adoption: DSA appeared in other organisations' models (GLM-5 and GLM-5.2, Tencent's Hy4-preview, NaiveAI's N0.5-Flash) [2ND] [65], in a serving framework's documentation (the vLLM page for index caching [51]), and in follow-up research aimed at the indexer's cost (HISA, MISA, IndexCache). In other words, the field's reception of DSA is best measured by what others built on it.

Adoption has a particular character here: the DSA family is not only reused, it is *amended*. IndexCache and GLM-5.2's IndexShare remove most indexer passes, and DeepSeek-V4 compresses before indexing. I read this as evidence that the lightning indexer was accepted as the right abstraction but not as a finished design: the quadratic indexer scan became the thing to optimise within a year, as Chapter 3 predicts from the cost structure. [DER]

16.2 Episode 2: MiniMax, from lightning attention to full attention to sparse attention

MiniMax's M2 post explained a retreat from linear attention in production because of evaluation and infrastructure gaps [5]. It was widely read as an authoritative statement of the "full attention still wins" position: an independent commentary was titled to that effect [69] (only the title was retrieved [2ND]). Within months, the same lab published MSA and released

M3 with sparse attention [58, 66]. Two readings are consistent with the evidence. The charitable reading is that the post never said efficient attention was wrong; it argued that the existing linear hybrids lacked reliable evaluation at scale, and sparse attention—which keeps exact softmax over selected tokens and so avoids the evaluation and numerical-precision issues the post listed—was the more conservative step. The sceptical reading is that the cost pressure of long-context agentic workloads forced a reversal and that the M2 post was, in part, a statement about timing. The retrieved sources do not distinguish these; what they do show is that the lab's internal assessment moved within a few months, and that a public rationale existed for each position. For readers this is a lesson in the half-life of architectural conclusions in this field.

16.3 Episode 3: TurboQuant, publicity and dispute

TurboQuant was first posted in April 2025 and became widely discussed in March 2026 following a Google blog describing 3-bit KV quantisation without accuracy loss, at least 6× memory reduction on needle tasks, and an 8× logit-computation speedup on H100 [63][VENDOR]. A May 2026 blog survey recorded at least ten community implementations within a month, a measured compression of about 4.7× at 3 bits on one Apple laptop chip (less than the headline), and a serving-framework integration not yet merged [2ND] [62]. The same post relayed an accusation by the authors of RaBitQ that TurboQuant mischaracterised RaBitQ and compared against a degraded implementation, and that a complaint was reported to the venue's ethics process; Google had not publicly responded at the time of writing [2ND]. I have not independently verified these allegations and report them only as part of the reception.

Three observations follow. (i) The community response combined enthusiasm with rapid reimplementation, which is the strongest kind of independent test; the headline numbers were partly reproduced and partly reduced to more modest figures. (ii) The framing of the 8× result (against 32-bit unquantised keys in JAX) is the type of claim whose comparison basis matters more than its value (Chapter 13). (iii) The dispute concerns attribution and fair baselines, not a numerical error, which is a reminder that for compression work the baseline implementation is as important as the method. My own real-tensor experiment (Table 7.3 [SIM]) adds a data point on a different question: for post-RoPE keys of a small model, the MSE-only rotation scheme did not beat a simple per-channel scale, which shows that comparisons depend on baselines and that the effect of the full method on end tasks must be assessed with the full pipeline.

16.4 Episode 4: SubQ and the 12-million-token claim

SubQ was the most contested announcement of the period. Subquadratic emerged from stealth in May 2026 with a model claimed to use dynamic sparse attention with a 12M-token context, and published figures that included a 56× speedup, a 98% needle score at 6M and 12M tokens, and an inference cost comparison (Section 8.9 and Chapter 8) [56][VENDOR]. The reception recorded by MIT Technology Review is summarised by four reactions: a former OpenAI researcher said sparse attention had been tried widely and was very hard to make work, and that public evidence did not support a claim that the quadratic bottleneck was solved; the third-party evaluator's research director distinguished independent from

company-provided claims; critics noted the reuse of an Alibaba Qwen model's weights, which sat uneasily with a claim of a new architecture; and the company's CTO acknowledged that third-party benchmarks at launch would have reduced scepticism [56]. An aggregator headline quoting a larger, "roughly 1,000×" figure was not traceable [2ND] [67].

What the episode illustrates. The scepticism was not an objection to the possibility of sparse attention, which the same year's open work (DSA, MSA, NSA) had established, but to the *evidence standard*: undisclosed method, limited access, benchmarks (needle retrieval) that sparse methods pass easily (Chapter 4), and a headline speedup against an unspecified baseline. The criteria the sceptics invoked are the criteria of this report: trained from published recipe, comparable baselines, evaluation on tasks that stress tail mass and non-literal retrieval, independent replication. Weights reuse is not itself a defect (DSA and CLSA were adapted from dense models); the objection was to the claim that it was a fundamentally new architecture. As of the time of writing I found no independent evaluation of SubQ on RULER-type multi-key tasks or NoLiMa.

16.5 Episode 5: the critical literature on evaluation

A distinctive feature of 2025–2026 is that critical analyses appeared almost as fast as the methods. Four strands stand out.

- **Benchmarks are too easy.** RULER showed that most models claiming 32K or more did not maintain performance on harder tasks [38]; NoLiMa removed lexical overlap and found large drops [39]; the lost-in-the-middle phenomenon [37] and the comparison of retrieval-augmented and long-context approaches [40] frame the question of whether long contexts are used well even by dense models. For sparse attention the practical reading is that a method evaluated only on needle retrieval is unevaluated.
- **Sparse attention at scale is not free.** The Sparse Frontier study examined the trade-offs of sparse attention for long-context LLMs and concluded that sparse patterns' benefit depends on the task and phase (prefill versus decode) [41].
- **KV compression pitfalls.** A systematic evaluation found that compression methods degrade in multi-instruction prompting and similar settings [64].
- **Random Attention.** A September 2026 paper reported that on reasoning traces a random selection of retained tokens can rival engineered scores; its authors take this to mean that the benchmarks and the redundancy of reasoning traces, not the sophistication of the scoring, drive the results [60]. Whatever one thinks of the specifics, this paper changed the burden of proof: new eviction methods must show a gap over random.

16.6 Themes

1. **Open weights and open reproduction convert claims into evidence.** The most credible results are those that outsiders could re-run (DSA models and follow-ups; reproductions of TurboQuant), and the least credible are those that cannot be accessed (SubQ).

Bet	Perception (as represented in sources)	Basis
Learned token-level sparse attention (DSA family)	accepted and spreading; the indexer cost is the main complaint	adoption by several organisations; IndexCache/HISA/MISA
Block-level learned sparse (NSA, MoBA, MSA)	accepted, used in production in some labs (Kimi; MiniMax-M3)	MoBA deployment claim; MSA and M3
Hybrid linear (Kimi Linear, SALA, Qwen3-Next)	promising, with public caution	M2 post; SALA's MRCR gap; limited independent evaluation
Compression plus selection (DeepSeek-V4)	too recent for much independent assessment in retrieved sources	single technical report
Training-free eviction	sceptical after Random Attention and Pitfalls	critical papers
Data-oblivious quantisation (TurboQuant)	enthusiastic adoption; contested attribution; headline numbers moderated	community implementations; dispute
Vendor sparse-attention claims at 12M (SubQ)	unproven	press coverage

Table 16.1: Judged perception of technical bets, with the basis for each judgement.

2. **Vendor speedups are read sceptically.** The community pattern in the sources is to ask for the baseline (32-bit JAX; FlashAttention; dense with offload) and the level (operator or end-to-end). The comparison basis column of Table 13.3 is the practical encoding of this reading.
3. **Architecture changes quickly and reversals are normal.** MiniMax's three architectures in about a year, and DeepSeek's move from selection (V3.2) to compression plus selection (V4), indicate that no design is settled. The period's lesson is to treat conclusions about which architecture "wins" as provisional.
4. **Attention to cost structure.** The rapid emergence of indexer-focused papers after DSA shows that practitioners computed the cost structure and found the scan.
5. **Concern about what benchmarks measure.** The consistent theme in the critical literature is that long-context claims need multi-key, non-literal and aggregative tasks and independent evaluation.

16.7 Perception of the technical bets

Table 16.1 summarises, as a judgement from the sources above, how each bet appears to be regarded at the end of October 2026. These are my assessments, not measurements of opinion.

Chapter 17

Case Studies

Each case study follows the same template: what was claimed, what the evidence is and where it comes from, what the cost model and measurements of this report say about it, and what remains unverified. All numbers attributed to sources are as retrieved; computations marked [DER] or [SIM] are mine.

17.1 Case 1: DeepSeek V3.2-Exp to V3.2 to V4

Claim. DSA lowers long-context cost with negligible quality change (V3.2-Exp, September 2025) [45]; a production model with DSA is competitive with frontier systems (V3.2, December 2025) [3]; and a further design with compression in front of the indexer reduces cost and cache again by large factors (V4, 2026) [59].

Evidence. The V3.2-Exp model card gives paired benchmarks (Table 8.1): on the tabulated tasks the continued-trained sparse model is higher on some and lower on others by up to a few points, with no systematic direction. The technical report adds a cost-per-million-tokens plot from 0 to 128K on H800s [3] and the price reduction [45, 46]. For V4 the evidence is a technical report with FLOP and cache ratios relative to V3.2 (V4-Pro: 27% of single-token FLOPs and 10% of KV cache at 1M) [59].

What the cost model says. For a V3.2-style layer (Table 3.2) at 128K tokens, the sparse path reads 19.7 MB per decoded token against 151.0 MB dense, a 7.7× reduction *in bytes*, of which 17.3 MB (88%) is index-key traffic. At 1M the reduction is 8.6× and the indexer is 98% of the bytes. Hence the cost curve of a DSA model is, to first order, linear in T with a smaller slope rather than flat, and the structure of the V4 design is predictable: the only ways to bend the curve further are to scan fewer keys (compress by m) and to store fewer bytes per key (FP4 index keys, which V4 does). [DER] V4's reported 10% cache relative to V3.2 and 27% FLOPs are of the order such changes give: compression by $m = 4$ on CSA layers and $m' = 128$ on HCA layers together with narrower storage (see the consistency check in Chapter 8); the 27% FLOPs figure indicates that compute savings are less than cache savings, as expected when the MoE feed-forward cost is unchanged.

What is not verified. (i) The V3.2 comparison is internal. (ii) The cost figures are model-level at the vendor's chosen hardware and utilisation; price is not cost. (iii) V4's long-context *quality* was not available to me beyond the report's summary statements, and its arXiv identifier and date are inconsistent. (iv) No independent evaluation of V4 was retrieved.

Lesson. The sequence shows the characteristic dynamics: a mechanism that replaces an $O(T^2)$ cost with an $O(T)$ scan within 12 months is itself compressed or shared, and adoption outside the lab (GLM-5 family, Hy4-preview) is the best evidence that the first step was sound.

17.2 Case 2: MiniMax, M2 to M3

Claim. Lightning-attention hybrids in MiniMax-01 (4M-token inference context) [24]; then M2 with full attention because evaluation and infrastructure for efficient attention were immature [5]; then MSA and M3 with block-sparse attention (128-token blocks, 16 selected; 2048-token budget) [58, 66].

Evidence. The M2 post reports no numbers for its own models but describes a method (proxy metrics for multi-hop reasoning) and obstacles. MSA reports on a 109B MoE trained for 3T tokens with 28.4× lower attention compute and 14.2×/7.6× prefill/decode wall-clock speedups at 1M on H800 [58]. The production model's dimensions come from a secondary source [2ND].

Analysis. Block-sparse attention addresses the M2 post's three obstacles directly. Evaluation: because the attention is exact over the selected blocks, quality is a function of captured mass, which is measurable (Chapter 4), and the failure surface is the same as the dense model's plus selection misses. Numerical precision: softmax over selected keys carries the same numerics as dense attention. Infrastructure: prefix caching and speculative decoding for a stateless-cache model are those of existing attention engines, with the addition of index keys. These are the reasons for which a lab that rejected linear hybrids might adopt sparse attention without contradicting itself; the retrieved sources do not state the lab's reasoning, so this is an interpretation. [DER]

Cost-model prediction. At 1M tokens and 128-token blocks with an index key per block (if the MSA design pools keys into one vector per block; I assumed one index head per GQA group), the scan is $T/128 \approx 7800$ keys, about 128× fewer than a token-level scan; with 16 blocks of 128 tokens the data read is the same 2048 tokens as DSA. For the hybrid cost table (Table 5.2) the block-idx column makes the same point: at 1M tokens the decode bytes of a 1:1 sparse:linear hybrid fall from 2366 MB to 186 MB per token when the index scan moves from tokens to 64-token blocks. [DER]The price is the block-granularity handicap (Proposition 4.4).

Not verified. Quality of M3 relative to M2 on long-context tasks; the independent speedup in serving.

17.3 Case 3: MiniCPM-SALA, a sparse–linear hybrid at 9B

Claim. A 9B hybrid with 25% sparse and 75% linear layers converted from a dense checkpoint at about a quarter of the training cost achieves RULER 128K of 89.37 against 71.74

Standard	SubQ (as retrieved)	Comparison: DSA / MSA / CLSA
Method disclosed	no detail on selection	arXiv technical reports
Weights or kernels available	waitlist	open weights (DSA), reports with kernels
Baseline specified	56× vs "FlashAttention-based" (theoretical)	hardware and baseline stated
Quality evidence beyond needle	LiveCodeBench 89.7% (company)	RULER/LongBench/ablations
Independent evaluation	one (Appen), needle and speed	third-party papers (HISA, MISA)
Cost claim basis	company figure	GPU-hour accounting in report

Table 17.1: SubQ against the evidence standards applied to the other methods in this report.

for Qwen3-8B, NoLiMa 128K 23.86 against 11.25, up to 3.5× faster than Qwen3-8B at 256K, and supports 1M and 2M-token contexts where the dense baseline runs out of memory [50].

Evidence and caveats. Everything is author-reported, with author-run baselines. Standard benchmark averages are comparable across models trained on different data. On MRCR the model trails the best baseline. The scaling to 2048K (RULER 81.6) is the largest-context result in the retrieved literature that is accompanied by a number rather than only a needle test.

Cost-model check. In my hybrid cost model, a 32-layer model with one sparse layer in four stores ≈33 GiB at 1M tokens (idealised token-index layout) against 128 GiB for dense GQA of similar dimensions (Table 5.2). A 4× reduction in stored cache and about 4× in layers that scan is what the architecture gives before any sparse read; the reported 3.5× speedup at 256K is of the order the cost model suggests for a bandwidth-bound decode with prefill included. [DER]A reproduction would need identical serving stacks, which I could not check.

Reading. The informative results are the NoLiMa and MRCR contrast: non-literal retrieval benefits from the hybrid (consistent with retrieval being concentrated in few attention layers), multi-round co-reference does not. This is the profile predicted by the finite-state analysis of Chapter 5: tasks needing many precisely separated items stress the state, tasks needing one or a few items do not. [DER]

17.4 Case 4: SubQ

Claim. Dynamic sparse attention with 12M-token context, 56× faster than FlashAttention-based models, 98% needle recall at 6M and 12M, RULER-128 for $8 against $2,600 on a frontier closed model [56][VENDOR].

Assessment against the report's standards.

What the claims would imply. A 12M-token cache for a model with GQA-8, $d = 128$ and 36 layers would be 12M × 147 KB ≈1.7 TB in BF16 (Table 3.1 gives the per-token bytes; [DER]), far above a single GPU, so any such deployment needs either offload, compression, a small number of cached layers, or a hybrid. A selector scanning 12M index keys per decode step would itself read of the order of 1.6 GB (132 B/key) per layer, so the 56× speedup is unlikely to be a speedup of an end-to-end decode step relative to dense attention; the article describes it as theoretical. The cost model therefore says which questions to ask: where is the cache stored, how is the selector amortised, and what is the baseline.

Status. Unverified. Needle results at 12M are compatible with very low coverage of the attention mass (Chapter 4); only multi-key and aggregation tasks would differentiate.

17.5 Case 5: TurboQuant

Claim. Data-oblivious 3-bit KV quantisation with no accuracy loss, ≥4.5× (paper) or ≥6× (blog) memory reduction, up to 8× logit speedup [36, 63].

Verified here. The synthetic experiments of Chapter 7 reproduce the core statistical statements for Gaussian-like vectors: random rotation yields coordinates close to Gaussian, the Lloyd–Max MSE at 1–4 bits is 0.3606, 0.1158, 0.0340, 0.00938 (consistent with the known distortion of optimal scalar quantisers for a Gaussian source, which I computed, not copied), rotation improves the MSE on outlier-heavy inputs by 11×, 6.9×, 5.0× at 2, 3 and 4 bits, MSE-only quantisation produces biased inner products with slope 0.885, 0.965, 0.991 (shrinkage), and the QJL residual removes the bias at the price of 2–2.4× higher RMSE in my implementation [SIM].

Not reproduced. The end-task claims (LongBench, needle) were not reproduced. The real-tensor test on a 0.6B model shows that MSE-only rotation to 3–4 bits is better than per-token min–max but worse than per-channel min–max in attention-output error on post-RoPE keys (Table 7.3), which does not contradict the paper (a different pipeline, key placement and model) but shows the result is baseline-dependent.

Memory arithmetic. The accounting of ≥4.5× vs ≥6× is mostly metadata: 3.5 bits per channel with outlier handling is 16/3.5 ≈ 4.6×; 3 bits without metadata would be 5.3×, and the blog's ≥ 6× would need below 2.7 bits per entry; the reported measurement of 4.7× at 3 bits on one laptop is consistent with 16/3.4 (3 bits + norm/metadata) [2ND]. [DER]

17.6 Case 6: GLM-5.2 and IndexShare

Claim. A production model (GLM-5.2) with DSA where three of every four layers reuse the previous layer's indices (`full, shared, shared, shared`), with a reported 2.9× reduction in per-token FLOPs at 1M and negligible quality loss in the IndexCache study on a 30B DSA model (75% of indexer passes removed; 1.82× prefill, 1.48× decode) [51, 68][2ND].

Analysis. The cost model gives the expected effect: sharing over $m = 4$ reduces the scan to a quarter and lifts the byte reduction at 128K from 7.7× to 22.6× for a V3.2-like layer (Table 3.3 [DER]). The reported 1.48× decode speedup at 200K is much smaller than the 2.9× ratio (22.6/7.7) of the attention-byte model for a single layer because of everything outside attention-index traffic (MoE experts, weights, communication). An Amdahl-style bound: if the sparse layer's attention is a fraction f of decode time before sharing and falls by 2.9×, the end-to-end speedup is $1/(1 - f + f/2.9)$; setting this equal to 1.48 gives $f \approx 0.49$, i.e. about half of decode time attributable to attention in that configuration. This is an inference from two numbers drawn from different sources and settings, and is offered only to show the consistency of the claims with a plausible time split. [DER]

Cross-layer overlap. The secondary report of 70–100% top-k overlap between adjacent layers is a measurement that I tried to cross-check on a different model; see Section 6.6. My result is lower once the shared sink and local window are removed, which is no contradiction (small model, short context, head-averaged attention), but it shows that overlap figures need a stated control.

17.7 Cross-case lessons

1. **The cost model predicts the next move.** In each case the observed follow-up design (IndexShare, CSA, block-sparse index) is the one the byte accounting suggests.
2. **Parity claims are comparison-dependent.** The baselines range from the same lab's previous model to a different architecture on other data.
3. **Operator-level speedups are not end-to-end.** The Amdahl check in Case 6 is a template for reading others.
4. **Independent replication is rare and valuable.** Third-party sparse-indexer papers and TurboQuant reimplementations carry more weight than any number in a launch post.
5. **Absence of detail is information.** In the SubQ case the lack of a method description is the reason the claim cannot be assessed.

Chapter 18

Open Problems

The problems below follow from gaps identified in Chapters 4–17. For each I state the question, why current evidence does not answer it, what a decisive experiment would be, and (where I can) a partial derivation or measurement from this report. Items are my own synthesis; none is claimed as an established open problem in the cited papers unless said.

18.1 Evaluation

P1. Benchmarks that discriminate sparse from dense at long context. Needle tests are passed by methods with low mass coverage (Chapter 4); standard average-score benchmarks have differences within noise (Chapter 13). Needed: a benchmark family with controlled *tail-mass* demand, i.e. tasks whose answer requires r tokens each with small attention weight, with r as a dial, and non-literal retrieval at contexts of 10^5–10^6 tokens. The mass-based analysis gives a prediction to test: error scales with discarded mass (ratio near 1.8 in Table 4.5), so a task with aggregation over r items should degrade when kr/T-type budgets are exceeded. [DER]

P2. Independent 1M-token evaluation. Every long-context quality claim at $\geq$1M tokens in the retrieved material is by the model's authors or vendor. A standing third-party evaluation with fixed harness and public prompts, on at least RULER-style multi-key tasks, NoLiMa-style non-literal retrieval and multi-round co-reference, would resolve several disputes at once (SALA's MRCR gap, SubQ, V4).

P3. Statistical power. Few sources report seeds or confidence intervals. How large are seed-to-seed differences for sparse-trained models at 1–10B scale, and are the reported parity results beyond them? A set of 5 seeds at 1B scale for NSA, DSA-style and dense models would calibrate the field's noise floor.

18.2 Mathematics and theory

P4. Gradient flow through selection. Top-k is piecewise constant in the scores; the index set changes discontinuously. DSA trains the indexer by KL imitation of dense attention rather than by the language-modelling loss, GSA by a gate, NSA by a compressed branch. Which objective yields the best selection for the final loss? A theory of selection learning would relate the KL-optimal indexer (Proposition 4.6) to the loss-optimal one; they coincide only if the output error is monotone in captured mass, whereas the real-model measurement shows the error is proportional to tail mass with a constant depending on value coherence ($\approx$1.8 in Qwen3-0.6B). A selection criterion that includes $\|v_i - \bar{o}\|$ (value-aware) would differ

from attention-mass selection; whether value-aware indexers help is untested in the retrieved literature. [DER] [SIM]

P5. Tight bounds for hierarchical and shared selection. Proposition 6.1 bounds the loss from sharing by total variation, which is loose. A bound in terms of measurable layer-wise statistics (the generalised Jensen–Shannon divergence of (6.4)) that predicts when sharing is free would let one choose sharing groups without end-to-end evaluation.

P6. Capacity of hybrid states. Proposition 5.2 and the simulation show the cosine of recall decaying as $1/\sqrt{1+(n-1)/d}$ for random keys. Trained keys are not random: they can be arranged to be nearly orthogonal. What is the achievable recall capacity of a trained gated-delta layer as a function of head dimension and number of heads, and how does it compare with the information-theoretic ceiling of the state size? Controlled synthetic recall tasks (multi-query associative recall) can measure this, but the link from capacity to downstream task performance is not established.

P7. Optimal layer ratio and placement in hybrids. The 72-model study suggests 3:1 to 6:1 linear-to-full ratios give Transformer-level recall at $\leq$1.3B parameters [27]; SALA uses 3:1 linear:sparse at 9B. How does the optimal ratio depend on scale, on sparse versus full attention as the exact layers, and on context length? With selection cost included, sparse layers are cheaper than full layers, so the optimum should shift toward more exact layers; no study tests this.

18.3 Training

P8. Training sparse from scratch versus adapting. DSA and CLSA adapt dense models; MSA, V4, NSA and GSA train sparse natively. A matched comparison at a fixed budget (same data, same compute) of native sparse, adapted sparse and dense would settle whether adaptation leaves quality on the table. No such matched comparison was retrieved.

P9. Train–inference mismatch. Low-precision (FP8/FP4) indexers, kernel nondeterminism and tie-breaking mean the inference-time selection differs from the training-time selection. The effect on long reasoning traces and RL fine-tuning is not characterised in retrieved sources; MiniMax's account of precision problems in RL training for linear attention (Chapter 10) is a related caution.

P10. Reinforcement learning with sparse attention. Agentic and reasoning post-training generate long traces; rollouts and trainer must use the same attention to avoid off-policy errors. Whether sparse selection is stable under policy-gradient training is unexamined here.

18.4 Systems

P11. Composition of quantisation and selection. Quantised keys shift the logits used for ranking; by the noisy-indexer analysis (Proposition 4.5) the budget needed for a given recall rises with noise. A systematic grid (bits × budget × indexer precision) on a trained sparse model would quantify the interaction. My real-tensor experiments (Table 7.3) measure only attention-row KL and output error without a learned indexer.

P12. Multi-turn and agentic cache policies. Eviction decisions that are optimal for one query are wrong for the next (Proposition 7.3). Selection with a full cache is safe but memory-hungry; offload designs (SparDA, AsyncTLS) store the cache off-GPU. What is the best policy when the cache must persist across turns, tool calls and sessions, with prefix caching? The metrics should be end-to-end task success across turns, not retained mass.

P13. Speculative decoding with sparse and recurrent layers. Multi-token verification asks several queries to attend with a shared state: for sparse layers the union of selected tokens grows with the number of queries, reducing the sparse advantage; for linear layers the verification needs state snapshots or rollback. The analyses of Chapter 3 suggest the bytes advantage shrinks as verification width grows, but I have no measurement. [DER]

P14. Prefix cache cost for hybrids. Prefix caching stores the state at each cached prefix boundary; for linear-state layers the state is of fixed size per boundary, so the storage per cached prefix is large relative to the KV of a short prefix. The trade-off between the number of snapshots and recompute cost is a scheduling problem without published solutions in the retrieved material.

P15. Kernel portability. Reported kernel speedups are tied to specific hardware (H800, H200, A100, A6000-class); the gather and top-k stages may behave differently on different accelerators or with different memory hierarchies. A cross-hardware benchmark of sparse-attention kernels is missing.

18.5 Scientific questions about the models

P16. Why are trained attention maps so compressible? Real attention concentrates strongly (Table 4.5: 128 keys capture 92.6% of the mass at 1K–2K context in a small model), with sink and local structure that depends on the layer (Figure 4.5). Does the coverage fall with context length as the Gaussian-logit scaling (Proposition 4.3) suggests, or do models learn to preserve coverage at long context? CLSA's coverage figures at 32K (76–84% at 2048 tokens) suggest partial decay. A scaling study of coverage versus context length at fixed budget for a trained long-context model is a clean experiment.

P17. What do retrieval heads need? Retrieval behaviour appears to be carried by a small number of heads [33]. If so, a hybrid or sparse design could preserve exact full attention for those heads only (DuoAttention's idea [32]) and compress the rest. How do learned sparse and hybrid designs differ from this head-level view?

P18. Aggregative tasks. Counting, summarisation and frequency estimation over a long context depend on many weak attenders. Do sparse models fail on them in proportion to the discarded mass, and can a small number of summary tokens (compression layers as in V4) restore them? This is where HCA's dense attention over heavily compressed entries should help; V4's reports do not isolate it in the retrieved text.

18.6 Claims that would change this report's conclusions

- An independent evaluation showing SubQ-like systems pass RULER multi-key and NoLiMa at $\geq$1M would strengthen the case for 10M-scale sparse contexts; failure would support the sceptics.
- A matched-compute study showing that native-sparse training clearly beats dense-to-sparse adaptation would change recommendations for labs with existing checkpoints.
- A demonstration that shared indices cause failures on agentic traces (tool calls scattered across long contexts) would qualify the IndexShare results.
- A published, reproducible end-to-end serving comparison of DSA-with-sharing, V4-style compression and a hybrid at 1M tokens on the same hardware.

Chapter 19

Conclusion

The period from October 2025 to October 2026 changed learned sparse attention from a research proposal into a component of deployed long-context models. This report has examined that transition with a cost model, derivations, simulations on synthetic and real tensors, and a review of retrieved sources with their evidential status marked.

19.1 What can be said with confidence

1. **Decode is a bytes problem and bytes come in two kinds.** The speedup of any attention variant at decode is bounded by the ratio of bytes read; capacity depends on bytes stored. Learned sparse attention reduces the former, not the latter, and adds index keys to the latter (Chapter 3, Table 3.2). [DER]
2. **The indexer becomes the bill.** In a V3.2-style layer at 128K the index scan is 88% of the bytes read; the 2026 designs (IndexCache, CLSA, HISA, MISA, CSA) all attack that cost. Sharing one scan over four layers raises the byte reduction from 7.7× to 22.6× (Chapter 6). [DER]
3. **Top-k error is governed by discarded mass and by value coherence.** The error of renormalised top-k equals the tail weight times the distance between tail and head averages (Proposition 4.1). On a real small model the output error was about 1.8× the tail mass over two orders of magnitude of budget, and the coverage of a fixed budget was strongly heterogeneous across heads. [DER] [SIM]
4. **Block selection cannot beat token selection at equal token budget,** and a noisy indexer needs a margin to retrieve a needle (Propositions 4.4 and 4.5). [DER]
5. **Finite-state layers trade recall for memory.** The recall cosine of an additive state decays as $1/\sqrt{1+(n-1)/d}$ for random keys, and hybrids preserve exact recall in the attention layers. [DER] [SIM]
6. **Trained sparse and hybrid models match dense ones on average benchmarks in the author-reported comparisons** at the scales and context lengths tested, with the stronger caveats about aggregation, non-literal retrieval and extrapolation to 10^6 tokens (Chapter 13).
7. **Eviction and quantisation are conditional.** Eviction is irreversible and its heuristics are fragile; quantisation depends on outlier structure (Chapters 7 and 11; Tables 7.3 and 7.4). [SIM]

19.2 What the evidence does not settle

- Whether any claimed end-to-end speedup holds at ≥1M tokens on independent systems.

- Whether learned sparse, compression or hybrid is the better route; each has strengths, and the design space is converging on combinations (sparse + sharing + compression; hybrids with sparse layers).
- The quality of V4, M3, SubQ and 10M-token claims on discriminating tasks.
- The interaction of quantisation and selection, and of sparse attention with speculative decoding, prefix caching and reinforcement learning.

19.3 Practical guidance

For a practitioner: measure the byte split of the target workload before choosing; if decode latency at long context dominates and retraining is possible, a learned token-level selector with shared or hierarchical scan is the best-supported design; if capacity is the constraint, combine with compression or quantisation after testing the combination; if retraining is impossible, training-free selectors and per-channel quantisation are the safe first steps; for any deployment, evaluate on multi-key, non-literal and aggregative tasks, not on needles, and treat vendor speedups according to their baseline and level.

19.4 Limitations of this report

(i) Most of the empirical content is from abstracts and full-text summaries retrieved by web search, not from re-running the systems; several retrieved summaries contained internal inconsistencies, which are noted. (ii) My experiments use a 0.6B model, 2048-token windows, one text genre and a CPU; they illustrate and test the mathematics but do not measure frontier-scale behaviour. (iii) Community perception is drawn from a small set of secondary sources. (iv) Cost-model results depend on assumptions about layouts (e.g. index-key bytes) flagged where used. (v) Dates of preprints and versions are as listed in the retrieved pages.

Appendix A

Extended Derivations

This appendix collects derivations that are quoted in the main text without full proofs, or that are used to generate the tables. Notation follows Chapters 3 and 4.

A.1 Arithmetic intensity of decode attention

Consider one decoder layer with H query heads, G KV heads (so H/G query heads share each KV head), head dimension d, context length T and b bytes per cached element. For one decoded token and dense attention:

$$\begin{aligned}\text{FLOPs} &= \underbrace{2HTd}_{qK^\top} + \underbrace{2HTd}_{\text{weights}\cdot V} = 4HTd \quad \text{(softmax work is } O(HT)\text{, neglected)},\\ \text{Bytes read} &= \underbrace{GTd\,b}_{K} + \underbrace{GTd\,b}_{V} = 2GTd\,b.\end{aligned}$$

Hence the arithmetic intensity is

$$I = \frac{4HTd}{2GTdb} = \frac{2H}{Gb}\ \text{FLOP/byte}, \tag{A.1}$$

independent of T and d. For $H/G = 4$ and BF16 ($b = 2$), $I = 4$ FLOP/B. With a device of peak compute P and memory bandwidth W, the roofline ridge point is P/W; for $P = 989$ TFLOP/s (BF16 dense, H100 SXM) and $W = 3.35$ TB/s, $P/W \approx 295$ FLOP/B. Decode attention is therefore about 74× below the ridge for GQA-4: it runs at roughly $I{\cdot}W/P \approx 1.4\%$ of peak compute, and the time per layer is $2GTdb/W$ to a good approximation. For MLA in absorbed form the same count applies with $G = 1$, $d = c_{KV} + c_R$ for the keys and $d_v = c_{KV}$ for values, with $H = 128$, so that the FLOPs per token per layer are $HT(2(c_{KV}+c_R)+2c_{KV})$ and the bytes $T(c_{KV} + c_R)b$; with $H = 128$, $c_{KV} = 512$, $c_R = 64$, $b = 2$ this is $I \approx 242$ FLOP/B (matching the 36.5 GFLOP and 151 MB of Table 3.2 at 128K), close to the ridge, so MLA decode approaches the ridge; this is the reason for the remark in Chapter 3 that for MLA both FLOPs and bytes matter. These values are device specifications used for illustration; the report does not claim measured utilisation. [DER]

A.2 Amdahl bound for end-to-end speedup

Let a fraction f of decode time be spent in the part that a method accelerates by a factor s. The end-to-end speedup is

$$S = \frac{1}{(1-f) + f/s}, \qquad S < \frac{1}{1-f}. \tag{A.2}$$

For $f = 0.5$ no operator speedup can give more than 2× end-to-end; for the Case 6 numbers of Chapter 17 ($S = 1.48$, $s = 2.9$) one solves $f = (1 - 1/S)/(1 - 1/s) = 0.324/0.655 \approx 0.49$.

In a batch setting, f itself depends on the batch size and context length: weights are read once per step independent of the batch (so their share falls as the batch grows), while the KV traffic scales with the batch. Hence the benefit of reducing KV traffic increases with batch size and context length.

A.3 Throughput under a capacity limit

Let device memory be M, weight bytes W_m, cache bytes per sequence $C(T)$. The feasible batch is $B^\star = \lfloor (M - W_m)/C(T) \rfloor$. If the step time is bandwidth-bound, $t_{\text{step}} = (W_m + B\,R(T))/\beta$, where $R(T)$ are the bytes read per sequence per step and β the bandwidth, the throughput in tokens/s is

$$\Theta(B) = \frac{B}{t_{\text{step}}} = \frac{\beta B}{W_m + BR(T)} \xrightarrow{B\to\infty} \frac{\beta}{R(T)}, \qquad B \le B^\star. \tag{A.3}$$

Θ is increasing and concave in B: when $B^\star R(T) \ll W_m$ (weights dominate) throughput is proportional to $B^\star$, hence to $1/C(T)$; when $B^\star R(T) \gg W_m$ it saturates at $\beta/R(T)$. This yields two regimes for any KV method. Reducing C (stored bytes) helps in the first regime, reducing R (read bytes) helps in the second. Learned sparse attention without compression changes only R and so helps only after enough sequences fit. CLSA/YOCO changes C by $L/2$ and R simultaneously, which explains why a combined architecture can show a larger reported throughput gain than either mechanism alone. [DER]

A.4 Euler–Maclaurin for Zipf-distributed attention

Let the sorted attention weights be $p_i = i^{-\alpha}/Z_T$, $i = 1, \dots, T$, with $Z_n = \sum_{i=1}^{n} i^{-\alpha}$. The mass of the top k is $m_k = Z_k/Z_T$. For $\alpha \neq 1$ the Euler–Maclaurin formula gives

$$Z_n = \frac{n^{1-\alpha}}{1-\alpha} + \zeta(\alpha) + \frac{1}{2}n^{-\alpha} + \frac{\alpha}{12}n^{-\alpha-1} + O(n^{-\alpha-3}), \tag{A.4}$$

where ζ is the Riemann zeta function, continued analytically for $\alpha < 1$. *Derivation.* Apply $\sum_{i=1}^{n} f(i) = \int_1^n f + \frac{1}{2}(f(1)+f(n)) + \frac{1}{12}(f'(n) - f'(1)) - \cdots$ with $f(x) = x^{-\alpha}$: the integral is $(n^{1-\alpha}-1)/(1-\alpha)$; collect the constants from the lower limit into $C(\alpha) = \lim_n \left(Z_n - \frac{n^{1-\alpha}}{1-\alpha}\right)$. For $\alpha > 1$ the limit of Z_n is $\zeta(\alpha)$ and the first term vanishes, so $C(\alpha) = \zeta(\alpha)$; for $0 < \alpha < 1$ the identity $C(\alpha) = \zeta(\alpha)$ follows by analytic continuation of the Dirichlet series (the difference is analytic in α and agrees on $\alpha > 1$). Dropping terms beyond $\frac{1}{2}n^{-\alpha}$ reproduces the simulated mass to three digits in Table 4.1 [SIM], including $\alpha > 1$ where my first closed form (an integral approximation without the ζ constant) failed. As $T \to \infty$ with $\alpha > 1$,

$$1 - m_k \;\to\; 1 - \frac{Z_k}{\zeta(\alpha)} \;\approx\; \frac{k^{1-\alpha}}{(\alpha-1)\zeta(\alpha)},$$

so the tail mass decays polynomially in the budget but does *not* depend on T: for a Zipf exponent above one, a fixed budget captures a fixed fraction regardless of context length. For $\alpha < 1$ the mass is $m_k \approx (k/T)^{1-\alpha}$, which tends to zero at fixed k. The Gaussian-logit model of the next section interpolates between these behaviours. [DER]

A.5 Mass of the top fraction under Gaussian logits

Assume the attention logits are i.i.d. $N(0, \sigma^2)$ (for a fixed query, after subtracting a constant that cancels in the softmax) and let $T \to \infty$. The weights are proportional to e^{s_i}. The top fraction q of positions by logit is the set $\{s \geq \sigma z_q\}$ with $z_q = \Phi^{-1}(1-q)$. The captured mass converges to

$$m(q) = \frac{\mathbb{E}[e^{\sigma Z}; Z \geq z_q]}{\mathbb{E}[e^{\sigma Z}]} = \frac{e^{\sigma^2/2}\ \Pr(Z + \sigma \geq z_q)}{e^{\sigma^2/2}} = \Phi(\sigma - z_q),$$

using $\mathbb{E}[e^{\sigma Z}\mathbf{1}\{Z \geq z\}] = e^{\sigma^2/2}\Pr(Z \geq z - \sigma)$ (completing the square). This is Proposition 4.3. For fixed budget k and context T, $q = k/T$, and $z_q \approx \sqrt{2\ln(T/k)}$ for small q, so $m \approx \Phi(\sigma - \sqrt{2\ln(T/k)})$: the captured mass falls as T grows, with the rate set by σ (the spread of logits). The Gaussian model is a model, not a fit; in the real model the relevant quantity is the empirical m_k of Table 4.5. [DER]

A.6 Error of random-value truncation

Let the values be independent random vectors with $\mathbb{E}v_i = 0$ and $\mathbb{E}v_i v_i^\top = \sigma_v^2 I$. The error of renormalised truncation to S is

$$o - \hat{o}_S = \sum_{i \in S}\Big(p_i - \frac{p_i}{m}\Big)v_i + \sum_{i \notin S} p_i v_i = -\frac{\tau}{m}\sum_{i \in S} p_i v_i + \sum_{i \notin S} p_i v_i,$$

using $1 - 1/m = -\tau/m$. By independence,

$$\mathbb{E}\|o - \hat{o}_S\|^2 = \sigma_v^2\Big[\sum_{i \notin S} p_i^2 + \Big(\frac{\tau}{m}\Big)^2 \sum_{i \in S} p_i^2\Big], \tag{A.5}$$

which is eq. (4.6). Two consequences: (i) the error depends on the ℓ_2 norm of the weights, not only on the tail mass τ; a tail made of many small weights contributes $\sum p_i^2 \leq \tau \max_{i \notin S} p_i$, which is far smaller than τ^2 when the tail is spread; (ii) when values are *not* independent but share a common direction, the first sum is replaced by $\|\sum_{i \notin S} p_i v_i\|^2 \approx \tau^2 \|\bar{v}_{S^c}\|^2$ and the error is of order τ, which is what the real-model measurement shows (ratio ≈ 1.8). The truncation identity $o - \hat{o} = \tau(\bar{o}_{S^c} - \hat{o}_S)$ holds in both cases. [DER]

A.7 KL divergence from logit noise

If logits are perturbed by δ_i, the perturbed distribution is $p'_i = p_i e^{\delta_i}/Z$ with $Z = \mathbb{E}_p[e^\delta]$. Then

$$\mathrm{KL}(p\|p') = \sum_i p_i \log \frac{p_i}{p'_i} = \log \mathbb{E}_p[e^\delta] - \mathbb{E}_p[\delta] = \tfrac{1}{2}\mathrm{Var}_p(\delta) + O(\kappa_3), \tag{A.6}$$

since $\log \mathbb{E}e^\delta$ is the cumulant generating function of δ under p at 1, whose expansion is $\kappa_1 + \kappa_2/2 + \kappa_3/6 + \cdots$. Note that only the *p-weighted variance of the noise* enters: noise on negligible-weight keys does not matter to second order, and a constant shift of all logits has no effect. For a quantised key $\hat{k}_i = k_i + e_i$ with quantisation error e_i independent of the query, $\delta_i = q^\top e_i/\sqrt{d}$, so $\mathrm{Var}_p(\delta)$ is governed by the error in the directions where the query is large. This explains why per-channel scales (which reduce error in the outlier channels that dominate $q^\top k$) beat uniform per-token scales in Table 7.3. [DER]

A.8 Lloyd–Max quantisation of a Gaussian source

For a scalar random variable $X \sim N(0,1)$ and a 2^b-level quantiser with decision boundaries t_j and reconstruction levels c_j, the mean-squared error is minimised by the fixed point of

$$t_j = \tfrac{1}{2}(c_{j-1} + c_j), \qquad c_j = \mathbb{E}[X \mid t_j \le X < t_{j+1}],$$

which Lloyd's algorithm iterates. At high resolution the Panter–Dite formula gives the distortion of the optimal fixed-rate quantiser of a density f:

$$D \approx \frac{1}{12\,N^2}\Big(\int f(x)^{1/3}dx\Big)^3, \qquad N = 2^b.$$

For $f = N(0,1)$, evaluating the Gaussian integral gives $\big(\int f^{1/3}dx\big)^3 = 6\sqrt{3}\,\pi \approx 32.65$ (checked numerically), so

$$D \approx \frac{6\sqrt{3}\,\pi}{12}\,2^{-2b} = \frac{\sqrt{3}\,\pi}{2}\,2^{-2b} \approx 2.72 \cdot 2^{-2b}. \tag{A.7}$$

My numerical Lloyd–Max values (0.3606, 0.1158, 0.0340, 0.00938 for $b = 1, \dots, 4$) are below this asymptote at low b (0.0425 at $b = 3$, 0.0106 at $b = 4$), as expected for an approximation valid as $b \to \infty$, and approach it from below as b increases: ratios 0.80 ($b = 3$) and 0.88 ($b = 4$). After a random rotation of a unit vector, each coordinate is approximately $N(0, 1/d)$; quantising it with the scaled Gaussian codebook gives error D/d per coordinate and hence a total squared error D for the unit vector. The constant $\sqrt{3}\pi/2 \approx 2.72$ is the quantity I interpret as the "gap of about 2.7" in the TurboQuant paper's comparison with the rate–distortion lower bound 2^{-2b}. [DER] [2ND]

A.9 Chunkwise form of linear attention and the gated delta rule

The recurrent form of (ungated) linear attention is $S_t = S_{t-1} + k_t v_t^\top$, $o_t = S_t^\top q_t$. Unrolling, $o_t = \sum_{s \le t}(q_t^\top k_s)v_s$, the parallel form $O = (QK^\top \odot M)V$ with the causal mask M. Split the sequence in chunks of length C and write $Q_{[c]}, K_{[c]}, V_{[c]} \in \mathbb{R}^{C \times \cdot}$ and $S_{[c]}$ for the state at the start of chunk c. Then within the chunk

$$O_{[c]} = Q_{[c]}S_{[c]} + \big(Q_{[c]}K_{[c]}^\top \odot M\big)V_{[c]}, \qquad S_{[c+1]} = S_{[c]} + K_{[c]}^\top V_{[c]}, \tag{A.8}$$

which uses dense matrix products of size $C \times d$ (tensor-core friendly) and one state update per chunk. The cost per token is $O(Cd + d^2)$. The simulation of Chapter 5 confirms numerically that the three forms agree to 2.6×10^{-13} in float64 [SIM].

For the delta rule, $S_t = S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$ in the orientation where S maps keys to values; chunking requires representing the product of Householder-like factors $\prod(I - \beta_s k_s k_s^\top)$ compactly (the WY representation), which is what lets the delta rule use matmuls inside the chunk [21]. The gated variant multiplies the state by a data-dependent scalar $\alpha_t \in (0,1)$ before the update [22]; the cumulative products of the α_t inside a chunk enter as diagonal scalings of the intra-chunk matrices, which is why gating adds little to the cost. I have not re-derived the WY form here beyond this outline.

A.10 Interference in an additive associative memory

Store n pairs (k_j, v_j) with k_j independent uniform on the unit sphere of $\mathbb{R}^d$ and v_j independent unit vectors: $S = \sum_j v_j k_j^\top$. Recalling with k_1 gives $\hat{y} = Sk_1 = v_1 + \sum_{j>1}(k_j^\top k_1)v_j$. Since $\mathbb{E}(k_j^\top k_1)^2 = 1/d$ and the cross terms between distinct j vanish in expectation for independent zero-mean-direction values,

$$\mathbb{E}\|\hat{y} - v_1\|^2 = \frac{n-1}{d}, \qquad \|\hat{y}\|^2 \approx 1 + \frac{n-1}{d},$$

and, by concentration for large d,

$$\cos(\hat{y}, v_1) \approx \frac{1}{\sqrt{1 + (n-1)/d}}, \tag{A.9}$$

which is eq. (5.4). At $n = d$ the cosine is about 0.71; at $n = 4d$, 0.50. The simulation of Chapter 5 agrees with this formula; the delta rule, which overwrites along k_j, recalls the most recent pair exactly but loses older pairs when $n > d$. [DER] [SIM]

A.11 Hierarchical selection: optimal block size

A two-stage selector first scores T/B block summaries and then scores the κB tokens in κ surviving blocks. The scan cost in index keys read is $T/B + \kappa B$, minimised over B at

$$B^\star = \sqrt{T/\kappa}, \qquad \text{cost}^\star = 2\sqrt{\kappa T}. \tag{A.10}$$

For $T = 10^6$ and $\kappa = 16$, $B^\star = 250$ and the scan costs 8000 keys, against 10^6 for the flat scan: a 125× reduction before accounting for recall loss. A third level gives $3(\kappa_1\kappa_2 T)^{1/3}$ keys at the optimum, a diminishing return. The price is the recall of the first stage: a needle inside a block is found only if the block summary ranks among the top κ, which Proposition 4.4 bounds above by the flat selection. HISA's two-stage design follows this scheme [52]. [DER]

A.12 From KL to captured mass: Pinsker and its limits

Let q be the indexer's distribution and $S = \text{top-}k(q)$. For the true attention p,

$$p(S) \geq q(S) - \text{TV}(p, q) \geq q(S) - \sqrt{\tfrac{1}{2}\,\text{KL}(p\|q)}$$

by Pinsker's inequality. If the indexer is trained to a KL of ε nats, the captured mass is within $\sqrt{\varepsilon/2}$ of the indexer's own mass on its top-k. This is a worst-case statement: for $\varepsilon = 0.02$ it allows a loss of 0.1 of mass, whereas the distilled indexers' practical loss is smaller since the top-k set only needs to be right at the top and not in the tail. The bound also shows what KL training does *not* control: it penalises errors in proportion to p_i and so ignores small-weight keys, exactly the keys that a long-tail aggregation needs. [DER]

Appendix B

Experiments: Setup, Results and Code

This appendix documents every experiment that produced a number in the main text marked [SIM], lists the scripts verbatim, and states the limitations of each. All scripts are in `experiments/` of the working directory; results are JSON files in `results/`; tables and figures generated from them are in `tex/gen/` and `figures/`.

B.1 Environment

Python 3 with NumPy 2.5, SciPy 1.18, Matplotlib 3.11, PyTorch 2.14 (CPU build) and Transformers 5.21. The hardware was a CPU-only virtual machine with 2 cores and about 8 GB of memory; this limited the real-model experiments to a 0.6B-parameter model, FP32 eager attention and 2048-token windows. The real-model experiments use the Hugging Face checkpoint `Qwen/Qwen3-0.6B` (28 layers, 16 query heads, 8 KV heads, head dimension 128) and a public-domain English novel from Project Gutenberg as text; windows start at token offsets 0, 6000 and 14000 after the first line "CHAPTER I". Random seeds are fixed in each script (see listings). No GPU measurement of any kernel was made, and no result of this report is a latency or throughput measurement.

B.2 Summary of experiments and results

B.3 Limitations specific to the experiments

- **Scale.** The real-model experiments use a 0.6B model at 2048 tokens. Attention concentration, sink behaviour and cross-layer similarity at 128K–1M tokens in 100B+ models may differ materially.
- **Text.** A single genre (fiction) from one source; code, mathematics and multi-document prompts may have different attention structure.
- **Windows.** Three windows for top-k and cross-layer statistics; the quantisation and eviction experiments used tensors saved by an *earlier version* of `exp_real_qwen.py` for layers 2, 9, 16 and 23 (a single window); the saving code was subsequently disabled to reduce memory use, so re-running `exp_f_real_quant_evict.py` requires re-enabling it. The eviction experiment used layers 9 and 16, heads 0, 5, 10 and 15 only.
- **Proxy metrics.** Retained attention mass and attention-output error are proxies for task quality; no downstream task was evaluated.

Experiment	Question	Principal result
A (synthetic top-k)	does the closed-form mass match simulation?	Euler–Maclaurin Zipf mass agrees to three digits; lognormal formula validated; incoherent-value error formula within ratio 0.91–1.19
B (block, needle)	block versus token selection; needle probability	closed form agrees within about three standard errors
C (quantisation)	rotation and Lloyd–Max	MSE $0.3606, 0.1158, 0.0340, 0.00938$ for $b = 1..4$; outlier test $11\times$ /$6.9\times$ /$5.0\times$ better with rotation at 2/3/4 bits; MSE-only inner-product slopes 0.885/0.965/0.991; QJL unbiased, RMSE 2–2.4× higher
D (linear capacity)	recall of additive and delta states	matches $1/\sqrt{1+(n-1)/d}$; delta rule worse than additive for $n > d$; chunkwise error 2.6×10^{-13}
E (cost model)	bytes and FLOPs per layer	indexer is 88% of bytes at 128K; 7.7× byte reduction, 22.6× with $m = 4$; CLSA 73× fewer attention bytes than the Transformer at 128K
Real top-k	coverage of Qwen3-0.6B attention	$m_{128} = 0.926$, worst head 0.447; error/tail ratio ≈ 1.8; sink mass 0.44
Real cross-layer	overlap of shared sets	raw overlap 0.61 (gap 1) falls to 0.44 when sink and local window are removed; mass ratio $0.94 \to 0.70$
Real quant / evict	key quantisation, eviction policies	see Tables 7.3 and 7.4

Table B.1: Summary of experiments (details in the listed chapters).

- **Quantisation baselines.** The per-channel baseline estimates its scales over the whole sequence including future tokens, which is not causal; the rotation scheme omits the QJL stage and bit-packing.
- **Eviction policies.** The H2O-style and random policies are my simple implementations of the described ideas, not the original code.

B.4 Script listings

B.4.1 exp_a_synthetic_topk.py

Synthetic top-k mass and output error under Zipf, Gaussian-logit and lognormal attention (Chapter 4). Output: `results/synthetic_topk.json`.

```
"""Synthetic top-k attention approximation experiments (own simulation)."""
import json, numpy as np
rng = np.random.default_rng(0)
OUT = "/home/claude/paper3_sparse_attn/results/"
N = 32768; d = 64
KS = [16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192]
res = {}
# (1) Zipf attention p_i ~ rank^-alpha
V = rng.standard_normal((N, d)) / np.sqrt(d)    # values with E||v||^2 = 1
zipf = {}
```

```
for a in [0.6, 0.8, 1.0, 1.2, 1.5]:
    p = np.arange(1, N + 1, dtype=float) ** (-a); p /= p.sum()
    perm = rng.permutation(N)                    # random location of ranks
    o = (p[:, None] * V[perm]).sum(0)
    row = {}
    for k in KS:
        m = p[:k].sum()
        oh = (p[:k, None] * V[perm][:k]).sum(0) / m
        row[k] = {"mass": m, "tail": 1 - m, "err_renorm": float(np.linalg.norm(o - oh)), "err_noren"
    : float(np.linalg.norm(o - (p[:k, None] * V[perm][:k]).sum(0))),
                  "rel_err_renorm": float(np.linalg.norm(o - oh) / np.linalg.norm(o))}
    zipf[a] = row
res["zipf"] = zipf
# (2) Gaussian logits N(0, sigma^2)
gauss = {}
for s in [1.0, 2.0, 3.0, 4.0, 5.0]:
    masses = {k: [] for k in KS}
    for _ in range(20):
        z = rng.standard_normal(N) * s; p = np.exp(z - z.max()); p /= p.sum(); ps = np.sort(p)[::-1]
        for k in KS: masses[k].append(ps[:k].sum())
    gauss[s] = {k: float(np.mean(v)) for k, v in masses.items()}
res["gauss"] = gauss
# (3) Indexer noise: true logits Gaussian sigma=3 (dense-ish) ; indexer score = z + noise*tau ;
    select top-k by score
noise = {}
k = 2048
for s in [2.0, 3.0]:
    for tau in [0.0, 0.5, 1.0, 2.0, 4.0]:
        ms = []; rec = []
        for _ in range(20):
            z = rng.standard_normal(N) * s; p = np.exp(z - z.max()); p /= p.sum()
            sc = z + tau * rng.standard_normal(N)
            sel = np.argpartition(-sc, k)[:k]
            ms.append(p[sel].sum())
            orac = np.argpartition(-z, k)[:k]
            rec.append(len(np.intersect1d(sel, orac)) / k)
        noise[f"s{s}_tau{tau}"] = {"mass": float(np.mean(ms)), "recall_vs_oracle": float(np.mean(rec
    ))}
res["indexer_noise"] = noise
# (4) Needle: one key gets logit boost Delta over a background of N(0,1) logits. Probability needle
    in top-k selected by noisy indexer.
needle = {}
for delta in [2.0, 4.0, 6.0, 8.0]:
    for tau in [1.0, 2.0, 3.0]:
        hit = 0; trials = 400; capt = []
        for _ in range(trials):
            z = rng.standard_normal(N); z[0] += delta
            sc = z + tau * rng.standard_normal(N)
            sel = np.argpartition(-sc, 2048)[:2048]
            hit += int(0 in sel)
            p = np.exp(z - z.max()); p /= p.sum(); capt.append(p[0])
        needle[f"delta{delta}_tau{tau}"] = {"hit": hit / trials, "needle_attn_mass_mean": float(np.
    mean(capt))}
res["needle"] = needle
json.dump(res, open(OUT + "synthetic_topk.json", "w"), indent=1)
print("ok")
```

B.4.2 exp_b_block_needle.py

Block versus token selection and needle retrieval through a noisy indexer, with Monte-Carlo checks of the closed forms of Propositions 4.4 and 4.5. Output: `results/block_needle.json`.

```
"""(1) Block-granular vs token-granular oracle selection under spatial clustering of heavy tokens (
    own simulation).
```

```
(2) Validation of the closed-form needle-hit probability Phi(D/sqrt(1+t^2) - Phi^-1(1-q)).
(3) Validation of the lognormal-mass formula m = Phi(sigma - Phi^-1(1-q))."""
import json, numpy as np
from scipy.stats import norm
rng = np.random.default_rng(7)
OUT = "/home/claude/paper3_sparse_attn/results/"
N = 32768; res = {}
# (1) Zipf weights placed with spatial clustering: ranks are laid out along a 1-D random walk of cluster centres.
def make_p(alpha, cluster):
    p = np.arange(1, N + 1, dtype=float) ** (-alpha); p /= p.sum()
    if cluster == 0:
        perm = rng.permutation(N)                       # iid placement
    else:
        # heavy tokens appear in runs of length `cluster`: assign ranks in chunks to contiguous runs at random locations
        nchunk = N // cluster; order = rng.permutation(nchunk)
        perm = np.concatenate([np.arange(c * cluster, (c + 1) * cluster) for c in order])
    q = np.empty(N); q[perm] = p     # q[position]=weight ; rank r goes to position perm[r]
    return q
blk = {}
for alpha in [0.8, 1.0, 1.2]:
    for cluster in [0, 8, 32, 128]:
        q = make_p(alpha, cluster)
        tok_sorted = np.sort(q)[::-1]
        for B in [16, 64, 128]:
            bm = q.reshape(-1, B).sum(1); bs = np.sort(bm)[::-1]
            for ktok in [512, 2048]:
                nb = ktok // B
                blk[f"a{alpha}_c{cluster}_B{B}_k{ktok}"] = {"token_oracle": float(tok_sorted[:ktok].sum()), "block_oracle": float(bs[:nb].sum())}
res["block_vs_token"] = blk
# (2) needle closed form vs 4000-trial simulation
Nn = 32768; k = 2048; qf = k / Nn
needle = {}
for delta in [2.0, 4.0, 6.0, 8.0]:
    for tau in [1.0, 2.0, 3.0]:
        trials = 3000; hit = 0
        for _ in range(trials):
            z = rng.standard_normal(Nn); z[0] += delta
            sc = z + tau * rng.standard_normal(Nn)
            thr = np.partition(sc, Nn - k)[Nn - k]
            hit += int(sc[0] >= thr)
        pred = norm.cdf(delta / np.sqrt(1 + tau ** 2) - norm.ppf(1 - qf))
        needle[f"d{delta}_t{tau}"] = {"sim": hit / trials, "pred": float(pred), "se": float(np.sqrt(pred * (1 - pred) / trials))}
res["needle_check"] = needle
# (3) lognormal mass formula
ln = {}
for s in [1.0, 2.0, 3.0, 4.0]:
    for N_ in [4096, 32768, 262144]:
        ms = []
        for _ in range(8):
            z = rng.standard_normal(N_) * s; p = np.exp(z - z.max()); p /= p.sum(); ps = np.sort(p)[::-1]; ms.append(ps[:2048 if N_ > 2048 else N_].sum())
        ln[f"s{s}_N{N_}"] = {"sim": float(np.mean(ms)), "pred": float(norm.cdf(s - norm.ppf(1 - 2048 / N_)))}
res["lognormal_check"] = ln
json.dump(res, open(OUT + "block_needle.json", "w"), indent=1)
for k_, v in needle.items(): print(k_, round(v["sim"], 3), round(v["pred"], 3), round(v["se"], 3))
for k_, v in ln.items(): print(k_, round(v["sim"], 3), round(v["pred"], 3))
for k_ in ["a1.0_c0_B64_k2048", "a1.0_c32_B64_k2048", "a1.0_c128_B64_k2048", "a1.0_c0_B16_k2048", "a1.0_c0_B128_k2048", "a1.2_c0_B128_k2048", "a1.2_c128_B128_k2048"]: print(k_, blk[k_])
```

B.4.3 exp_c_quant.py

Random rotation and Lloyd–Max quantisation: MSE for 1–4 bits, outlier test, inner-product bias and the QJL residual (Chapter 7). Output: `results/quant_sim.json`.

```
"""Random-rotation scalar quantization (TurboQuant-style stage 1) + 1-bit QJL residual: own re-implementation / simulation."""
import json, numpy as np
rng = np.random.default_rng(1)
OUT = "/home/claude/paper3_sparse_attn/results/"
d = 128; n = 4000

def lloyd_max(b, iters=200):
    x = np.sort(rng.standard_normal(2_000_000)); L = 2 ** b
    c = np.quantile(x, (np.arange(L) + 0.5) / L)
    for _ in range(iters):
        bnd = (c[:-1] + c[1:]) / 2
        idx = np.searchsorted(bnd, x)
        c = np.array([x[idx == j].mean() for j in range(L)])
    return c
books = {b: lloyd_max(b) for b in [1, 2, 3, 4]}
def quant(z, cb):   # z standard-normal scaled coordinates
    bnd = (cb[:-1] + cb[1:]) / 2
    return cb[np.searchsorted(bnd, z)]
def rand_rot(d):
    q, r = np.linalg.qr(rng.standard_normal((d, d))); return q * np.sign(np.diag(r))
res = {"mse": {}, "paper_bounds_unit": {1: 0.36, 2: 0.117, 3: 0.03, 4: 0.009}, "lower_bound_4^-b": {b: 4.0 ** -b for b in [1, 2, 3, 4]}}
# unit vectors
X = rng.standard_normal((n, d)); X /= np.linalg.norm(X, axis=1, keepdims=True)
Rm = rand_rot(d)
for b in [1, 2, 3, 4]:
    Z = (X @ Rm.T) * np.sqrt(d)            # coordinates ~ N(0,1)
    Zq = quant(Z, books[b]); Xh = (Zq / np.sqrt(d)) @ Rm
    res["mse"][b] = float(np.mean(np.sum((X - Xh) ** 2, axis=1)))
# adversarial/outlier data: few huge channels. Compare (i) naive per-vector uniform min-max quantization, (ii) rotation+Lloyd-Max
def naive_uniform(X, b):
    lo = X.min(1, keepdims=True); hi = X.max(1, keepdims=True); L = 2 ** b - 1
    q = np.round((X - lo) / (hi - lo) * L); return q / L * (hi - lo) + lo
Xo = rng.standard_normal((n, d)); Xo[:, :4] *= 25.0   # 4 outlier channels, 25x magnitude
Xo /= np.linalg.norm(Xo, axis=1, keepdims=True)
out = {}
for b in [2, 3, 4]:
    e_naive = np.mean(np.sum((Xo - naive_uniform(Xo, b)) ** 2, 1))
    Z = (Xo @ Rm.T) * np.sqrt(d); Xh = (quant(Z, books[b]) / np.sqrt(d)) @ Rm
    e_rot = np.mean(np.sum((Xo - Xh) ** 2, 1))
    out[b] = {"naive_uniform_mse": float(e_naive), "rotated_lloydmax_mse": float(e_rot)}
res["outlier"] = out
# inner product bias & variance: x quantized (MSE-only b bits) vs prod variant ((b-1) bit MSE + 1-bit QJL on residual)
Y = rng.standard_normal((n, d)); Y /= np.linalg.norm(Y, axis=1, keepdims=True)
true_ip = np.sum(X * Y, 1)
ip = {}
S = rng.standard_normal((d, d))
for b in [2, 3, 4]:
    # MSE-only at b bits
    Z = (X @ Rm.T) * np.sqrt(d); Xh_b = (quant(Z, books[b]) / np.sqrt(d)) @ Rm
    est_mse = np.sum(Xh_b * Y, 1)
    # prod: (b-1)-bit MSE + QJL residual
    Zm = quant(Z, books[b - 1]); Xh_m = (Zm / np.sqrt(d)) @ Rm
    r = X - Xh_m; rn = np.linalg.norm(r, axis=1, keepdims=True)
    sgn = np.sign(r @ S.T)                  # 1 bit per coord
    qjl = np.sqrt(np.pi / 2) / d * rn[:, 0] * np.sum((Y @ S.T) * sgn, 1)
    est_prod = np.sum(Xh_m * Y, 1) + qjl
    def stats(est):
```

```
        err = est - true_ip
        slope = np.sum(est * true_ip) / np.sum(true_ip ** 2)    # regression slope (1 = unbiased scale)
        return {"mean_err": float(err.mean()), "rmse": float(np.sqrt((err ** 2).mean())), "slope": float(slope)}
    ip[b] = {"mse_only": stats(est_mse), "prod": stats(est_prod)}
res["inner_product"] = ip
res["ip_paper_bound_x_over_d"] = {1: 1.57, 2: 0.56, 3: 0.18, 4: 0.047}
# attention-logit effect on correlated high-dim data with softmax: keys cluster (non-zero mean), query random
json.dump(res, open(OUT + "quant_sim.json", "w"), indent=1)
print(json.dumps(res, indent=1)[:2500])
```

B.4.4 exp_d_linear_capacity.py

Recall capacity of additive and delta-rule linear states, and equivalence of the recurrent, parallel and chunkwise forms (Chapter 5). Output: `results/linear_capacity.json`.

```
"""Associative-recall capacity of fixed-size states: vanilla linear attention, delta rule, gated delta rule vs softmax KV cache. Own simulation."""
import json, numpy as np
rng = np.random.default_rng(2)
OUT = "/home/claude/paper3_sparse_attn/results/"
def run(d, n, trials=20, beta=1.0, alpha=1.0, beta_sm=None):
    out = {"linear": [], "delta": [], "gated_delta": [], "softmax": []}
    for _ in range(trials):
        K = rng.standard_normal((n, d)); K /= np.linalg.norm(K, axis=1, keepdims=True)
        Vv = rng.standard_normal((n, d)); Vv /= np.linalg.norm(Vv, axis=1, keepdims=True)
        S_lin = np.zeros((d, d)); S_del = np.zeros((d, d)); S_gd = np.zeros((d, d))
        for t in range(n):
            k, v = K[t], Vv[t]
            S_lin += np.outer(v, k)
            S_del = S_del + beta * np.outer(v - S_del @ k, k)                    # S_t = S_{t-1}(I - beta k k^T) + beta v k^T
            S_gd = alpha * S_gd; S_gd = S_gd + beta * np.outer(v - S_gd @ k, k)  # gated: decay then delta (alpha<1 => forgets)
        def score(S):
            Vh = K @ S.T
            cos = np.sum(Vh * Vv, 1) / (np.linalg.norm(Vh, axis=1) * np.linalg.norm(Vv, axis=1) + 1e-12)
            return cos.mean()
        out["linear"].append(score(S_lin)); out["delta"].append(score(S_del)); out["gated_delta"].append(score(S_gd))
        # softmax retrieval over stored n pairs with sharp temperature
        sc = 30.0 * (K @ K.T); sc -= sc.max(1, keepdims=True); P = np.exp(sc); P /= P.sum(1, keepdims=True)
        Vh = P @ Vv
        out["softmax"].append((np.sum(Vh * Vv, 1) / (np.linalg.norm(Vh, axis=1))).mean())
    return {k: float(np.mean(v)) for k, v in out.items()}
res = {}
for d in [32, 64, 128]:
    res[d] = {n: run(d, n, trials=10 if d < 128 else 6) for n in [4, 8, 16, 32, 48, 64, 96, 128, 192, 256, 384, 512] if n >= 4}
# gated delta with decay: recall of the *oldest* items vs newest, d=64 n=128, alpha in {1,0.99,0.97}
dec = {}
d = 64; n = 128
for alpha in [1.0, 0.99, 0.97, 0.9]:
    cs_old, cs_new = [], []
    for _ in range(10):
        K = rng.standard_normal((n, d)); K /= np.linalg.norm(K, axis=1, keepdims=True)
        Vv = rng.standard_normal((n, d)); Vv /= np.linalg.norm(Vv, axis=1, keepdims=True)
        S = np.zeros((d, d))
        for t in range(n):
            S = alpha * S; S = S + np.outer(Vv[t] - S @ K[t], K[t])
```

```
        Vh = K @ S.T; cos = np.sum(Vh * Vv, 1) / (np.linalg.norm(Vh, axis=1) + 1e-12)
        cs_old.append(cos[:16].mean()); cs_new.append(cos[-16:].mean())
    dec[alpha] = {"oldest16": float(np.mean(cs_old)), "newest16": float(np.mean(cs_new))}
res["decay"] = dec
# numerical equivalence parallel vs recurrent linear attention (causal, no normalization), float64
T, d = 256, 32
Q = rng.standard_normal((T, d)); K = rng.standard_normal((T, d)); Vv = rng.standard_normal((T, d))
par = np.tril(Q @ K.T) @ Vv
S = np.zeros((d, d)); rec = np.zeros((T, d))
for t in range(T):
    S += np.outer(K[t], Vv[t]); rec[t] = Q[t] @ S
# chunkwise form with chunk size C
C = 32; S = np.zeros((d, d)); chk = np.zeros((T, d))
for c in range(0, T, C):
    q, k, v = Q[c:c+C], K[c:c+C], Vv[c:c+C]
    inter = q @ S; intra = np.tril(q @ k.T) @ v
    chk[c:c+C] = inter + intra; S += k.T @ v
res["equivalence"] = {"max_abs_diff_recurrent": float(np.abs(par - rec).max()), "
    max_abs_diff_chunkwise": float(np.abs(par - chk).max())}
json.dump(res, open(OUT + "linear_capacity.json", "w"), indent=1)
for d in [32, 64, 128]:
    print(d, {n: {k: round(v, 3) for k, v in r.items()} for n, r in res[d].items() if n in (8, 32,
    64, 128, 256)})
print(res["decay"]); print(res["equivalence"])
```

B.4.5 exp_e_costmodel.py

Byte and FLOP cost model for dense GQA, MLA, DSA-like layers, indexer sharing and sparse/linear hybrids (Chapter 3); generates the `tab_*.tex` files. Output: `results/costmodel.json`.

```
"""Analytical FLOP/byte cost models for decode-time attention (own derivations; hardware numbers are
    public NVIDIA H100 SXM spec assumptions)."""
import json, numpy as np
OUT = "/home/claude/paper3_sparse_attn/results/"; GEN = "/home/claude/paper3_sparse_attn/tex/gen/"
BW = 3.35e12; PEAK = 989e12          # H100 SXM: HBM3 bytes/s, BF16 dense FLOP/s (public spec
    assumption)
Ts = [8192, 32768, 131072, 524288, 1048576]
res = {}
def tstr(T): return f"{T//1024}K" if T < 1048576 else "1M"
# ---- configs (verified from HF config.json where noted)
qwen = dict(L=36, H=32, G=8, dh=128)                    # Qwen3-8B config.json
dsv = dict(L=61, H=128, c_kv=512, c_rope=64, HI=64, dI=128, k=2048)   # DeepSeek-V3.2-Exp config.
    json
# ---- KV bytes per token and totals
def kv_gqa(cfg, b=2): return 2 * cfg["L"] * cfg["G"] * cfg["dh"] * b
def kv_mla(cfg, b=2): return cfg["L"] * (cfg["c_kv"] + cfg["c_rope"]) * b
def idx_cache(cfg, bytes_per=128 + 4): return cfg["L"] * bytes_per     # FP8 index key (128 B) + 4B
    scale (assumed layout)
mem = {"qwen3_8b_bf16": kv_gqa(qwen), "qwen3_8b_int4_approx": kv_gqa(qwen, 0.5), "dsv32_mla_bf16":
    kv_mla(dsv), "dsv32_indexer_fp8": idx_cache(dsv),
      "clsa_4b_global_kv": 2 * 4 * 128 * 2, "transformer_4b_equiv": 2 * 32 * 4 * 128 * 2}
res["bytes_per_token"] = mem
res["kv_gib"] = {name: {tstr(T): v * T / 2**30 for T in Ts} for name, v in mem.items()}
# ---- decode per-layer bytes/flops
def dense_gqa(T, cfg=qwen, b=2):
    return 2 * cfg["G"] * cfg["dh"] * T * b, 4 * cfg["H"] * cfg["dh"] * T
def dense_mla(T, cfg=dsv, b=2):
    c = cfg["c_kv"] + cfg["c_rope"]
    return c * b * T, 2 * cfg["H"] * (c + cfg["c_kv"]) * T
def dsa(T, cfg=dsv, b=2, idx_b=132):
    c = cfg["c_kv"] + cfg["c_rope"]; k = min(cfg["k"], T)
    ib, ifl = idx_b * T, 2 * cfg["HI"] * cfg["dI"] * T
```

```
    sb, sfl = c * b * k, 2 * cfg["H"] * (c + cfg["c_kv"]) * k
    return ib + sb, ifl + sfl, ib, ifl, sb, sfl
def blocksparse(T, cfg=qwen, b=2, dI=128, k_tok=2048):
    ib, ifl = dI * b * T, 2 * cfg["G"] * dI * T          # one index query head per KV group, one shared index key head
    sb, sfl = 2 * cfg["G"] * cfg["dh"] * min(k_tok, T) * b, 4 * cfg["H"] * cfg["dh"] * min(k_tok, T)
    return ib + sb, ifl + sfl, ib, ifl, sb, sfl
def tmax(by, fl): return max(by / BW, fl / PEAK)
rows = {}
for T in Ts:
    d1b, d1f = dense_gqa(T); d2b, d2f = dense_mla(T); s = dsa(T); bsp = blocksparse(T)
    rows[tstr(T)] = {"gqa_dense": [d1b, d1f], "mla_dense": [d2b, d2f], "dsa_total": [s[0], s[1]], "dsa_indexer": [s[2], s[3]], "dsa_sparse": [s[4], s[5]],
                     "blk_total": [bsp[0], bsp[1]], "blk_indexer": [bsp[2], bsp[3]], "blk_sparse": [bsp[4], bsp[5]]}
res["decode_per_layer"] = rows
res["intensity"] = {"gqa_dense_flop_per_byte": qwen["H"] / qwen["G"], "mla_dense_flop_per_byte": dense_mla(1)[1] / dense_mla(1)[0], "ridge": PEAK / BW}
# ---- shared routing amortization (DSA-like, T=128K): per-layer total bytes when indexer shared over m layers
T = 131072; s = dsa(T); amort = {}
for m in [1, 2, 4, 8, 16, 61]:
    by = s[2] / m + s[4]; fl = s[3] / m + s[5]
    amort[m] = {"bytes": by, "flops": fl, "speedup_vs_dense_bytes": dense_mla(T)[0] / by, "indexer_share": (s[2] / m) / by}
res["amortization_dsa_128K"] = amort
# CLSA-style: 32 layers; first 16 SWA (window 512), 16 cross layers share KV + indexer
def clsa(T, m=16, W1=512, W2=2048, L=32, G=4, dh=128, H=20, b=2, dI=128):
    self_b = (L // 2) * 2 * G * dh * min(W1, T) * b
    ib = dI * b * T
    cross_b = (L // 2) * 2 * G * dh * min(W2, T) * b
    return self_b + cross_b + ib
def dense_t(T, L=32, G=4, dh=128, b=2): return L * 2 * G * dh * T * b
res["clsa_decode_bytes"] = {tstr(T): {"clsa": clsa(T), "transformer": dense_t(T), "ratio": dense_t(T) / clsa(T)} for T in Ts}
# ---- hybrid sparse+linear: L=32, Qwen3-8B-like dims, fraction f of layers sparse(top-k=2048 + indexer), rest linear w/ state H_l*dk*dv fp32? use bf16 state
def hybrid(T, f, L=32, G=8, dh=128, Hl=32, dk=128, dv=128, k=2048, b=2, idx_per_tok=132.0):
    """f = fraction of layers that are sparse-attention layers (token budget k + index scan at idx_per_tok bytes/token/layer);
    remaining layers are linear-attention layers with a per-head state Hl*dk*dv elements (BF16 state assumed; real kernels often keep FP32)."""
    ns = int(round(f * L)); nl = L - ns
    sp = ns * (2 * G * dh * min(k, T) * b + idx_per_tok * T)
    ln = nl * (Hl * dk * dv * b)
    stored = ns * (2 * G * dh * b + idx_per_tok) * T + nl * Hl * dk * dv * b
    return sp + ln, stored
hyb = {}
for f in [1.0, 0.5, 0.25, 0.125, 1/16.0, 0.0]:
    hyb[f"{f:.4f}"] = {tstr(T): {"decode_bytes_tok": hybrid(T, f)[0], "decode_bytes_blk": hybrid(T, f, idx_per_tok=132.0/64)[0], "stored_gib": hybrid(T, f)[1] / 2**30,
                                 "dense_decode_bytes": 32 * 2 * 8 * 128 * 2 * T} for T in [131072, 1048576]}
res["hybrid"] = hyb
# ---- prefill FLOPs (per layer, causal), dense vs token-sparse with indexer
def prefill(T, H=32, dh=128, k=2048, HI=64, dI=128):
    dense = 2 * H * dh * T * T                       # causal QK^T + PV: 2 * (2 H dh T^2 / 2)
    idx = HI * dI * T * T                            # causal: 2*HI*dI*T^2/2
    sp = 4 * H * dh * T * min(k, T)
    return dense, idx, sp
res["prefill_flops_per_layer"] = {tstr(T): dict(zip(["dense", "indexer", "sparse"], prefill(T))) for T in Ts}
json.dump(res, open(OUT + "costmodel.json", "w"), indent=1, default=float)
# ---- LaTeX tables
def e(x): return f"{x:.3g}"
with open(GEN + "tab_kvmem.tex", "w") as f:
```

```
    f.write("\\begin{tabular}{lrrrrr}\\toprule\nConfiguration & B/token & 32K & 128K & 512K & 1M (GiB)\\\\\\\\midrule\n")
    labels = {"qwen3_8b_bf16": "Qwen3-8B GQA, BF16", "qwen3_8b_int4_approx": "Qwen3-8B GQA, 4-bit (ideal, no scales)", "dsv32_mla_bf16": "DeepSeek-V3.2 MLA latent, BF16",
              "dsv32_indexer_fp8": "DSA indexer keys, FP8 (assumed 132 B/layer)", "transformer_4b_equiv": "4B transformer (32L, 4 KV heads), BF16", "clsa_4b_global_kv": "4B YOCO/CLSA shared global KV, BF16"}
    for kname, lab in labels.items():
        v = mem[kname]; g = res["kv_gib"][kname]
        f.write(f"{lab} & {v:,.0f} & {g['32K']:.2f} & {g['128K']:.2f} & {g['512K']:.2f} & {g['1M']:.2f}\\\\\n")
    f.write("\\bottomrule\\end{tabular}\n")
with open(GEN + "tab_decode_mla.tex", "w") as f:
    f.write("\\begin{tabular}{lrrrrrr}\\toprule\n & \\multicolumn{2}{c}{Dense MLA} & \\multicolumn{2}{c}{DSA total} & \\multicolumn{2}{c}{of which indexer}\\\\\n$T$ & MB & GFLOP & MB & GFLOP & MB & GFLOP\\\\\\midrule\n")
    for T in Ts:
        r = rows[tstr(T)]
        f.write(f"{tstr(T)} & {r['mla_dense'][0]/1e6:.1f} & {r['mla_dense'][1]/1e9:.2f} & {r['dsa_total'][0]/1e6:.1f} & {r['dsa_total'][1]/1e9:.2f} & {r['dsa_indexer'][0]/1e6:.1f} & {r['dsa_indexer'][1]/1e9:.2f}\\\\\n")
    f.write("\\bottomrule\\end{tabular}\n")
with open(GEN + "tab_amort.tex", "w") as f:
    f.write("\\begin{tabular}{rrrrr}\\toprule\nShare group $m$ & Bytes/layer (MB) & GFLOP/layer & Indexer share of bytes & Byte reduction vs dense\\\\\\midrule\n")
    for m, r in amort.items():
        f.write(f"{m} & {r['bytes']/1e6:.2f} & {r['flops']/1e9:.2f} & {100*r['indexer_share']:.0f}\\% & {r['speedup_vs_dense_bytes']:.1f}$\\times$\\\\\n")
    f.write("\\bottomrule\\end{tabular}\n")
with open(GEN + "tab_hybrid.tex", "w") as f:
    f.write("\\begin{tabular}{rrrrrrr}\\toprule\nSparse layers & Sparse:linear & \\multicolumn{2}{c}{Decode MB/token @128K} & \\multicolumn{2}{c}{Decode MB/token @1M} & Stored GiB @1M\\\\\n & & token-idx & block-idx & token-idx & block-idx & (token-idx)\\\\\\midrule\n")
    for fr, r in hyb.items():
        fv = float(fr); ns = int(round(fv * 32)); ratio = f"{ns}:{32-ns}"
        f.write(f"{fv:.3f} & {ratio} & {r['128K']['decode_bytes_tok']/1e6:.0f} & {r['128K']['decode_bytes_blk']/1e6:.0f} & {r['1M']['decode_bytes_tok']/1e6:.0f} & {r['1M']['decode_bytes_blk']/1e6:.0f} & {r['1M']['stored_gib']:.2f}\\\\\n")
    r = hyb["1.0000"]
    f.write(f"\\midrule dense GQA (ref.) & 32 full & \\multicolumn{{2}}{{c}}{{{r['128K']['dense_decode_bytes']/1e6:.0f}}} & \\multicolumn{{2}}{{c}}{{{r['1M']['dense_decode_bytes']/1e6:.0f}}} & {r['1M']['dense_decode_bytes']/2**30:.0f}\\\\\n")
    f.write("\\bottomrule\\end{tabular}\n")
print(json.dumps(res["intensity"]), json.dumps(res["clsa_decode_bytes"]["128K"]))
print({m: (round(r["bytes"]/1e6,2), round(r["speedup_vs_dense_bytes"],1), round(r["indexer_share"],2)) for m, r in amort.items()})
print(rows["128K"]["dsa_indexer"], rows["128K"]["dsa_sparse"], rows["128K"]["mla_dense"])
```

B.4.6 exp_real_qwen.py

Real-model attention statistics on Qwen3-0.6B (top-k mass and error, sink and locality, cross-layer overlap). Run as three processes, one per window (`run_real.sh`). Output: `results/real_qwen_stats.json`.

```
"""Real-model attention statistics on Qwen3-0.6B (CPU, eager attention).
Measures: top-k attention mass, top-k output error, sink/local mass, cross-layer top-k overlap,
and saves K/V/Q + attention tensors for quantization/eviction post-analysis.
"""
import json, sys, time, numpy as np, torch, torch.nn.functional as F
import transformers.models.qwen3.modeling_qwen3 as M
from transformers import AutoTokenizer, AutoModelForCausalLM
torch.set_num_threads(2)
OUT = "/home/claude/paper3_sparse_attn/results/"
```

```
SCR = "/tmp/claude-0/-home-claude/edf7f09b-fbfa-5fa7-86ee-2b925800d2e1/scratchpad/"
T = 2048; R0 = 1024
KS = [16, 32, 64, 128, 256, 512, 1024]
SAVE_LAYERS = [2, 9, 16, 23]
name = "Qwen/Qwen3-0.6B"
tok = AutoTokenizer.from_pretrained(name)
model = AutoModelForCausalLM.from_pretrained(name, dtype=torch.float32, attn_implementation="eager").eval()
text = open(SCR + "book.txt", encoding="utf-8", errors="ignore").read()
text = text[text.find("CHAPTER I"):]
ids_all = tok(text, return_tensors="pt").input_ids[0]
print("tokens available", len(ids_all))
state = {}
def patched(module, query, key, value, attention_mask, scaling, dropout=0.0, **kw):
    li = module.layer_idx
    kk = M.repeat_kv(key, module.num_key_value_groups); vv = M.repeat_kv(value, module.num_key_value_groups)
    H_ = query.shape[1]
    outs = []
    st = {"mass": {k: np.zeros(H_) for k in KS}, "err": {k: np.zeros(H_) for k in KS}}
    sink = np.zeros(H_); loc = 0.0
    Asum = torch.zeros(T - R0, T)
    for h0 in range(0, H_, 4):
        w = torch.matmul(query[:, h0:h0+4], kk[:, h0:h0+4].transpose(2, 3)) * scaling
        if attention_mask is not None: w = w + attention_mask
        w = F.softmax(w, dim=-1, dtype=torch.float32)
        outs.append(torch.matmul(w, vv[:, h0:h0+4]))
        Ph = w[0, :, R0:, :]; Vh = vv[0, h0:h0+4]
        o_full = torch.matmul(Ph, Vh)
        srt, idx = torch.topk(Ph, max(KS), dim=-1)
        for k in KS:
            mk = srt[..., :k].sum(-1)
            mask = torch.zeros_like(Ph).scatter_(-1, idx[..., :k], 1.0)
            Ps = Ph * mask / mk.unsqueeze(-1)
            rel = (torch.matmul(Ps, Vh) - o_full).norm(dim=-1) / o_full.norm(dim=-1)
            st["mass"][k][h0:h0+4] = mk.mean(-1).numpy(); st["err"][k][h0:h0+4] = rel.mean(-1).numpy()
        sink[h0:h0+4] = Ph[..., 0].mean(-1).numpy()
        loc += np.mean([Ph[:, i, max(0, R0 + i - 63):R0 + i + 1].sum(-1).mean().item() for i in range(0, Ph.shape[1], 16)]) * 4
        Asum += Ph.sum(0)
        del w, Ph, srt, idx, o_full, mask, Ps
    st["sink"] = sink; st["local64"] = loc / H_
    st["A"] = (Asum / H_).numpy().astype(np.float16)
    state.setdefault("layers", {})[li] = st
    out = torch.cat(outs, dim=1)
    return out.transpose(1, 2).contiguous(), None
M.eager_attention_forward = patched
SEG = int(sys.argv[1]); results = json.load(open(OUT + 'real_qwen_stats.json')) if SEG > 0 else []
for seg, off in [(SEG, [0, 6000, 14000][SEG])]:
    ids = ids_all[off:off + T].unsqueeze(0)
    state.clear(); state["seg"] = seg
    t0 = time.time()
    with torch.no_grad(): model(ids)
    L = state["layers"]; nl = len(L)
    rec = {"seg": seg, "off": off}
    rec["mass"] = {k: np.array([L[l]["mass"][k] for l in range(nl)]).tolist() for k in KS}   # [layer, head]
    rec["err"] = {k: np.array([L[l]["err"][k] for l in range(nl)]).tolist() for k in KS}
    rec["sink"] = np.array([L[l]["sink"] for l in range(nl)]).tolist()
    rec["local64"] = [float(L[l]["local64"]) for l in range(nl)]
    # cross-layer analysis on head-averaged attention
    kset = [64, 256]
    cross = {}
    for k in kset:
        tops = [np.argsort(-L[l]["A"].astype(np.float32), axis=-1)[:, :k] for l in range(nl)]
```

```
        own = np.array([np.take_along_axis(L[l]["A"].astype(np.float32), tops[l], -1).sum(-1).mean() for l in range(nl)])
        # for gap g: mass of layer l+g captured by layer l's top-k set; and Jaccard overlap
        gaps = {}
        for g in [1, 2, 3, 4, 8]:
            ms, js = [], []
            for l in range(nl - g):
                A2 = L[l + g]["A"].astype(np.float32)
                ms.append(np.take_along_axis(A2, tops[l], -1).sum(-1).mean())
                inter = np.mean([len(set(tops[l][r]) & set(tops[l + g][r])) for r in range(0, tops[l].shape[0], 8)])
                js.append(inter / k)
            gaps[g] = {"mass_shared": float(np.mean(ms)), "overlap_frac": float(np.mean(js))}
        cross[k] = {"own_mass_by_layer": own.tolist(), "gaps": gaps}
    rec["cross"] = cross
    results.append(rec)
    json.dump(results, open(OUT + 'real_qwen_stats.json', 'w'))
    import gc; gc.collect()
    print("seg", seg, "done", time.time() - t0, flush=True)
json.dump(results, open(OUT + "real_qwen_stats.json", "w"))
print("saved")
```

B.4.7 exp_real_cross_ctrl.py

Cross-layer overlap with a control for the sink and the last 64 tokens, and per-head overlap for layers 6, 12 and 18 (Section 6.6). Output: `results/cross_ctrl_seg*.json`.

```
"""Cross-layer top-k overlap on Qwen3-0.6B with a control that removes the trivially shared positions
(token 0 = sink, and the 64 most recent tokens). Also per-head (not head-averaged) overlap for a few layers."""
import json, sys, time, numpy as np, torch, torch.nn.functional as F
import transformers.models.qwen3.modeling_qwen3 as M
from transformers import AutoTokenizer, AutoModelForCausalLM
torch.set_num_threads(2)
OUT = "/home/claude/paper3_sparse_attn/results/"; SCR = "/tmp/claude-0/-home-claude/edf7f09b-fbfa-5fa7-86ee-2b925800d2e1/scratchpad/"
T = 2048; R0 = 1024; SEG = int(sys.argv[1]); OFF = [0, 6000, 14000][SEG]
tok = AutoTokenizer.from_pretrained("Qwen/Qwen3-0.6B")
model = AutoModelForCausalLM.from_pretrained("Qwen/Qwen3-0.6B", dtype=torch.float32, attn_implementation="eager").eval()
text = open(SCR + "book.txt", encoding="utf-8", errors="ignore").read(); text = text[text.find("CHAPTER I"):]
ids = tok(text, return_tensors="pt").input_ids[0][OFF:OFF + T].unsqueeze(0)
As = {}; PH = {}
def patched(module, query, key, value, attention_mask, scaling, dropout=0.0, **kw):
    li = module.layer_idx
    kk = M.repeat_kv(key, module.num_key_value_groups); vv = M.repeat_kv(value, module.num_key_value_groups)
    H_ = query.shape[1]; outs = []; Asum = torch.zeros(T - R0, T); ph = []
    for h0 in range(0, H_, 4):
        w = torch.matmul(query[:, h0:h0+4], kk[:, h0:h0+4].transpose(2, 3)) * scaling
        if attention_mask is not None: w = w + attention_mask
        w = F.softmax(w, dim=-1, dtype=torch.float32)
        outs.append(torch.matmul(w, vv[:, h0:h0+4]))
        Asum += w[0, :, R0:, :].sum(0)
        if li in (6, 12, 18):   # keep a few per-head rows (every 8th query) for per-head overlap
            ph.append(w[0, :, R0::8, :].clone())
    As[li] = (Asum / H_).numpy()
    if ph: PH[li] = torch.cat(ph, 0).numpy()
    return torch.cat(outs, 1).transpose(1, 2).contiguous(), None
M.eager_attention_forward = patched
with torch.no_grad(): model.model(ids)
nl = len(As); rows = T - R0
```

```
pos = np.arange(T)[None, :]; qpos = (R0 + np.arange(rows))[:, None]
trivial = (pos == 0) | (pos > qpos - 64) | (pos > qpos)      # sink, last 64 (incl. self), and non-causal
def topk_idx(A, k, mask=None):
    B = A.copy()
    if mask is not None: B[mask] = -1
    return np.argpartition(-B, k, axis=1)[:, :k]
res = {"seg": SEG}
for k in [64, 256]:
    for ctrl in [False, True]:
        tops = [topk_idx(As[l], k, trivial if ctrl else None) for l in range(nl)]
        gaps = {}
        for g in [1, 2, 4, 8]:
            ov, ms, own_l, sh_l = [], [], [], []
            for l in range(nl - g):
                A2 = As[l + g].copy()
                if ctrl: A2[trivial] = 0
                own = np.take_along_axis(A2, tops[l + g], 1).sum(1) ; sh = np.take_along_axis(A2, tops[l], 1).sum(1)
                ov.append(np.mean([len(np.intersect1d(tops[l][r], tops[l + g][r])) for r in range(0, rows, 8)]) / k)
                own_l.append(own.mean()); sh_l.append(sh.mean())
            gaps[g] = {"overlap": float(np.mean(ov)), "mass_ratio": float(np.sum(sh_l) / np.sum(own_l)), "own_mass": float(np.mean(own_l))}
        res[f"k{k}_ctrl{int(ctrl)}"] = gaps
# remaining mass share held by trivial positions (head-avg)
res["trivial_mass_share_by_layer"] = [float(np.where(trivial, As[l], 0).sum(1).mean()) for l in range(nl)]
# per-head overlap between consecutive anchor layers (6->12 and 12->18), k=64, control applied
ph = {}
for (a, b) in [(6, 12), (12, 18)]:
    Pa, Pb = PH[a], PH[b]; H_ = Pa.shape[0]; r_ = Pa.shape[1]
    tr = trivial[::8][:r_]
    ovs = []; mrs = []
    for h in range(H_):
        A1 = Pa[h].copy(); A2 = Pb[h].copy(); A1[tr] = -1; A2[tr] = 0
        t1 = np.argpartition(-A1, 64, axis=1)[:, :64]; t2 = np.argpartition(-np.where(tr, -1, Pb[h]), 64, axis=1)[:, :64]
        ovs.append(np.mean([len(np.intersect1d(t1[r], t2[r])) for r in range(r_)]) / 64)
        mrs.append(np.take_along_axis(A2, t1, 1).sum(1).mean() / np.take_along_axis(A2, t2, 1).sum(1).mean())
    ph[f"{a}->{b}"] = {"overlap_mean": float(np.mean(ovs)), "overlap_min": float(np.min(ovs)), "overlap_max": float(np.max(ovs)), "mass_ratio_mean": float(np.mean(mrs)), "mass_ratio_min": float(np.min(mrs))}
res["per_head_k64_ctrl"] = ph
json.dump(res, open(OUT + f"cross_ctrl_seg{SEG}.json", "w"), indent=1)
print(json.dumps(res["k64_ctrl1"]), json.dumps(res["k64_ctrl0"]), json.dumps(ph))
```

B.4.8 exp_f_real_quant_evict.py

Key quantisation and eviction policies on saved Qwen3-0.6B tensors (Section 7.6). Output: `results/real_quant_evict.json`.

```
"""Post-hoc analyses on saved Qwen3-0.6B tensors: (1) key-quantization effect on attention; (2) KV eviction policies (retained attention mass)."""
import json, numpy as np
rng = np.random.default_rng(3)
SCR = "/tmp/claude-0/-home-claude/edf7f09b-fbfa-5fa7-86ee-2b925800d2e1/scratchpad/"; OUT = "/home/claude/paper3_sparse_attn/results/"
LAYERS = [2, 9, 16, 23]; R0 = 1024; T = 2048
def lloyd_max(b, iters=100):
    x = np.sort(rng.standard_normal(1_000_000)); L = 2 ** b
    c = np.quantile(x, (np.arange(L) + 0.5) / L)
    for _ in range(iters):
```

```
        bnd = (c[:-1] + c[1:]) / 2; idx = np.searchsorted(bnd, x); c = np.array([x[idx == j].mean() for j in range(L)])
    return c
books = {b: lloyd_max(b) for b in [2, 3, 4]}
def q_cb(z, cb): return cb[np.searchsorted((cb[:-1] + cb[1:]) / 2, z)]
def naive_token(X, b):      # per-token (per-vector) min-max uniform
    lo = X.min(-1, keepdims=True); hi = X.max(-1, keepdims=True); L = 2 ** b - 1
    return np.round((X - lo) / (hi - lo + 1e-9) * L) / L * (hi - lo) + lo
def naive_channel(X, b):    # per-channel min-max over the token axis (KIVI-style for keys, no residual window)
    lo = X.min(-2, keepdims=True); hi = X.max(-2, keepdims=True); L = 2 ** b - 1
    return np.round((X - lo) / (hi - lo + 1e-9) * L) / L * (hi - lo) + lo
d = 128
Qm, Rr = np.linalg.qr(rng.standard_normal((d, d))); Qm = Qm * np.sign(np.diag(Rr))
def rot_lm(X, b):
    nrm = np.linalg.norm(X, axis=-1, keepdims=True); U = X / (nrm + 1e-9)
    Z = (U @ Qm.T) * np.sqrt(d); return (q_cb(Z, books[b]) / np.sqrt(d)) @ Qm * nrm
res = {"quant": {}, "evict": {}}
def attn(Q, K, V, rows0=R0):
    # Q: [H,rows,d], K/V: [Hkv,T,d]; GQA groups
    H = Q.shape[0]; g = H // K.shape[0]
    Kr = np.repeat(K, g, 0); Vr = np.repeat(V, g, 0)
    lg = np.einsum("hrd,htd->hrt", Q, Kr) / np.sqrt(Q.shape[-1])
    mask = np.arange(K.shape[1])[None, :] > (rows0 + np.arange(Q.shape[1]))[:, None]
    lg = np.where(mask[None], -1e30, lg); lg -= lg.max(-1, keepdims=True); P = np.exp(lg); P /= P.sum(-1, keepdims=True)
    return P, np.einsum("hrt,htd->hrd", P, Vr)
for li in LAYERS:
    z = np.load(SCR + f"layer{li}.npz"); K, V, Q = z["K"].astype(np.float64), z["V"].astype(np.float64), z["Q"].astype(np.float64)
    P0, O0 = attn(Q, K, V)
    for b in [2, 3, 4]:
        for name, fn in [("naive_per_token", naive_token), ("naive_per_channel", naive_channel), ("rot_lloydmax", rot_lm)]:
            Kq = fn(K, b); P1, O1 = attn(Q, Kq, V)
            kl = np.sum(P0 * (np.log(P0 + 1e-30) - np.log(P1 + 1e-30)), -1).mean()
            rel = (np.linalg.norm(O1 - O0, axis=-1) / np.linalg.norm(O0, axis=-1)).mean()
            res["quant"].setdefault(f"{name}_b{b}", []).append({"layer": li, "kl": float(kl), "rel_out_err": float(rel)})
    # K statistic: outlier-ness (max |channel| / median |channel| in mean abs per channel)
    ch = np.abs(K).mean(1).mean(0); res.setdefault("key_channel_stats", {})[li] = {"max_over_median_channel": float(ch.max() / np.median(ch))}
# ---- eviction: retained attention mass under budget B, using the dense attention probabilities P (all rows)
def evict_sim(P, B, policy, sinks=4, recent=32):
    # P: [T,T] causal dense attention for one head. Returns mean retained mass over rows t>=R0 of the cache set that a policy would hold.
    Tn = P.shape[0]; acc = np.zeros(Tn); cache = []; ret = []
    inset = np.zeros(Tn, bool)
    for t in range(Tn):
        inset[t] = True
        if inset.sum() > B:
            idxs = np.where(inset)[0]
            if policy == "stream":
                keep = set(idxs[:sinks]) | set(idxs[-(B - sinks):]); drop = [i for i in idxs if i not in keep][:1]
                for i in drop: inset[i] = False
            elif policy == "h2o":
                prot = set(idxs[:sinks]) | set(idxs[-recent:]); cand = [i for i in idxs if i not in prot]
                j = cand[int(np.argmin(acc[cand]))]; inset[j] = False
            elif policy == "random":
                prot = set(idxs[:sinks]) | set(idxs[-recent:]); cand = [i for i in idxs if i not in prot]
                inset[cand[rng.integers(len(cand))]] = False
        # attention at step t over cached set (renormalised): update H2O accumulators
```

```
        row = P[t] * inset; s = row.sum(); row = row / max(s, 1e-12)
        acc += row
        if t >= R0: ret.append(P[t][inset].sum())
    return float(np.mean(ret))
def oracle(P, B):
    vals = []
    for t in range(R0, P.shape[0]):
        vals.append(np.sort(P[t, : t + 1])[::-1][:B].sum())
    return float(np.mean(vals))
for li in [9, 16]:
    z = np.load(SCR + f"layer{li}.npz"); Pfull = z["P"].astype(np.float64)   # [H,T,T]
    for B in [128, 256, 512]:
        for pol in ["stream", "h2o", "random"]:
            vals = [evict_sim(Pfull[h], B, pol) for h in [0, 5, 10, 15]]
            res["evict"].setdefault(f"{pol}_B{B}", []).append({"layer": li, "retained_mass": float(
    np.mean(vals))})
        res["evict"].setdefault(f"oracle_topk_B{B}", []).append({"layer": li, "retained_mass": float
    (np.mean([oracle(Pfull[h], B) for h in [0, 5, 10, 15]]))})
json.dump(res, open(OUT + "real_quant_evict.json", "w"), indent=1)
for k, v in res["quant"].items(): print(k, [round(x["rel_out_err"], 4) for x in v], [round(x["kl"],
    5) for x in v])
for k, v in res["evict"].items(): print(k, [round(x["retained_mass"], 3) for x in v])
print(res["key_channel_stats"])
```

Appendix C

Evidence Ledger

The ledger lists the principal quantitative or factual claims used in this report with their source type and verification status. *Derived* means proved in this report; *Measured* means produced by my own experiment (with its limits); *Author-reported* means taken from the authors' paper as retrieved (abstract or full text), not re-run; *Vendor* and *Secondary* are as defined in Chapter 1. No claim in this ledger is verified by independent replication unless stated. Retrieval was by web search and page fetch on or before 27 October 2026; summaries were produced from retrieved pages, and some papers were read only at abstract level.

Claim	Source	Status	Notes
Top-k truncation identity $o - \hat{o} = \tau(\bar{o}_{S^c} - \hat{o}_S)$	derivation (Ch. 4)	Derived; checked numerically	exact, no assumptions
Zipf top-k mass via Euler–Maclaurin	derivation	Derived; simulated	matches to 3 digits
Gaussian-logit mass $\Phi(\sigma - \Phi^{-1}(1-q))$	derivation	Derived; simulated	asymptotic in T
Block selection $\leq$ token selection at equal token budget	derivation	Derived	exact for oracle scores
Needle selection probability through noisy indexer	derivation	Derived; simulated within 3 s.e.	Gaussian model
Real top-k coverage: $m_{128} = 0.926$, worst head 0.447	own experiment	Measured	Qwen3-0.6B; 3 windows of 2048 tokens
Error/tail-mass ratio ≈ 1.8	own experiment	Measured	same
Sink mass 0.44; layer-dependent locality	own experiment	Measured	same
Cross-layer top-64 overlap 0.61 (raw), 0.44 (controlled), gap 1	own experiment	Measured	small model; short context
Key quantisation: per-channel beats rotation + Lloyd–Max on real keys	own experiment	Measured	4 layers; one window; non-causal scales
H2O-style score retains less mass than window or random	own experiment	Measured	2 layers $\times$ 4 heads; own implementation
Indexer 88% of bytes at 128K (V3.2-style layer)	own cost model	Derived	assumes 132-byte FP8 index key
Sharing over $m = 4$: 22.6$\times$ byte reduction	own cost model	Derived	same assumption
CLSA: 73$\times$ fewer attention bytes at 128K	own cost model	Derived	uses CLSA dimensions from the paper
Hybrid 3:1 linear:sparse stores 33 GiB at 1M	own cost model	Derived	dimension assumptions in Table 5.2
NSA: 9.0$\times$ fwd, 6.0$\times$ bwd, 11.6$\times$ decode (64K, A100)	[42]	Author-reported	pretraining tokens 260B vs 270B inconsistent in retrieved summaries
NSA: avg. 0.456 vs 0.443 full attention	[42]	Author-reported	27B/3B-active MoE
MoBA: RULER 128K 0.7818 vs 0.7849	[43]	Author-reported	Llama-8B-1M conversion
MoBA deployed in Kimi's long-context service	[43]	Author-reported	not independently verified
InfLLM-V2: $\sim 4\times$ speedup, dense–sparse switchable	[44]	Author-reported (abstract only)	details not retrieved
V3.2-Exp matches V3.1-Terminus on benchmarks (mixed $\pm$1–3 pts)	[45, 71]	Vendor, primary table	in-house comparison
DSA: 2.1B warm-up + 943.7B sparse training tokens	[3]	Author-reported	tech report

Claim	Source	Status	Notes
API price reduction by more than half; new rates	[45, 46]	Vendor / secondary	price is not cost
DSA adopted by GLM-5/5.2, Hy4-preview, Naive-N0.5-Flash	[65]	Secondary	not verified from primary model cards
MSA: 28.4× compute, 14.2× prefill, 7.6× decode at 1M	[58]	Author-reported	H800; tables not fully retrieved
MiniMax-M3: 428B/23B active, 128-token blocks, 16 selected	[66]	Secondary	single blog source
GSA: RULER 128K 62.2 vs 31.7	[47]	Author-reported	baseline YaRN-extended; theory section has mismatches
DeepSeek-V4: 27% FLOPs, 10% KV vs V3.2 at 1M (Pro)	[59]	Author-reported	arXiv id/date inconsistent; no independent evaluation retrieved
CLSA: RULER 32K 53.1 vs 52.3 (YOCO), 46.2 (Transformer)	[1]	Author-reported	4B; single seed
CLSA: 7.6× decode, 17.1× overall throughput at 128K	[1]	Author-reported	combines YOCO and sparse
IndexCache: 75% indexer ops removed, 1.82× prefill, 1.48× decode	[51, 68]	Author-reported / secondary	200K figure from secondary
Adjacent-layer top-k overlap 70–100%	[68]	Secondary	my own control measurement is lower (Sec. 6.6)
GLM-5.2 deploys full/shared/shared/shared; 2.9× fewer FLOPs at 1M	[68]	Secondary	not primary
HySparse: KV $\sim$ 10× lower; 5 full layers of 49	[48]	Author-reported	80B MoE
HySparse2: RULER-v2 256K 58.45 vs 32.61	[61]	Author-reported (secondary reader)	read via alphaxiv
SparDA: 1.7× decode vs sparse+offload; 5.3× vs non-offload	[57]	Author-reported	H100; 128K
HISA / MISA reproduce DSA indexer choices training-free	[52, 55]	Author-reported	MISA 3.82× indexer kernel on H200
AsyncTLS: op. 1.2–10.0×, e2e 1.3–4.7×	[54]	Author-reported	ranges inconsistent across sections
TSA: up to 3.23× attention; RULER 87.02 vs 87.01	[49]	Author-reported	abstract vs table maxima differ
MiniCPM-SALA: RULER 128K 89.37; NoLiMa 128K 23.86; 3.5× at 256K	[50]	Author-reported	author-run baselines; MRCR below best
Kimi Linear: −75% KV, up to 6× decode at 1M	[4]	Author-reported (abstract)	3:1 ratio not confirmed in retrieved text
Qwen3-Next: $\sim$ 10× cheaper training and inference vs Qwen3-32B	[70]	Vendor / secondary	ratio of layers not retrieved
MiniMax M2 post: reasons for full attention	[5]	Vendor, primary text	no numbers
MiniMax reversed to sparse attention in M3	[58, 66]	Author-reported / secondary	interpretation of motive is mine
TriAttention: 2.5× throughput / 10.7× KV (AIME25)	[53]	Author-reported	AIME24 figure 42.1 vs 45.8 inconsistent
Random Attention: random retention rivals engineered scores on reasoning	[60]	Author-reported	math-heavy tasks
KV-cache compression pitfalls in multi-instruction prompts	[64]	Author-reported	abstract level
TurboQuant: LongBench 49.44 at 2.5 bits vs 50.06 full	[36]	Author-reported	not reproduced
TurboQuant: $\geq$ 6× memory, 8× logit speedup	[63]	Vendor	baseline 32-bit JAX
TurboQuant: $\sim$ 4.7× at 3 bits on M4 Max; ten implementations	[62]	Secondary	unverified
RaBitQ authors' accusation against TurboQuant	[62]	Secondary, relayed	not verified; included as reception only

Claim	Source	Status	Notes
SubQ: 12M context, 56×, 98% needle, $8 vs $2,600	[56]	Vendor	unverified; only Appen speed/needle test is third party
SubQ “roughly 1,000×” headline	[67]	Secondary	basis untraceable
NoLiMa: GPT-4o 99.3%→69.7% at 32K	[39]	Author-reported	

C.1 Known inconsistencies in retrieved sources

- NSA: pre-training token count given as 260B in one place and 270B in another.
- TriAttention: AIME24 score of 42.1 in one place, 45.8 in another.
- AsyncTLS: speedup ranges differ between abstract, introduction and results.
- TSA: abstract 3.23× versus table maxima of 1.36–2.76×.
- GSA: parts of the theory section do not match the stated setup in the retrieved text.
- DeepSeek-V4: arXiv identifier (2606.19348) and listed submission date (26 April 2026) are inconsistent.
- Kimi Linear: a 3:1 linear-to-full ratio appeared only in a search snippet and is not used as fact.

Appendix D

Glossary

Arithmetic intensity
FLOPs performed per byte moved from memory; decode attention has $2H/(Gb)$.

Attention sink
A position (typically the first token) that receives a large share of attention regardless of content.

Block selection
Choosing contiguous groups of tokens as the unit of sparsity; selection scores come from pooled block keys.

Captured mass
$m(S) = \sum_{i \in S} p_i$, the attention weight falling in the selected set.

Chunkwise form
A way to compute linear attention by processing chunks with dense matrix products and passing a state between chunks.

CSA / HCA
DeepSeek-V4's compressed sparse attention and heavily compressed attention.

Cross-decoder
In YOCO, the layers that read a single shared KV cache produced by the self-decoder.

Delta rule
Associative-memory update that overwrites the stored value along the current key direction.

DSA
DeepSeek Sparse Attention: a trained lightning indexer selects the top-k tokens per query.

GQA / MQA / MLA
Grouped-query, multi-query and multi-head latent attention: ways to reduce cache entries per token.

Hybrid model
A model mixing exact (full or sparse) attention layers with linear-state or sliding-window layers.

Indexer
A small learned module that scores candidate keys to choose which tokens a layer attends to.

IndexCache / IndexShare
Reusing the indices selected at one layer in subsequent layers.

KL divergence
$\mathrm{KL}(p\|q) = \sum_i p_i \log(p_i/q_i)$; the training objective of distilled indexers.

KV cache
Stored keys and values of past tokens for each attention layer.

Lightning attention
A linear-attention variant with a block-wise computation, used in MiniMax-01 and MiniCPM-SALA.

Lloyd–Max quantiser
Scalar quantiser minimising mean-squared error for a given source density.

Mass ratio
Fraction of a layer's own top-k mass recovered by another layer's selected set (eq. (6.3)).

NoLiMa
Long-context benchmark that removes literal overlap between question and answer.

Oracle top-k
The k largest true attention weights; an upper bound for any selector.

Prefill / decode
Processing the prompt in parallel versus generating tokens one at a time.

QJL
Quantised Johnson–Lindenstrauss one-bit residual correction used to remove inner-product bias.

Roofline
Performance model bounded by peak compute and memory bandwidth; ridge point P/W.

RULER
Synthetic long-context benchmark with retrieval, tracing and aggregation tasks.

SALA
MiniCPM's sparse and linear attention hybrid.

Sliding-window attention
Each token attends only to the most recent w tokens.

State (linear attention)
Fixed-size matrix summarising the past in a linear-attention layer.

Tail mass
$\tau = 1 - m(S)$, attention weight outside the selected set.

Top-k
Selection of the k highest-scoring positions.

Vendor claim
A claim by the organisation selling or releasing a product, not independently verified (tag [VENDOR]).

YOCO
You Only Cache Once: a decoder-decoder architecture with a single global KV cache.

Bibliography

[1] Sun et al. Cross-layer sparse attention (clsa) on yoco, 2026. arXiv:2606.06467; title abbreviated; author list abbreviated.

[2] Huiqiang Jiang et al. Minference 1.0: Accelerating pre-filling for long-context llms via dynamic sparse attention, 2024. arXiv:2407.02490.

[3] DeepSeek-AI. Deepseek-v3.2 technical report (deepseek sparse attention), 2025. arXiv:2512.02556; title abbreviated.

[4] Kimi Team. Kimi linear: An expressive, efficient attention architecture, 2025. arXiv:2510.26692.

[5] MiniMax. Why did m2 end up as a full attention model? https://huggingface.co/blog/MiniMax-AI/why-did-m2-end-up-as-a-full-attention-model, 2025. vendor blog post.

[6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2017. arXiv:1706.03762; NeurIPS 2017.

[7] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2021. arXiv:2104.09864.

[8] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers, 2019. arXiv:1904.10509.

[9] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer, 2020. arXiv:2004.05150.

[10] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer, 2020. arXiv:2001.04451.

[11] Noam Shazeer. Fast transformer decoding: One write-head is all you need, 2019. arXiv:1911.02150.

[12] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints, 2023. arXiv:2305.13245.

[13] DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model, 2024. arXiv:2405.04434.

[14] Yutao Sun, Li Dong, Yi Zhu, Shaohan Huang, Wenhui Wang, Shuming Ma, Quanlu Zhang, Jianyong Wang, and Furu Wei. You only cache once: Decoder-decoder architectures for language models, 2024. arXiv:2405.05254.

[15] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness, 2022. arXiv:2205.14135.

[16] Woosuk Kwon et al. Efficient memory management for large language model serving with pagedattention, 2023. arXiv:2309.06180.

[17] Jordan Juravsky, Bradley Brown, Ryan Ehrlich, Daniel Y. Fu, Christopher Ré, and Azalia Mirhoseini. Hydragen: High-throughput llm inference with shared prefixes, 2024. arXiv:2402.05099.

[18] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention, 2020. arXiv:2006.16236.

[19] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2023. arXiv:2312.00752.

[20] Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality, 2024. arXiv:2405.21060.

[21] Songlin Yang et al. Parallelizing linear transformers with the delta rule over sequence length, 2024. arXiv:2406.06484.

[22] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule, 2024. arXiv:2412.06464.

[23] Zhen Qin et al. Lightning attention-2: A free lunch for handling unlimited sequence lengths in large language models, 2024. arXiv:2401.04658.

[24] MiniMax. Minimax-01: Scaling foundation models with lightning attention, 2025. arXiv:2501.08313.

[25] Opher Lieber et al. Jamba: A hybrid transformer-mamba language model, 2024. arXiv:2403.19887.

[26] Samy Jelassi, David Brandfonbrener, Sham M. Kakade, and Eran Malach. Repeat after me: Transformers are better than state space models at copying, 2024. arXiv:2402.01032.

[27] Wang et al. A systematic analysis of hybrid linear attention, 2025. arXiv:2507.06457; author list abbreviated.

[28] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks, 2023. arXiv:2309.17453.

[29] Zhenyu Zhang et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models, 2023. arXiv:2306.14048.

[30] Yuhong Li et al. Snapkv: Llm knows what you are looking for before generation, 2024. arXiv:2404.14469.

[31] Jiaming Tang et al. Quest: Query-aware sparsity for efficient long-context llm inference, 2024. arXiv:2406.10774.

[32] Guangxuan Xiao et al. Duoattention: Efficient long-context llm inference with retrieval and streaming heads, 2024. arXiv:2410.10819.

[33] Wenhao Wu, Yizhong Wang, Guangxuan Xiao, Hao Peng, and Yao Fu. Retrieval head mechanistically explains long-context factuality, 2024. arXiv:2404.15574.

[34] Yizhao Gao et al. Seerattention: Learning intrinsic sparse attention in your llms, 2024. arXiv:2410.13276.

[35] Zirui Liu et al. Kivi: A tuning-free asymmetric 2bit quantization for kv cache, 2024. arXiv:2402.02750.

[36] Amir Zandieh et al. Turboquant: Online vector quantization with near-optimal distortion rate, 2025. arXiv:2504.19874; first posted April 2025; ICLR 2026.

[37] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts, 2023. arXiv:2307.03172.

[38] Cheng-Ping Hsieh et al. Ruler: What's the real context size of your long-context language models?, 2024. arXiv:2404.06654.

[39] Ali Modarressi et al. Nolima: Long-context evaluation beyond literal matching, 2025. arXiv:2502.05167.

[40] Zhuowan Li et al. Retrieval augmented generation or long-context llms? a comprehensive study and hybrid approach, 2024. arXiv:2407.16833.

[41] Piotr Nawrot et al. The sparse frontier: Sparse attention trade-offs in transformer llms, 2025. arXiv:2504.17768.

[42] Jingyang Yuan et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention, 2025. arXiv:2502.11089.

[43] Enzhe Lu et al. Moba: Mixture of block attention for long-context llms, 2025. arXiv:2502.13189.

[44] Weilin Zhao et al. Infllm-v2: Dense-sparse switchable attention for seamless short-to-long adaptation, 2025. arXiv:2509.24663.

[45] DeepSeek. Deepseek-v3.2-exp release announcement and api price reduction. https://deepseek.com/en/news/v3-2-exp/, 2025. web page; accessed during retrieval for this report.

[46] The Decoder. Deepseek slashes api prices by up to 75 percent with its latest v3.2 model. https://the-decoder.com/deepseek-slashes-api-prices-by-up-to-75-percent-with-its-latest-v3-2-model/, 2025. news report (secondary).

[47] Shen et al. Gated sparse attention (gsa), 2026. arXiv:2601.15305; title abbreviated; author list abbreviated.

[48] Gao et al. Hysparse: hybrid sparse attention with oracle selection and kv sharing, 2026. arXiv:2602.03560; title abbreviated; author list abbreviated.

[49] Jo et al. Token sparse attention (tsa), 2026. arXiv:2602.03216; title abbreviated; author list abbreviated.

[50] MiniCPM Team. Minicpm-sala: hybrid sparse and linear attention, 2026. arXiv:2602.11761; title abbreviated.

[51] Bai et al. Indexcache: reusing sparse-attention indices across layers, 2026. arXiv:2603.12201; title abbreviated; author list abbreviated.

[52] Xu et al. Hisa: hierarchical indexed sparse attention, 2026. arXiv:2603.28458; title abbreviated; author list abbreviated.

[53] Mao et al. Triattention: pre-rope trigonometric kv compression, 2026. arXiv:2604.04921; title abbreviated; author list abbreviated.

[54] Hu et al. Asynctls: two-level sparse attention with asynchronous offload, 2026. arXiv:2604.07815; title abbreviated; author list abbreviated.

[55] Zhou et al. Misa: mixture of indexer heads for sparse attention, 2026. arXiv:2605.07363; title abbreviated; author list abbreviated.

[56] MIT Technology Review. A startup claims it broke through a bottleneck that is holding back llms. https://www.technologyreview.com/2026/06/19/1139313/a-startup-claims-it-broke-through-a-bottleneck-thats-holding-back-llms/, 2026. press article, 19 June 2026 (secondary).

[57] Fu et al. Sparda: forecast-based decoupled sparse attention, 2026. arXiv:2606.04511; title abbreviated; author list abbreviated.

[58] MiniMax. Minimax sparse attention (msa), 2026. arXiv:2606.13392; title abbreviated.

[59] DeepSeek-AI. Deepseek-v4 technical report (compressed sparse attention and heavily compressed attention), 2026. arXiv:2606.19348; identifier and date as listed, which are mutually inconsistent; title abbreviated.

[60] Wang et al. Random attention: random kv retention rivals engineered eviction scores, 2026. arXiv:2609.03430; title abbreviated; author list abbreviated.

[61] Wei et al. Hysparse2: follow-up to hysparse, 80b-a3b moe to 256k context, 2026. arXiv:2609.26368; title abbreviated; author list abbreviated; read via alphaxiv.

[62] KeepCoding. Turboquant, un mes despues: implementaciones, controversia, benchmarks. https://keepcoding.io/blog-frr/turboquant-un-mes-despues-implementaciones-controversia-benchmarks/, 2026. blog post in Spanish (secondary); dispute allegations are relayed, not verified.

[63] Google Research. Turboquant: Redefining ai efficiency with extreme compression. https://research.google/blog/turboquant-redefining-ai-efficiency-with-extreme-compression/, 2026. vendor blog post.

[64] Chen et al. Pitfalls of kv cache compression, 2025. arXiv:2510.00231; title abbreviated; author list abbreviated.

[65] Sebastian Raschka. Llm architecture gallery: Deepseek sparse attention. https://www.sebastianraschka.com/llm-architecture-gallery/deepseek-sparse-attention/, 2026. secondary.

[66] Kili Technology. Data story: Minimax-m3. https://kili-technology.com/blog/data-story-minimax-m3, 2026. secondary blog post.

[67] Runtimewire. Subquadratic, mit, llm bottleneck and skepticism. https://runtimewire.com/article/subquadratic-mit-llm-bottleneck-skepticism, 2026. aggregator (secondary); basis of its numbers not traceable.

[68] Sebastian Raschka. Llm architecture gallery: Indexshare. https://sebastianraschka.com/llm-architecture-gallery/indexshare/, 2026. secondary.

[69] The Kaitchup. Commentary on minimax m2 and full attention (only title retrieved). Blog commentary (URL not retained), 2025. secondary; only the title was retrieved.

[70] Simon Willison. Qwen3-next. https://simonwillison.net/2025/Sep/12/qwen3-next/, 2025. blog post (secondary).

[71] DeepSeek-AI. Deepseek-v3.2-exp model card. https://huggingface.co/deepseek-ai/DeepSeek-V3.2-Exp, 2025. model card with benchmark table and configuration.